SKALA

KDT DevOps

DevOps 이해 및 활용

2025.09

본 문서는 SK(주)C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

1일차

1. 실습환경 구성하기
2. DevOps 개요
3. CI 전용 컨테이너빌드 도구
4. Kustomize 이해
5. 패키지 매니저 Helm
6. Helm chart 기본 문법

2 일차

1. Jenkins CI/CD
2. Jenkins CI
3. Jenkin PollSCM
4. k8s에 Jenkins 서버 배포
5. gitOps 기반 CD 구성하기
6. 나만의 파이프라인 구성하기

실습 환경 구성하기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

확인!!! - 실습 코드 내 {학번} 부분 변경 방법

sk{학번} → **sk000** 형태로 표시해주세요

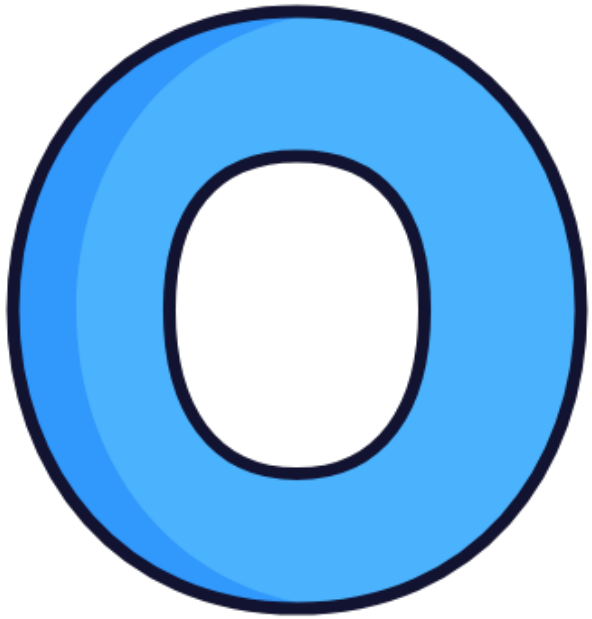
예를 들어, 99 번이면 **sk099**

확인!!! - 이렇게 하지 마세요.

p099 , u099, g099

sk0099

확인!!! - 이렇게 하세요!!!!



소문자 3자리 고유번호

sk099

kubernetes 실습 환경 구성 [판교]

kubernetes 실습을 위한 설치 및 설정 가이드

kubectl 설치

```
brew install kubectl tree helm
```

접속 인증 정보 다운로드

```
# 자신의 작업 디렉토리로 이동
# cd ~/workspace/cloud/

# 다운로드
> curl -f -O https://skala-fileserver.skala-ai.com/cloud/k8s-basic/setup-k8s-pg.tar
> tar xvf setup-k8s-pg.tar
```

또는 아래 링크로 접속후 다운로드 : **https://skala-fileserver.skala-ai.com**

kubernetes 인증 설정 확인하기 [판교]

```
# 디렉토리 이동
> cd ./setup-k8s-pg

# 자신의 클래스를 선택
> chmod +x setup.sh

# 자신의 클래스를 선택후 실행
# 1반 : class-1   교육생 번호: 0000
> ./setup.sh class-1  sk000

# kubernetes 접속을 위한 token & CA 정보가 포함되어 있는지 확인하기
> ls ~/.kube/config

# namespace 변경하기
#namespace 는 cloud: class-cloud     skala 1반: class-1,  2반: class-2   3반 : class-3
> kcns    class-1

# kubernetes 접속 인증 확인
> kubectl get serviceaccount

# namespace skala에 배포되어 있는 pod 검색
> k get pod -n skala

# gen-yaml.sh 파일 위치 확인하기
> which gen-yaml.sh
/Users/{계정}/.kube/gen-yaml.sh
```

kubernetes 실습 환경 구성 [울산]

kubernetes 실습을 위한 설치 및 설정 가이드

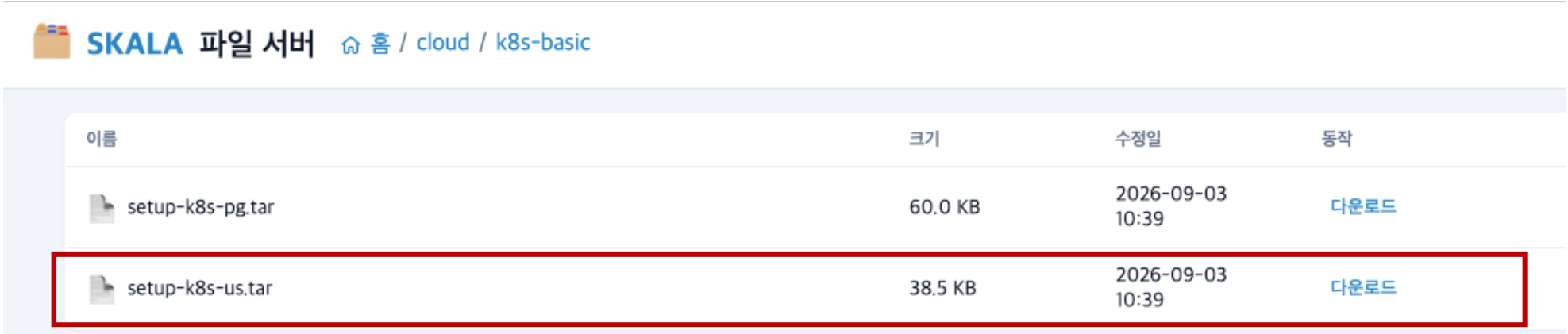
kubectl 설치

```
brew install kubectl
```

접속 인증 정보 다운로드

```
# 자신의 작업 디렉토리로 이동
# cd ~/workspace/cloud/

# 다운로드
> curl -f -O https://skala-fileserver.skala-ai.com/cloud/k8s-basic/setup-k8s-us.tar
> tar xvf setup-k8s-us.tar
```

또는 아래 링크로 접속후 다운로드 : **https://skala-fileserver.skala-ai.com**



kubernetes 인증 설정 확인하기 [울산]

```
# 디렉토리 이동
> cd ./setup-k8s-us

# 자신의 클래스를 선택
> chmod +x setup.sh

# 자신의 클래스를 선택후 실행
# 1반 : class-1   교육생 번호: 0000
> ./setup.sh class-1  sk000

# kubernetes 접속을 위한 token & CA 정보가 포함되어 있는지 확인하기
> ls ~/.kube/config

# namespace 변경하기
#namespace 는 cloud: class-cloud     skala 1반: class-1,  2반: class-2   3반 : class-3
> kcns    class-1

# kubernetes 접속 인증 확인
> kubectl get serviceaccount

# namespace skala에 배포되어 있는 pod 검색
> k get pod -n skala

# gen-yaml.sh 파일 위치 확인하기
> which gen-yaml.sh
/Users/{계정}/.kube/gen-yaml.sh
```

~/.kube/config 인증정보 확인하기

```
# 디렉토리 이동
> cat ~/.kube/config
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0t…klDQVRFLS0tLS0K
    server: https://F6CF0573B26EB0F515847B76DBFD3019.gr7.ap-northeast-2.eks.amazonaws.com
  name: skala-us
contexts:
- context:
    cluster: skala-us
    namespace: class-0
    user: skala-admin-sa
  name: skala-admin-sa@class-0
current-context: skala-admin-sa@class-0
kind: Config
users:
- name: skala-admin-sa
  user:
    token: eyJhbGciOiJSUzI1NiI_K…_ssEarilauQjH6A_mJez2Q
```

실습 코드 다운로드

실습 디렉토리 구성하기

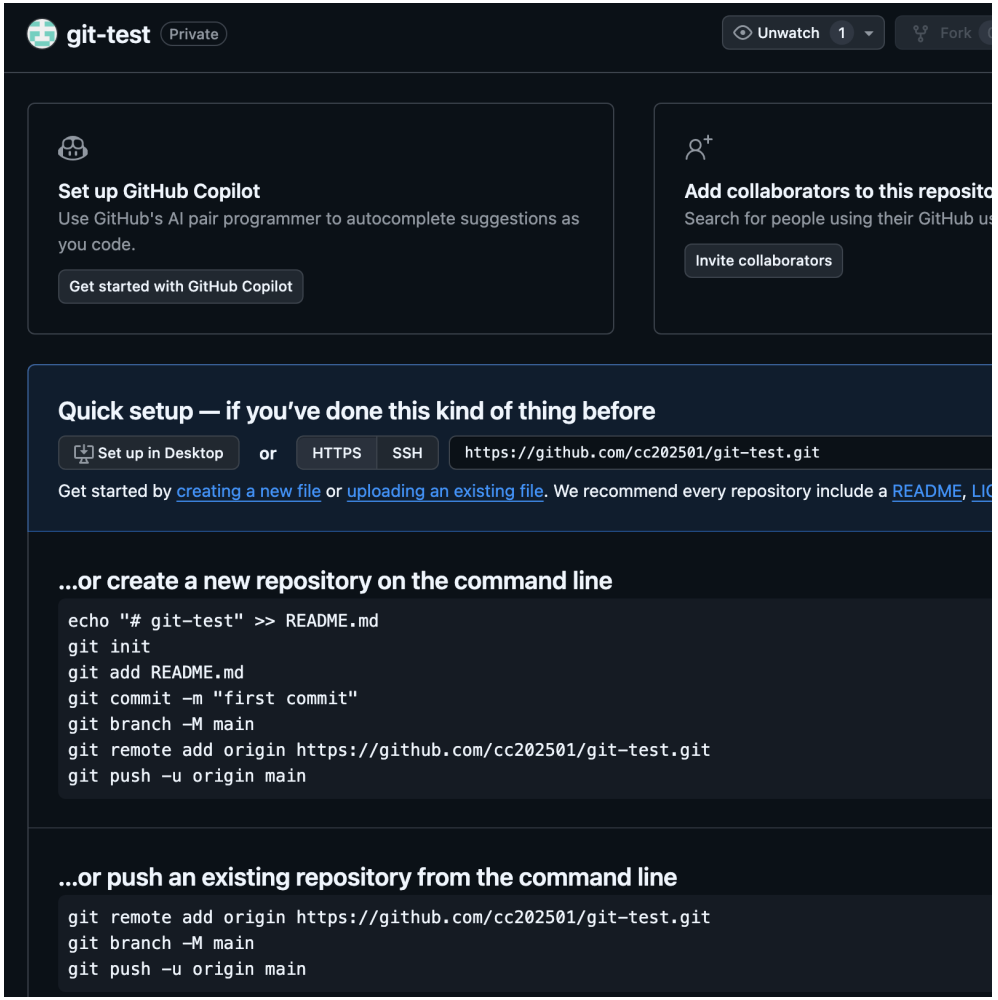
```
# setup reload
> source ~/.zshrc

# home 디렉토리로 이동
> cd

# 작업 디렉토리가 없으면 아래처럼 만들어보자 있다면 스킵
> mkdir workspace
> cd workspace


# 소스 코드 git clone
> git clone https://github.com/himang10/skala-devops.git

> cd skala-devops
```

myfirst-api-server git repository 만들기 (1/2)

github에 자신의 Git repositor를 생성하고 로컬 소스 코드를 push

1. github.com에 로그인
2. github에 repository 생성
   git repository 이름은 “**myfirst-api-server**”

생성하게 되면 다음과 같이 절차가 나온다.



myfirst-api-server git repository 만들기 (2/2)

DevOps 실습을 위한 샘플 코드를 복사하고 이것을 자신의 Git Repository에 등록합니다.

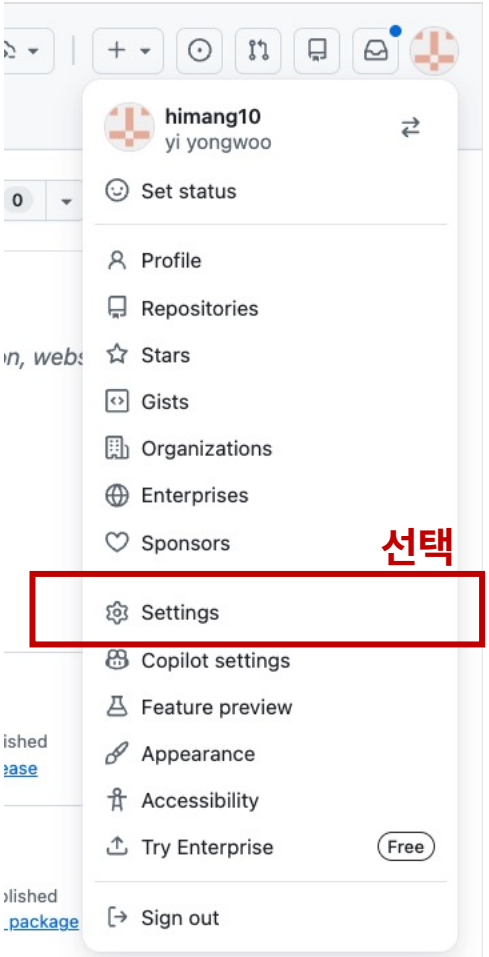
1. git 초기화 및 등록

```
> mkdir ~/workspace/myfirst-api-server
> cd ~/workspace/myfirst-api-server
> git init
> echo "sample code for devops" > README.md
> git add .
> git commit -m "first commit"
> git branch -M main
> git remote add origin https://github.com/{계정이름}/myfirst-api-server.git
```

Peronal Access Token(PAT) 생성하기 (1/5)

Git에서 원격 저장소(특히 GitHub·GitLab) 접근 시 사용자 계정 대신 사용할 수 있는 토큰 기반 인증 수단
ID/Password 대신 토큰 문자열을 이용해 인증

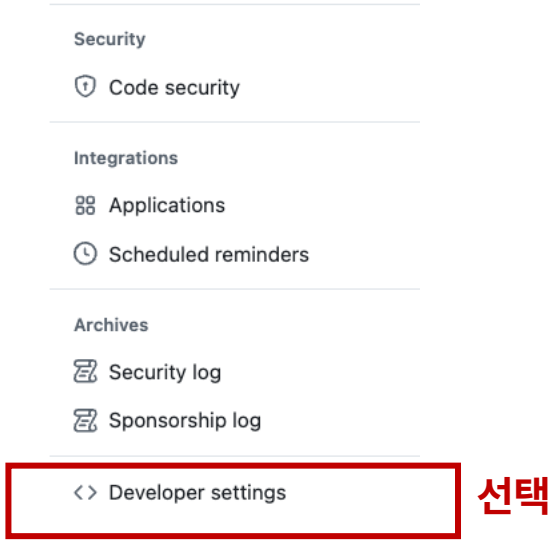
1. github.com/{자신계정}/myfirst-api-server.git 접속하기
2. 오른쪽 상단 계정 버튼 클릭 후 settings 선택



Peronal Access Token(PAT) 생성하기 (2/5)

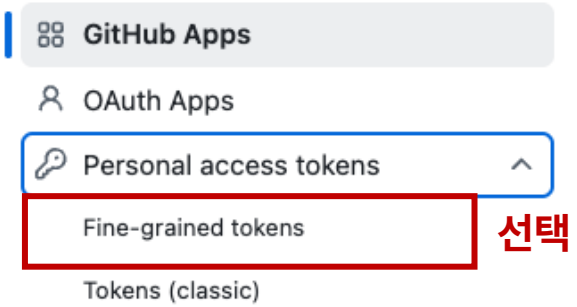
Git에서 원격 저장소(특히 GitHub·GitLab) 접근 시 사용자 계정 대신 사용할 수 있는 토큰 기반 인증 수단
ID/Password 대신 토큰 문자열을 이용해 인증

3. 왼쪽 메뉴 최 하단의 “Developer setttings” 선택



4. 왼쪽 ”Personal Access Token” 선택

5. Fine-grained tokens 선택



6. generate new token 버튼 클릭

Peronal Access Token(PAT) 생성하기 (3/5)

7. token name 등록

New fine-grained personal access token

Create a fine-grained, repository-scoped token suitable for personal API use and for using Git over HTTPS.

Token name *

skala-devops-test

skala-devops-test

8. Expiration 선택 (기본값 사용)

Expiration

30 days (Oct 28, 2026)

The token will expire on the selected date

9. 접근 가능 Repository 선택

Only Select Repositories에서 myfirst-api-server 만 선택

Repository access

- Public repositories
  Read-only access to public repositories.
- All repositories
  This applies to all current and future repositories you own. Also includes public re
- Only select repositories
  Select at least one repository. Max 50 repositories. Also includes public repositori

Select repositories 1

Select repositories

myfirst-api-server

himang10/myfirst-api-server

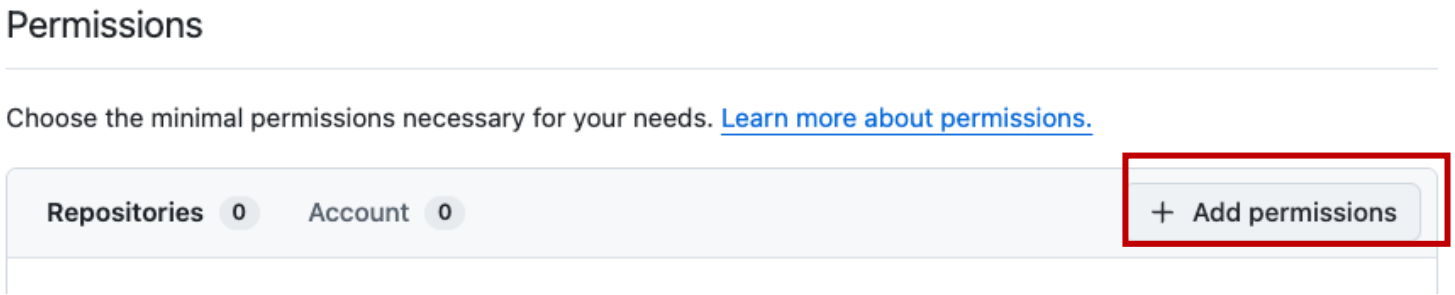
no description

Peronal Access Token(PAT) 생성하기 (4/5)

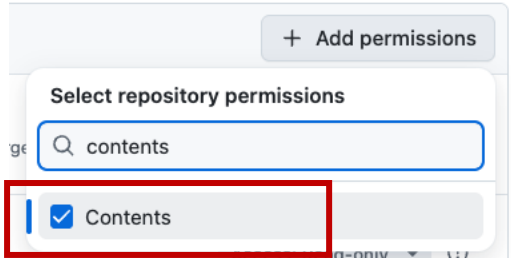
10. Permissions 에 접근 권한 선택

Content의 Read And Write 를 선택해서 Pull & Push 가능하도록 설정

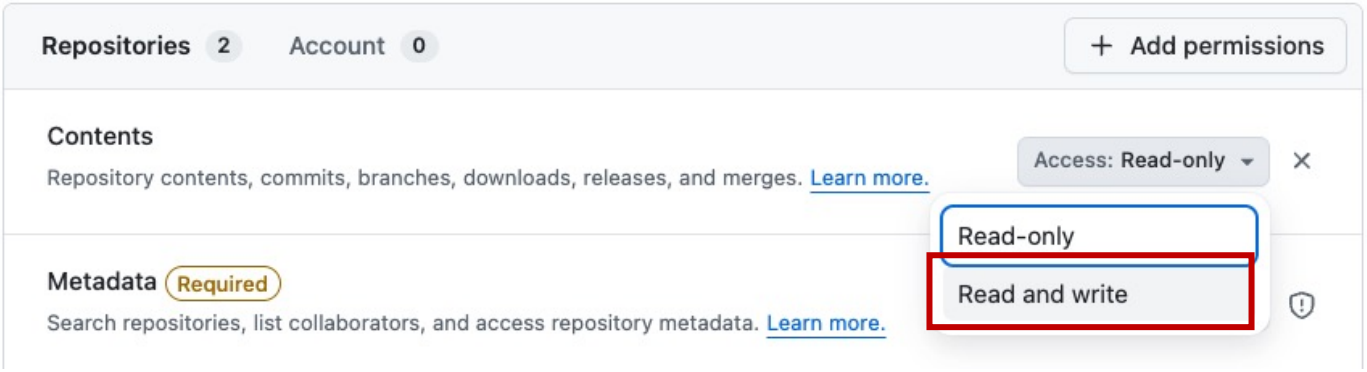
Add permissions 클릭



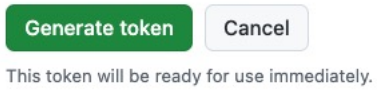
contents 검색 및 선택



read and write 선택

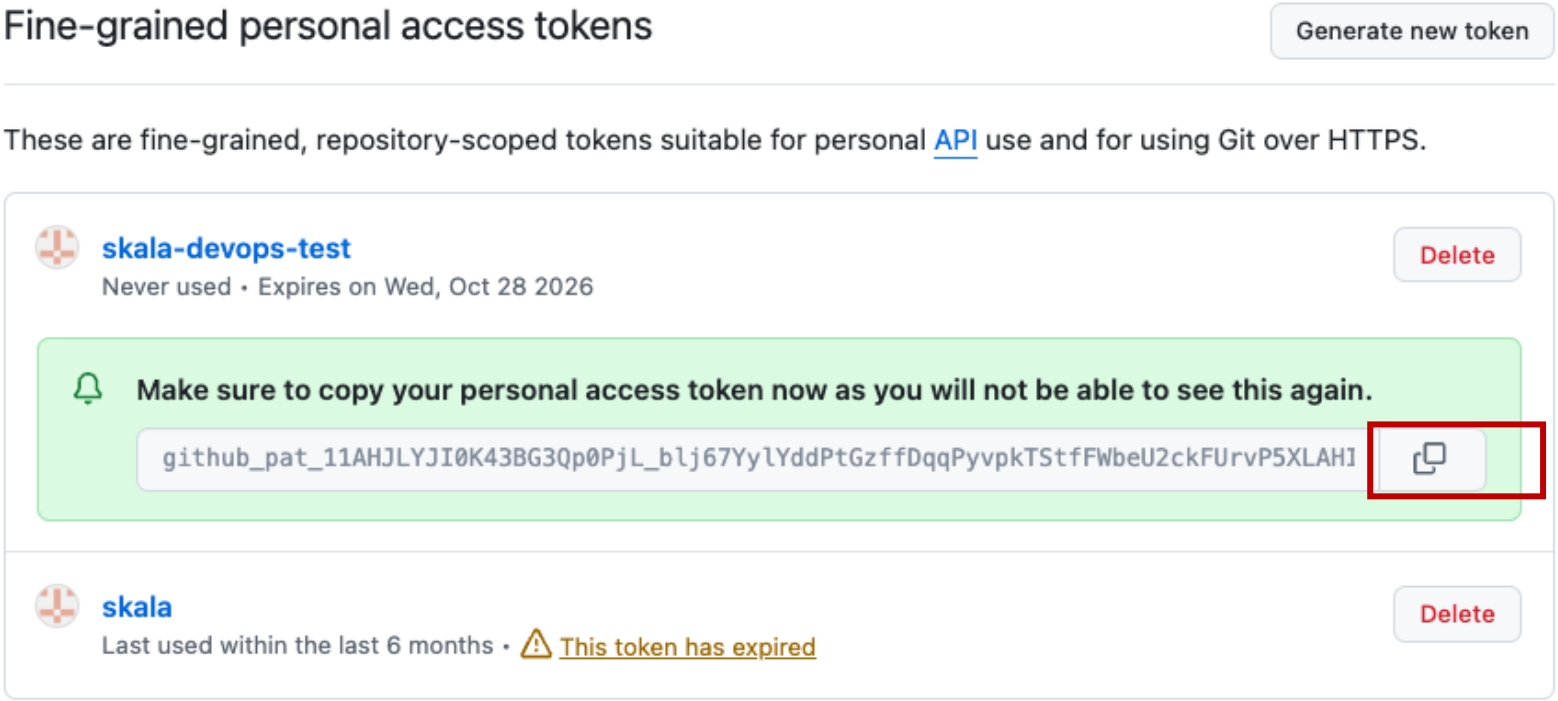


Key Generation



Peronal Access Token(PAT) 생성하기 (5/5)

11. 생성된 Token을 복사해서 잊어버리지 않도록 한다



나의 실습 코드 생성하기 (1/2)

skala-devops/ 아래에 env.properties 파일을 읽어서 아래의 붉은색 부분을 변경해야 합니다.

```
> ls env.properties
> vi env.properties
```

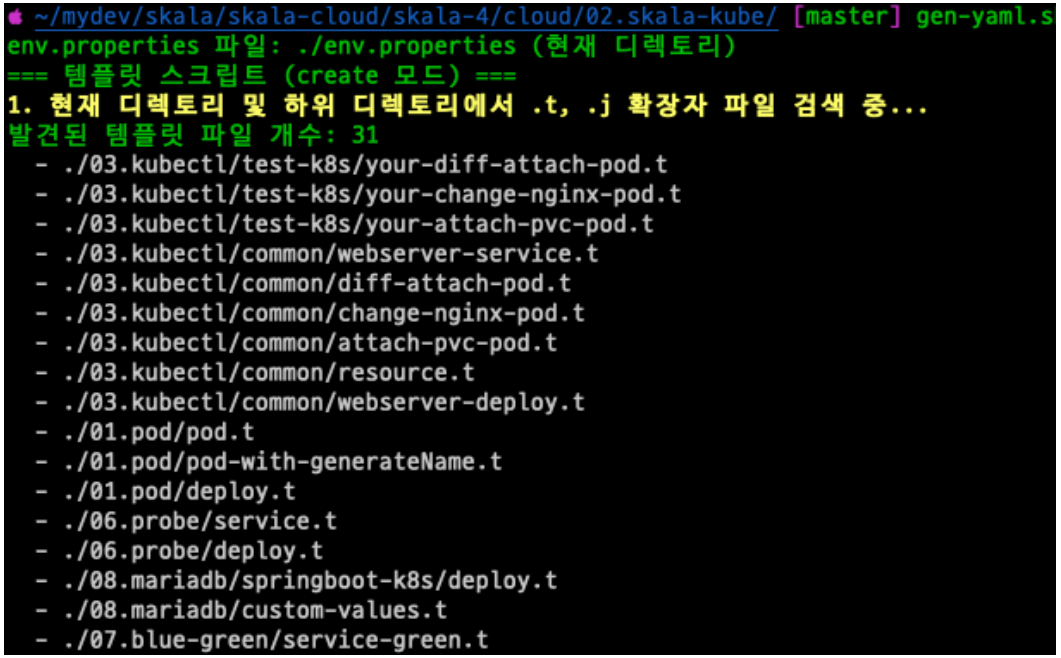
env.properties 내용

```
SERVICE_NAME="myfirst-api-server"

IMAGE_NAME="webserver"
VERSION="1.0"

DOCKER_REGISTRY=${DOCKER_REGISTRY}
DOCKER_REGISTRY_USER=${DOCKER_REGISTRY_USER}
DOCKER_REGISTRY_PASSWORD=${DOCKER_REGISTRY_PASSWORD}
USER_NAME=${STUDENT_NUM}
REPLICAS=1
NAMESPACE=${CLASS_NAME}
PROJECT_NAME=${CLASS_NAME}

GIT_REPO_URL="github.com/{자신의 계정}/myfirst-api-server.git"
GIT_BRANCH="master"
GIT_USERNAME="himang10"
GIT_PASSWORD="PAT Token을 여기에 붙여 주세요”
```

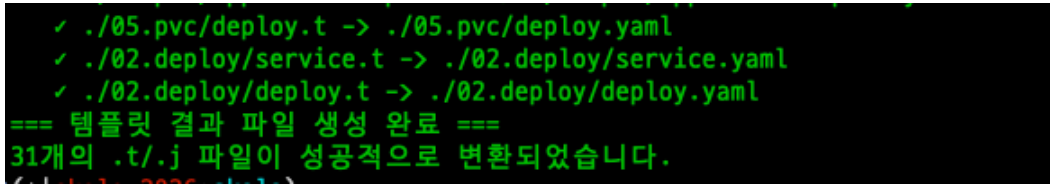
나의 실습 코드 생성하기 (2/2)

env.properties가 있는 디렉토리 위치에서 gen-yaml.sh 를 실행합니다.

```
> gen-yaml.sh
```



...



KDT DevOps – DevOps

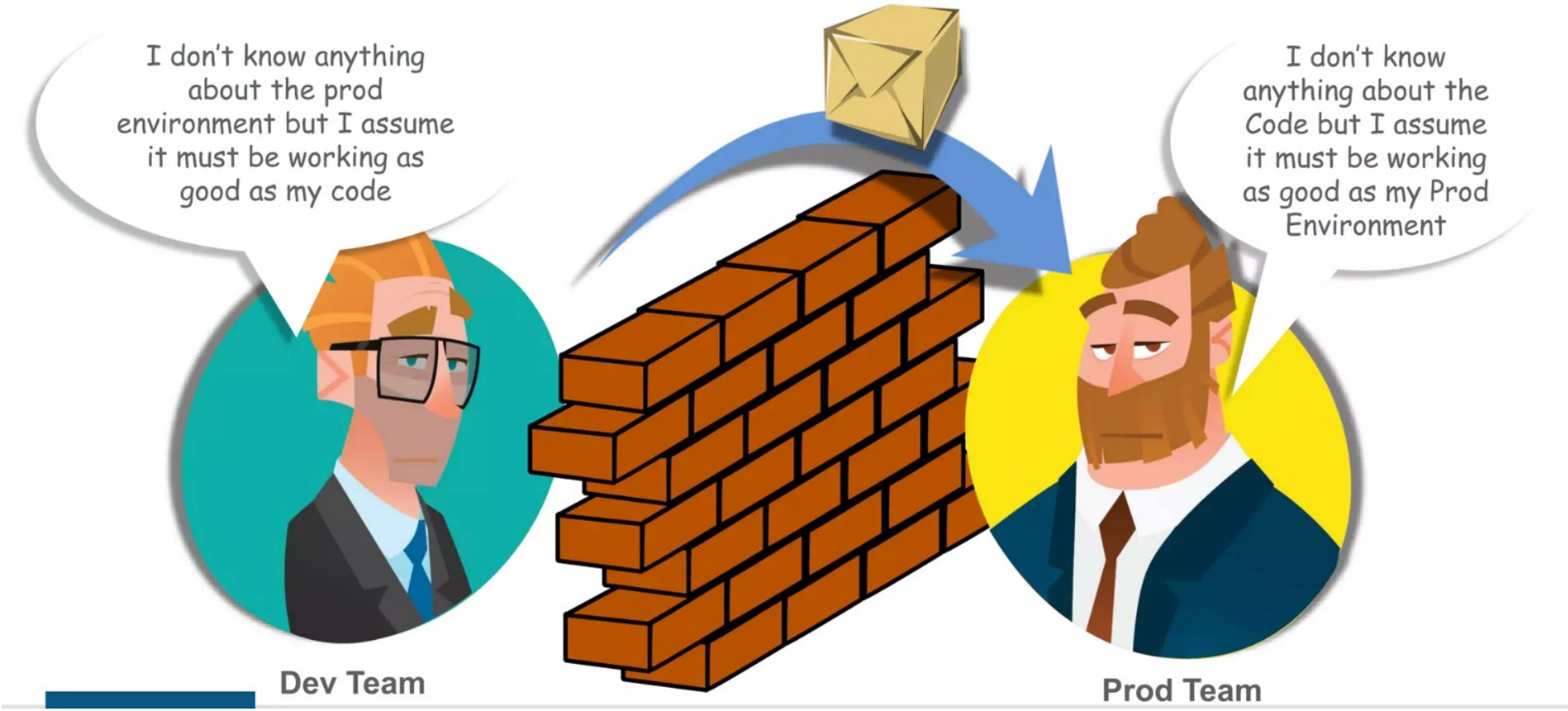
DevOps 개요

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

전통적인 개발 프로세스

개발은 잘 동작하는데, ... 오픈 시 항상 장애가...

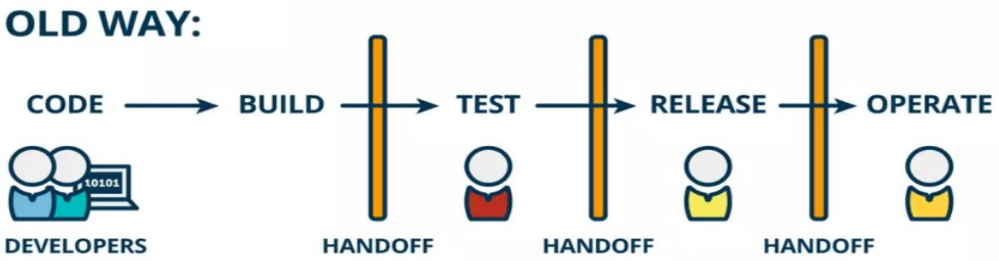


DevOps 정의

Dev | Ops

- 책임 전가 / 책임 회피
- 난 운영 몰라/난 개발 안했는데,
- 던지면 그만/쓰레기 던지지마..
- 신기술 도입해야 지 / 검증안된 기술 도입 안돼

weeks/months/years

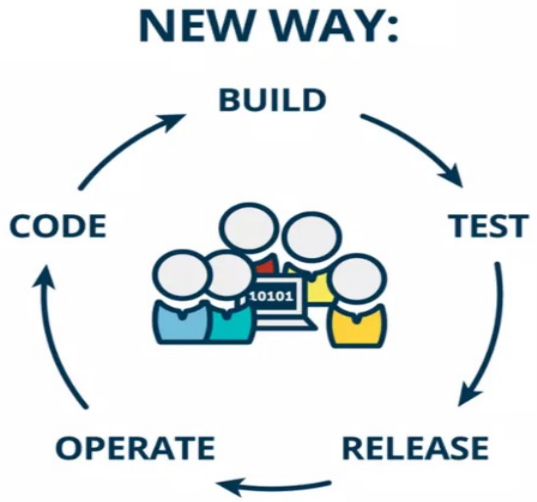


개발 과 운영이 분리
서로 다른 목표 (KPI)

Dev + Ops

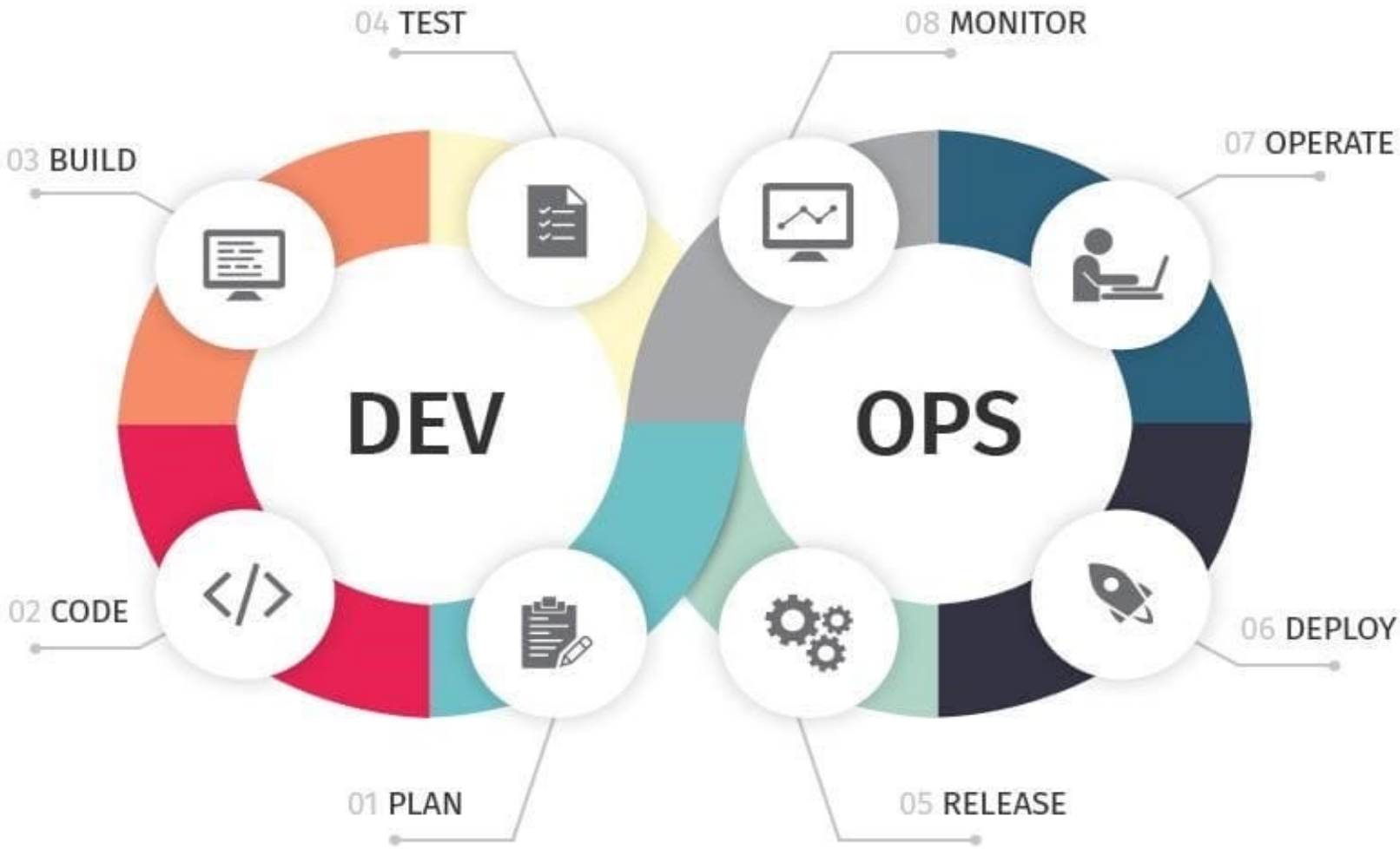
- 협업/협동을 통한 문제 해결
- 내가 끝까지 책임져야지 (Full Stack 인력)
- 처음부터 쓰레기를 만들지 말자..
- 신기술 도입해서 검증까지 해보자 (Fast Failed)

hours/days!



개발과 운영 통합
같은 목표 (KPI)

Development + Operations



DevOps 실현을 위한 핵심 요소

DevOps 목표는 **개발과 운영의 업무 프로세스를 개선**하는 것

문화와 조직 관점의 개선:

> DevOps는 협업과 커뮤니케이션을 중시하는 문화

> 개발자와 운영자가 함께 일하며, 공동의 목표를 설정하고 이를 달성하기 위해 협력

> 지속적인 피드백과 개선을 통해 팀의 효율성 극대화

프로세스와 자동화 도입:

> 빌드, 테스트, 배포, 모니터링 등 SW 개발/운영의 다 단계를 자동화하여 휴먼 에러를 줄이고 효율성을 극대화

> CI/CD(지속적 통합 및 지속적 배포) 파이프라인을 통해 코드 변경 사항을 자동으로 테스트하고 운영 배포

DevOps의 프로세스

소프트웨어의 개발부터 테스트, 배포, 운영까지 전체 과정을 표준화하고 지속적으로 개선하여 빠르고 안정적으로 소프트웨어를 제공하기 위한 업무 절차

DevOps 프로세스 자동화 : CICD

CI/CD는 DevOps를 실현하기 위한 대표적인 자동화 프로세스

CICD 적용 시 이점

- **빠른 배포 속도**: 자동화된 파이프라인을 통해 소프트웨어를 더 빠르게 배포
- **높은 품질**: 지속적인 테스트와 모니터링을 통해 소프트웨어의 품질을 유지하고 개선
- **효율성 향상**: 반복적이고 수동적인 작업을 자동화하여 팀의 생산성 극대화
- **리스크 감소**: 작은 단위의 변경 사항을 자주 배포함으로써 리스크를 줄이고, 문제 발생 시 빠르게 대응
- **고객 만족도 향상**: 더 빠르고 안정적인 소프트웨어 릴리스를 통해 고객의 요구에 신속하게 대응

CI/CD 단계의 의미

CI/CD는 지속적 통합(Continuous Integration)과 지속적 배포(Continuous Deployment) 또는 지속적 전달(Continuous Delivery)을 포함

- **지속적 통합(Continuous Integration, CI)**

개념: 개발자들이 코드 변경 사항을 중앙 리포지토리에 자주 병합하는 프로세스이다.

목적: 코드 변경 사항을 자동으로 빌드하고 테스트하여 통합 시 발생할 수 있는 문제를 조기에 발견하고 해결한다.

도구: Jenkins, GitLab CI, Travis CI, CircleCI 등.

- **지속적 전달(Continuous Delivery, CD)**

개념: CI의 결과물을 자동으로 준비하여 프로덕션 환경에 배포할 수 있는 상태로 만드는 프로세스이다.

목적: 코드가 언제든지 프로덕션에 배포될 수 있도록 준비 상태를 유지한다.

특징: 운영자의 승인 (Manual Approval) 후 배포, 운영자가 상황보고 배포 여부 결정 가능

- **지속적 배포(Continuous Deployment, CD)**

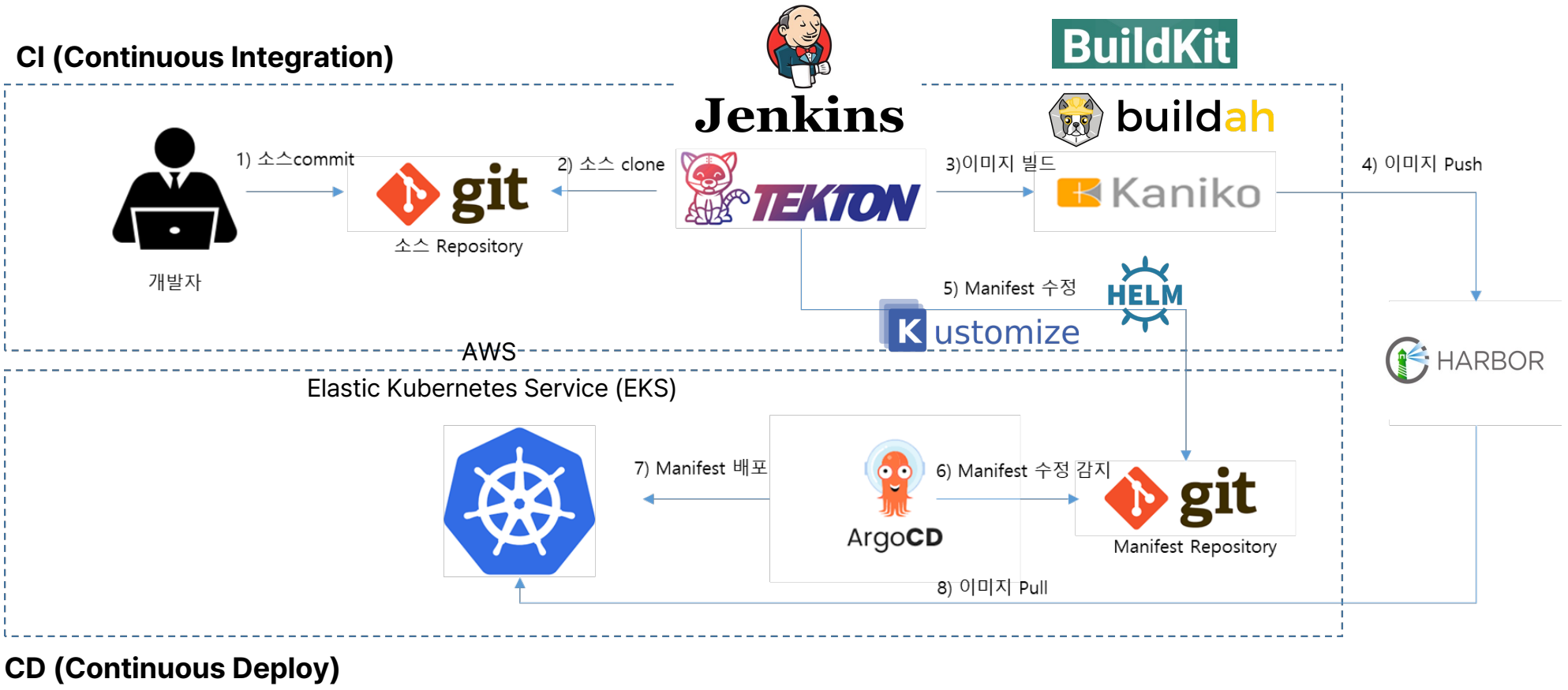
개념: 지속적 전달의 확장으로, 코드 변경 사항이 자동으로 프로덕션 환경에 배포되는 프로세스이다.

목적: 모든 코드 변경 사항이 자동으로 배포되어 사용자에게 즉시 제공된다.

특징: 완전한 자동화로, 수동 개입 없이 배포가 이루어진다.

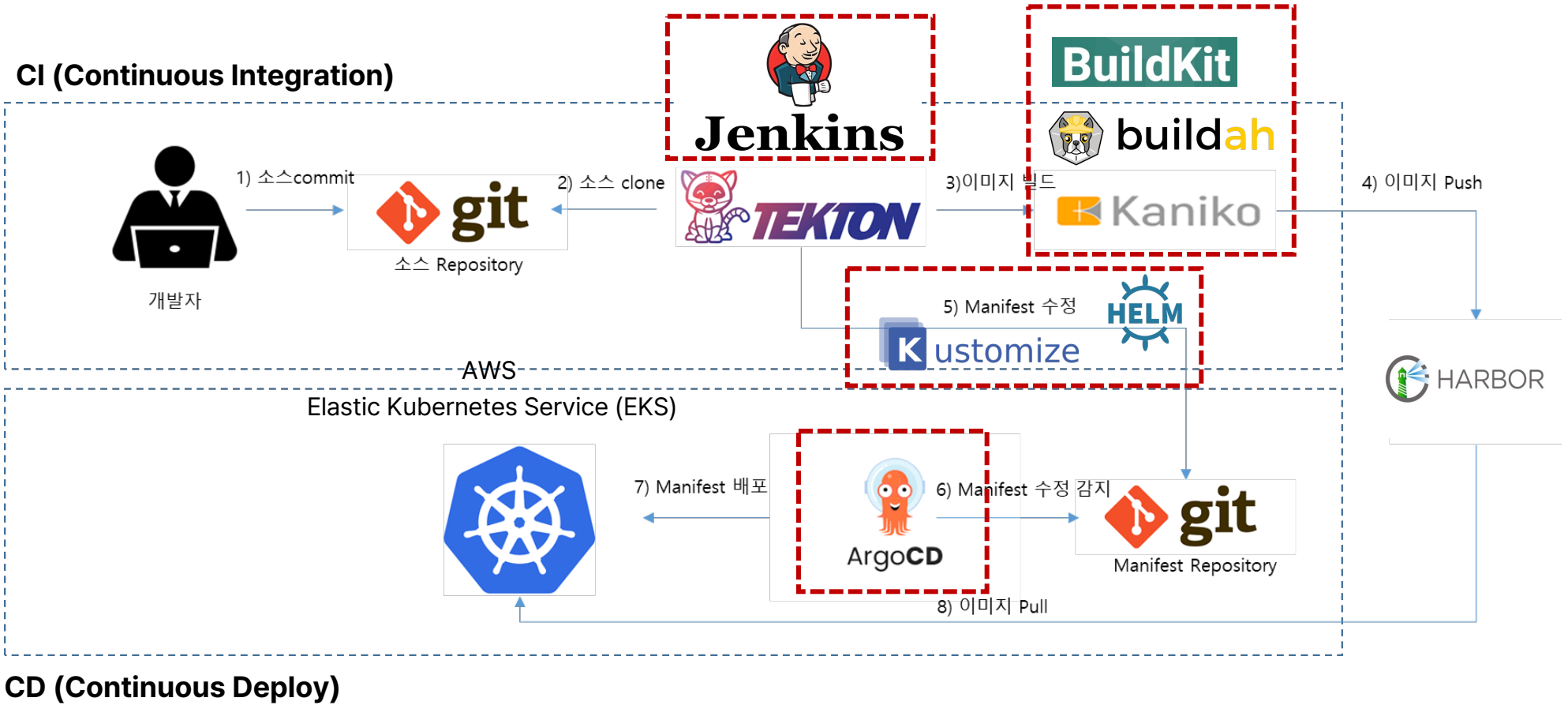
CI/CD 파이프라인 구성

자신의 Backend 와 Frontend 코드를 Kubernetes 환경에 배포할 수 있도록
Jenkins/Tekton + ArgoCD 기반으로 파이프라인을 구성



CI/CD 파이프라인 구성

자신의 Backend 와 Frontend 코드를 Kubernetes 환경에 배포할 수 있도록
Jenkins/Tekton + ArgoCD 기반으로 파이프라인을 구성



본 과정의 목표

본 과정을 마무리 하면

자신의 CI/CD 파이프라인은 구성

자신의 Frontend 와 Backend, AI 서비스를 자동 빌드/배포

KDT DevOps – DevOps

CI 전용 컨테이너 빌드 도구

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

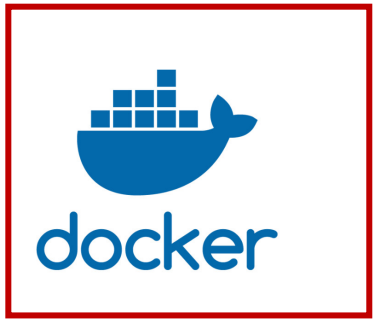
SK 주식회사 C&C

컨테이너 관리 도구

CICD 파이프라인 내에서 컨테이너를 빌드를 위한 도구

컨테이너 이미지 빌드 도구 (OCI 표준)

컨테이너 런타임 및 관리 도구 (CRI 표준)



컨테이너 플랫폼

BuildKit

buildah

Kaniko

컨테이너 빌드 도구

containerd

컨테이너 런타임 도구

컨테이너 빌드 도구들

CICD 파이프라인 내에서 컨테이너를 빌드를 위한 도구

순위	도구	추정 사용 점유율	사용률 증가 추세	비고 / 환경
1	**BuildKit / Docker Buildx**	**~65–75%**	**가장 빠름 (▲▲▲)**	Jenkins, Git Action, GitLab CI의 기본/ 표준 속도와 원격 캐시가 압도적
2	**Kaniko**	**~15–20%**	**하락세 (▼▼)**	K8s daemonless 빌드로 인기였으나, 구글 공식 아카이브 종료 후 BuildKit으로 전환 가속화
3	**Buildah / Podman**	**~8–12%**	**완만한 상승 (▲)**	Red Hat 계열(OpenShift, RHEL) 및 Rootless/Daemonless 보안 중시 환경

buildkit 과 buildah 비교

구분	BuildKit	Buildah
데몬 유무	**데몬 기반 (buildkitd)**	**완전 데몬리스 (단일 CLI 실행)**
Rootless	지원하나 설정 복잡도 있음	기본 철학으로 자연스럽게 지원
강점	압도적인 멀티스테이지 캐싱, 동시 빌드 속도	데몬 없는 단순성, 강력한 보안/비특권 빌드
주요 활용처	로컬 개발 환경, Docker 기본 엔진, 고성능 전용 CI 러너	Kube 파이프라인(Tekton 등), 보안이 엄격한 멀티테넌트 환경

kaniko 란?

kaniko는 Google이 만든 오픈소스 도구로 Kubernetes 같은 컨테이너화된 환경에 최적화된 컨테이너 빌드 도구
Docker, Podman 등은 빌드 환경을 구성하기 위해 만은 설정과 노력이 필요하지만, kaniko는 executor만 실행하면 끝....

Kaniko `gcr.io/kaniko-project/executor` 이미지를 이용하여 kaniko는 이미지로만 실행하는 방식

기능	제공 여부	설명
Dockerfile 기반 이미지 빌드	O	컨테이너 안에서 안전하게 빌드 가능
이미지 레지스트리 푸시	O	DockerHub, Harbor, GCR 등에 푸시
컨테이너 실행 (`run`)	✖	컨테이너를 실행하거나 테스트할 수 없음
이미지 관리 (`image list` , `rm` , `tag`)	✖	이미지 목록 보기, 태그 변경, 삭제 등의 기능 없음
로컬 캐시나 볼륨 마운트 같은 고급 기능	✖	단순한 빌드/푸시에 집중됨

실습: Kaniko 로 빌드하기

실습 코드 위치 : 02.build-platform/

[실습] kaniko 실행 방법 (1/3)

Kaniko

kaniko는 컨테이너 기반으로 빌드 및 푸시 기능을 제공

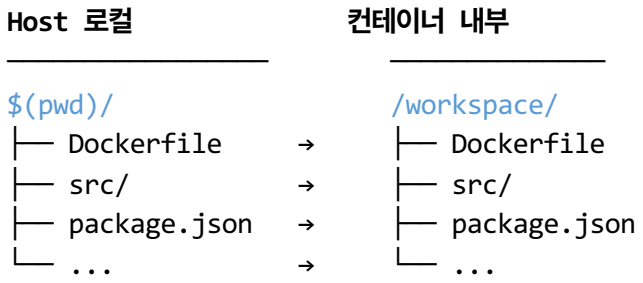
위치: 02.build-platform/build.push-with-kaniko.sh

```
mkdir -p auth
cat > auth/config.json <<EOF
{
  "auths": {
    "amdp-registry.skala-ai.com": {
      "username": "robot$skala25a",
      "password": "1qB9cyusbNComZPHAdjNIFWinf52xaBJ",
      "auth": "$(echo -n "robot$skala25a:1qB9cyusbNComZPHAdjNIFWinf52xaBJ" | base64)"
    }
  }
}
EOF

# docker run으로 kaniko 실행
docker run --rm \
  -v $(pwd):/workspace" \
  -v "$(pwd)/auth/config.json:/kaniko/.docker/config.json" \
  gcr.io/kaniko-project/executor:latest \
  --dockerfile=/workspace/Dockerfile \
  --context=dir:///workspace \
  --destination=amdp-registry.skala-ai.com/skala25a/sk000-myfirst-api-server:1.0.kaniko \
  --custom-platform=linux/amd64
```

kubernetes.io/dockerconfigjson 유형의 Container Image Registry 접속 정보

kaniko container image

build/push 되는 container image 명



[실습] kaniko 실행 방법 (2/3)

위치: 02.build-platform/build.push-with-kaniko.sh

1. run-with-kaniko.sh 파일 내용 확인해보기

```
> cat run-with-kaniko.sh
```

2. kaniko로 컨테이너 이미지 Build & Push 실행

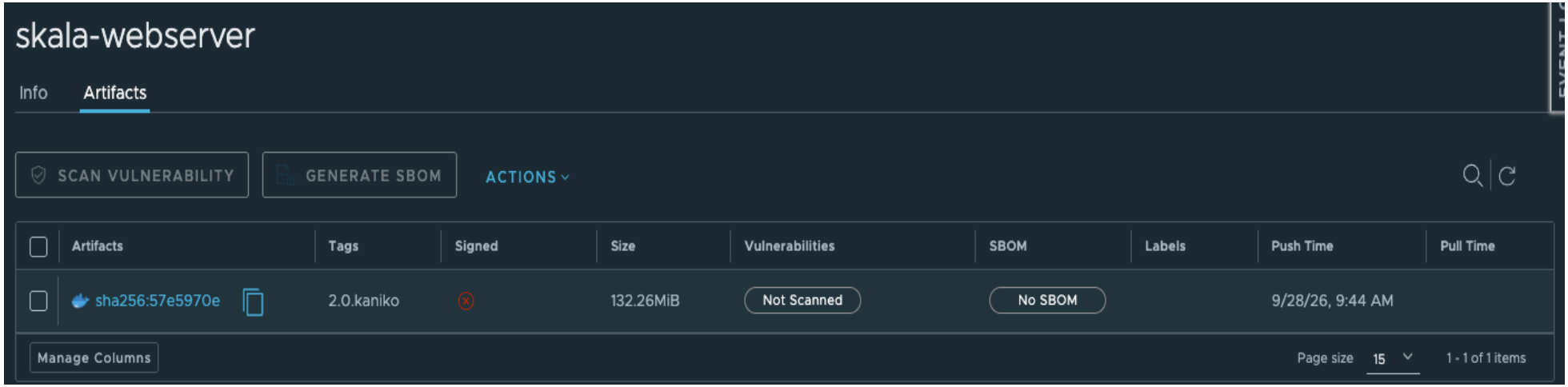
```
### kaniko 실행하기
$ ./build.push-with-kaniko.sh
```

```
Successfully installed dnspython-2.8.0 email-validator-2.3.0
INFO[0073] Taking snapshot of full filesystem...
INFO[0074] COPY fastserver.py fastserver.py
INFO[0074] Taking snapshot of files...
INFO[0074] CMD ["python3", "fastserver.py"]
INFO[0074] Pushing image to skala-registry.skala-ai.com/class-0/skala-webserver:2.0.kaniko
INFO[0083] Pushed skala-registry.skala-ai.com/class-0/skala-webserver@sha256:57e5970ea59f1d50f620bab4a13878b566d1823b3598f73
```

pushed image tag info

[실습] 배포 이미지 확인 (3/3)

https://skala-registry.skala-??.com



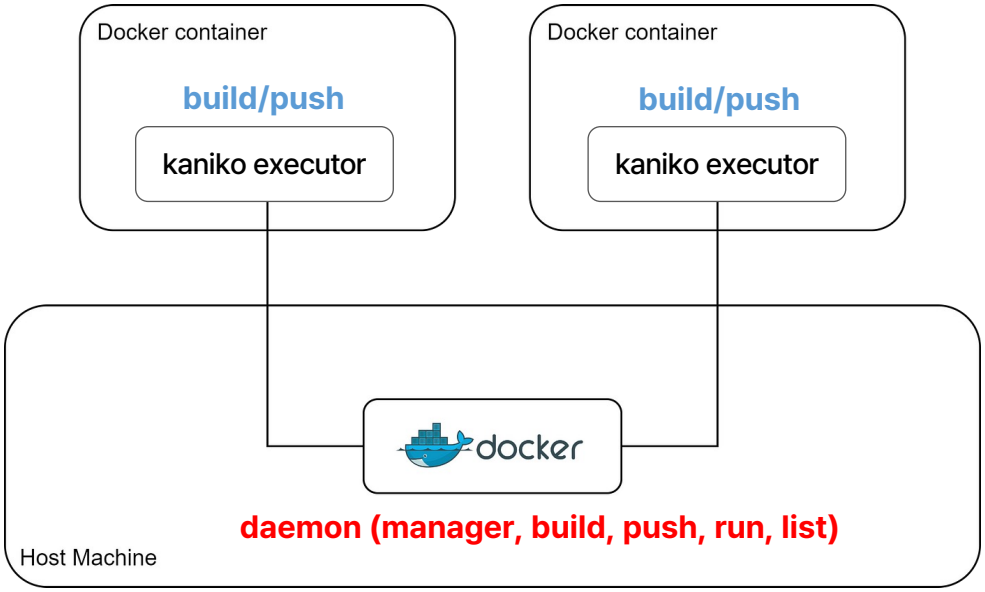
실습 끝

실습 코드 위치 : 02.build-platform/

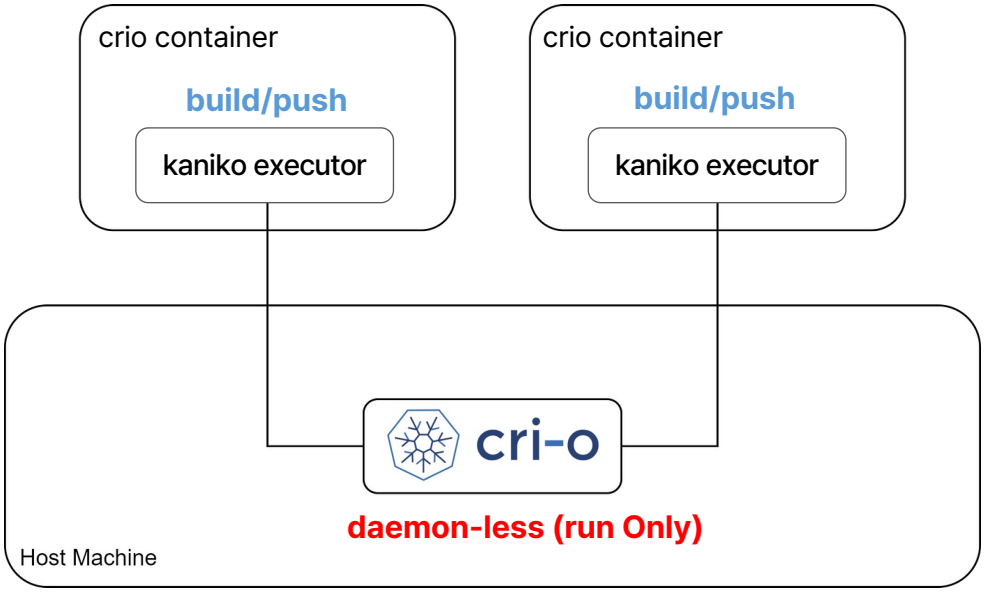
kubernetes 환경 기반 Docker 대체 build 도구

kaniko는 docker 환경에서 동작하는 것은 적합하지 않지만,
kubenetes 기반의 crio 와 containerd 환경에서는 최적화된 빌드 솔루션

Local PC 환경에서의 Kaniko 실행



kubernetes 환경에서의 Kaniko 실행



실습: Kaniko Pod로 빌드하기

실습 코드 위치 : 02.build-platform/

[실습] Pod 실행을 위한 사전 작업

위치: 02.build-platform/

Pod를 실행하기 위해서는 빌드를 위한 **Dockerfile과 코드 그리고 Harbor registry 접속 정보가 Pod에 제공**되어야 함

configmap.sh

Dockerfile과 fastserver.py 코드를 configmap으로 생성

```
> ./configmap.sh
> k get cm sk000-app-workspace -o yaml
```

```
#!/bin/bash

kubectl create configmap sk000-app-workspace \
  --from-file=Dockerfile=./Dockerfile \
  --from-file=fastserver.py=./fastserver.py \
  --namespace {자신의 Namespace}
```

harbor-registry-secret.sh (클래스별 한번만 배포)

```
> ./harbor-registry-secret.sh
> k get cm sk000-app-workspace -o yaml

> k get secret harbor-registry-secret -o json

> k get secret harbor-registry-secret \
  -o jsonpath='{.data.\.dockerconfigjson}' \
  | base64 -d | jq .
```

```
DOCKER_REGISTRY=”skala-registry.skala-??.com"
DOCKER_REGISTRY_USER="robot$skala25a"
DOCKER_REGISTRY_PASSWORD="1qB9cyusbNComZPHAdjNIFWinf52xaBJ"

kubectl create secret docker-registry harbor-registry-secret \
  --docker-server=${DOCKER_REGISTRY} \
  --docker-username=${DOCKER_REGISTRY_USER} \
  --docker-password=${DOCKER_REGISTRY_PASSWORD} \
  --docker-email=dev@skala.ai \
  --namespace ${NAMESPACE}
```

[실습] Pod로 Kaniko python 이미지 만들기 (1/2)

위치: 02.build-platform/pod-kaniko

kubernetes 환경에 kaniko를 Pod로 구성해서 빌드 요청하는 실습

명령 실행

```
### 위치
> cd ./pod-kaniko

### 실행 확인하기
> kubectl apply -f pod/kaniko-pod.yaml

### 실행 상황 확인하기
> k get pod sk000-kaniko-pod
> k logs -f sk000-kaniko-pod
```

kaniko-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sk000-kaniko-pod
  namespace: skala-practice

..

  containers:
  - name: kaniko
    image: gcr.io/kaniko-project/executor:latest
    args:
      - --dockerfile=/workspace/Dockerfile
      - --context=dir:///workspace
      - --destination={이미지 명}
      volumeMounts:
          - name: kaniko-secret
            mountPath: /kaniko/.docker
          - name: workspace
            mountPath: /workspace
      restartPolicy: Never
```

```
Successfully installed dnspython-2.8.0 email-validator-2.3.0
INFO[0017] Taking snapshot of full filesystem...
INFO[0019] COPY fastserver.py fastserver.py
INFO[0019] Taking snapshot of files...
INFO[0019] CMD ["python3", "fastserver.py"]
INFO[0019] Pushing image to skala-registry.skala-ai.com/class-0/skala-webserver:2.0.kaniko-pod
INFO[0020] Pushed skala-registry.skala-ai.com/class-0/skala-webserver@sha256:012edfabf32abeba6c8c9a44a7e4953dc82fd440d63226b
```

[실습] Pod로 Kaniko python 이미지 만들기 (2/2)

위치: 02.build-platform/pod-kaniko

kubernetes 환경에 kaniko를 Pod로 구성해서 빌드 요청하는 실습

자원 삭제 실행

```
#확인 후 삭제
> k get pod sk000-kaniko-pod
NAME               READY   STATUS      RESTARTS   AGE
sk000-kaniko-pod   0/1     Completed   0          2m28s

> k delete pod sk000-kaniko-pod
```

[실습] Kaniko POD로 spring boot 빌드하기 (1/3)

위치: 02.build-platform/pod-kaniko

myfirst-api-server java 코드를 git clone 후 직접 maven build하고 컨테이너 빌드/배포 실행

kaniko-pod-withgit.yaml

상세한 것은 코드 확인

```
apiVersion: v1
kind: Pod
metadata:
  name: sk000-kaniko-pod-springboot
  namespace: skala-practice

..
 # Init Container: Maven 빌드
  initContainers:
  - name: build
    image: maven:3.8.5-openjdk-17
    workingDir: /workspace
    command: ["/bin/bash"]
    args:
      - -c
      - |
        git clone https://github.com/${id}/xxx.git .
        mvn clean install –DskipTests

 # Main Container: Kaniko 빌드 (기본 entrypoint만 사용)
  containers:
  - name: kaniko
    image: gcr.io/kaniko-project/executor:latest
    args:
      - --dockerfile=/workspace/Dockerfile
      - --context=dir:///workspace
      - --destination={이미지 명}
```

[실습] Kaniko POD로 spring boot 빌드하기 (2/3)

1. Kaniko Pod 배포

```
> k apply -f kaniko-pod-withgit.yaml
```

2. init-container 내 git clone & mvn build 상태 확인하기

```
> k get pod sk000-kaniko-pod
> k logs -f sk000-kaniko-pod -c maven
```

```
~/mydev/skala/ywyi-skala-edu-code/cloud/devops/02.kaniko/pod/ [master*] k logs -f sk000-kaniko-build -c maven-build
=== Git Clone 시작 ===
Cloning into '.'...
=== Maven 빌드 시작 ===
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-starter-parent/3.4.3/spring-boot-starter-parent-3.4.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-starter-parent/3.4.3/spring-boot-starter-parent-3.4.3.pom (13 kB at 23 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-dependencies/3.4.3/spring-boot-dependencies-3.4.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-dependencies/3.4.3/spring-boot-dependencies-3.4.3.pom (97 kB at 1.4 MB/s)
```

3. spring boot 컨테이너 이미지 빌드 및 push 실행 확인하기

```
$ k logs -f sk000-kaniko-pod -c kaniko
```

```
~/mydev/skala/ywyi-skala-edu-code/cloud/devops/02.kaniko/pod/ [master*] k logs -f sk000-kaniko-build -c kaniko
DEBU[0000] Getting source context from dir:///workspace
DEBU[0000] Build context located at /workspace
DEBU[0000] Copying file /workspace/Dockerfile to /kaniko/Dockerfile
DEBU[0000] Skip resolving path /kaniko/Dockerfile
DEBU[0000] Skip resolving path /workspace
DEBU[0000] Skip resolving path /cache
DEBU[0000] Skip resolving path
DEBU[0000] Skip resolving path
DEBU[0000] Skip resolving path
DEBU[0000] Skip resolving path
```

[실습] Kaniko POD로 spring boot 빌드하기 (3/3)

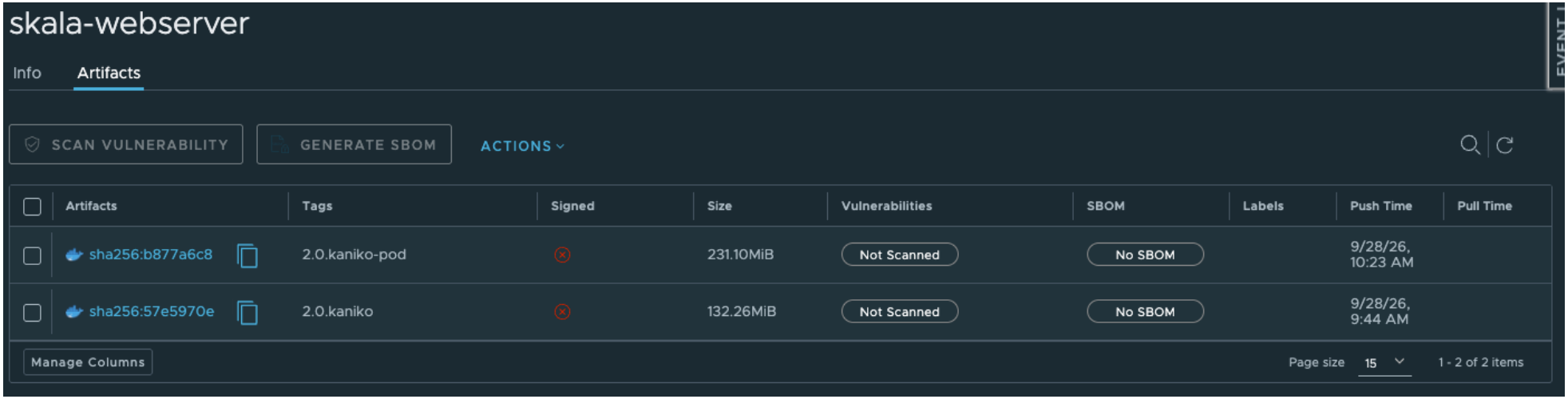
kubernetes 환경에 kaniko를 Pod로 구성해서 빌드 요청하는 실습

자원 삭제 실행

```
#확인 후 삭제
> k get pod sk000-kaniko-pod
NAME               READY   STATUS      RESTARTS   AGE
sk000-kaniko-pod   0/1     Completed   0          2m28s

> k delete pod sk000-kaniko-pod
```

[실습] Pod로 Kaniko 실행 결과 확인

https://skala-registry.skala-??.com 접속 후 자신의 프로젝트에서 이미지 확인하기



실습 끝

실습 코드 위치 : 02.build-platform/

Buildah

Buildah는 **Red Hat**이 주도적으로 개발한 **컨테이너 이미지 빌드 전용 도구**
기존 Docker와 달리 **데몬(daemon)이 필요하지 않고**, 가볍고 유연하게 **OCI(Open Container Initiative) 호환 이미지**를 생성

buildah `quay.io/buildah/stable:latest` 이미지를 이용할 수 있으며, 컨테이너 내부에서 buildah를 설치해서 사용 가능

항목	Kaniko	Buildah
개발 주체	Google	Red Hat
주요 목적	Dockerfile 기반 이미지 빌드, 특히 대량 처리시 적합 (특히 Kubernetes 내에서 안전하게 빌드)	Dockerfile 빌드 + 커맨드 기반 이미지 빌드 흐름을 사용자가 구성
동작 방식	전용 Kaniko Container 를 run 해서 동작 (주로 Kubernetes Pod로 실행)	호스트 CLI에서 직접 실행
데몬 필요 여부	Docker 데몬 불필요	Docker 데몬 불필요
스토리지 드라이버	내부적으로 tar 기반 (overlayfs 등 없음) → 단순히 파일시스템 copy/write	overlayfs, vfs 등 다양한 스토리지 드라이버 선택 가능
성능	빌드 성능 놓음 단, 큰 이미지에서는 느려질 수 있음	스토리지 드라이버에 따라 빠르거나 느릴 수 있음 overlayfs 사용 시 성능 우수
주요 사용 환경	Kubernetes CI 파이프라인 (Tekton, GitHub Actions, GitLab CI 등)	Jenkins CI stage 내 파이프라인 구성 (Jenkins, GitHub Actions, GitLab CI 등)
학습 난이도	낮음 (Dockerfile만 알면 됨)	중간 (Dockerfile 방식 + 수동 빌드 방식 모두 학습 가능)

실습: buildah로 빌드하기

실습 코드 위치 : 02.build-platform/

[실습] Buildah 기반으로 실습하기

1. build.push-with-buildah.sh 를 실행해서 컨테이너로 buildah 실행하고 Registry에 이미지를 확인해보자

```
> ./build.push-with-buildah.sh
```

2. ./pod-buildah 아래로 이동해서 python 코드를 컨테이너로 빌드/푸시하고, Registry에 이미지를 확인해보자

```
> cd ./pod-buildah
> k apply -f buildah-pod.yaml
> k get pod sk000-buildah-pod
> k logs -f sk000-buildah-pod
```

```
> k delete pod sk000-buildah-pod
```

3. spring boot 코드를 컴파일하고, 컨테이너로 빌드/푸시하고, Registry에 이미지를 확인해보자

```
> k apply -f buildah-pod-withgit.yaml
> k get pod sk000-buildah-pod
> k logs -f sk000-buildah-pod -c maven
> k logs -f sk000-buildah-pod -c buildah
```

```
> k delete pod sk000-buildah-pod
```

실습 끝

실습 코드 위치 : 02.build-platform/

멀티 플랫폼 빌드

동일한 소스 코드와 단일 이미지 태그를 기반으로 서로 다른 CPU 아키텍처(x86-64 계열의 amd64, ARM 계열의 arm64)에서 실행 가능한 컨테이너 이미지를 한 번에 생성하고 단일 패키지로 묶는 기술

1. 멀티 플랫폼 빌드의 개념과 동작 방식

독립 빌드 파이프라인(DAG) 생성:
빌드 엔진(BuildKit/Docker Buildx)은 입력받은 플랫폼 목록을 기반으로 아키텍처별 빌드 환경을 분리하여 병렬로 빌드를 수행

빌드인자 자동 주입:
Dockerfile 내에서 TARGETPLATFORM, TARGETARCH 등의 환경 변수가 각 플랫폼 빌드 스텝에 맞춰 자동으로 분기 적용

2. 이미지 레지스트리(Repository) 저장 구조

멀티 플랫폼 이미지를 푸시하면 레지스트리에는 개별 이미지들이 단순히 따로 들어가는 것이 아니라,
이를 묶어주는 **매니페스트 리스트(Manifest List)** 또는 **OCI Image Index**가 함께 생성되어 저장

```
[ Image Index (Manifest List) ]  <-- 태그: my-app:latest
  │
  ├── 1. Platform: linux/amd64
  │      └── Digest: sha256:aaaa... ──▶ [ amd64용 Manifest ] ──▶ [ amd64 레이어 Blobs ]
  │
  └── 2. Platform: linux/arm64
         └── Digest: sha256:bbbb... ──▶ [ arm64용 Manifest ] ──▶ [ arm64 레이어 Blobs ]
```

- **Image Index (메타데이터):**
  linux/amd64용 매니페스트의 식별자(sha256:aaaa)와 linux/arm64용 매니페스트의 식별자(sha256:bbbb) 목록을 보관
- **개별 Image Manifest:**
  해당 아키텍처에서 실행되는 컨테이너의 설정(Config) 및 실제 파일시스템 레이어(Layer Blobs)들의 해시 목록을 보관

Manifest List 와 OCI Image Index

여러 CPU 아키텍처(amd64, arm64 등)나 OS(Linux, Windows 등)별 개별 이미지 매니페스트들을
하나의 태그(예: my-app:1.0) 아래로 묶어주는 상위 메타데이터 목차(Index)를 의미

구분	Manifest List	OCI Image Index
제정 진영	Docker Inc. (도커 자체 규격)	OCI (Open Container Initiative) (국제 오픈 표준)
미디어 타입 (MIME Type)	application/vnd.docker.distribution.manifest.list.v2+json	application/vnd.oci.image.index.v1+json
등장 배경	도커가 멀티 아키텍처 지원을 위해 먼저 도입한 v2 schema 2 규격	리눅스 재단 산하 OCI에서 특정 벤더 종속성을 없애고 표준화한 규격

Client는 두가지 모두 지원 가능

Manifest List 구성 방식 (1/2)

Docker Buildx

단일 명령어로 두 아키텍처를 동시에 빌드하여 레지스트리로 바로 전송

```
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t skala-registry.skala-ai.com/class-0/my-app:1.0 \
  --push .
```

Buildah (과거)

manifest 생성을 위해 별로 명령어를 통해 두가지 플랫폼을 하나의 메니페스트로 통합해서 구성하는 방식 사용

```
# 1. 아키텍처별 개별 이미지 빌드
buildah bud --platform linux/amd64 -t my-app:1.0-amd64 .
buildah bud --platform linux/arm64 -t my-app:1.0-arm64 .

# 2. my-app:1.0.0 이라는 이름의 빈 Manifest List(인덱스) 생성
buildah manifest create my-app:1.0

# 3. 인덱스에 위에서 빌드한 amd64, arm64 이미지를 각각 등록
buildah manifest add my-app:1.0 my-app:1.0-amd64
buildah manifest add my-app:1.0 my-app:1.0-arm64

# 4. OCI Index 형식으로 Harbor에 일괄 푸시
buildah manifest push --format oci --all my-app:1.0.0 \
  docker://skala-registry.skala-ai.com/class-0/my-app:1.0
```

Manifest List 구성 방식 (2/2)

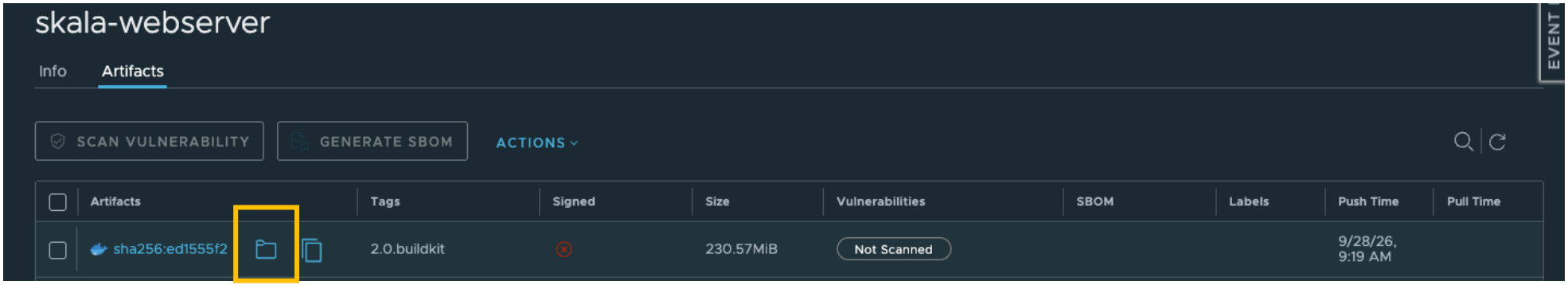
Buildah (개선 구조 : Docker buildx 와 유사)

개별적으로 이미지 빌드 및 manifest 에 add 하던 방식으로 아래의 같은 명령어로 통합 실행

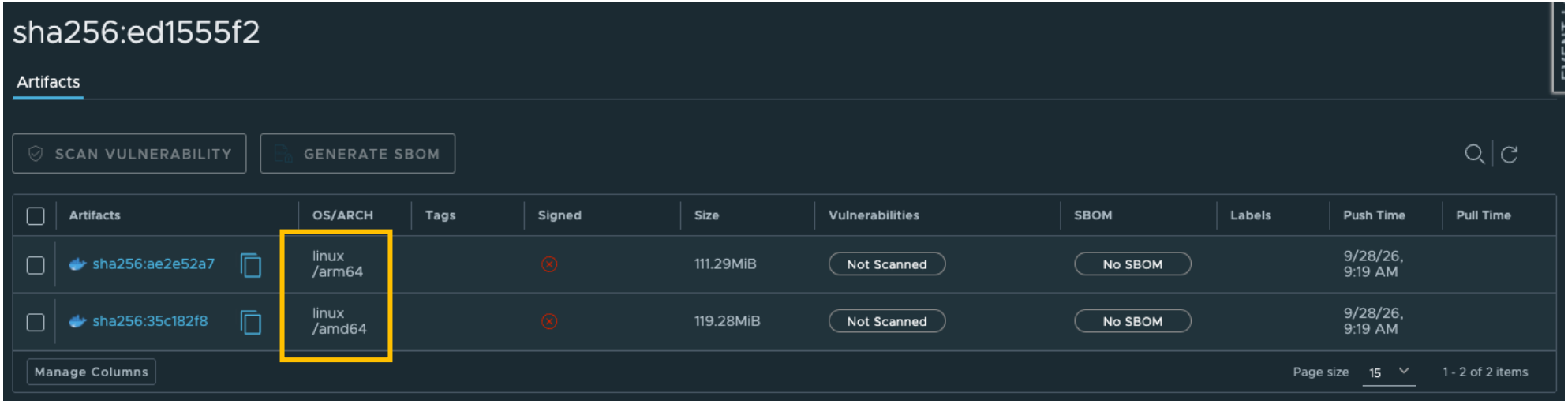
```
# 1. 아키텍처별 개별 이미지 빌드
buildah bud
  --platform linux/amd64,linux/arm64 \
  --manifest skala-registry.skala-ai.com/class-0/my-app:1.0 \
  .

# 2. OCI Index 형식으로 Harbor에 일괄 푸시
buildah manifest push --format oci --all my-app:1.0.0 \
  docker://skala-registry.skala-ai.com/class-0/my-app:1.0
```

Registry 내 Manifest List 저장 구조

multi-Platform Build 시 Manifest List 기준으로 하나의 이미지 테그 표시



Manifest List 내부에 플랫폼 별 이미지 포함 (Manifest 목록)



Buildkit

- Moby(Docker) 프로젝트에서 개발한 차세대 오픈소스 컨테이너 이미지 빌드 엔진
- 기존 Docker 빌더의 직렬 실행 방식과 비효율성을 개선하여 빌드 성능과 보안성을 극대화하기 위해 설계
- 현재 Docker Engine(v23.0+) 및 docker buildx의 기본 빌더로 작동

BuildKit

`docker.io/moby/buildkit:rootless` 이미지를 이용

비교 항목	BuildKit	Kaniko	Buildah
현재 상태 및 생태계	**사실상 표준 (De facto Standard)** Docker/Buildx, GitHub Actions 기본 탑재	**공식 아카이브 (유지보수 중단)** 기능 추가 및 최신 규격 반영 중단	**Red Hat 생태계 주력** OpenShift, Podman 기반 환경 중심
빌드 실행 방식	**DAG 기반 병렬 실행** 독립 스테이지/명령어 동시 처리	**순차(Sequential) 실행** Dockerfile 라인 단위 직렬 처리	**명령어 기반 커밋 / 직렬** 쉘 스크립트 기반 수동 최적화 가능
멀티 플랫폼 지원 (--platform)	**단일 명령어로 네이티브 지원** OCI Image Index 자동 패키징	**자체 지원 불가** 아키텍처별 분리 빌드 및 수동 병합 필요	**지원** buildah manifest CLI로 생성 가능
캐시 시스템	**고도화된 캐시** - 로컬/마운트/원격 캐시 - 세분화된 마운트 캐시(--mount=type=cache)	**기본 레이어 캐시** 원격 레지스트리 레이어 캐싱 위주	**로컬 스토리지 캐시 위주** 원격 캐시 연동 설정 상대적 복잡
빌드 & 푸시 성능	**파이프라이닝 최적화** 빌드 완료된 레이어부터 실시간 병렬 푸시	**순차 스냅샷 & 푸시** 빌드 완료 후 일괄 스냅샷/업로드	**프로세스 단위 처리** 빌드 완료 후 단일 커밋/푸시

실습: buildkit으로 빌드하기

실습 코드 위치 : 02.build-platform/

[실습] buildkit 기반으로 실습하기

1. build.push-with-buildkit.sh 를 실행해서 컨테이너로 buildah 실행하고 Registry에 이미지를 확인해보자

```
> ./build.push-with-buildkit.sh
```

2. ./pod-buildah 아래로 이동해서 python 코드를 컨테이너로 빌드/푸시하고, Registry에 이미지를 확인해보자

```
> cd ./pod-buildkit
> k apply -f buildkit-pod.yaml
> k get pod sk000-buildkit-pod
> k logs -f sk000-buildkit-pod
```

```
> k delete pod sk000-buildah-pod
```

3. spring boot 코드를 컴파일하고, 컨테이너로 빌드/푸시하고, Registry에 이미지를 확인해보자

```
> k apply -f buildkit-pod-withgit.yaml
> k get pod sk000-buildkit-pod
> k logs -f sk000-buildkit-pod -c maven
> k logs -f sk000-buildkit-pod -c buildkit
```

```
> k delete pod sk000-buildkit-pod
```

실습 끝

실습 코드 위치 : 02.build-platform/

KDT DevOps – DevOps

K8S 리소스 Kustomize하기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

쿠버네티스에 배포한다는 의미?

YAML 파일을 작성해서 클러스터에 적용하는 것

이 과정에서 가장 어려운 것은 첫 YAML 버전을 만드는 것

dev/nginx.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      component: nginx
  template:
    metadata:
      labels:
        component: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
```

stg/nginx.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      component: nginx
  template:
    metadata:
      labels:
        component: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
```

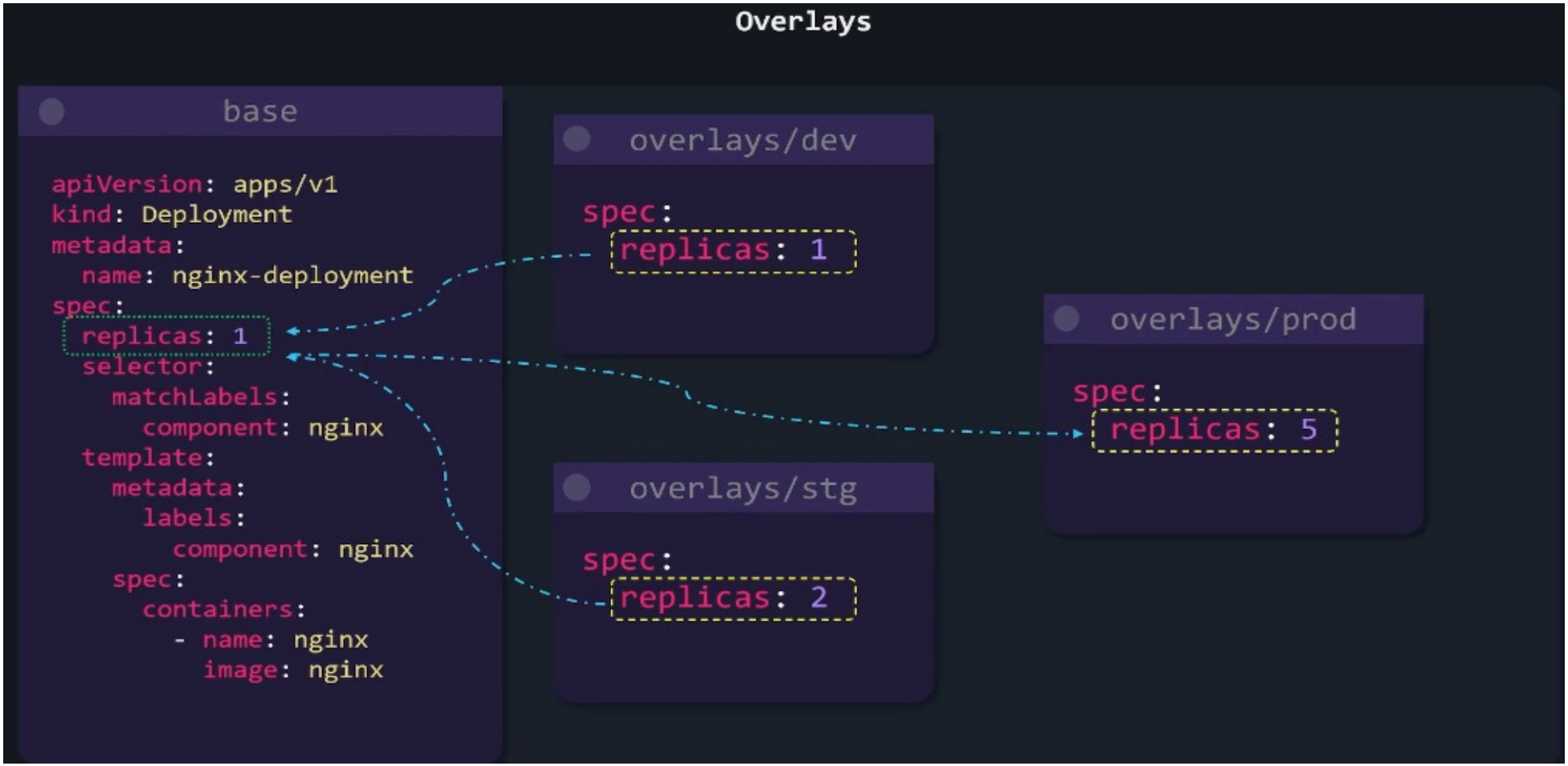
prod/nginx.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      component: nginx
  template:
    metadata:
      labels:
        component: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
```

Kustomize 란?

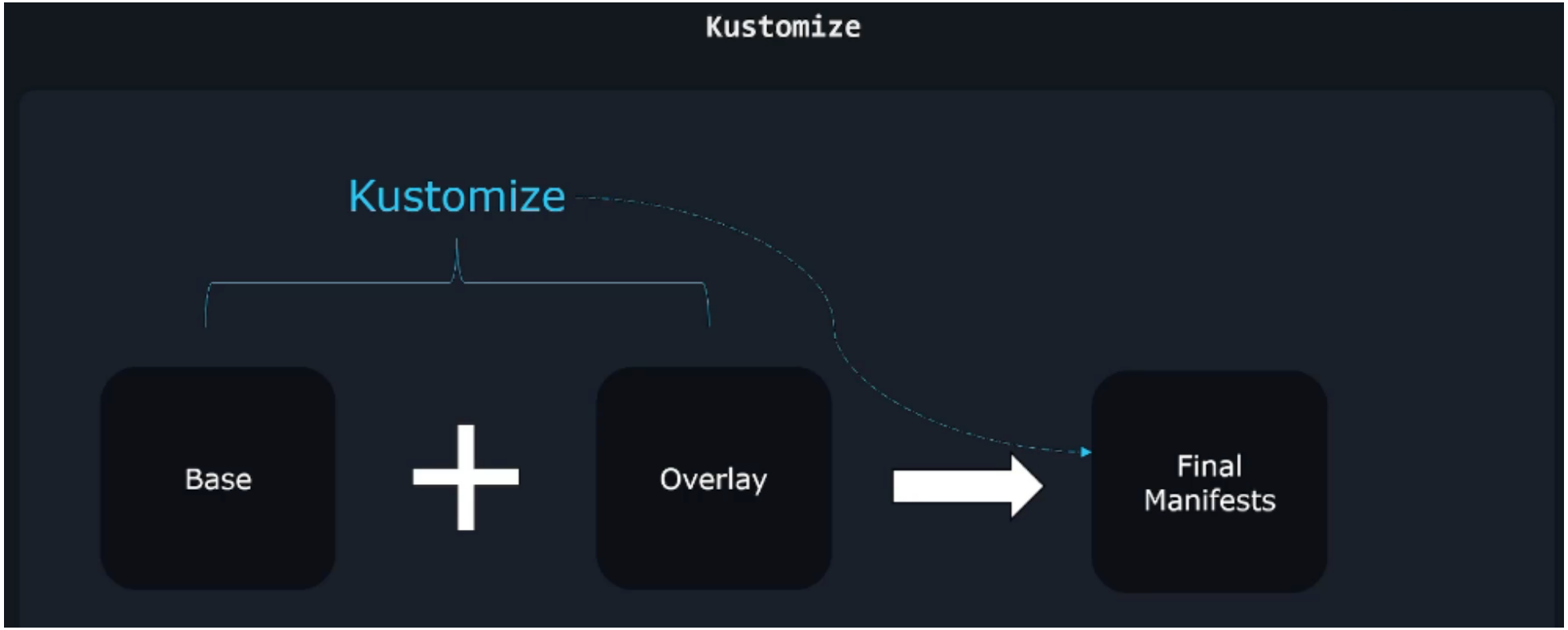
원본 YAML은 순수 쿠버네티스 표준 문법 그대로 두고 변경이 필요한 차이점(Delta)만 별도 파일로 덮어씌움

Kuberenetes + Customize = Kustomize



Kustomize 를 통한 Yaml 생성 방법

- Base Config 파일들을 base/ 디렉토리에 생성
- Overlays 폴더들을 overlays/ 디렉토리에 생성하고 환경별 설정을 정의
- kubectl apply -k overlay/dev 명령어를 사용하여 배포



Kustomize 특징

환경별 설정 분리

- dev, staging, prod 등의 환경에 따라 다른 설정이 필요할 때,
- 동일한 리소스에 대해 중복 없이 설정만 다르게 관리 가능

재사용성과 유지보수성

- 공통 base를 만들고 이를 여러 오버레이가 재사용하게 하여 유지보수 부담을 줄임.

템플릿 언어 없이 간편함

- Helm처럼 복잡한 템플릿 엔진 없이도 순수 YAML만으로 구성 가능
- 러닝 커브가 낮고, 디버깅이 쉬움.

보안 및 투명성

- 변수나 값을 외부에서 주입하거나 템플릿으로 감싸지 않기 때문에,
- 어떤 값이 최종 적용되는지 명확하게 확인 가능.

GitOps 및 CI/CD 적합

- Git에 선언형 YAML을 그대로 저장하고,
- 각 환경별로 적절한 overlay를 적용하여 자동 배포에 최적화

Kustomize 구조

위치: 01.spring-backend-v1.0/kustomize

- **base**: 공통으로 쓰이는 Kubernetes 리소스 정의 (예: Deployment, Service 등)
- **overlay**: 특정 환경(dev, stage, prod 등)에 따라 필요한 변경사항만 정의

kustomization.yaml 파일을 기준으로 최종적으로 적용될 YAML을 빌드

```
kustomize/
├── README.md
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
└── overlays/
    ├── dev/
    │   ├── configmap.yaml
    │   ├── patch-deployment.yaml
    │   └── kustomization.yaml
    └── prod/
        ├── configmap.yaml
        ├── patch-deployment.yaml
        └── kustomization.yaml
```

기존 Kubernetes 리소스

기존 Kubernetes 리소스에서 변경해야 할 설정 값

Kustomize 구조

Base의 kustomization.yaml의 역할

특정 경로를 읽을 때 그 디렉토리 안에 kustomization.yaml이 존재하는지 확인하여 유효한 Kustomize 모듈(타깃)인지 판단

```
kustomize/
├── base/
│   ├── kustomization.yaml
│   ├── deploy.yaml
│   └── service.yaml
└── overlays/
    └── dev/
        ├── kustomization.yaml
        ├── patch-deployment.yaml
        └── configmap.yaml
```

[참고]
labels의 includSelectors: true 인 경우
spec.selector.matchLabels,
spec.template.metadata.labels에 모두 추가됨

역할

- 포함해야 하는 쿠버네티스 원본 리소스 지정 (Resources)
- 공통 namespace 지정 (namespace)
- 공통 Label과 annotation 추가
  (labels, commonAnnotations)

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: class-0

labels:
  - pairs:
      skala.io/component: skala-devops
      skala.io/managed-by: class-0-skala
    includeSelectors: false   # spec.selector.matchLabels 에 미추가

commonAnnotations:
  skala.io/owner: skala
  skala.io/registry: skala-registry.skala-ai.com

resources:
  - deployment.yaml
  - service.yaml
```

Kustomize 구조

Overlays의 kustomization.yaml 역할

Base에 정의된 공통 뼈대를 가져와 해당 환경(Dev, Staging, Prod 등)의 조건에 맞게 변형·확장·격리하는 오케스트레이터 역할

역할

- Base 상속 선언 (resources)
- 환경별 격리 및 식별자 부여 (namespace, namePrefix / nameSuffix)
- 공통 Label과 annotation 추가 (commonLabels, commonAnnotations)
- 세부 설정 덮어 쓰기 (patch)
- images 의 이름과 tag 덮어쓰기 (images)
  - newTag : tag 덮어쓰기
  - newName: name 덮어쓰기

```
kustomize/
├── base/
│   ├── kustomization.yaml
│   ├── deploy.yaml
│   └── service.yaml
└── overlays/
    └── dev/
        ├── kustomization.yaml
        ├── patch-deployment.yaml
        └── configmap.yaml
```

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: class-0

resources:
- ../../base
- configmap.yaml      # 필요시 해당 환경에만 존재하는 전용 리소스 추가 가능

patches:
- path: patch-deployment.yaml
  target:
    kind: Deployment
    name: skala-myfirst-api-server

images:
- name: skala-registry.skala-ai.com/class-0/skala-webserver
  newTag: "1.0"
```

Kustomize 구조 : Overlays resources (1/2)

Patch resource

```
kustomize/
├── base/
│   ├── kustomization.yaml
│   ├── deploy.yaml
│   └── service.yaml
└── overlays/
    └── dev/
        ├── kustomization.yaml
        ├── patch-deployment.yaml
        └── configmap.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    prometheus.io/scrape: 'true'
    prometheus.io/port: '8080'
    prometheus.io/path: '/actuator/prometheus'
    update: e773ddf4
  name: skala-myfirst-api-server
  namespace: class-0
spec:
  replicas: 1
  selector:
    matchLabels:
      app: skala-myfirst-api-server
  template:
    metadata:
      labels:
        app: skala-myfirst-api-server
    spec:
      # SIGTERM 전달 후 강제 종료(SIGKILL)까지 대기하는 최대 시간
      terminationGracePeriodSeconds: 20
      containers:
        - name: myfirst-api-server
          image: skala-registry.skala-ai.com/class-0/skala-webserver:1.0
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
          # SPRING_PROFILES_ACTIVE는 dev/prod overlay의 patch-deployment.yaml
          # 준비(readiness)가 끝나기 전에는 Service가 트래픽을 보내지 않는다.
          readinessProbe:
```

+

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: skala-myfirst-api-server
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: myfirst-api-server
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "prod"
          volumeMounts:
            - name: config-volume
              mountPath: /app/config
      volumes:
        - name: config-volume
          configMap:
            name: skala-backend-configmap
```

replicas patch

env patch

configmap patch

Kustomize 구조 : Overlays resources (2/2)

```
> kubectl kustomize overlay/dev > output.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    prometheus.io/path: /actuator/prometheus
    prometheus.io/port: "8080"
    prometheus.io/scrape: "true"
    skala.io/owner: skala
    skala.io/registry: skala-registry.skala-ai.com
    update: e773ddf4
  labels:
    skala.io/component: skala-devops
    skala.io/managed-by: class-0-skala
  name: skala-myfirst-api-server
  namespace: class-0
spec:
  replicas: 2
  selector:
    matchLabels:
      app: skala-myfirst-api-server
  template:
    metadata:
      annotations:
        skala.io/owner: skala
        skala.io/registry: skala-registry.skala-ai.com
      labels:
        app: skala-myfirst-api-server
    spec:
      containers:
      - env:
        - name: SPRING_PROFILES_ACTIVE
          value: prod
        image: skala-registry.skala-ai.com/class-0/skala-webserver:1.0
        imagePullPolicy: Always
…
```

```
livenessProbe:
          failureThreshold: 3
          httpGet:
            path: /actuator/health/liveness
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
          timeoutSeconds: 5
        name: myfirst-api-server
        ports:
        - containerPort: 8080
        readinessProbe:
          failureThreshold: 3
          httpGet:
            path: /actuator/health/readiness
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
          timeoutSeconds: 5
        volumeMounts:
          - mountPath: /app/config
            name: config-volume
      volumes:
       - name: config-volume
         configMap:
           name: skala-backend-configmap
```

[실습] kustomize 를 통한 output 생성하기

kustomize를 통해 생성결과 확인하기

1. 01.spring-backend-v1.0/kustomize를 통해 yaml을 생성해보자

```
> cd ~/workspace/skala-devops/01.spring-backend-v1.0/kustomize

# 생성된 결과 확인하기
> kubectl kustomize overlays/dev > output.yaml

# 내용 확인하기
> cat output.yaml
```

2. yaml의 유효성을 검증해보자

```
# 생성된 yaml이 Validation 한지를 확인한다
> kubectl apply -k overlays/dev/ --dry-run=server
configmap/skala-backend-configmap created (server dry run)
service/skala-myfirst-api-server configured (server dry run)
deployment.apps/skala-myfirst-api-server configured (server dry run)
```

아래와 유사한 내용이 있다면 Invalid

The Deployment "skala-myfirst-api-server" is invalid: spec.selector: Invalid value: {"matchLabels":{"app":"skala-myfirst-api-server","skala.io/component":"skala-devops","skala.io/managed-by":"class-0-skala"}}: field is immutable

[실습] kustomize로 배포해보기

이미지를 kustomize를 통해 배포해보자

1. kustomize를 통해 직접 배포하기

```
> kubectl apply -k overlays/dev

> kubectl get pod -l app=sk000-myfirst-api-server
skala-myfirst-api-server-66f949d849-dnhtt   1/1     Running   0          29s
skala-myfirst-api-server-66f949d849-wxd7x   1/1     Running   0          15s
```

2. configmap 과 Service 확인하기

```
# service 검색하기
> kubectl get service  | grep sk000

# configmap 검색하기
> kubectl get service  | grep sk000
```

3. 배포된 자원 삭제

```
kubectl delete -k overlays/dev
```

[실습] kustomization.yaml 의 tag 변경하기

이미지를 kustomize를 통해 배포해보자

1. kustomize 설치하기

kustomization.yaml 의 images newTag 변경 기능은 kubectl에서 제공하지 않고 kustomize 에서만 지원

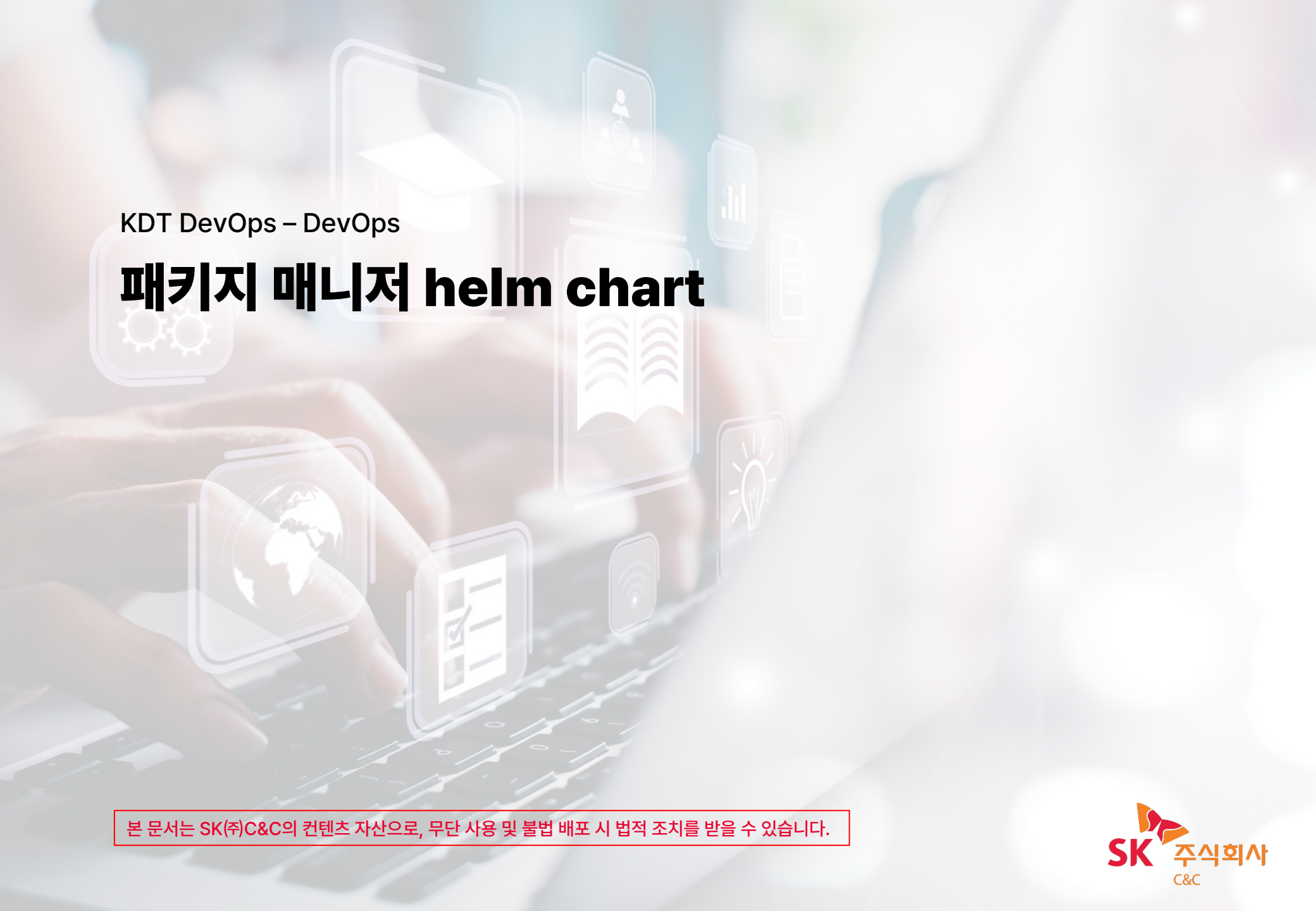
```
# kustomize 설치하기
> brew install kustomize
> kustomize version
```

2. overlays/dev/kustomization.yaml의 newTag 변경하기

환경에 대한 kustomization.yaml의 images의 newTag를 명려어로 변경해보자

```
kustomize edit set image <Current_Image_이름>=<New_Image_이름>:<새_태그>
```

```
# image tag를 변경해보자
> (cd overlays/dev && kustomize edit set image skala-registry.skala-?.com/class-0/sk000-webserver=skala-registry.skala-?.com/class-0/sk000-webserver:2.0 )

> cat overlays/dev/kustomization.yaml
…
- name: skala-registry.skala-ai.com/class-0/skala-webserver
  newName: skala-registry.skala-ai.com/class-0/skala-webserver
  newTag: "2.0"
```



KDT DevOps – DevOps

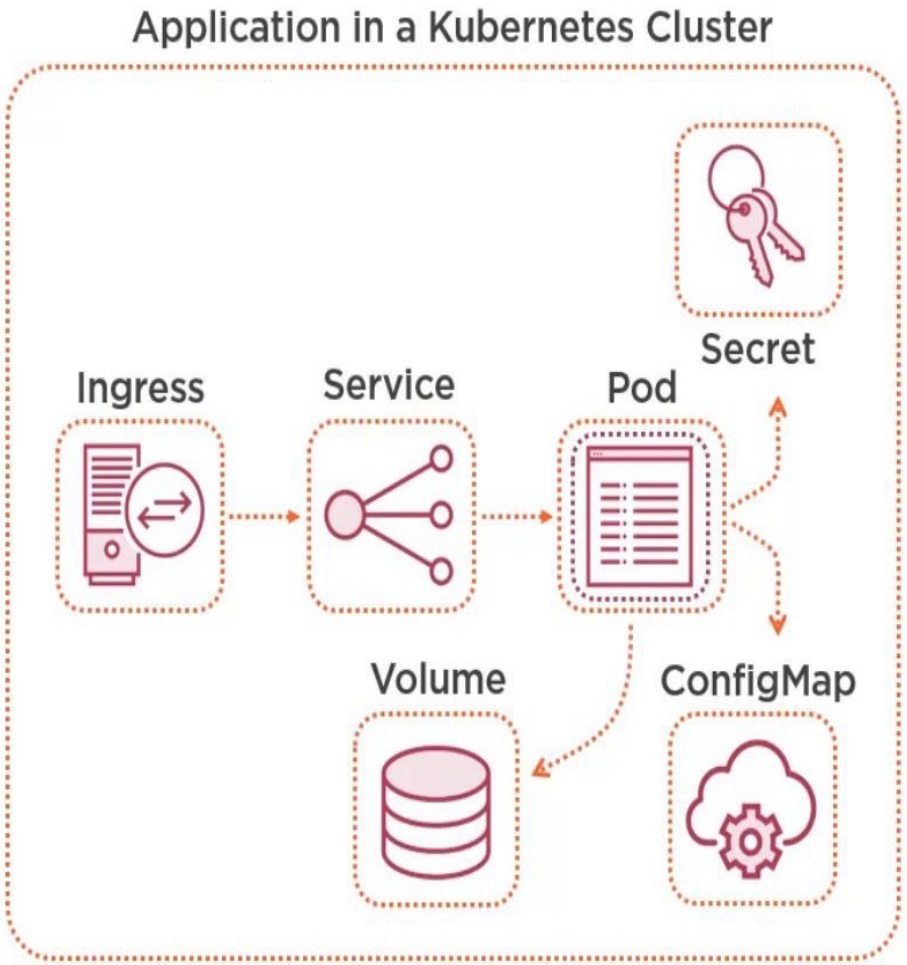
패키지 매니저 helm chart

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

k8s application 패키지 문제

애플리케이션을 kubernetes에 배포하기 위해 많은 리소스를 개별적으로 배포해야 함으로써 아래와 같은 제약 존재



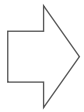
- 쿠버네티스에 배포되는 애플리케이션은 많은 Kubernetes Object를 포함
  - Pod
  - Services
  - Ingress
  - PV, PVC
  - Configmap, Secret and many more...
- **Packaing**: 각 오브젝트는 **서로 종속되어 있더라도 개별적으로 배포/**설치해야 하는 순서도 중요
- **Versioning**: k8s 오브젝트 버전이 독립적 관리되어 애플리케이션 단위 버전 개념 존재하지 않음.
  애플리케이션 단위 롤백 불가
- **Templating**:
  환경별 (dev/stage/prod) 별 다른 값 적용 어려움
- **Sharing**:
  팀 간 또는 외부에 공통 배포 패턴을 공유할 표준 방법이 없음

Template 기반 패키지 구조

중복되는 YAML에서 변경되는 값만 변수로 빼는 구조

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: my-app
          image: myregistry/my-app:1.0
```



templates/deployment.yaml

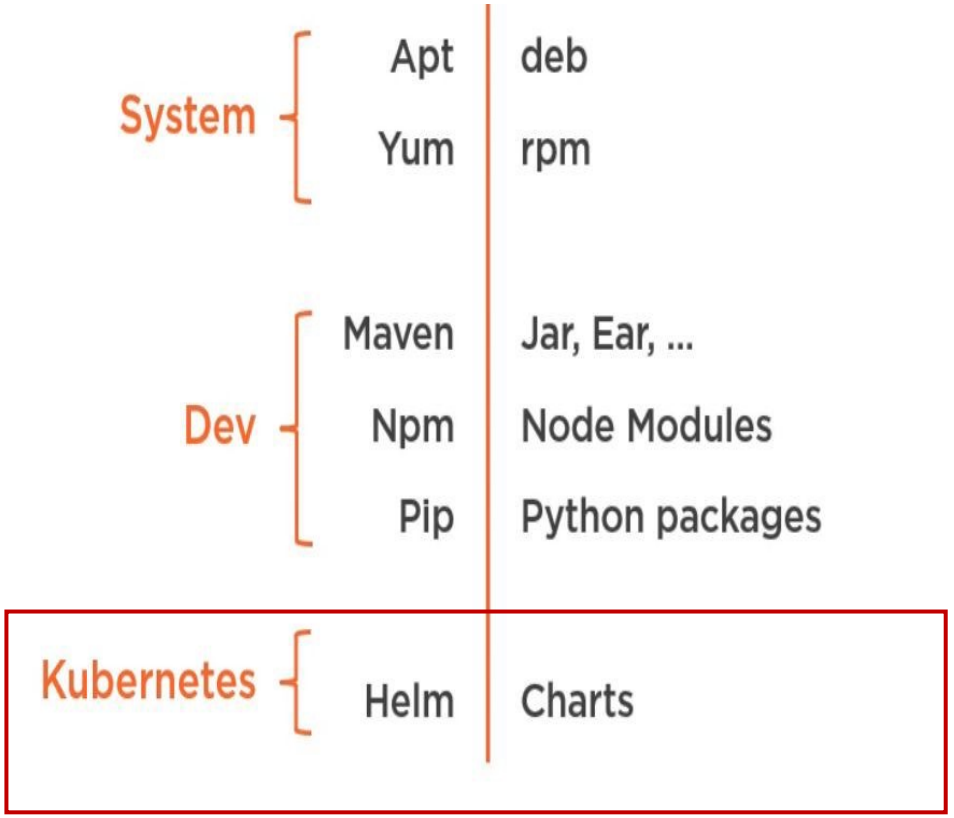
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.name }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: {{ .Values.name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

values.yaml

```yaml
name: my-app     {{ .Values.name }}

replicaCount: 3  {{ .Values.replicaCount }}

image:
  repository: myregistry/my-app {{ .Values.image.repository }}
  tag: "1.0"                    {{ .Values.image.tag }}
```

Helm

애플리케이션을 k8s에 배포하기 위한 여러 리소스를 패키지로 묶어서 통합 배포를 지원하는 k8s를 위한 **Package Manager**

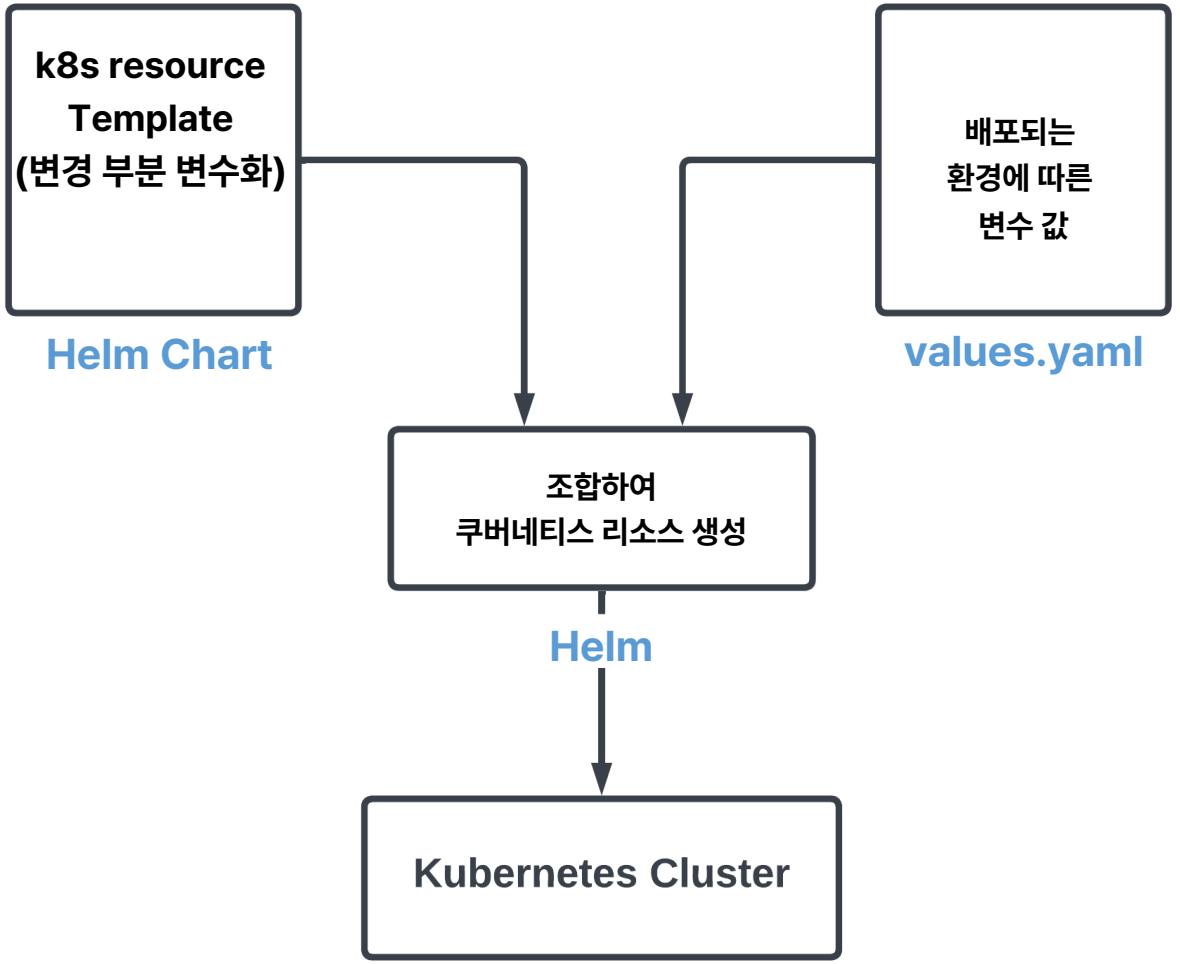


- Helm은 Kubernetes를 위한 애플리케이션 리소스를 패키징해서 통합 배포하는 **Package Manager**
- 애플리케이션 설치, 업그레이드, 삭제, 설정값 추적 및 롤백 등의 애플리케이션 패키지 기능 제공
- 하나의 애플리케이션이 필요로 하는 모든 오브젝트(리소스)를 패키징 설계를 chart로 제공

```
mychart/
├── Chart.yaml          # 차트 메타정보
├── values.yaml         # 사용자 설정값 (기본값)
├── templates/          # 쿠버네티스 리소스 템플릿 (Deployment, Service 등)
└── charts/             # 의존 차트 (서브 차트)
```

Helm Chart 동작 방식

- **Helm Chart**: 애플리케이션 배포를 위한 오브젝트를 패키지화 (템플릿과 생성을 위한 데이터 조립 방식 정의)
- **values.yaml** : 변경해야 할 변수들에 대한 Instance 실제값 정의



Helm Chart 작동 방식

helm chart는 Templates과 values Chart 정보를 구분되며 이것을 merge해서 동작

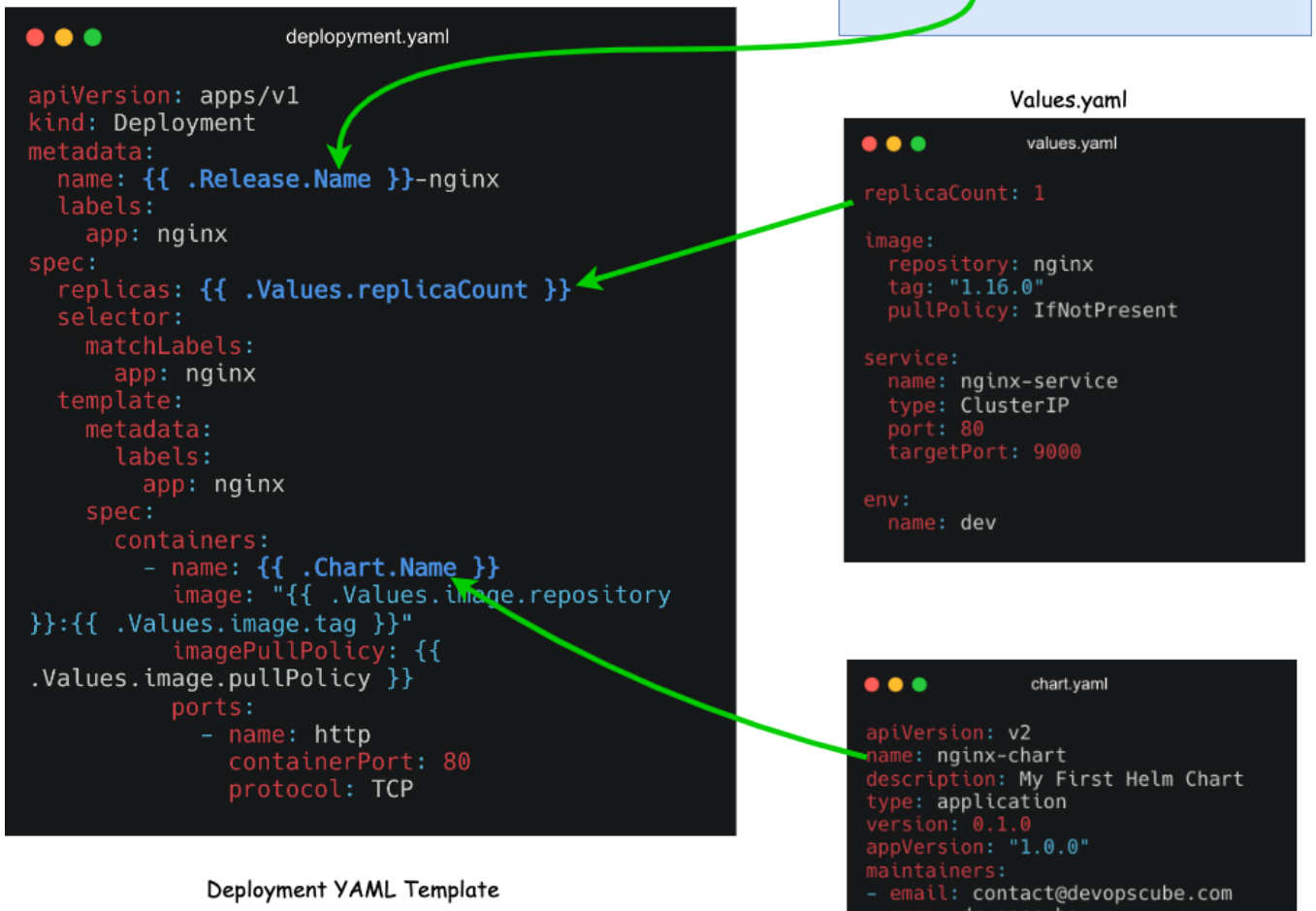
핵심 개념

values.yaml :
- 인스턴스 실제 설정값 저장소

templates/
- K8s 리소스 생성을 위한 동적 YAML 틀

Chart.yaml
- 패키지 메타데이터 (이름·버전)

```
my-nginx/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── hpa.yaml
└── charts/
```

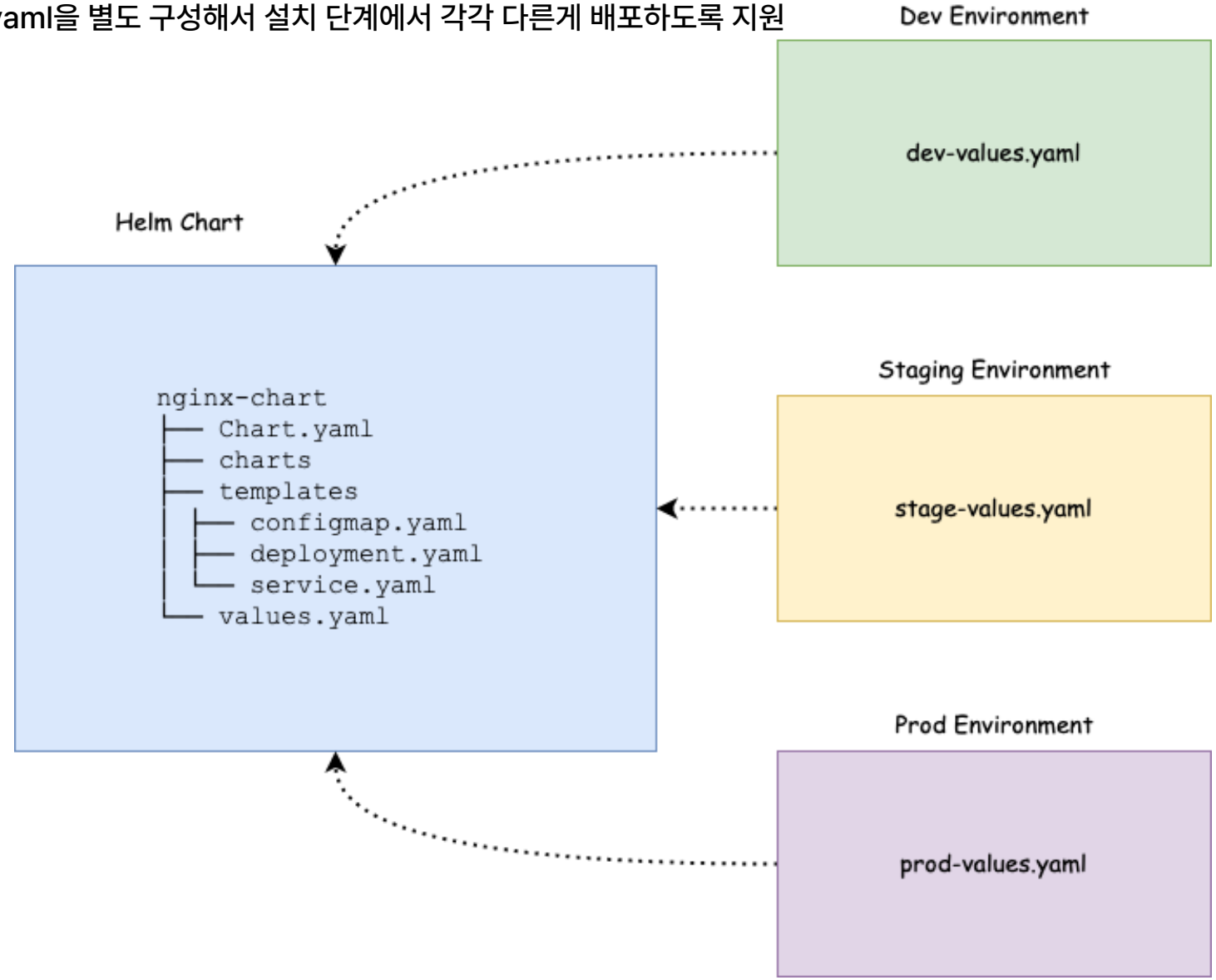


렌더링 흐름

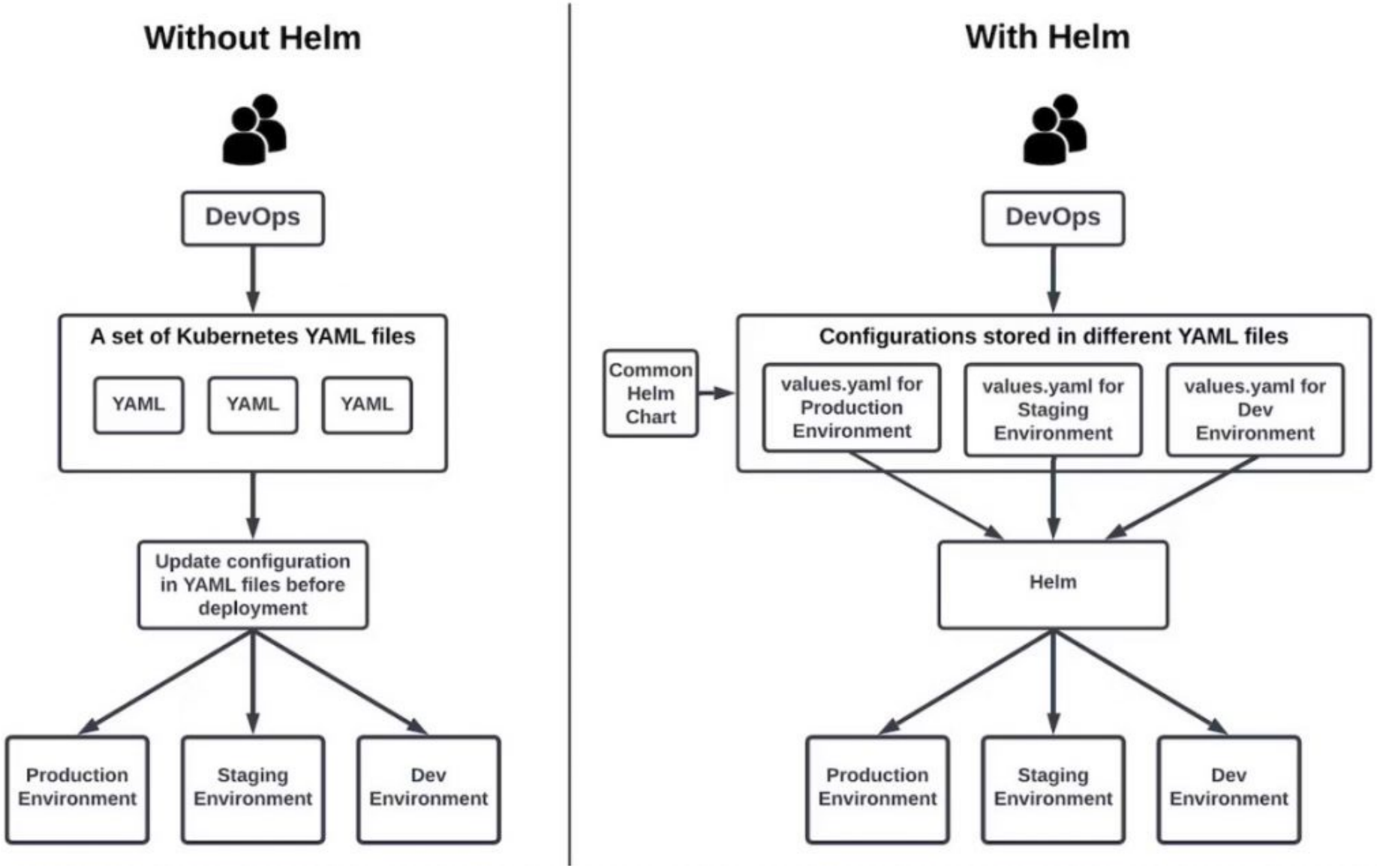
Chart (my-nginx) **+ values.yaml** → Helm 렌더링 → K8s 배포

Helm Chart : profile 별 배포 방식

프로파일 별 values.yaml을 별도 구성해서 설치 단계에서 각각 다른게 배포하도록 지원



with and Without Helm



Helm Command

목적	명령어	예시
릴리즈 설치	`helm install [release] [chart]`	`helm install myapp ./mychart  -f values.yaml`
업그레이드	`helm upgrade [release] [chart]`	`helm upgrade myapp ./mychart -f values.yaml`
Rollback	`helm rollback [release] [revision]`	`helm rollback myapp 2`
이력 확인	`helm history [release]`	`helm history myapp`
상태 확인	`helm status [release]`	`helm status myapp`
상세 정보 확인	`helm get [release]`	`helm get all myapp`
릴리즈 삭제	`helm uninstall [release]`	`helm uninstall myapp`
릴리즈 목록 확인	`helm list`	`helm list -A`

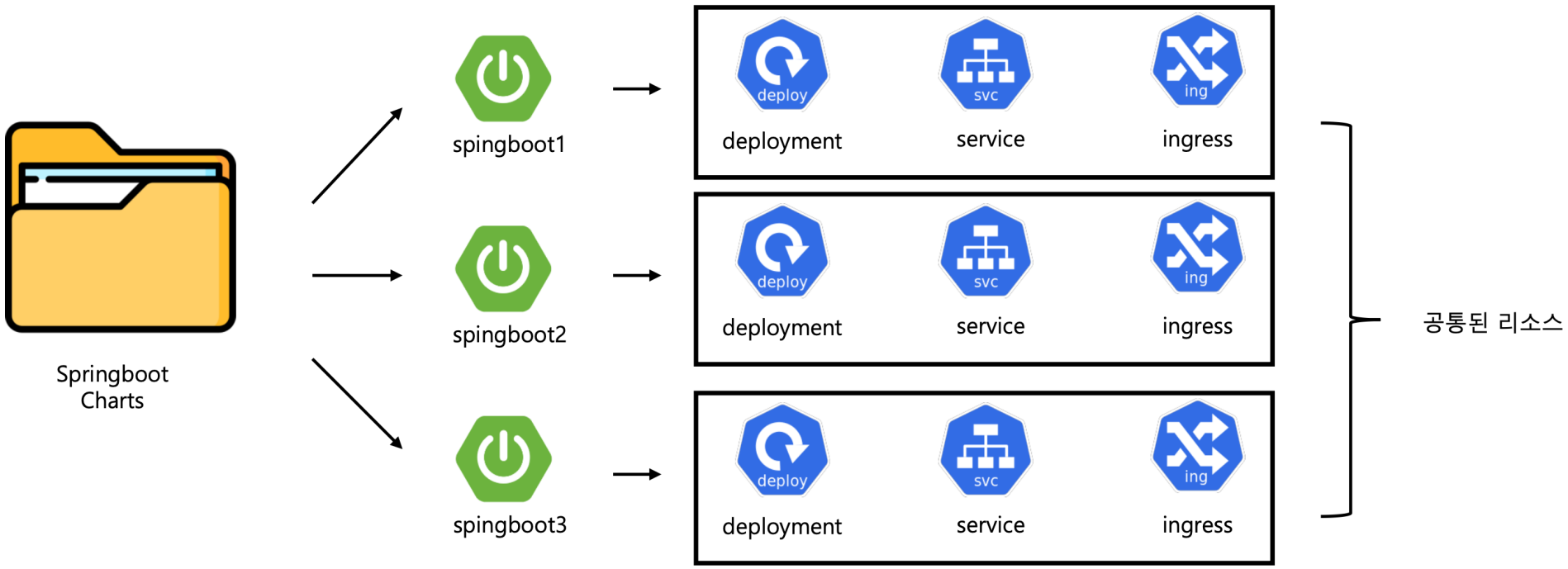
```
myapp/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── hpa.yaml
└── charts/
```

values.yaml

```yaml
replicaCount: 2

image:
  repository: skala-registry/myapp
  tag: "1.0"

service:
  type: ClusterIP
  port: 8080
```

Helm 도입 효과

언제 Helm 이 유용한가?
공통된 쿠버네티스 리소스를 반복적으로 배포하고자 하는 경우 유용함



Helm의 주요 특징

4가지 핵심 요소

복잡도 관리 (Manage Complexity)

여러 YAML을 묶어 규격화

“밀키트 하나로 요리 끝내기”

수십 개의 YAML 파일을 관리할 필요 없이, 차트 1개로 복잡한 인프라를 한 번에 배포

쉬운 업데이트 (Easy Updates)

템플릿(Template) 과 설정 (Value)의 분리

"버전 숫자 하나만 바꿔 재배포"

설정값(values.yaml)만 변경하여 연결된 모든 리소스를 안전하고 통일되게 업그레이드 가능

손쉬운 공유 (Simple Sharing)

패키지 Registry 기반 공유

"앱스토어에서 앱 다운받듯 설치"

검증된 설정을 압축 패키징(tgz)하여, 앱스토어처럼 레지스트리를 통해 팀/조직 간 손쉽게 공유

손쉬운 롤백 (Easy Rollback)

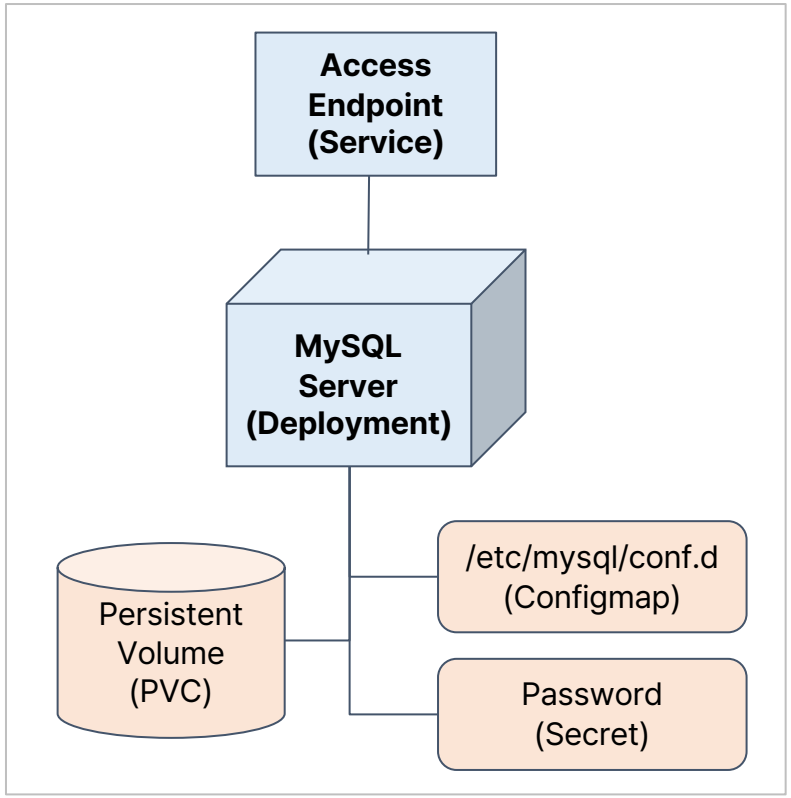
릴리스 이력 관리

"배포 망쳤을 때 누르는 실행 취소(Ctrl+Z)"

배포 실패 시 직전 정상 상태로 즉시 되돌릴 수 있는 원클릭 실행 취소(Undo)를 제공

Manage Complexity

애플리케이션 배포를 위한 다양한 리소스를 패키지화해서 관리 가능



[기존 배포 방식] 배포 환경에 따라 개별적으로 배포 지정

```
//Kubernetes yaml
> kubectl apply -f   deployment.yaml
> kubectl apply -f   svc.yaml
> kubectl apply -f   pvc.yaml
> kubectl apply -f   secret.yaml
> kubectl apply -f   configmap.yaml
```

helm chart 기반 배포

```
> helm repo add  bitnami https://charts.bitnami.com/bitnami
> helm repo update
> heml install  my-nginx bitnami/nginx
or
> helm  install my-nginx bitnami/nginx -f custom-values.yaml
```

Easy Update

애플케이션 전체를 하나의 버전으로 관리하고 업그레이드 지원

Upgrade 명령을 사용하여 설정이나 버전을 쉽게 업데이트

```
# 기존 설정에서 일부 설정만 변경
helm upgrade my-nginx bitnami/nginx \
     --reuse-values \
     --set mysqlRootPassword="password”

# 새로운 설정으로 변경
helm upgrade my-nginx bitnami/nginx \
     -f values-prod.yaml

# 지정한 특정 차트 버전으로 변경
helm upgrade mysql-release stable/mysql   --version x.x.x
```

Simple Sharing

애플리케이션 단위로 패키지화해서 다양한 형태로 공유 가능

Package

```
helm package ./mycharts
myapp-1.1.0.tgz 생성
```

※ tgz 으로 패키징되며 이것은 artifecthub.io 또는 harbor의 registry에 등록 가능

Share

```
helm repo index . --url https://himang10.github.io/mycharts # chart 설명 문서
git push origin master
```

index.yaml
- 어떤 Chart가 있는지
- 버전이 몇 개인지
- tgz 파일이 어디 있는지

github

```
mycharts/
├── index.yaml
├── myapp-1.0.0.tgz
├── myapp-1.1.0.tgz
└── nginx-1.0.0.tgz
```

Download

```
# helm repo 등록. myrepo 이름을 사용
helm repo add myrepo https://himang10.github.io/mycharts

# index.yaml download
helm repo update

# myapp-1.1.0.tgz download
helm install myrepo/myapp --version 1.1.0
```

Easy Rollback

특정 버전으로 rollback 지원

Rollback 명령을 사용하여 이전 버전 또는 특정 버전으로 쉽게 복구

```
$ helm history mycharts
```

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
1	2024-04-01 10:00:00 +0900	deployed	nginx-13.1.0	1.23.1	Install complete
2	2024-04-03 15:20:00 +0900	superseded	nginx-13.1.1	1.24.0	Upgrade complete
3	2024-04-04 17:00:00 +0900	deployed	nginx-13.1.2	1.25.0	Upgrade complete

```
$ helm rollback mycharts 2
```

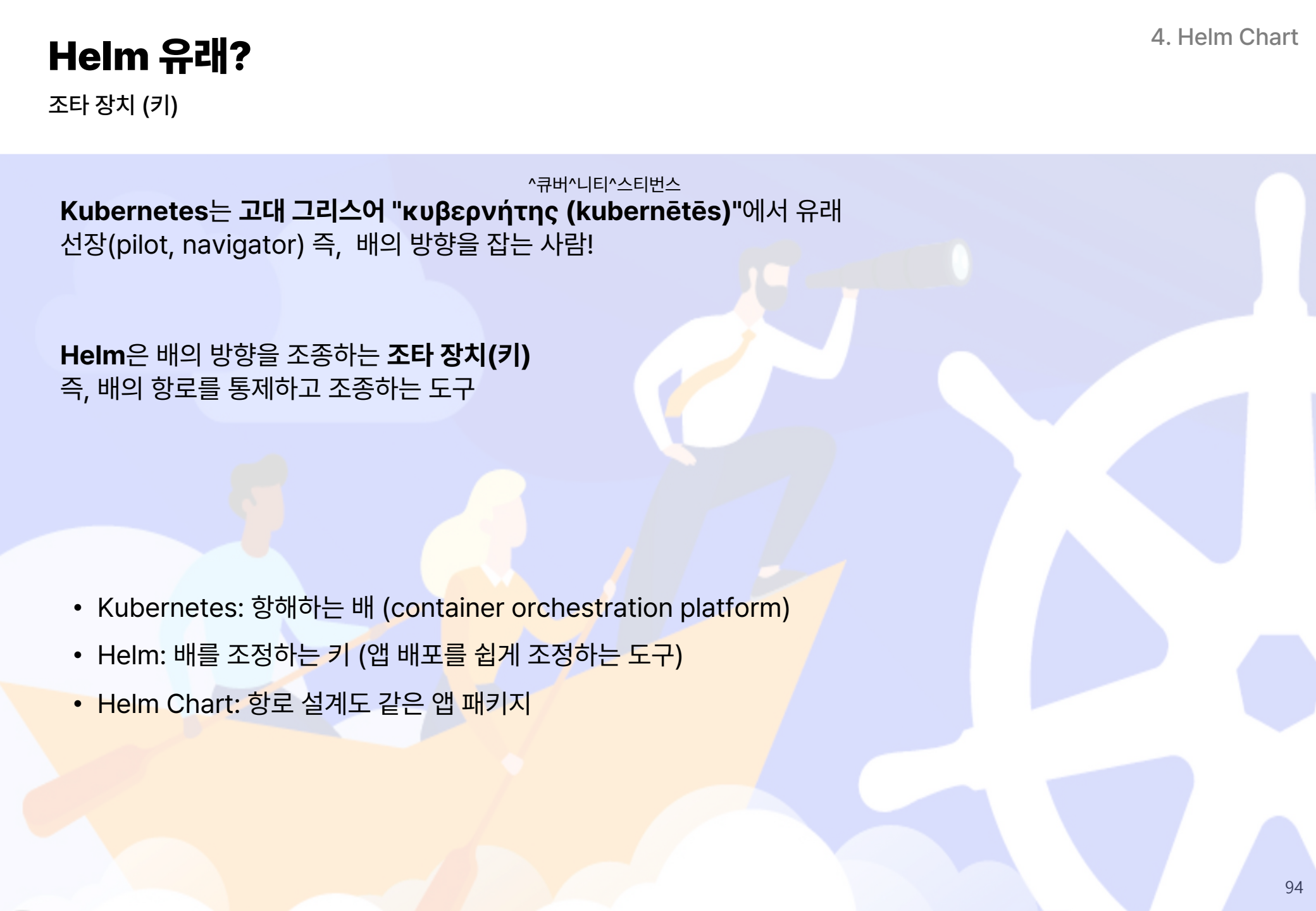
Helm 유래?

조타 장치 (키)

^큐버^니티^스티번스

Kubernetes는 **고대 그리스어 "κυβερνήτης (kubernētēs)"**에서 유래
선장(pilot, navigator) 즉, 배의 방향을 잡는 사람!

Helm은 배의 방향을 조종하는 **조타 장치(키)**
즉, 배의 항로를 통제하고 조종하는 도구



- Kubernetes: 항해하는 배 (container orchestration platform)
- Helm: 배를 조정하는 키 (앱 배포를 쉽게 조정하는 도구)
- Helm Chart: 항로 설계도 같은 앱 패키지

[쉬어가는 페이지]

IZZAT BABA

maje

PARIS

EIDER

[실습] helm chart를 만들어보자

위치: 03.helm

1. 03.helm 으로 이동 후 ,
helm create 명령을 실행해서 새로운 chart를 생성할 수 있음

```
> helm create mychart
> tree mychart
```

디렉토리 구조

```
mychart/
├── Chart.yaml              # 차트 메타데이터 (필수)
├── values.yaml             # 기본 설정값 (사용자 커스터마이징 가능)
├── charts/                 # 의존(서브) 차트 디렉토리
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml        # Kubernetes 리소스 템플릿
│   ├── _helpers.tpl        # 재사용 템플릿 함수 정의
│   └── NOTES.txt           # 설치 완료 후 출력 메시지
└── .helmignore             # 패키징 제외 파일 목록
```

Chart 디렉토리 구조: Chart.yaml

Chart 자체에 대한 Meta data를 포함한 기본 파일.

```
# Chart.yaml 예시
apiVersion: v2           # Helm 3: v2, Helm 2: v1
name: myfirst-api-server  # 차트 이름
description: xxx
type: application        # application 또는 library
version: 0.0.1           # 차트 패키지 버전 (SemVer)
appVersion: "1.0.0"      # 배포되는 앱의 실제 버전
```

version(차트 버전)과 appVersion(앱 버전)은 독립적
앱 코드는 그대로여도 차트 구조 변 경 시 version을 올림

Chart 디렉토리 구조: values.yaml

Helm Chart Template에 주입되는 기본 환경 설정값을 모아둔 파일이며, 배포 환경 별로 설정을 분리해서 사용

values.yaml

```yaml
replicaCount: 1

image:
  repository: sk000-registry.skala-ai.com/class-0/skala-webserver
  tag: "1.0.buildkit-pod”
  pullPolicy: Always

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: true
  className: "public-nginx"
  hosts:
    - host: sk000-ingress.class-0.skala-ai.com
      paths:
        - path: /api
          pathType: Prefix
  tls: []
    - secretName: chart-example-tls
      hosts:
        - chart-example.local
```

Chart 디렉토리 구조: templates

values.yaml의 필드와 결합될 kubernetes resource template을 담고 있는 디렉토리. go 템플릿 언어를 사용

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myfirst-chart.fullname" . }}
  labels:
    {{- include "myfirst-chart.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "myfirst-chart.selectorLabels" . | nindent 6 }}
  template:
…
 spec:
..
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
```

내장 변수: .Chart, .Release, .Release.Namespace

1. Releass.Name과 Release.Namespace 는 helm install 단계에 주입

```
> helm template myapp ./mychart --namespace dev
```

Release.Name

Release.Namespace

2. Chart.Name 은 Chart.yaml의 name 값

```
apiVersion: v2
name: mychart
appVersion: "1.16.0"
```

3. templates/deployment.yaml

```
metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
  namespace: {{ .Release.Namespace }}
  labels:
    app.kubernetes.io/name: {{ .Chart.Name }}
    app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
```

4. 렌더링 결과

```
metadata:
  name: myapp-nginx
  namespace: dev
  labels:
    app.kubernetes.io/name: mychart
    app.kubernetes.io/version: "1.16.0"
```

Chart 디렉토리 구조: _helpers.tpl

공통적으로 사용하는 Template 함수를 정의 (templates 아래 리소스 템플릿에 공통으로 사용할 템플릿 정의)

templates/_helpers.tpl

```
{{- define "myfirst-chart.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}
..
{{- define "myfirst-chart.chart" -}}
{{- printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" | trunc 63 | trimSuffix "-" }}
{{- end }}

{{- define "myfirst-chart.labels" -}}
helm.sh/chart: {{ include "myfirst-chart.chart" . }}
{{ include "myfirst-chart.selectorLabels" . }}
{{- if .Chart.AppVersion }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
{{- end }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}
```

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myfirst-chart.fullname" . }}
  labels:
    {{- include "myfirst-chart.labels" . | nindent 4 }}
```

helm 명령어 실행 방법

1. helm chart 설치하기

```
# -f 옵션을 주고 override values.yaml을 설정해서 install
helm install -f <override values.yaml> <Release 이름> <chart 경로>
예)  helm install -f myvalues.yaml my-mariadb ./mariadb
```

2. values.yaml field overriding 방법

```
# --set 옵션으로 필드 값을 overriding
helm install --set <overriding field>=<value> <Release 이름> <chart 경로>
예)  helm install --set image.repository=mariadb --set image.tag="1.0" my-mariadb ./mariadb
```

3. Helm Chart 병합 우선순위

기본 values.yaml 은 가장 낮은 우선순위 , -f 로 넘긴 파일이 더 높은 우선순위, --set otion이 가장 높은 우선순위
우선순위에 따라 3개 모두 병합 실행

[실습] Generated Yaml을 만들어보자

1. 앞에서 만들었던 myfirst-chart 로 이동해보자

```
> cd mychart/
```

2. template/deployment.yaml 의 값 확인

내용 확인하기

3. values.yaml 확인

```
# This will set the replicaset count more information can be found here:
https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/
replicaCount: 1

# This sets the container image more information can be found here: …
image:
  repository: nginx
  # This sets the pull policy for images.
  pullPolicy: IfNotPresent
  # Overrides the image tag whose default is the chart appVersion.
  tag: ""
```

4. template 기반 yaml 생성해보기

```
> helm template . --show-only templates/deployment.yaml
```

[실습] helm으로 나의 skxxx-webserver 배포하기 (1/2)

1. mychart 상단으로 이동한다.

```
> cd skala-devops
```

2. 현재 위치에서 myfirst-values.yaml을 만들어보자

```
replicaCount: 1

image:
repository: sk000-registry.skala-??.com/class-0/skala-webserver
tag: "1.0.buildkit-pod"

livenessProbe: null
readinessProbe: null
startupProbe: null
```

3. helm으로 직접 설치해보자

```
helm upgrade --install skala-myfirst-api-server \
 --wait \
 --atomic \
 ./myfirst-chart \
 -f myfirst-values.yaml
```

[실습] helm으로 나의 skxxx-webserver 배포하기 (2/2)

4. helm으로 직접 설치해보자

```
helm upgrade --install skala-myfirst-api-server \
 --wait \
 --atomic \
 ./myfirst-chart \
 -f values.yaml
```

5. helm 설치 확인 하기

```
> helm list
NAME                        NAMESPACE     REVISION      UPDATED                                   STATUS        CHART
sk000-myfirst-api-server    class-0       1             2026-10-09 15:34:43.37394 +0900 KST       deployed      myfirst-chart-0.1.0

> k get pod
NAME                                                  READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-myfirst-chart-55697ccb69-zpt44   1/1     Running   0          4m4s
```

6. helm 삭제하기

```
> helm delete sk000-myfirst-api-server
NAME                        NAMESPACE     REVISION      UPDATED                                   STATUS        CHART
sk000-myfirst-api-server    class-0       1             2026-10-09 15:34:43.37394 +0900 KST       deployed      myfirst-chart-0.1.0
```

[실습] myfirst-api-server helm 만들기 (1/4)

위치: 03.helm

1. 자신의 helm chart만들어보자

```
helm create myfirst-helmchart
```

2. myfirst-helmchart/values.yaml 확인하기

```yaml
image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: ""

fullnameOverride: ""

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false
  className: ""
  annotations: {}
    # kubernetes.io/ingress.class: nginx
    # kubernetes.io/tls-acme: "true"
  hosts:
    - host: chart-example.local
      paths:
        - path: /
          pathType: ImplementationSpecific
  tls: []
  #  - secretName: chart-example-tls
  #    hosts:
  #      - chart-example.local
```

03.helm/skala-value/skala-values.yaml 를 확인해보자

```yaml
image:                                  판교: ai    울산: us
  repository: skala-registry.skala-??.com/class-0/sk000-webserver
  pullPolicy: Always
  tag: "1.0"
fullnameOverride: "sk000-myfirst-helmchart"

service:
  type: ClusterIP
  port: 8080

ingress:
  enabled: true
  className: ""
  annotations: {}
    # kubernetes.io/ingress.class: nginx
    # kubernetes.io/tls-acme: "true"
  hosts:                                        판교: ai    울산: us
    - host: sk000-ingress.class-0.skala-??.com
      paths:
        - path: /
          pathType: Prefix
  tls: []                                     판교: ai    울산: us
    - secretName: wildcard-class-0-skala-??-com-tls
      hosts:
        - sk000-ingress.class-0.skala-??.com
```

[실습] myfirst-api-server helm 만들기 (2/4)

위치: 03.helm

2. ./myfirst-helmchart/Chart.yaml에서 chart 이름을 변경해보자

```
apiVersion: v2
name: myfirst-helmchart   → sk000-myfirst-helmchart  로 변경하자
description: A Helm chart for Kubernetes

# A chart can be either an 'application' or a 'library' chart.
#
# Application charts are a collection of templates that can be packaged into versioned archives
# to be deployed.
#
# Library charts provide useful utilities or functions for the chart developer. They're included as
# a dependency of application charts to inject those utilities and functions into the rendering
# pipeline. Library charts do not define any templates and therefore cannot be deployed.
type: application

# This is the chart version. This version number should be incremented each time you make changes
# to the chart and its templates, including the app version.
# Versions are expected to follow Semantic Versioning (https://semver.org/)
version: 0.1.0

# This is the version number of the application being deployed. This version number should be
# incremented each time you make changes to the application. Versions are not expected to
# follow Semantic Versioning. They should reflect the version the application is using.
# It is recommended to use it with quotes.
appVersion: "1.16.0"
```

[실습] myfirst-api-server helm 만들기 (3/4)

위치: 03.helm

1. skala-value/skala-values.yaml 을 기반으로 myfirst-api-server를 배포해보자

```
> helm upgrade --install ${STUDENT_NUM}-myfirst-helm ./myfirst-helmchart \
--version 1.0.0 \
--atomic  \                                  # 업그레이드 실패하면 자동으로 이전 정상 버전으로 롤백
--timeout  3m \                              # 리소스가 정상 실행될때까지 기다리는 시간
-f ./skala-value/skala-values.yaml
```

2. https://sk000-ingress.class-0.skala-??.com 접속하기

[참고] 01.local-install.sh 를 직접 실행해서 위의 명령어를 수행할 수 있다.

```
> cat 01.local-install.sh

> chmod + *.sh
> ./01.local-install.sh
```

[실습] myfirst-api-server helm 만들기 (4/4)

위치: 03.helm

배포 현황 확인하기

1. 배포 결과 확인하기

```
# 설치된 helm 목록 확인하기
helm list | grep sk000

# 릴리스 상태 요약
helm status sk000-myfirst-helm

# 릴리스 이력(리비전) 확인
helm history sk000-myfirst-helm

# (필요 시) 실제 배포된 매니페스트 확인
helm get manifest sk000-myfirst-helm | less

# (필요 시) 적용된 values 확인
helm get values sk000-myfirst-helm
```

[참고] Helm Dry-run

```
# 검증하기. 설치하지 않고 설치전까지만 진행
helm upgrade --install sk000-my-webserver . -f values.yaml --namespace skala-practice --debug --dry-run

# 설치작업
helm upgrade --install sk199-my-webserver . -f values.yaml --namespace skala-practice –wait --atomic
```

Dry-run 이란?

dry-run은 실제로 Kubernetes 클러스터에 리소스를 배포하지 않고, 배포 시 어떤 일이 발생할지 미리 확인할 수 있는 기능

Dry-run 동작 방식

1. **템플릿 렌더링**
   Helm은 차트의 모든 템플릿을 렌더링하여 YAML 매니페스트를 생성

2. **서버 측 유효성 검사**
   --dry-run: 클라이언트 측에서만 템플릿을 렌더링하고 검증.
   이는 리소스가 실제로 생성되지는 않지만, 구문과 리소스 요구 사항을 확인

3. **결과 출력**
   생성될 YAML 매니페스트와 함께 검증 결과를 출력합니다. 오류가 있을 경우, 어떤 문제가 있는지 표시합니다.

[실습] helm upgrade

위치: 04.devops/03.helm

-f skala-value/skala-values-2.0.yaml의 설정에 따라 upgrade

```
> helm upgrade ${STUDENT_NUM}-myfirst-helm ./myfirst-api-server \
--reuse-values \
-f ./skala-value/skala-values-2.0.yaml \
--atomic  \
--timeout  3m
```

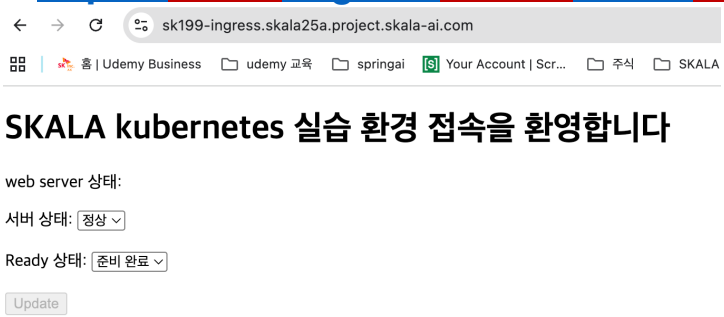
skala-values-2.0.yaml

```
# 컨테이너 이미지 태그를 2.0으로 변경
image:
  tag: "2.0"

# 레플리카를 2개로 증가
replicaCount: 2
```

2. https://sk000-ingress.class-0.skala-??.com 접속하기

index.html이 캐쉬되어 있으므로 캐시 제거및 리로드 필요

Command + shift + R



[참고] 01.local-upgrade.sh 를 직접 실행해서 위의 명령어를 수행할 수 있다.

```
> cat 02.local-upgrade.hs

> chmod + *.sh
> ./02.local-upgrade.hs
```

[실습] helm rollback

위치: 04.devops/03.helm

1. 로컬에 Helm chart를 1.0으로 Rollback하기

```
> helm history ${STUDENT_NUM}-myfirst-helm
REVISION    UPDATED                     STATUS      CHART                     APP VERSION  DESCRIPTION
1           Sat Mar 14 14:04:42 2026    superseded  myfirst-helmchart-0.1.0   1.0.0        Install complete
2           Sat Mar 14 14:15:28 2026    deployed    myfirst-helmchart-0.1.0   1.0.0        Upgrade complete
```

2. 아래와 같이 rollback 실행하기

```
> helm rollback ${STUDENT_NUM}-myfirst-helm 1 \
  --wait \
  --timeout 3m
```

2. https://sk000-ingress.class-0.skala-??.com 접속하기

[참고] 01.local-rollback.sh 를 직접 실행해서 위의 명령어를 수행할 수 있다.

```
> cat 01.local-rollback.sh

> chmod + *.sh
> ./01.local-rollback.sh
```

[참고] 자주 쓰는 패턴/팁

value file의 여러개 병합하기

```
helm upgrade --install sk199-my-webserver ./mychart \
  -n class-0 \
  -f values.yaml \
  -f values-prod.yaml \
  --wait --timeout 5m --atomic
```

- values.yaml : 기본 값
- values-prod.yaml : 운영 환경에 맞게 override

일회성 덮어쓰기 (--set)로 image.tag=2.0. 을 강제

```
helm upgrade --install sk199-my-webserver ./mychart \
  -n skala-practice -f values.yaml \
  --set image.tag=2.0.0 \
  --wait --timeout 5m --atomic
```

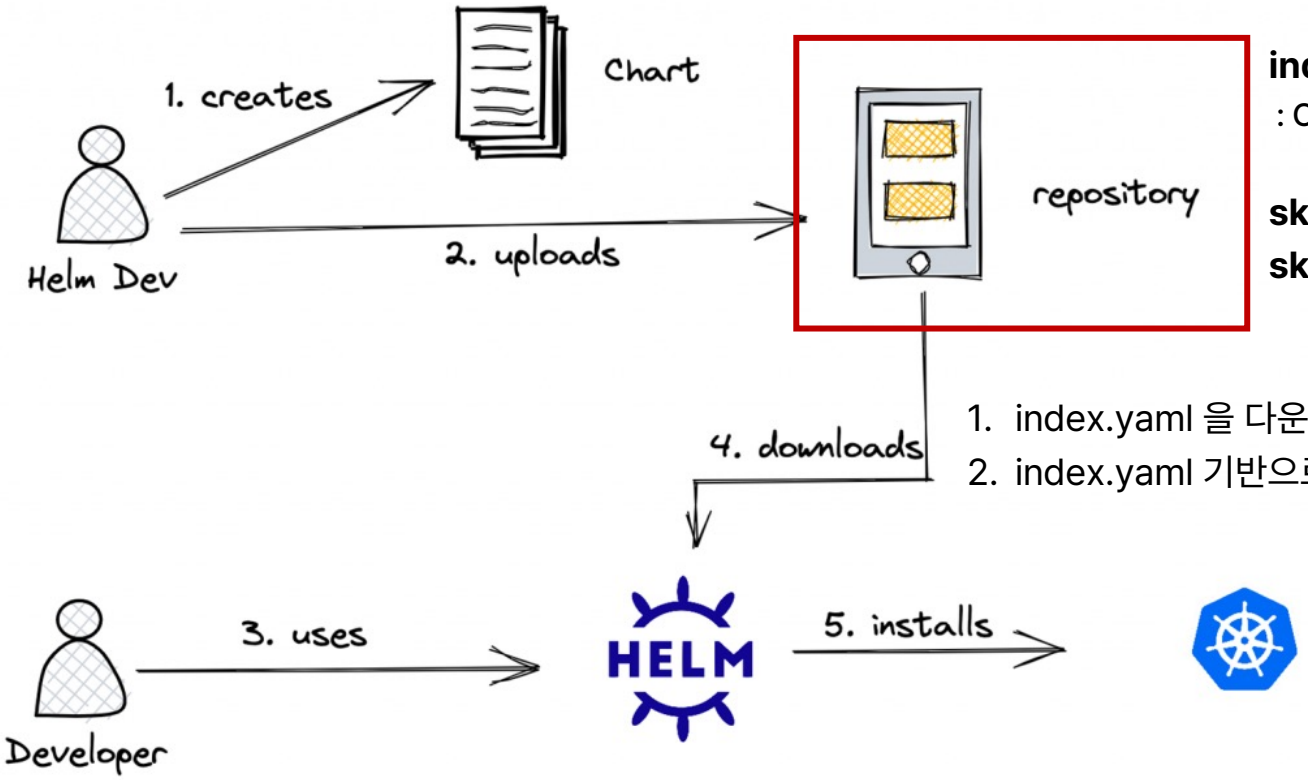
--set 과의 우선순위

Helm values 우선순위는 아래와 같음 (뒤로 갈수록 우선순위 ↑):

- Chart의 values.yaml (차트 안에 있는 기본값)
- -f values.yaml 같은 외부 values 파일
- -f values-prod.yaml 같은 추가 values 파일 (뒤에 오는 파일이 앞을 override)
- --set key=value (명령줄에서 지정한 값이 최종 우선)

Helm Chart Repository

k8s 애플리케이션을 배포하기 위한 Helm Chart 패키지를 저장하고, 버전별로 관리하며, 시스템에 배포할 수 있도록 제공하는 저장소



index.yaml

: Chart 이름, 버전, 패키지 URL 등의 메타데이터

skala-myfirst-helmchart-0.0.1.tgz

skala-myfirst-helmchart-0.0.2.tgz

1. index.yaml 을 다운로드
2. index.yaml 기반으로 tgz download 및 설치 실행

Helm Chart Repository: index.yaml

Chart 이름, 버전, 패키지 URL 등의 메타데이터

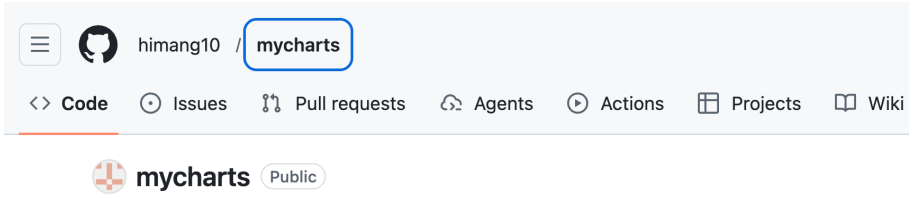
```
apiVersion: v1
entries:
  skala-myfirst-helmchart:
  - apiVersion: v2
    appVersion: 1.16.0
    created: "2026-10-05T11:38:16.983816+09:00"
    description: A Helm chart for Kubernetes
    digest: 4abdfaa7e2bad1c4ada738f4dd2f58a34e73f1b85f78cafbeb9e4f811f2b8970
    name: skala-myfirst-helmchart
    type: application
    urls:
    - https://skala-fileserver.skala-ai.com/mycharts/skala-myfirst-helmchart-0.0.2.tgz
    version: 0.0.2
  - apiVersion: v2
    appVersion: 1.16.0
    created: "2026-10-05T11:38:16.983508+09:00"
    description: A Helm chart for Kubernetes
    digest: 4902e658e2a666b3228be71d0a4ed2aeda8e4fe539e25ba4a637076c52ab8b47
    name: skala-myfirst-helmchart
    type: application
    urls:
    - https://skala-fileserver.skala-ai.com/mycharts/skala-myfirst-helmchart-0.0.1.tgz
    version: 0.0.1
generated: "2026-10-05T11:38:16.982751+09:00"
```

[실습] github 기반 Repository 구성 (1/4)

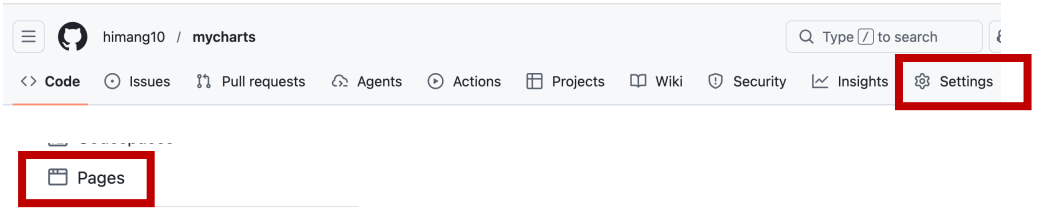
github 를 이용해서 helm repository를 위한 **정적 웹서버를 구성**해보자

1. github에 mycharts repository 생성하기

```
a. github에 mycharts repository를 생성
b. public으로 구성하고
```



2. github page 지정

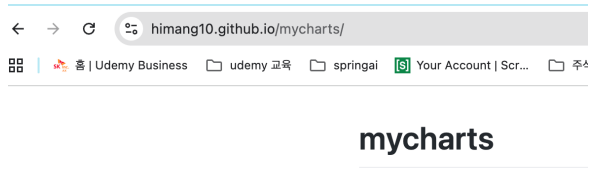


3. github page 만들기

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at https://himang10.github.io/mycharts/
Last deployed by himang10 18 minutes ago
Visit site | Unpublish site

4. page를 만들면 {your git name}.github.io/mycharts 접속 가능



[실습] github 기반 Repository 구성 (2/4)

위치: 03.helm

github page를 이용해서 나만의 helm chart repository를 만들고, 여기에 나의 helm chart를 push 해보자

5. skala-value/skala-values.yaml을 myfirst-helmchart/values.yaml로 복사하기

```
> cp -rf ./skala-value/skala-values.yaml ./myfirst-helmchart/values.yaml
> cat ./myfirst-helmchart/values.yaml
```

6. local mycharts repositor에 04.devops/03.helm/myfirst-helmchart 복사

```
> git clone https//github.com/{yourname}/mycharts.git
> cd mycharts
> cp -rf ~/workspace/04.devops/03.helm/myfirst-helmchart .
```

[실습] github 기반 Repository 구성 (3/4)

위치: 03.helm

local에서 test를 한 myfirst-helmchart helm chart를 패키지화해서 git 기반 helm chart repository에 업로드해보자

1. helm package

```
# package 만들기 (tgz)
> helm package ./myfirst-helmchart --version 0.0.1
> ls *.tgz (tar + zip)
```

2. index.yaml 생성하기

```
# index.yaml 파일을 생성하고 tgz을 찾기 위한 base url 정보를 등록
> helm repo index . --url https://himang10.github.io/mycharts # chart 설명 문서
```

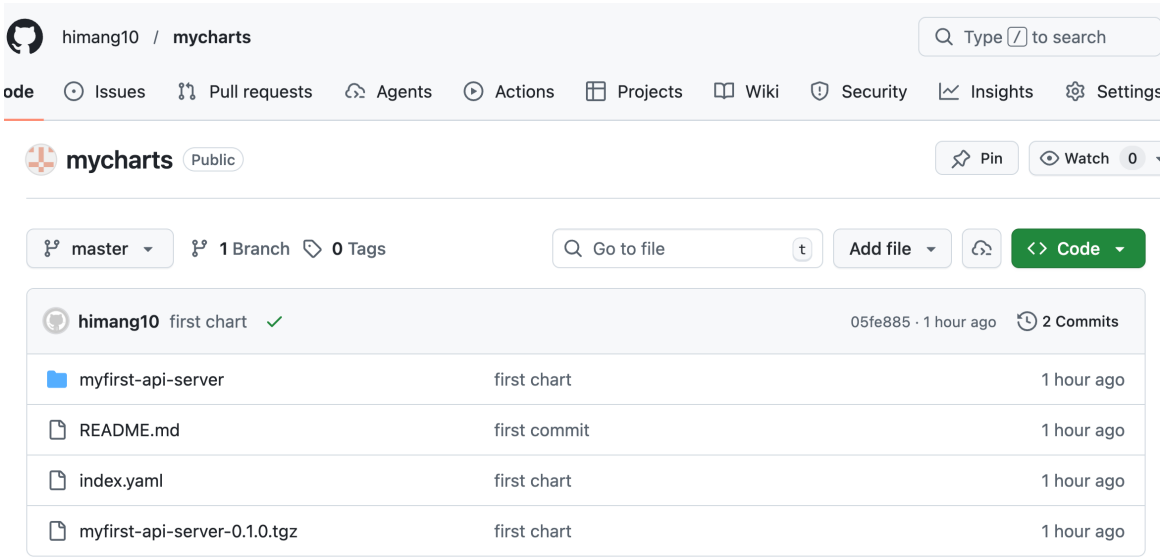
3. index.yaml 내용 확인하기

```
> cat index.yaml
```

index.yaml digest : shasum –a 256 {helmcode}.tgz

4. git 기반 helm chart repository에 push하기

```
> git add .
> git commit -m "first chart"
> git push origin main
```



[실습] github 기반 Repository 구성 (4/4)

위치: 03.helm

git repository로부터 직접 pull해서 설치하기

1. 사전 작업 (기존 helm chart를 제거하자)

```
> helm delete sk000-myfirst-helm
```

2. 설치 실행

```
helm repo add myrepo https://{자기 git 계정 이름}.github.io/mycharts
helm repo update
helm search repo myrepo/sk000-myfirst-helm --versions
NAME                                        CHART VERSION         APP VERSION           DESCRIPTION
nginxrepo/sk000-myfirst-helmchart   0.0.2                 1.16.0      A Helm chart for Kubernetes
nginxrepo/sk000-myfirst-helmchart   0.0.1                 1.16.0      A Helm chart for Kubernetes

helm upgrade --install sk000-myfirst-helm myrepo/sk000-myfirst-helmchart  \
             --version 0.0.1  \
             --namespace  ${CLASS_NAME} \
             --atomic \
             --timeout 3m
```

3. https://sk000-ingress.class-0.skala-??.com 접속하기

[참고] 이것을 실행하게되면 version 0.0.2 helm chart로 실행

```
> cat 04.install-git.sh
> ./04.install-git.sh
```

참고: helm repo add 시 index.yaml 을 .cache/helm/repository에 저장
helm install 시 .tgz을 download해서 .cache에 임시 저장후 설치 진행

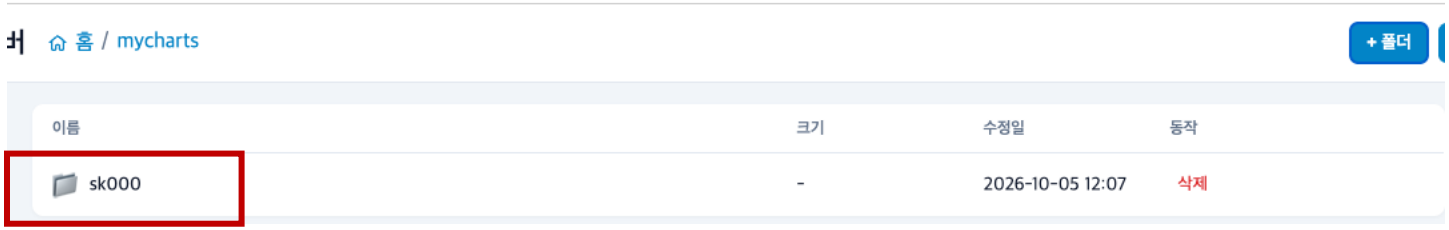
[실습] nginx 기반 file-server 를 repo로 만들기

https://skala-fileserver.skala-??.com 은 nginx 기반 정적 웹서버로 만들어져 있음
파일 서버로써 역할 수행

1. 로그인 :

2. 폴더 생성

3. 생성한 폴더 선택



4. index.yaml 과 tgz 업로드

[데모] nginx 기반 file-server 구조 검토하기

접속권한이 없기때문에 데모 형태로 보여주기

```
> k exec -it skala-fileserver-0 -n skala -- /bin/bash
---- container 내부 ------
:# ls /data/mycharts
index.yaml  skala-myfirst-helmchart-0.0.1.tgz  skala-myfirst-helmchart-0.0.2.tgz

:# cat /etc/nginx/nginx.conf
…
# 실제 파일 경로 그대로 직접 다운로드 (UI와 별개, /data/ 접두사 없이)
        # 예: curl -f -O https://skala-fileserver.skala-ai.com/cloud/k8s-basic/setup.tar
        # 위의 명시적 location(=/, =/app.js, =/style.css, =/healthz, /data/)에 걸리지 않는
        # 나머지 모든 경로가 여기로 들어오며, 다운로드/목록조회(GET)만 허용하고 쓰기는 막는다.
        location / {
            root /data;
            autoindex on;
            autoindex_format json;

            limit_except GET {
                deny all;
            }

            add_header Cache-Control "no-store";
        }

:# ps -ef
```

[실습] file-server 기반 helm 설치하기

위치: 03.helm

git repository로부터 직접 pull해서 설치하기

1. 사전 작업 (기존 helm chart를 제거하자)

```
> helm delete sk000-myfirst-helm
```

2. 설치 실행

판교: ai, 울산: us

자신이 만든 디렉토리

```
helm repo add nginxrepo https://skala-fileserver.skala-??.com/mycharts/sk000
helm repo update
helm search repo nginxrepo/sk000-myfirst-helm --versions
NAME                                        CHART VERSION         APP VERSION           DESCRIPTION
nginxrepo/sk000-myfirst-helmchart   0.0.2                 1.16.0      A Helm chart for Kubernetes
nginxrepo/sk000-myfirst-helmchart   0.0.1                 1.16.0      A Helm chart for Kubernetes

helm upgrade --install sk000-myfirst-helm nginxrepo/sk000-myfirst-helmchart  \
             --version 0.0.1  \
             --namespace  ${CLASS_NAME} \
             --atomic \
             --timeout 3m
```

3. https://sk000-ingress.class-0.skala-??.com 접속하기

[참고] 이것을 실행하게되면 version 0.0.2 helm chart로 실행

```
> cat 05.install-fileserver.sh

> ./05.install-fileserver.sh
```

Umbrella Chart

위치: 03.helm-umbrella

여러 개의 하위 차트(subchart)를 하나의 상위 차트가 감싸서 함께 배포·관리하는 구조를 의미

```
umbrella-chart/          ← Umbrella Chart (부모)
├── Chart.yaml
├── values.yaml
└── charts/
    ├── springboot/      ← Sub-chart (자식)
    ├── vuejs/           ← Sub-chart (자식)
    ├── python/          ← Sub-chart (자식)
    └── mariadb-x.x.tgz ← Dependency Chart (외부 의존성)
```

용어	설명
Umbrella Chart	여러 서브차트를 하나로 묶어 통합 배포하는 부모 차트
Sub-chart	charts/ 디렉토리 안에 직접 작성된 자식 차트
Dependency Chart	Chart.yaml의 dependencies:에 선언해 외부에서 다운로드하는 차트 (예: Bitnami MariaDB)

샘플코드 03.helm-umbrella를 검토해보자

Chart.yaml

```yaml
apiVersion: v2
name: myfirst-helm
description: Spring Boot API, Vue.js Frontend, Python App, Ma
type: application
version: 0.1.0
appVersion: "1.0.0"

dependencies:
  - name: mariadb
    version: "20.5.5"
    repository: https://charts.bitnami.com/bitnami
    condition: mariadb.enabled
```

global 사용방법

위치: 03.helm-umbrella

global 은 Helm이 모든 subchart에서 공통으로 볼 수 있게 열어주는 특별한 values 영역
하지만 **자동으로 image.repository 앞에 붙여준다 같은 동작은 하지 않음**

01.umbrella-chart/values.yaml

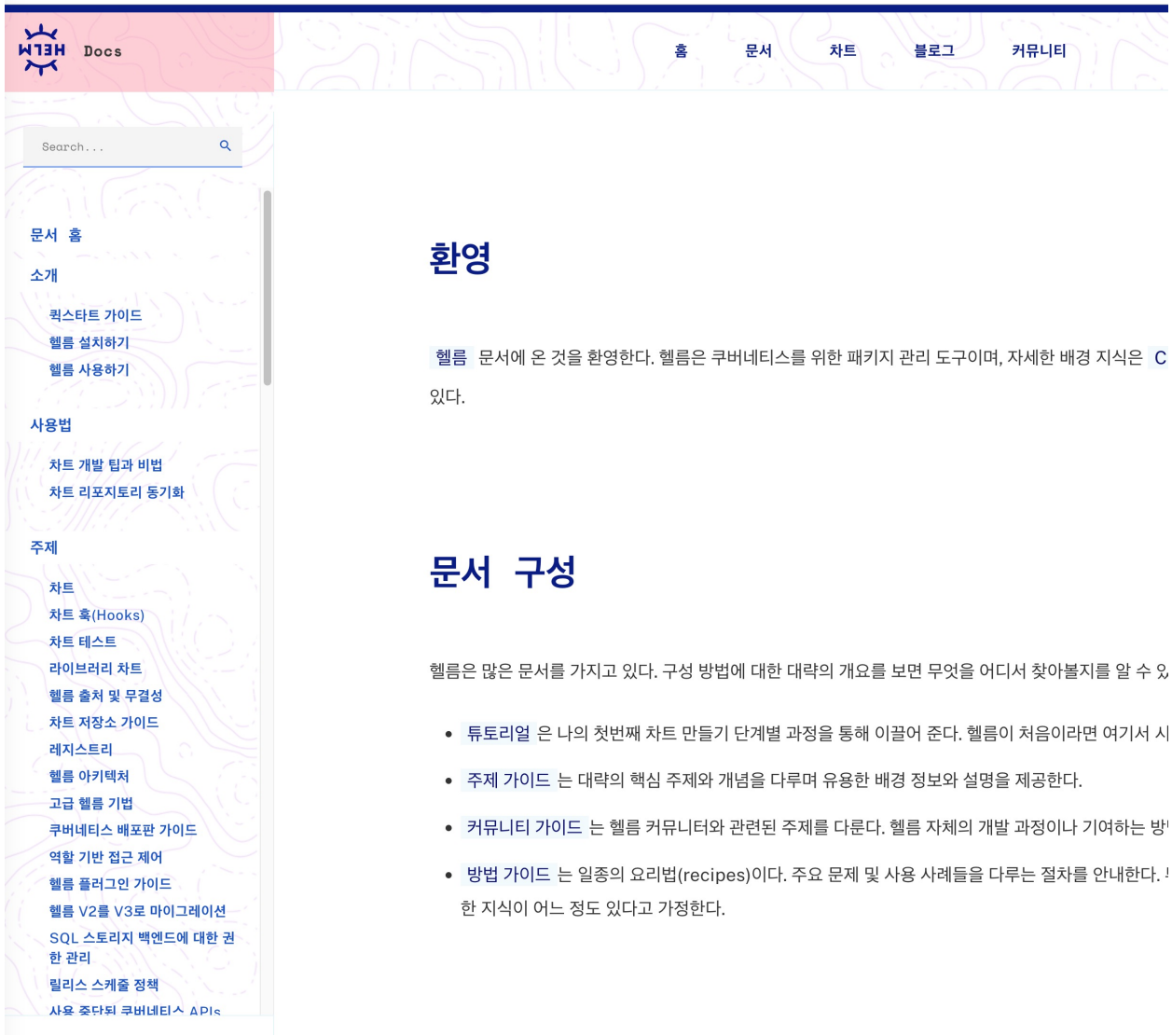
```yaml
global:
  username: "sk000"
  namespace: "class-0"
  registry: skala-registry.skala-??/class-0

backend:
  image:
    repository: backend
    tag: "1.0.0"
```

하위 템플릿 설정 방법 (예. backend/templates/deployment.yaml)

```
image: "{{ .Values.global.registry }}/{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

[참고] Helm 참고 사이트

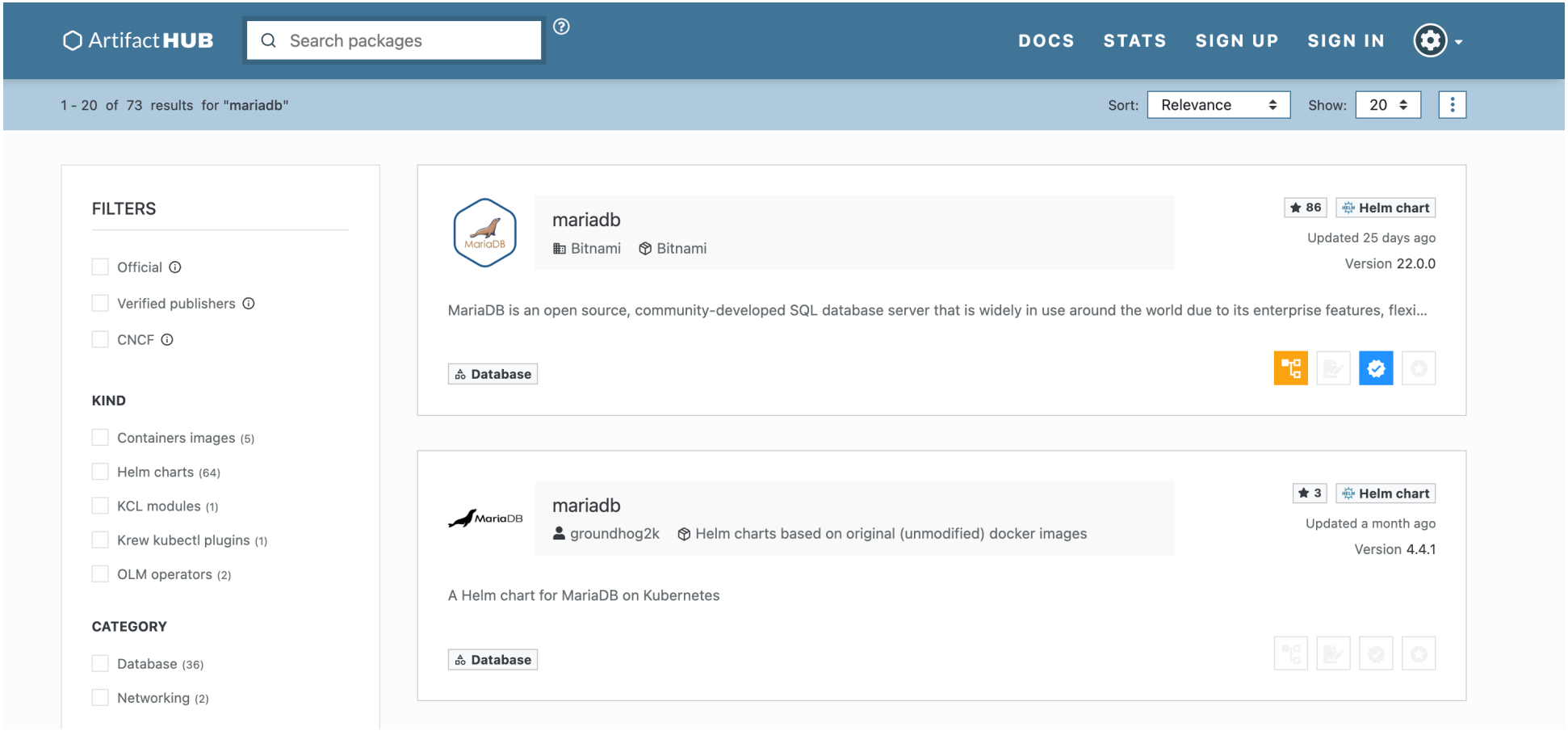
https://helm.sh/ko/docs



Artifact HUB

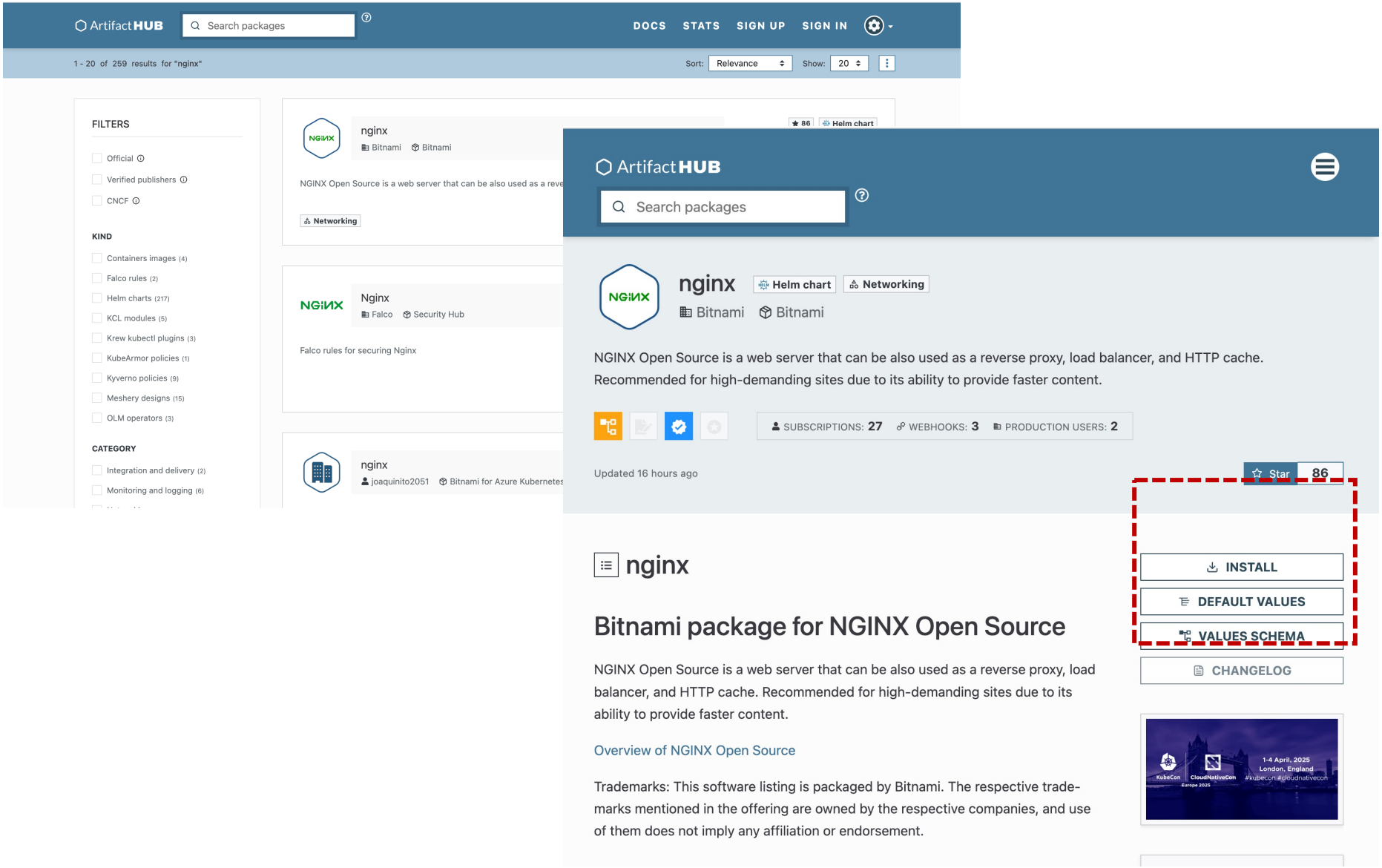
자기가 만든 템플릿을 공유할 수 있도록 저장소 운영. 많은 오픈소스 솔루션들이 artifacthub에 등록 관리

https://artifacthub.io/



Artifact HUB

Docker hub와 유사



KDT DevOps – DevOps

helmchart 기본 문법

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Helm Template 의 .과 $의 의미

Helm Chart는 Go Template 문법을 사용하며, 템플릿에서 .과 $는 데이터를 참조하는 **기준 위치(Context)**를 나타냄

- **. :** 현재 Context를 의미
- **$:** 템플릿 실행이 시작될 때의 최상위 Context(Root Context)를 의미

표현	의미
.	현재 Context (변경 가능)
.Values.name	현재 Context의 Values.name
.name	현재 Context의 name 필드 참조
$	최상위 Context (Root Context)
$.Values.name	최상위 Context의 Values.name 참조

values.yaml

```
users:
  - name: alice
  - name: bob
global:
  name: hey
```

templates/configmap.yaml (일부)

```
{{- range .Values.users }}
- name: {{ .name }}
  global: {{ $.Values.global.name }}
{{- end }}
```

range: forEach 와 비슷하게 동작

Helm Template 의 공백과 줄 바꿈 제어 ({{- , -}})

- 템플릿을 렌더링할 때 공백(Space)과 줄바꿈(Newline)도 결과에 반영
- 이때 {{-와 -}}를 사용하면 템플릿 실행 과정에서 불필요한 공백과 줄바꿈을 제거
- 특히 YAML은 들여쓰기에 민감하므로, Helm Template에서는 공백 및 줄바꿈 제어가 중요

예시 **공백제거 없음** **결과**

```
start\n
{{  if  true  }} \n
line\n
{{ end }} \n
end\n
```

```
start

line

end
```

왼쪽 공백 제거

```
start\n
{{-  if  true  }} \n
line\n
{{- end }} \n
end\n
```

```
start
line
end
```

양쪽 공백 제거

```
start\n
{{-  if  true  -}} \n
line\n
{{ end }} \n
end\n
```

```
startline

end
```

오른쪽 공백 제거

```
start\n
{{-  if  true  -}} \n
line\n
{{- end }} \n
end\n
```

```
startline
end
```

- {{ ... }} | 공백 제거 없음 — 그대로 출력
- {{- ... }} | 왼쪽(앞) 공백·줄바꿈 제거
- {{ ... -}} | 오른쪽(뒤) 공백·줄바꿈 제거
- {{- ... -}} | 양쪽 공백·줄바꿈 모두 제거

작성시 잘못 설정하는 경우?

```
metadata:
  name: {{- .Values.name -}}
  namespace: {{ .Values.namespace }}
```

결과는?

[실습] Helm Template 을 테스트해보자

위치: 03.helm/template-test

helm 실행을 테스트해보자

1. 03.helm/template-test/values.yaml

```
test:
  enabled: true
replicaCount: 1
autoscaling:
     enabled: false
```

2. 03.helm/ template-test /templates 아래에 test1.yaml을 작성해보자

```
metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
{{- if .Values.test.enabled }}
labels:
  {{- include "mychart.labels" . | nindent 2 }}
{{- end }}
```

3. 실행하기

```
cd 03.helm/template-test

helm  template . --debug
helm  template . --debug --show-only  templates/test1.yaml
```

[실습] Helm Template : 조건문

위치: 03.helm/template-test

If / else / end - 조건문

```
# values.yaml
service:
  enabled: true
adminUser: admin
```

templates/if-else.yaml

```
# template
{{- if .Values.service.enabled }}
apiVersion: v1
kind: Service
{{- else }}
# Service disabled
{{- end }}
```

실행하기

```
cd 03.helm/template-test

helm  template  .  --debug
helm  template  .  --debug --show-only  templates/if-else.yaml
```

[실습] Helm Template : default & required

위치: 03.helm/template-test

default / required

default — 기본값 지정, **required** — 필수값 강제

default-required.yaml

```
# port가 values.yaml에 없으면 80을 기본값으로 사용
containerPort: {{ default 80 .Values.port }}

# adminUser가 비어있으면 에러 발생 후 배포 중단
admin: {{ required "adminUser is required!" .Values.adminUser }}
```

실행하기

```
cd 03.helm/template-test

helm  template  .  --debug
helm  template  .  --debug --show-only  templates/default-required.yaml
```

Helm Template의 Pipeline과 주요 함수

Pipeline은 앞에서 처리한 결과를 다음 함수의 인자로 전달하여 연속적으로 처리하는 문법

```
{{ .Values.키_이름 | 함수1 | 함수2 }}
```

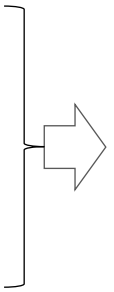
values.yaml

```
name: my-app
```

Template

대문자 큰따옴표

```
app: {{ .Values.name | upper | quote }}
```



Rendering 후

```
app: "MY-APP"
```

Helm Template의 Pipeline과 주요 함수

Values에 정의한 Map, List 같은 복합 데이터를 YAML 형식으로 변환할 때 toYaml을 사용

```
{{   toYaml   .Values.xxx   |   indent   N   }}
```

변환된 YAML의 각 줄 앞에 지정한 개수만큼 공백을 추가한다.

함수	의미	예시
toYaml	Map, List 등의 데이터를 YAML 문자열로 변환	toYaml .Values.config
indent N	모든 줄 앞에 N개의 공백 추가	indent 4
nindent N	맨 앞에 줄바꿈을 추가하고 N칸 들여쓰기	nindent 4

values.yaml

```
config:
  setting1: "on"
  setting2: "off"
```

Template

```
data:
  config.yaml: |
{{ toYaml .Values.config | indent 4 }}
```

Rendering 후

```
data:
  config.yaml: |
    setting1: "on"
    setting2: "off"
```

[실습] pipeline & toYaml 적용하기

위치: 03.helm/template-test

template-test 에서 pipeline.yaml을 검토하고 실행해보자

pipeline.yaml

```
app: {{ .Values.name | upper | quote }}
data:
  config.yaml: |
{{ toYaml .Values.config | indent 4 }}
```

실행하기

```
cd 03.helm/template-test

helm  template  .  --debug
helm  template  .  --debug --show-only  templates/pipeline.yaml
```

[참고] 파이프라인 함수

Go 템플릿의 내장 함수

카테고리	함수	사용법	설명	예시
문자열 처리	trunc	trunc 63	지정된 길이로 문자열 자르기	"hello-world" | trunc 5 → "hello"
	trimSuffix	trimSuffix "-"	끝에서 특정 문자열 제거	"app-name-" | trimSuffix "-" → "app-name"
	trimPrefix	trimPrefix "prefix-"	시작에서 특정 문자열 제거	"prefix-app" | trimPrefix "prefix-" → "app"
	upper	upper	대문자로 변환	"hello" | upper → "HELLO"
	lower	lower	소문자로 변환	"Hello" | lower → "hello"
	title	title	각 단어의 첫글자 대문자	"hello world" | title → "Hello World"
	quote	quote	따옴표로 감싸기	hello | quote → "hello"
	replace	replace "old" "new"	문자열 교체	"hello-world" | replace "-" "_" → "hello_world"
들여쓰기	indent	indent 4	지정된 칸수만큼 들여쓰기	"text" | indent 2 → " text"
	nindent	nindent 4	줄바꿈 + 들여쓰기	"text" | nindent 2 → "\n text"
조건/기본값	default	default "기본값"	값이 없으면 기본값 사용	.Values.name | default "myapp"
	required	required "에러메시지"	필수값 체크, 없으면 에러	.Values.image | required "image is required"
포매팅	printf	printf "%s-%s"	문자열 포맷팅	printf "%s-%s" .Release.Name .Chart.Name
배열/리스트	join	join "-"	배열을 구분자로 연결	list "a" "b" "c" | join "-" → "a-b-c"

```
# 여러 함수를 연결해서 사용
name: {{ .Release.Name ¦ lower ¦ trunc 50 ¦ trimSuffix "-" }}

# 라벨에서 자주 사용되는 패턴
labels:
  {{- include "myapp.labels" . ¦ nindent 4 }}
```

Helm Template의 with 구문

지정한 값이 비어 있지 않으면 블록을 실행하고, 해당 객체를 현재 Context(.)로 설정하는 구문

```
{{- with .Values.객체명 }}
  {{ .필드명 }}
{{- end }}
```

values.yaml (값이 있을때)

```
resources:
  limits:
    cpu: 100m
    memory: 128Mi
```

Template

```
resources:
  {{- with .Values.resources }}
  limits:
    cpu: {{ .limits.cpu }}
    memory: {{ .limits.memory }}
  {{- end }}
```

Rendering 후

```
resources:
  limits:
    cpu: 100m
    memory: 128Mi
```

values.yaml (값이 없을때)

```
resources: {}
```

Rendering 후

```
resources:
```

Helm Template의 with와 else 구문

with는 else를 사용하여 값이 없을 때의 동작을 정의할 수 있음

values.yaml (Values.resources가 있을때)

```
resources:
  limits:
    cpu: 100m
    memory: 128Mi
```

Template

```
resources:
  {{- with .Values.resources }}
  limits:
    cpu: {{ .limits.cpu }}
    memory: {{ .limits.memory }}
  {{- else }}
  limits:
    cpu: 50m
    memory: 64Mi
  {{- end }}
```

values.yaml (Values.resources가 없을때)

```
resources: {}
```

Rendering 후

```
resources:
  limits:
    cpu: 100m
    memory: 128Mi
```

Rendering 후

```
resources:
  limits:
    cpu: 50m
    memory: 64Mi
```

Helm Template의 Range 구문

List, Array, Map 등의 요소를 하나씩 순회하면서 반복적으로 처리

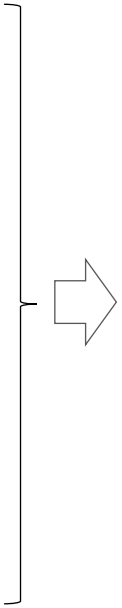
```
{{- range .Values.목록명 }}
  {{ .필드명 }}
{{- end }}
```

values.yaml

```
envs:
  - name: ENV1
    value: "value1"
  - name: ENV2
    value: "value2"
```

Template

```
env:
{{- range .Values.envs }}
  - key: {{ .name }}
    secret: {{ .value }}
{{- end }}
```



Rendering 후

```
env:
  - key: ENV1
    secret: value1
  - key: ENV2
    secret: value2
```

[실습] with else 적용하기

위치: 03.helm/template-test

with else 동작을 확인해보자

with-else.yaml

```
resources:
  {{- with .Values.resources }}
  limits:
    cpu: {{ .limits.cpu }}
    memory: {{ .limits.memory }}
  {{- else }}
  limits:
    cpu: 50m
    memory: 64Mi
  {{- end }}
```

실행하기

```
cd 03.helm/template-test

helm  template  .  --debug
helm  template  .  --debug --show-only  templates/with-else.yaml
```

[실습] range 적용하기

위치: 03.helm/template-test

range 동작을 확인해보자

range.yaml

```
env:
{{- range .Values.envs }}
  - key: {{ .name }}
    secret: {{ .value }}
{{- end }}
```

실행하기

```
cd 03.helm/template-test

helm  template  .  --debug
helm  template  .  --debug --show-only  templates/range.yaml
```

재사용 템플릿 함수: _helpers.tpl

여러 YAML 파일에서 반복적으로 사용하는 템플릿 코드를 _helpers.tpl 파일에 정의하고 재사용할 수 있음

- **define** : 재사용할 템플릿에 이름을 부여하여 정의
- **include** : 정의한 템플릿을 호출하고 그 결과를 현재 위치에 삽입

define (_helpers.tpl)

```
{{- define "템플릿이름" -}}
  템플릿 내용
{{- end -}}
```

include (templates)

```
{{ include "템플릿이름" . }}
```

예시)

_helpers.tpl

```
{{- define "mychart.fullname" -}}
{{- .Chart.Name -}}
{{- end -}}
```

Chart.yaml

```
apiVersion: v2
name: mychart
version: 1.0.0
```

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "mychart.fullname" . }}
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mychart
```

[실습] 재사용 템플릿 함수: _helpers.tpl

위치: 03.helm/template-test

_helpers.tpl 내용과 include.yaml을 확인해보자

 # _helpers.tpl

```
{{- define "mychart.fullname" -}}
{{- .Chart.Name -}}
{{- end -}}
```

 # deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "mychart.fullname" . }}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: {{ include "mychart.fullname" . }}
```

실행 결과

```
> helm template .  --show-only templates/include.yaml
# Source: template-test/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: template-test
```

KDT DevOps – DevOps

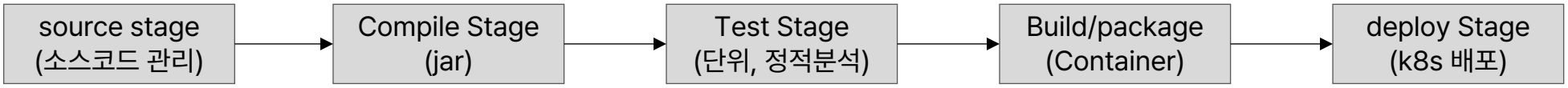
Jenkins를 이용한 CI/CD 구성

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

CICD Pipeline 이란?

소스 코드의 변경 사항이 빌드, 테스트를 거쳐 최종 운영 환경에 배포되기까지의 모든 과정을 자동화된 일련의 흐름으로 정의한 것



github, gitlab,
bitbucket
gitea

interpreter는 skip

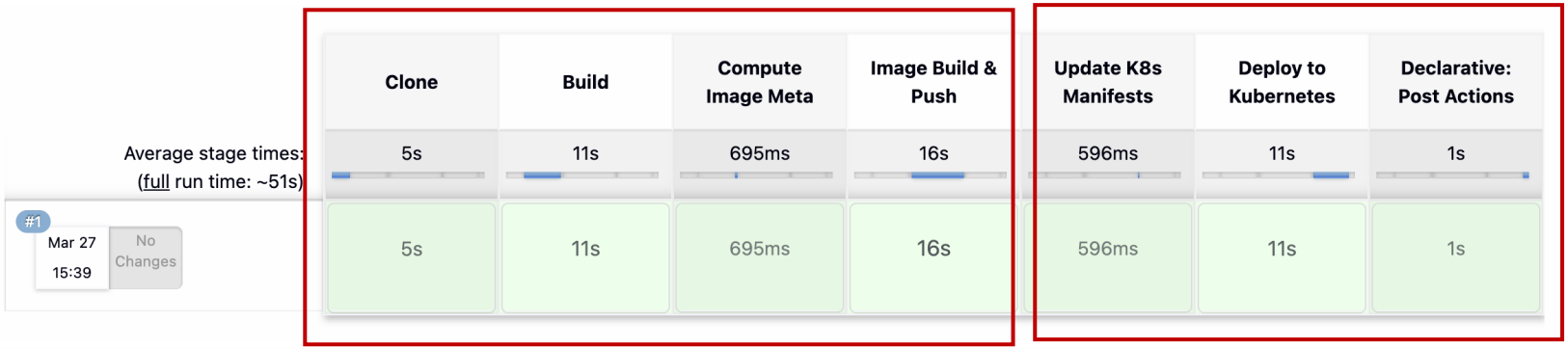
Junit, Sonaqube

docker, buildah,
kaniko

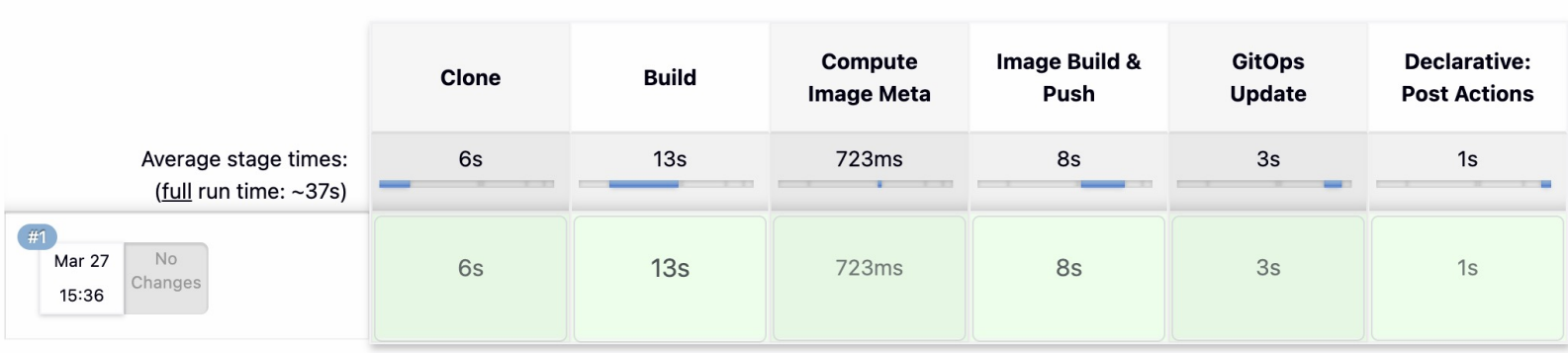
gitOps
ArgoCD, Flux

https://www.cncf.io/projects/flux/

CI/CD



CI



https://sk199-jenkins.skala25a.project.skala-ai.com/

Jenkins란?

소프트웨어 개발 과정에서 빌드(Build), 테스트(Test), 배포(Deploy) 등의 반복적인 작업을 자동화하기 위한 오픈소스 도구
CI/CD(Continuous Integration/Continuous deploy)를 지원하여 개발자가 자주 코드를 통합하고, 신속하게 배포하도록 지원

Jenkins의 주요 특징

플랫폼 독립성

- Java 기반 → Linux, Windows, macOS 어디서든 실행 가능
- 다양한 컨테이너 플랫폼(Docker, Podman, Kaniko etc)으로 컨테이너를 손쉽게 빌드/배포 가능

플러그인 확장성

- 1,800개 이상의 플러그인 제공 (Git, Maven, Docker, Kubernetes, Slack, SonarQube 등)
- 필요한 기능을 조합하여 맞춤형 CI/CD 파이프라인 구성 가능

Pipeline as Code

- Jenkinsfile(Groovy 기반, Domain Specific Language)을 사용해 빌드/배포 과정을 코드로 정의
- 버전 관리(Git)와 함께 운영 → Infrastructure as Code 방식

분산 빌드

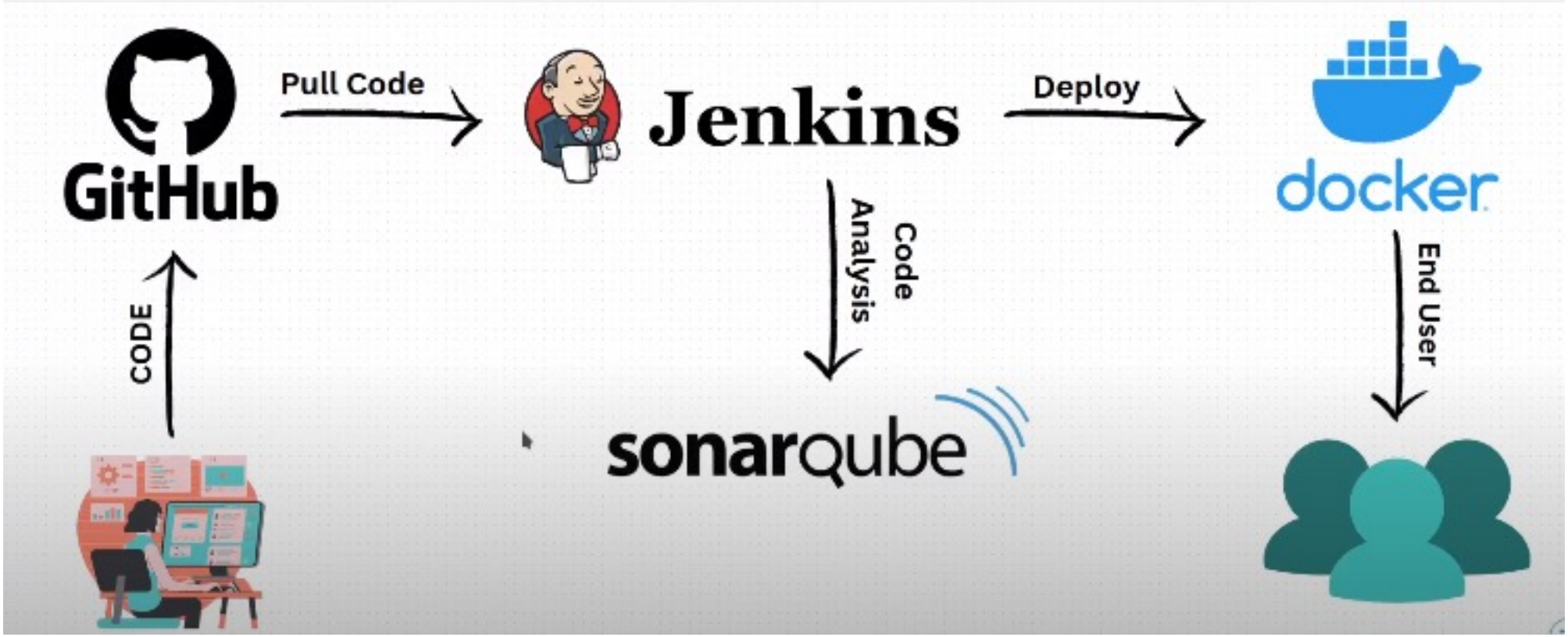
- Master/Controller + Agent 구조
- 빌드 작업을 여러 노드에서 병렬 처리 → 빠른 속도 & 확장성

광범위한 커뮤니티와 레퍼런스

- 전 세계적으로 가장 많이 쓰이는 CI/CD 도구
- 풍부한 문서, 블로그, Q&A 지원

Jenkins 흐름

나의 코드가 github에 push 되면 Jenkins는 자동 또는 수동으로 컴파일, 이미지 빌드, 릴리즈 실행



JCasC (Jenkins Configuration as Code)

Jenkins의 전체 시스템 설정 (보안, 사용자, 플러그인 설정, 전역 인증 정보 등)을 UI 가 아닌
단일 YAML (jenkins.yaml)로 정의하고 버전 관리할 수 있도록 지원하는 방식

- **다중 사용자 환경의 자동화된 권한 분리**:
  admin 외에 4명(skala01~skala04)의 사용자를 사전 생성하고,
  Job 생성 시 서로의 작업 공간을 침범하지 못하도록 Role-Based Matrix 규칙(folder-owner-*)을 자동 적용

- **작업 디렉터리(Folder) 자동 생성**:
  jobs 섹션의 Job DSL 스크립트를 통해 서버 실행 즉시 각 사용자의 독립 폴더(skala01~skala04)를 생성

- **공용 Credential 주입**:
  GitHub PAT(skala-github-id)와 Harbor 레지스트리 토큰(skala-image-registry-id)을 시작 시점에
  미리 등록해 사용자가 별도로 인증서를 등록하지 않고 즉시 파이프라인에서 호출할 수 있게 지원

- **서버 재현성(IaC)**:
  Jenkins 컨테이너가 삭제, 재배포되어도 YAML 파일 하나만 있으면 즉시 동일한 멀티유저 실습 환경으로 복원

JCasC (Jenkins Configuration as Code)

jeknis.yaml 내용 확인

04.jenkins/jenkins.yaml

```
jenkins:
  systemMessage: "SKALA DevOps Study - Jenkins based CI/CD"
  numExecutors: 2
  mode: NORMAL
```

- **systemMessage**: 대시보드 상단 안내문.
- **numExecutors =** 2: 컨트롤러(master) 노드에서 동시에 2개 작업 실행. 교육/소규모 OK.
  실전에서는 보통 0으로 두고 에이전트에서만 빌드 권장.
- **mode=**NORMAL: 컨트롤러에서도 작업 수행 가능(**EXCLUSIVE**는 Controller는 관리/조율만 하도록 지시)

```
 securityRealm:
    local:
      allowsSignup: false
      users:
      - id: "admin"
        password: "admin”
      - id: "skala01"
        password: "skala"
...
```

- **allowsSignup**: false는 관리자만이 계정 생성 지원
- **admin/admin**: 계정 및 패스워드

JCasC (Jenkins Configuration as Code)

jeknis.yaml 내용 확인

04.jenkins/jenkins.yaml

```
credentials:
  system:
    domainCredentials:
      - credentials:
          # ── GitHub PAT (username/password) ──────────────────────────────
          # Jenkinsfile: credentialsId: 'skala-github-id'
          - usernamePassword:
              scope: GLOBAL
              id: "skala-github-id"
              description: "GitHub Personal Access Token"
              username: "자신의 git 계정 이름"   # ← 실제 GitHub 사용자명으로 교체
              password: "자신의 myfirst-api-server Git PAT token"   # ← 실제 PAT로 교체

          # ── Harbor Registry (username/password) ─────────────────────────
          # Jenkinsfile: credentialsId: 'skala-image-registry-id'
          - usernamePassword:
              scope: GLOBAL
              id: "skala-image-registry-id"
              description: "Harbor Registry Credential"
              username: "robot$skala26b"
              password: "skala-registry 접속 API KEY"   # ← 실제 비밀번호로 교체
```

[참고] Jenkins vs Tekton

JCasC 이전의 Jenkins 설정은 **Snowflake Server(눈송이 서버)**를 만들기 딱 좋은 환경
똑같은 설정을 가진 서버를 다시 만들기 어려운 수준

비교 항목	Jenkins (with JCasC/Agent)	Tekton (Cloud Native CI)
아키텍처	**Master-Agent (Controller-Worker)**. 컨트롤러가 항상 떠 있어야 함.	**Serverless / CRD 기반** K8s Operator 기반으로 선언적 k8s 리소스 방식 지원
호출 방식	명령형 방식	선언적 방식
K8s 최적화	**플러그인**을 통한 지원 (Add-on 방식)	K8s 자체 리소스 사용 (Native 방식)
확장성	**Dynamic Agent (Pod)**로 확장 가능하지만, 컨트롤러 가용성 관리가 필요	**완전 무상태(Stateless)**. 리소스가 필요할 때만 생성되어 비용 효율적임.
학습 곡선	상대적으로 낮음 (GUI 및 풍부한 레퍼런스)	높음 (YAML 구조가 복잡하고 K8s 지식 필수)
설정 관리	**JCasC** (YAML) + **Groovy** (Shared Library)	**YAML** (Task, Pipeline 리소스 정의)
에코시스템	1,800개 이상의 방대한 플러그인	Tekton Hub를 통한 재사용 가능한 Task 제공
리소스 격리	Pod 단위 격리 가능하나, 설정이 복잡함.	기본적으로 모든 Task가 컨테이너로 격리됨.
장점	Legacy 부터 Cloud Native까지 모두 가능	표준적이고 가볍우며, K8S 환경에 최적화되어 있음
단점	Tekton 대비 무겁다	진입 장벽 있음

[실습] Jenkins 로컬 실행하기

로컬 환경에서 Jenkins를 Container로 실행해보자

04.jenkins/

1. Dockerfile 확인

skala-registry.skala-??/library/skala-custom-jenkins:1.0
를 생성하기 위한 Dockerfile

2. local에서 jenkins를 실행해보자

```
> ./local-run.sh
```

[실습 1] Jenkins 접속하기

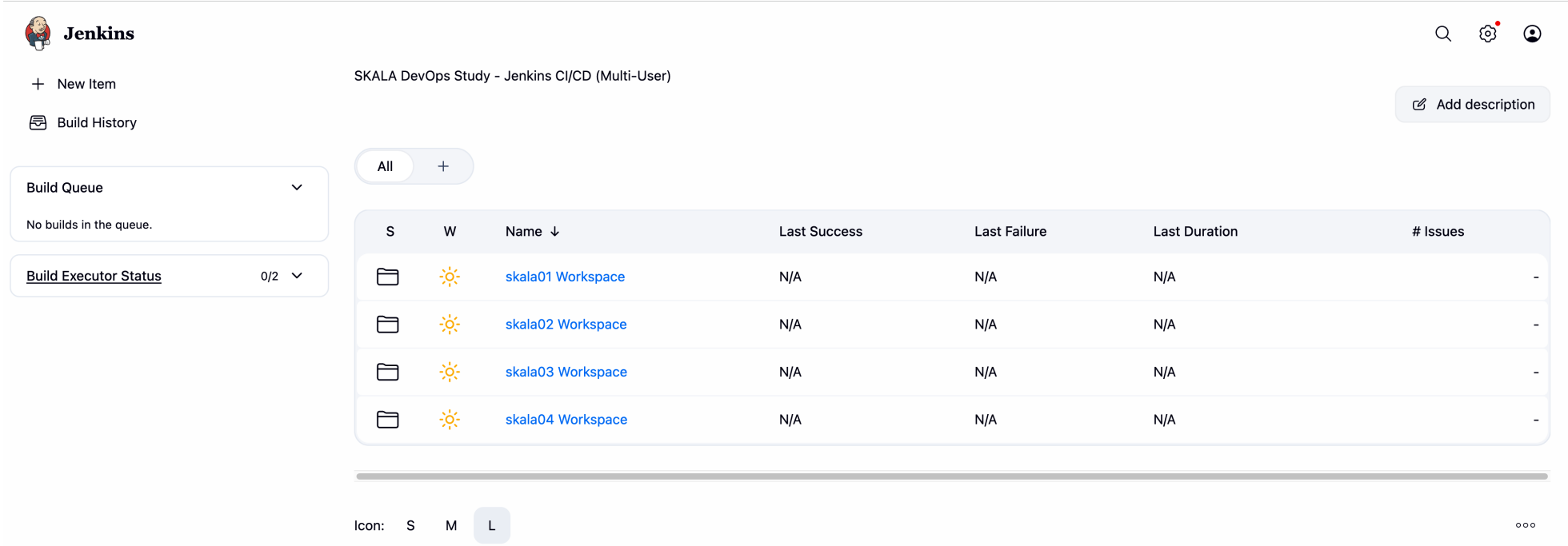
4. Jenkins 접속하기

Browser 를 통해 아래 링크로 접속

http://localhost:8888

jd: admin

pwd: admin



[참고] 사전 설치된 Plug-in

각 Stage는 아래와 같다.

Plugin 이름	주요 기능	사용 여부
workflow-aggregator	Jenkins Pipeline을 실행하기 위한 핵심 플러그인 묶음 (Declarative 및 Scripted Pipeline 지원)	전체 파이프라인 실행
git	Git 저장소에서 코드 clone	stage('Clone Repository')
git-client	Git CLI 명령어 실행에 필요한 클라이언트 기능 지원	git 플러그인 보조 역할
github-branch-source	GitHub 저장소를 멀티브랜치 또는 Organization 프로젝트로 인식	GitHub 연동 자동화에 유용 (Pipeline 자체에는 직접 사용 안되지만 멀티브랜치 빌드 시 중요)
docker-workflow	Docker 명령어 사용 (docker.build, docker.push)을 파이프라인에서 지원	stage('Docker Build & Push')
pipeline-stage-view	Jenkins UI에서 각 스테이지별 진행 상황 시각화	UI 시각화
blueocean	Jenkins용 현대적인 UI 플러그인 (시각적 파이프라인 뷰)	UI 개선
credentials-binding	Jenkins에서 보안 크리덴셜을 환경 변수처럼 바인딩해서 사용 가능	credentialsId로 PAT, Docker 인증정보 사용
timestamper	빌드 로그에 타임스탬프 추가	디버깅용 로그 개선
job-dsl	Jenkins job을 DSL로 정의 (코드로 Job 정의 가능)	템플릿 관리에 유용 – 이번 파이프라인에는 직접 사용 안됨
configuration-as-code	Jenkins 전체 설정을 YAML로 정의하고 관리	Jenkins 설정 자동화 (plugin 설치, system config 등)
kubernetes	Kubernetes Plugin Jenkins 에이전트를 Kubernetes Pod로 실행	Jenkins 에이전트가 K8s 위에서 실행된다면 필수
maven-plugin	Maven 프로젝트 자동 인식 및 Maven 설치 관리	Maven 명령어 수동 실행과 직접적 연관은 적지만 Maven 환경 세팅에 유용
junit	junit 테스트 결과 리포트 시각화	mvn test를 사용할 때 유용 (-DskipTests로 생략했으므로 현재는 미사용)
htmlpublisher	HTML 리포트 (예: Javadoc, Sonar, Test Report 등) 게시	현재 파이프라인에서는 사용되지 않음

Jenkinsfile 이란

Jenkinsfile은 **Jenkins Pipeline(자동화 빌드/배포 과정)**을 코드로 정의한 스크립트 파일

보통 프로젝트의 루트 디렉토리에 Jenkinsfile 이름으로 저장

```
pipeline {
    agent any
    environment {
        NODE_ENV = 'prod'
    }
    options {
        timeout(time: 10, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '20'))
    }

    stages {
        stage('Test') {
            steps {
                sh 'npm test'
                sh 'npm run lint'
            }
        }
    }

    post {
        success { echo '성공' }
        failure { echo '실패' }
        always {
            cleanWs()
        }
    }
}
```

pipeline

- Jenkins Declarative 파이프라인의 최상위 블록
- 파이프라인 전체 구조를 정의하는 시작점

agent any

Jenkins에서 실행할 노드(에이전트) 지정

- any : 모든 노드에서 실행 가능
- 특정 라벨이나 Docker 컨테이너도 지정 가능

stages

- 여러 단계(stage)를 묶는 블록
- 예: Build, Test, Deploy

stage('이름')

- 파이프라인의 개별 단계
- UI에 단계별로 표시되어 진행 상황 확인 가능
- stage안에는 agent, steps, environment, when, options, input 등이 들어갈 수 있음

steps

- stage 안에서 실제로 실행되는 작업 모음
- 예: sh, echo, checkout scm, archiveArtifacts

Jenkinsfile 주요 블록 : options

pipeline 블록이나 stage 블록 안에서 사용할 수 있는 부가 설정 블록

options {...}

파이프라인 또는 특정 stage의 실행 방식을 제어하는 옵션 설정

```
pipeline {
  agent any
  options {
    retry(2)                                              // 내부 실패 시 재시도 횟수
    timeout(time: 10, unit: 'MINUTES')                    // 전체 파이프라인 10분 제한
    buildDiscarder(logRotator(numToKeepStr: '5'))         // 최근 5개 빌드 이력만 보관
    disableConcurrentBuilds()                             // 동시에 두 번 실행 못 하게
    timestamps()                                          // 각 콜솔 로그 출력 라인 앞에 타임스탬프를 자동으로 추가
  }
  stages { ... }
}
```

- timeout : 실행 제한 시간
- retry(n) : 실패 시 n번까지 재시도
- buildDiscarder : 오래된 빌드 로그/아티팩트 삭제
- disableConcurrentBuilds() : 같은 Job이 동시에 실행되지 않도록 제한
- timestamps(): 콘솔 로그 출력 라인앞에 타임스탬프를 자동으로 추가

Jenkinsfile 주요 블록: environment

pipeline 블록이나 stage 블록 안에서 사용할 수 있는 부가 설정 블록

environment {...}

pipeline 또는 stage에서 사용할 환경 변수(Environment Variable)를 정의

```
pipeline {
  agent any

  environment {
    GLOBAL_ENV = 'hello'         // 전체 파이프라인에서 사용 가능
  }

  stages {
    stage('Build') {
      environment {
        BUILD_ENV = 'build-only' // 이 stage 안에서만 유효
      }
      steps {
        echo "전역 변수: ${env.GLOBAL_ENV}"
        echo "빌드 변수: ${env.BUILD_ENV}"
      }
```

Scope

- pipeline 최상위에 선언 → 전체 파이프라인에서 사용 가능
- stage 안에 선언 → 해당 stage에서만 유효

Jenkinsfile 주요 블록: 환경 변수 제공 방식

A. env.변수명 및 직접 참조 방식

```
pipeline {
    agent any

    environment {
        APP_NAME = 'my-api'
    }

    stages {
        stage('Example') {
            steps {
                // 1) ${env.변수명} 형태
                echo "env 객체 참조: ${env.APP_NAME}"

                // 2) ${변수명} 직접 참조
                echo "직접 변수 참조: ${APP_NAME}"

                // 3) sh 내부에서 셸 변수로 직접 접근 (작은따옴표 권장)
                sh 'echo "셸 환경변수: ${APP_NAME}"'
            }
        }
    }
}
```

Jenkinsfile 주요 블록: 환경 변수 제공 방식

A. env.변수명 및 직접 참조 방식

```
pipeline {
    agent any

    environment {
        APP_NAME = 'my-api'
    }

    stages {
        stage('Example') {
            steps {
                // 1) ${env.변수명} 형태
                echo "env 객체 참조: ${env.APP_NAME}"

                // 2) ${변수명} 직접 참조
                echo "직접 변수 참조: ${APP_NAME}"

                // 3) sh 내부에서 셸 변수로 직접 접근 (작은따옴표 권장)
                sh 'echo "셸 환경변수: ${APP_NAME}"'
            }
        }
    }
}
```

Jenkinsfile 주요 블록: 환경 변수 제공 방식

A. 런타임에 동적으로 환경 변수 만들기

```
pipeline {
    agent any

    stages {
        stage('Generate Version') {
            steps {
                script {
                    // 빌드 번호를 조합해 고유 버전 생성 (예: v1.0.42)
                    APP_VERSION = "v1.0.${BUILD_NUMBER}"
                }
                echo "생성된 버전: ${env.APP_VERSION}"
            }
        }

        stage('Deploy') {
            steps {
                // 이전 stage에서 만들어진 env 변수를 그대로 사용
                echo "이전 stage에서 전달받은 버전: ${env.APP_VERSION}"
                sh "echo Deploying version: ${env.APP_VERSION}"
                sh 'echo Deploying version: ${APP_VERSION}'
            }
        }
    }
}
```

env.APP_VERSION과
APP_VERSION의 차이 ????

[참고] “ ”와 ‘ ‘ 의 Jenkins 내부 해석 차이

“” 와 ‘’의 차이

구문 형태	변수 동작 여부	해석 주체 및 특징
`sh 'echo ${APP_NAME}'`	O	**Linux Shell**이 변수를 해석
`sh "echo ${APP_NAME}"`	O	Jenkins(Groovy)가 먼저 값을 치환 후 Shell 전달
`echo "${APP_NAME}"`	O	Jenkins Step의 표준 참조 방식
`echo '$APP_NAME'`	X	치환되지 않고 문자 그대로 $APP_NAME 출력
`echo $APP_NAME`	X	Groovy 문법 에러 발생

Jenkinsfile 주요 블록: post

pipeline 블록이나 stage 블록 안에서 사용할 수 있는 부가 설정 블록

post {...}

파이프라인이나 stage 실행 후 결과에 따라 실행할 동작 정의

```
pipeline {
  agent any
  stages {…}
  post {
    always { echo '항상 실행됨' }
    success { echo '성공했을 때 실행' }
    failure { echo '실패했을 때 실행' }
    aborted { echo '사용자가 중단했을 때 실행' }
  }
}
```

활용 예시

- 빌드 성공 시 슬랙/메일 알림
- 실패 시 로그/아티팩트 보관
- 항상 실행하는 cleanup 작업

Jenkins DSL (Domain-Specific Language)

Jenkinsfile 안에서 파이프라인을 구성하기 위해 사용하는 도메인 전용 언어(DSL)

Jenkins Pipeline 문법에서 제공하는 파이프라인용 명령 스타일의 요소를 의미

일반 Groovy 문법이라기보다 **Jenkins가 Pipeline 작성을 쉽게 하려고 제공한 구조적 문법**

- pipeline
- agent
- stages
- stage
- steps
- post
- environment
- when
- parameters

특징

- 파이프라인의 구조를 선언적으로 표현하기 쉽다.
- 사람이 읽기 쉽다.
- Jenkins가 검증하기 쉽다.
- UI와 연결되기 쉽다. 예: stage 표시

```
pipeline {
    agent any
    environment {
        APP_NAME = 'demo'
    }
    stages {
        stage('Test') {
            when {
                branch 'main'
            }
            steps {
                echo "Testing ${APP_NAME}"
            }
        }
    }
}
```

Jenkins Step

Step = Jenkins Pipeline에서 실제 작업을 수행하는 가장 작은 실행 명령 단위

```
Pipeline
 ├─ Stage
 └─ Stage
     └─ Steps
         ├─ Step
         ├─ Step
         └─ Step
```

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
                echo 'Build Complete'
            }
        }
    }
}
```

Jenkins Step 함수

기존 내장 Step 함수

- **echo** : Jenkins 콘솔 로그에 메시지를 출력하는 Step
  예: `echo '빌드 시작'`
- **sh** : Linux/Unix 에이전트에서 셸 명령을 실행하는 Step
  예: `sh 'mvn clean package'`
- **checkout** : SCM 설정에 따라 소스 코드를 체크아웃하는 Step
  예: checkout scm
- **git** : Git 저장소에서 코드를 간단히 가져오는 Step
  예: `git branch: 'main', url: 'https://github.com/example/repo.git'`
- **timeout** : 지정한 시간 안에 끝나지 않으면 중단시키는 Step
  예: timeout(time: 10, unit: 'MINUTES') { sh 'sleep 999' }
- **retry** : 실패한 블록을 지정한 횟수만큼 다시 시도하는 Step
  예: `retry(3) { sh 'curl -f http://service/health' }`
- **sleep** : 파이프라인 실행을 일정 시간 대기시키는 Step
  예: `sleep time: 5, unit: 'SECONDS'`
- **withCredentials** : Credentials Binding 플러그인을 통해 자격증명을 블록 내부 환경변수나 파일로 주입하는 Step
  예:
```
withCredentials([usernamePassword(
                                credentialsId: 'harbor-creds',
                                usernameVariable: 'USER',
                                 passwordVariable: 'PASS'
        )]) { sh 'echo $USER' }
```

[참고] step 함수 구조

Groovy에서는 함수 호출의 ()을 생략 가능하며, Key:Value 구조의 Map 문법 제공

Jenkins Pipeline Step은 일반 함수처럼 호출되지만,
Groovy DSL을 사용하기 때문에

- 괄호를 생략하고,
- Map으로 이름 있는 옵션을 전달하며,
- 필요한 경우 Closure를 마지막 인자로 전달하여 { } 블록 안의 작업까지 Step이 제어

함수 구조

```
git branch: 'main',
    url: 'http://github.com/example/repo.git'
```

내재된 의미

```
Map args = [
    branch: 'main',
    url: 'http://github.com/example/repo.git'
]
git(args)
```

```
withCredentials([
    usernamePassword(
        credentialsId: 'harbor-creds',
        usernameVariable: 'USER',
        passwordVariable: 'PASS'
    )
]) {
    sh 'echo $USER'
}
```

```
withCredentials(
    credentials,
    () -> {
        sh("echo $USER");
    }
);
```

closure

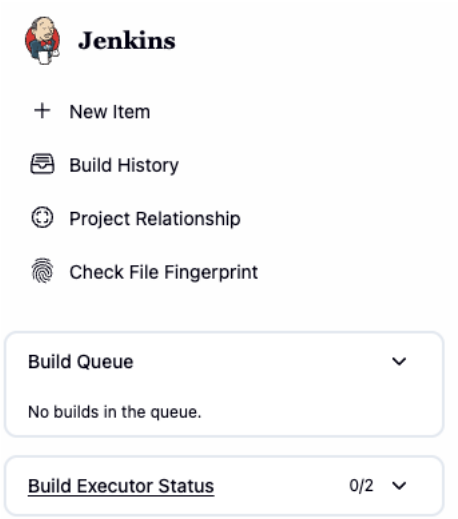
실습: Jenkins 처음 실행

실습 코드 위치 : 04.jenkins/

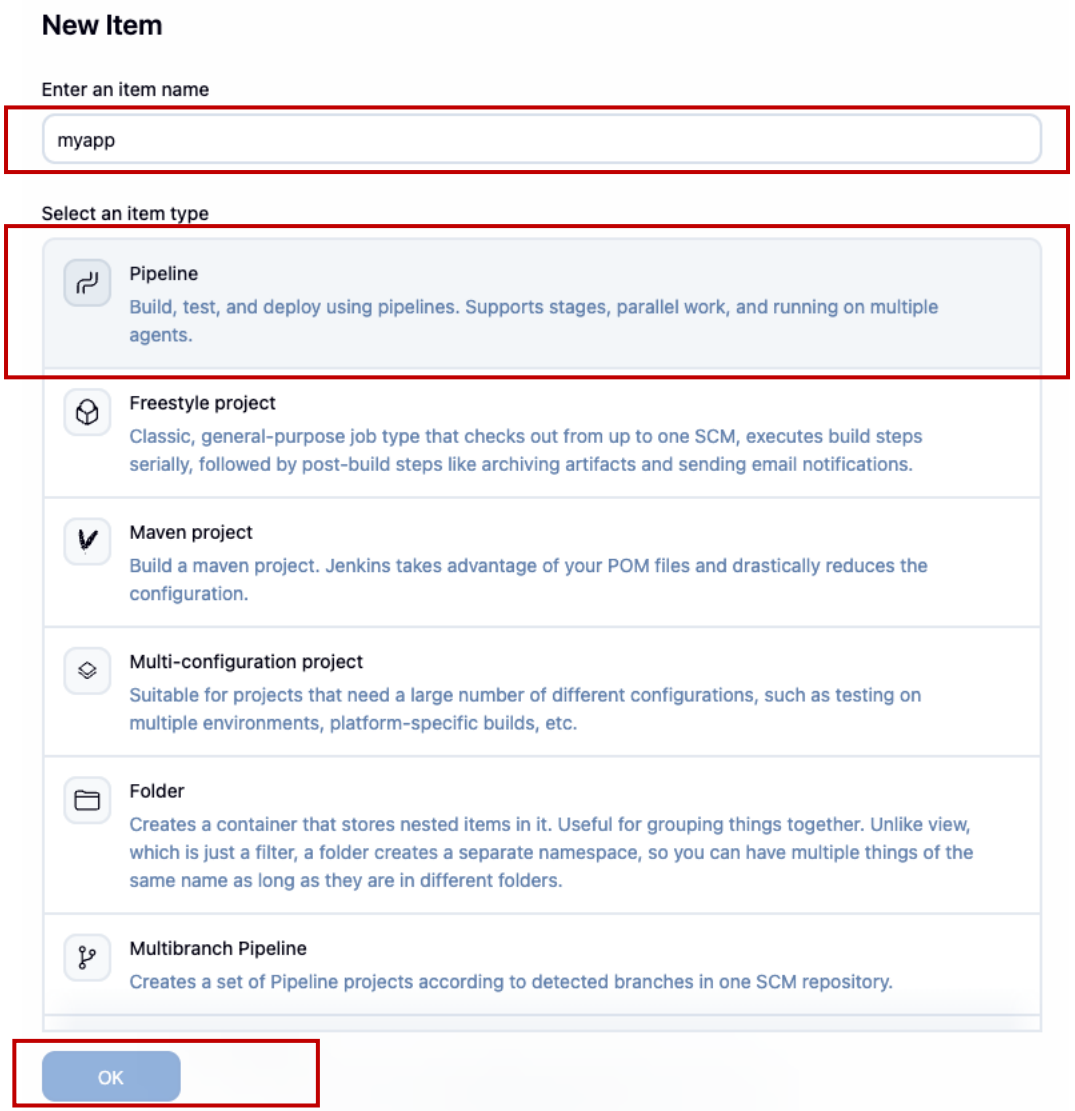
[실습] 샘플 베이직 파이프라인 구성하기

각 스탭 별 트린트하는 파이프라인 구성하기

1. New Item 선택

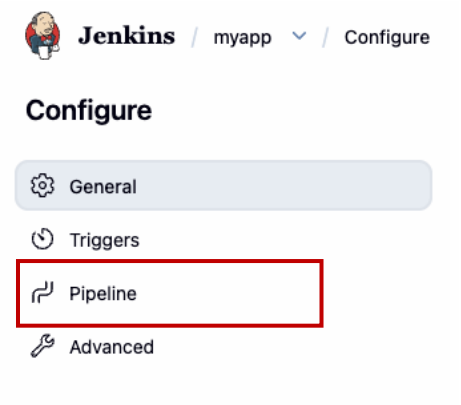


2. 파이프라인 생성하기



[실습] 샘플 베이직 파이프라인 구성하기

3. 파이프라인 선택



[실습] 샘플 베이직 파이프라인 구성하기

참고 코드 jenkinsfile/01.basic/Jenkinsfile-basic

Definition

Pipeline script

Script ?

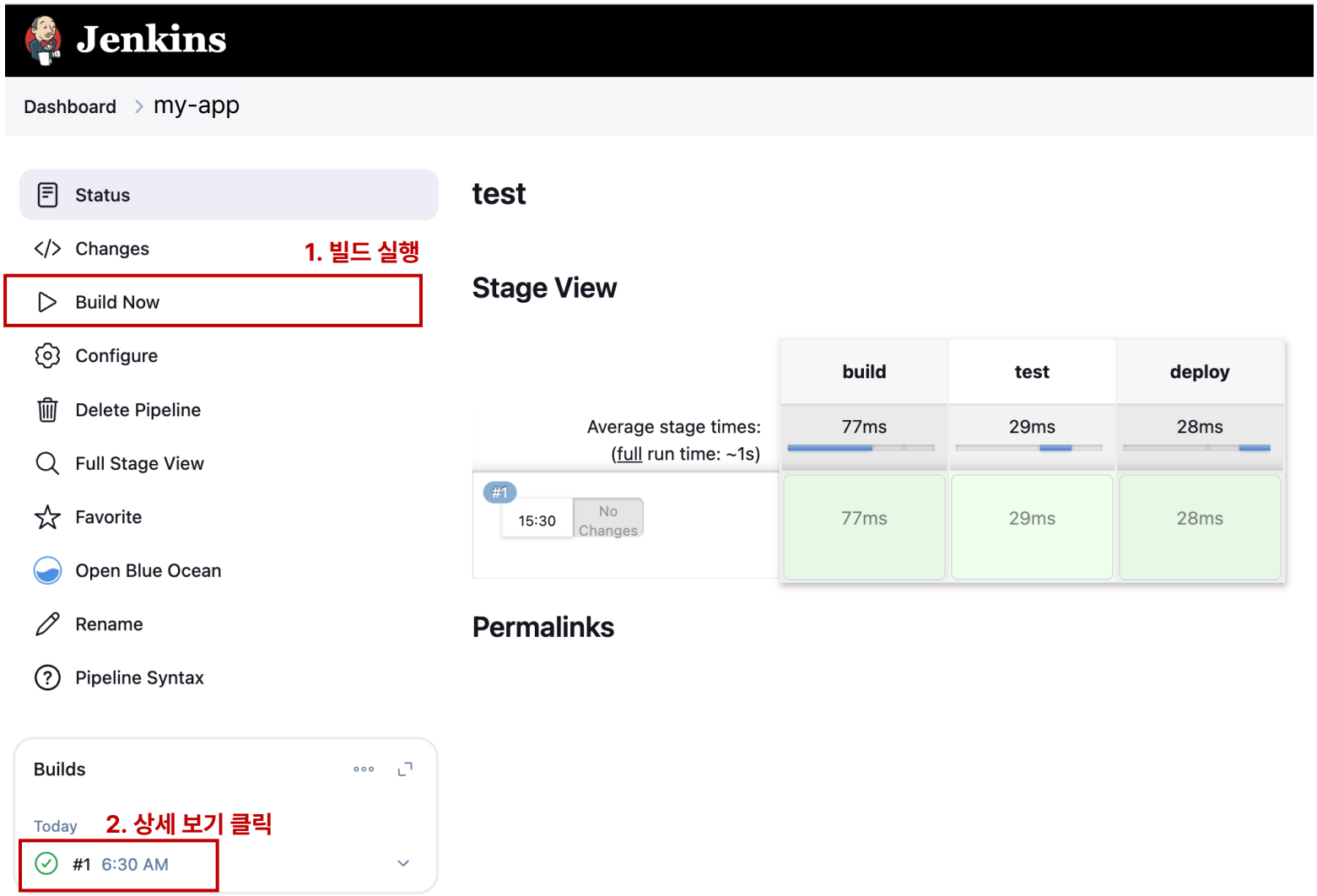
1

오른쪽 스크립트를
여기에 등록

Save

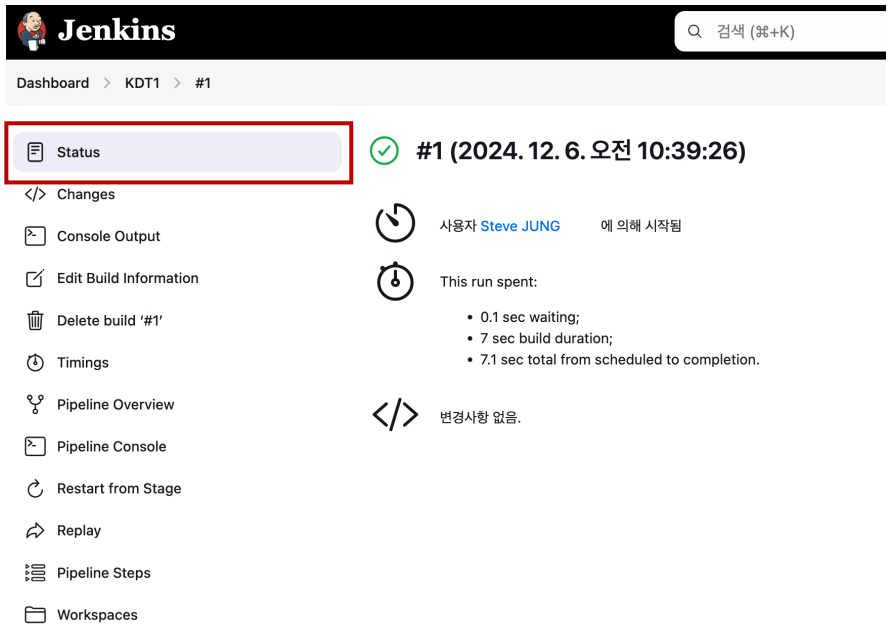
```
pipeline {
    agent any
    environment { APP_NAME = 'my-sample-app'; DEPLOY_ENV = 'dev' }
    options {
        timeout(time: 10, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '5'))
        disableConcurrentBuilds()
    }
    stages {
        stage('build') {
            environment { CURRENT_ENV = 'build' }
            steps {
                echo "Building ${env.APP_NAME}"
                echo "Current env: ${env.CURRENT_ENV}"
            }
            post {
                always { echo 'Build stage finished (regardless of result)' }
            }
        }

        stage('test') {
            steps {
                echo "Testing in ${env.DEPLOY_ENV}"
            }
            post {
                always { echo 'test finished' }
            }
        }

        stage('deploy') {
            steps {
                echo "Deploying ${env.APP_NAME} to ${env.DEPLOY_ENV}"
            }
            post {
                always { echo 'deploy finished' }
            }
        }
    }
    post {
        always  { echo 'Pipeline finished' }
        success { echo 'Success!' }
        failure { echo 'Failed!' }
    }
}
```

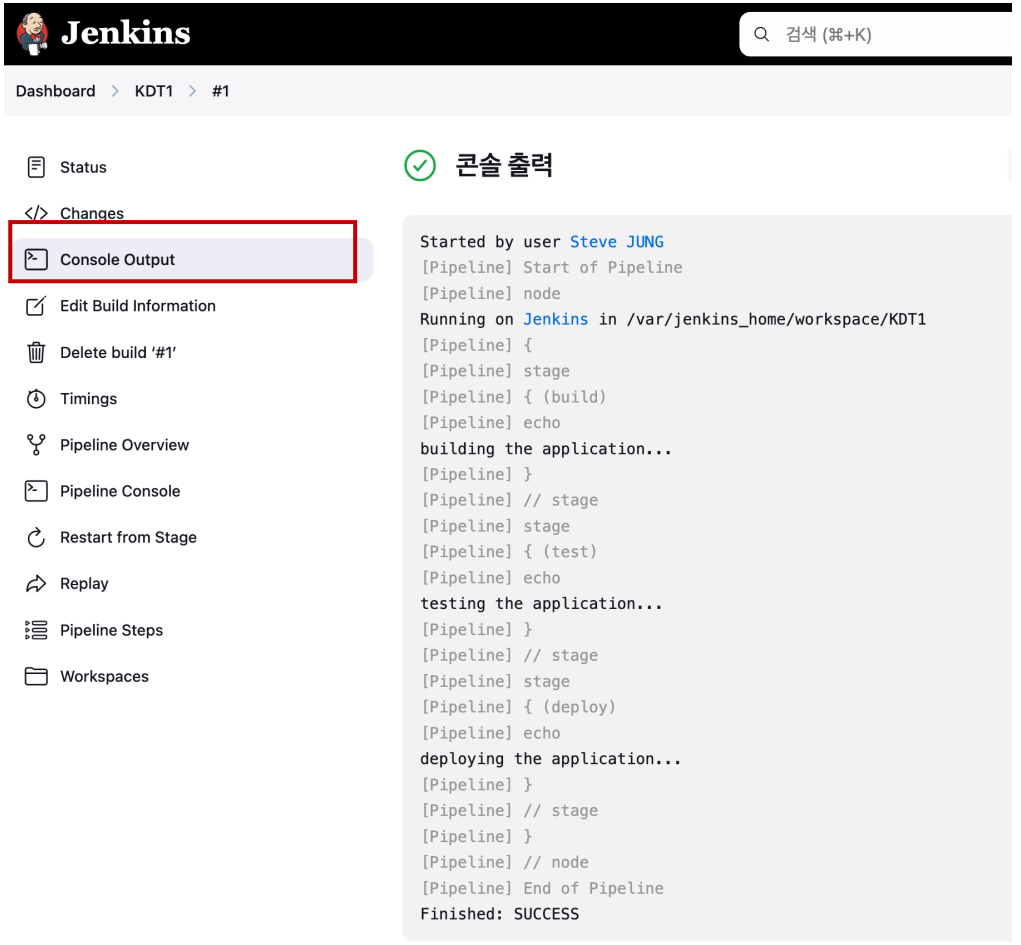
[실습] 샘플 베이직 파이프라인 구성하기



[실습] Jenkins 기본 환경 구성

왼쪽 탭에 위치한 Console Output 을 클릭해서 실행한 파이프라인의 전체 출력을 확인할 수 있다.



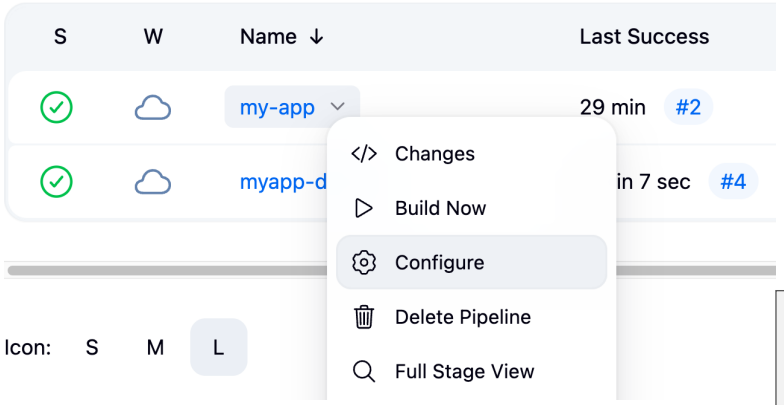


[실습] Agent를 docker로 실행해보기

agent any를 다음과 같이 수정한 후 다시 빌드해보자

참고 코드 jenkinsfile/basic/Jenkinsfile-basic-docker

1. test




```
agent {
      docker {
          image 'amdp-registry.skala-ai.com/library/alpine:3.20'
          args  '--entrypoint=""'
      }
  }
```

Pipeline script

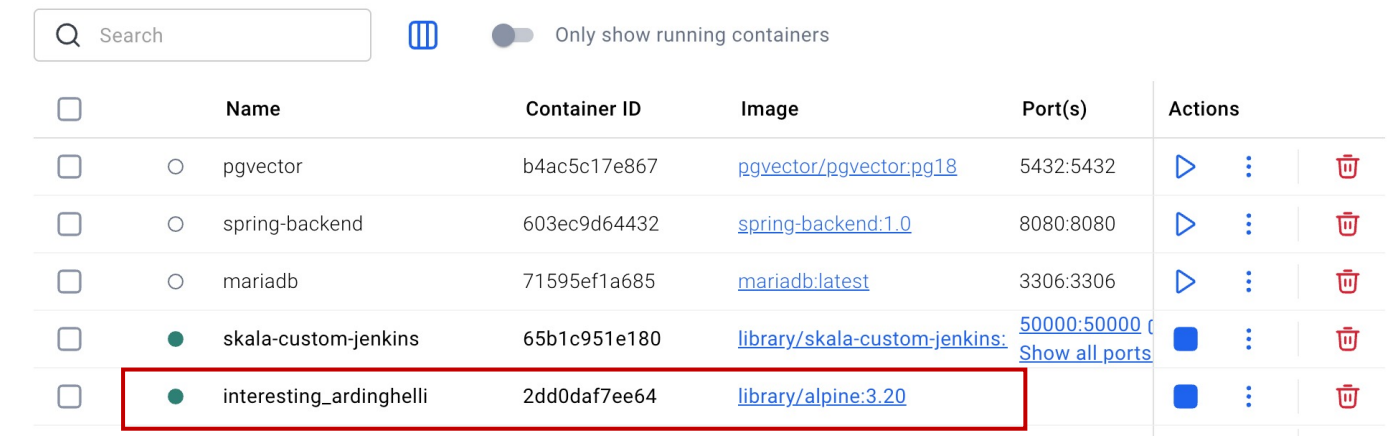
이것으로 수정

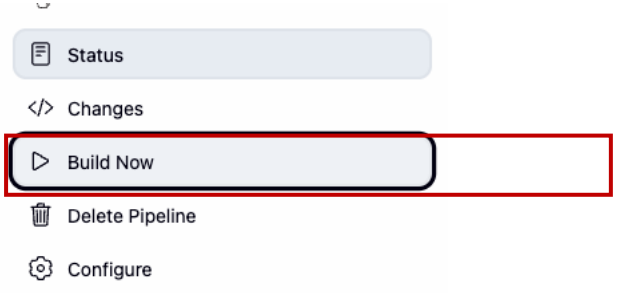
Script ?

```
1  pipeline {
2      agent any
3
4      environment { APP_NAME = 'my-sample-app'; DEPLOY_ENV = 'dev' }
5
6      options {
7          timeout(time: 10, unit: 'MINUTES')
8          buildDiscarder(logRotator(numToKeepStr: '5'))
9          disableConcurrentBuilds()
10     }
```

[실습] Agent를 docker로 실행해보기

docker desktop을 열고 난뒤 build를 실행해보자





실습 끝

Credential 등록하기

파이프라인을 구성하기 위한 Git Credential과 Harbor Registry를 등록한다.

Credential 등록

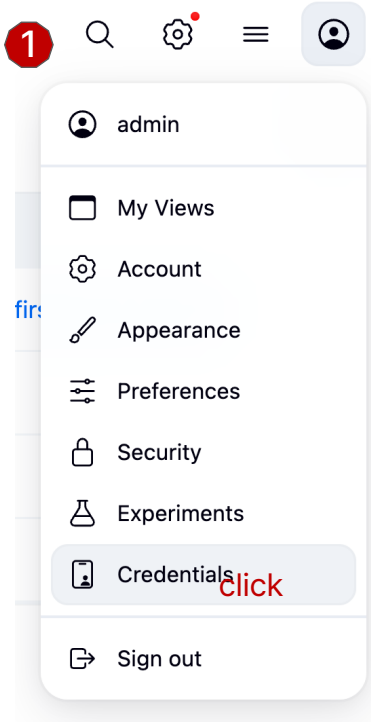
Jenkins.yaml JCaaS에 기 등록되어 있음

```yaml
credentials:
  system:
    domainCredentials:
      - credentials:
          # ── GitHub PAT (username/password) ──────────────────────────
          # Jenkinsfile: credentialsId: 'skala-github-id'
          # 운영 환경: password: "${GITHUB_PAT}"  (환경변수 또는 K8s Secret으로 주입)
          - usernamePassword:
              scope: GLOBAL
              id: "skala-github-id"
              description: "GitHub Personal Access Token"
              username: "your github username"   # ← 실제 GitHub 사용자명으로 교체
              password: "your github pat"   # ← 실제 PAT로 교체

          # ── Harbor Registry (username/password) ─────────────────────
          # Jenkinsfile: credentialsId: 'skala-image-registry-id'
          # 운영 환경: password: "${HARBOR_PASSWORD}"  (환경변수 또는 K8s Secret으로 주입)
          - usernamePassword:
              scope: GLOBAL
              id: "skala-image-registry-id"
              description: "Harbor Registry Credential"
              username: "robot$skala25a"
              password: "1qB9cyusbNComZPHAdjNIFWinf52xaBJ"   # ← 실제 비밀번호로 교체
```

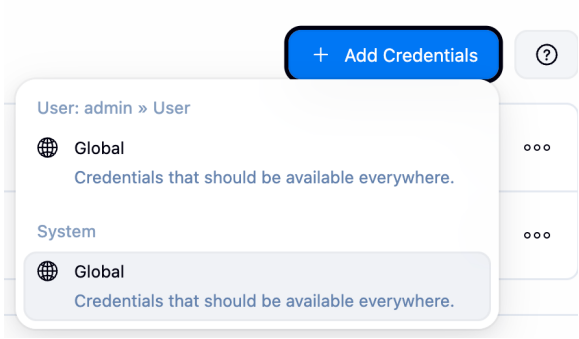
[참고] git credentials 추가하기

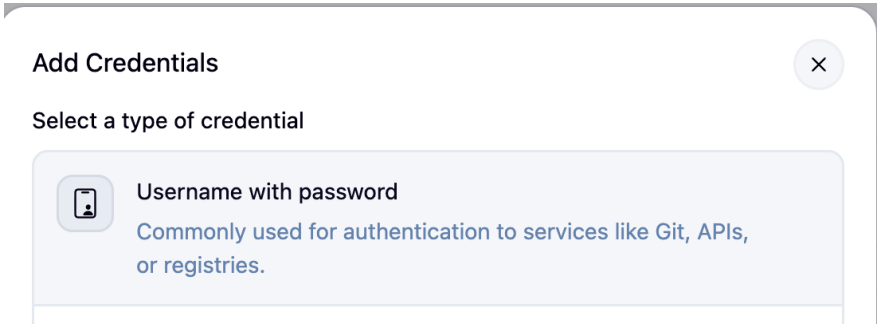
자신의 gitub 접속 정보를 sk199-github-id credentials 로 등록해보자

1

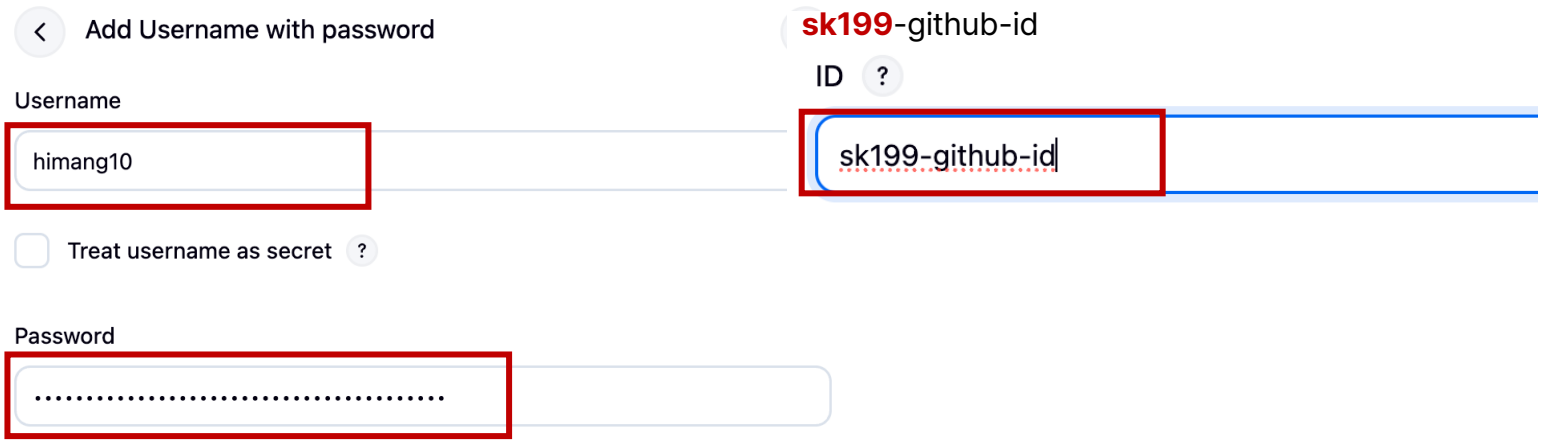


2 Username with password 추가하기





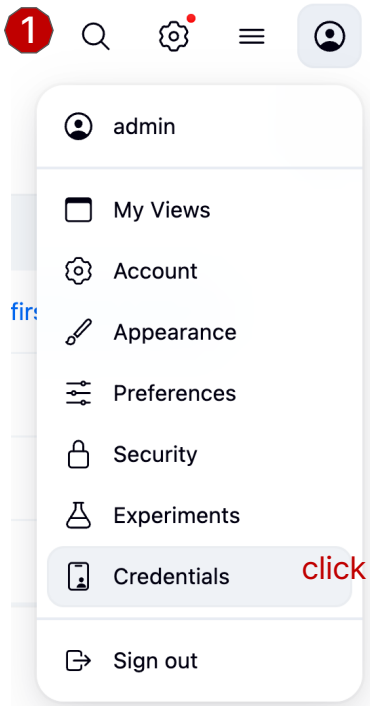
3 git username 과 PAT Token 그리고 id 추가하기



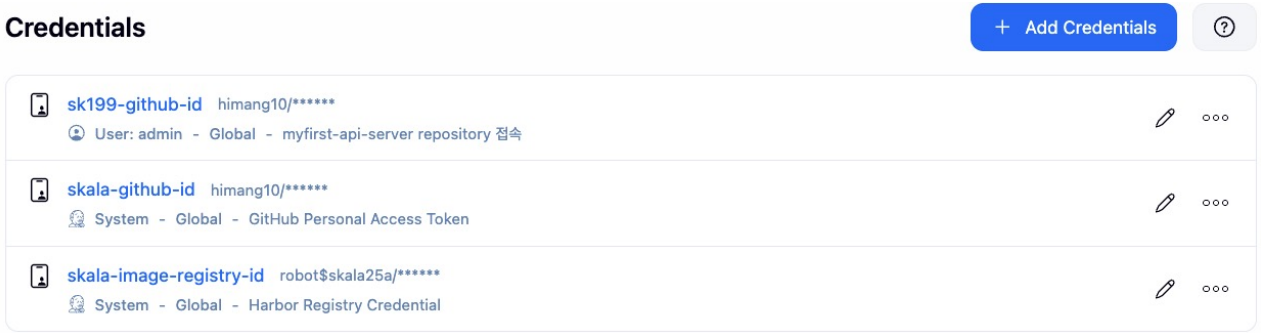
[참고] docker registry cred 확인하기

기존에 등록되어 있는 docker registry cred를 수정하기

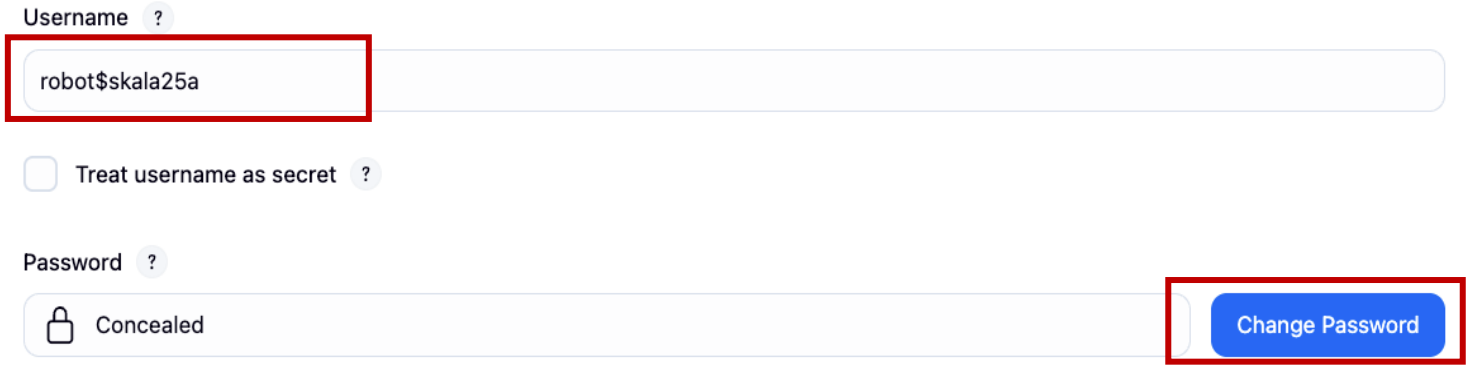
1



2 skala-image-registry-id 의 id/pwd(PAT) 확인



3 git username 과 Password 확인

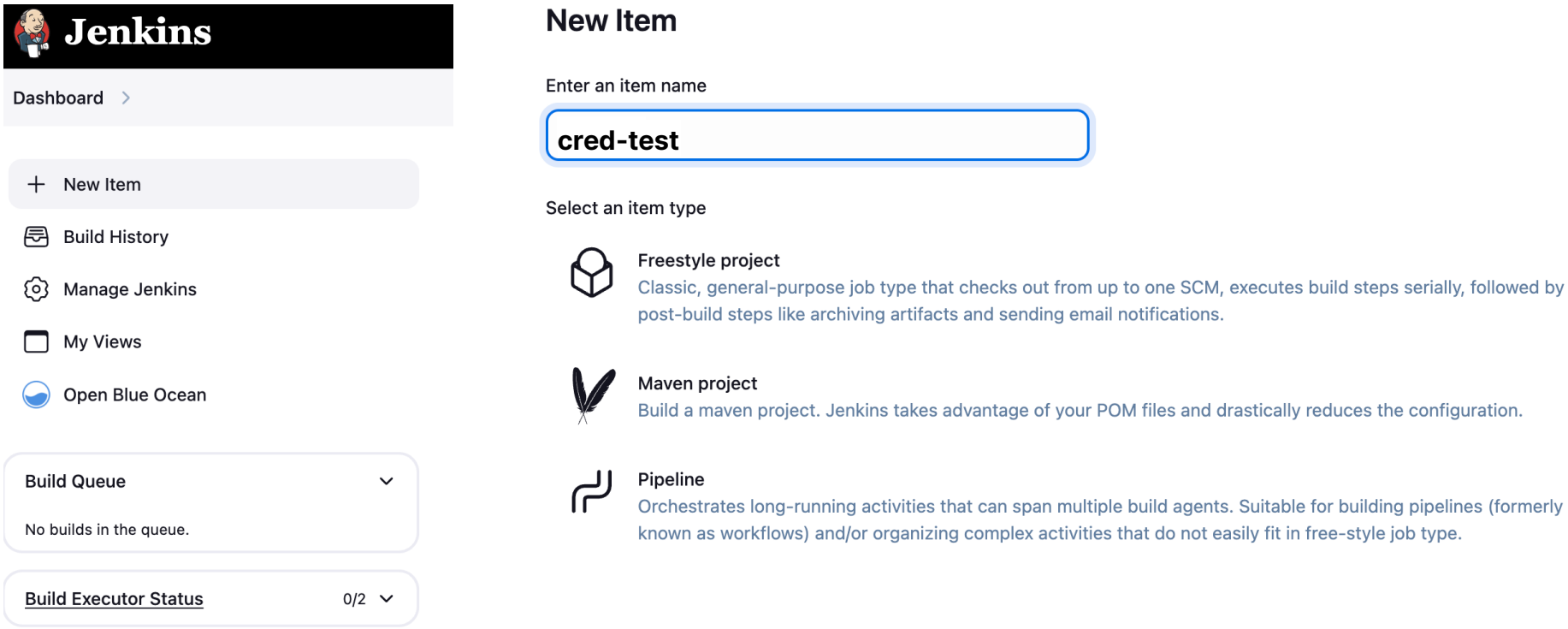


[검증 시험] Credentials 정보 검증하기

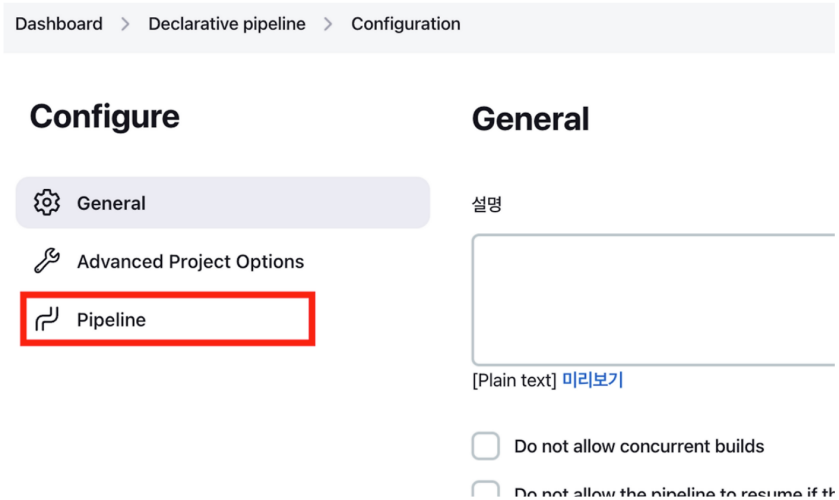
1. Jenkinsfile 복사하기

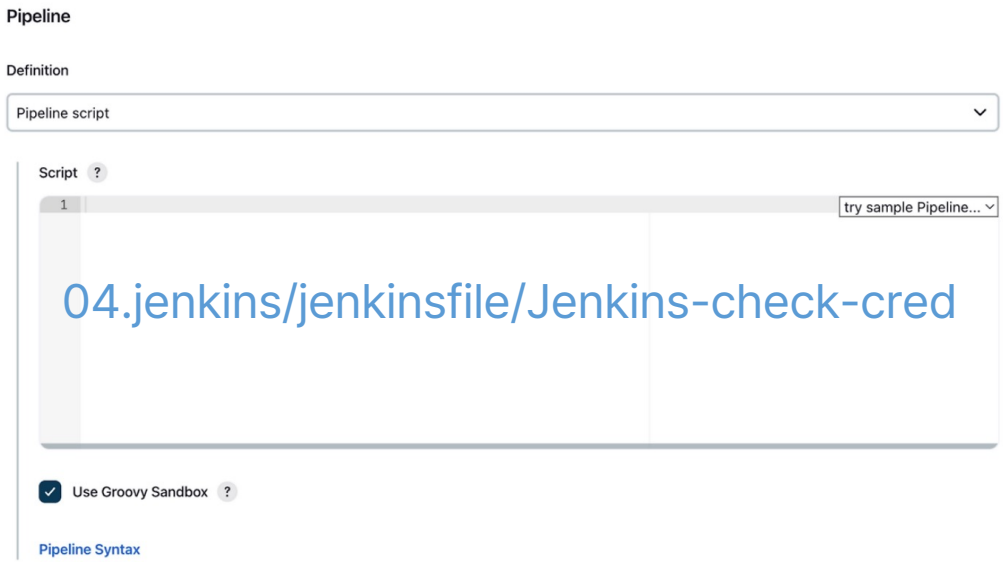
```
cd ~/workspace/devops/04.jenkins/jenkinsfile

ls Jenkinsfile-check-cred
```

2. Jenkinsfile-check-cred 내용으로 Credential Test



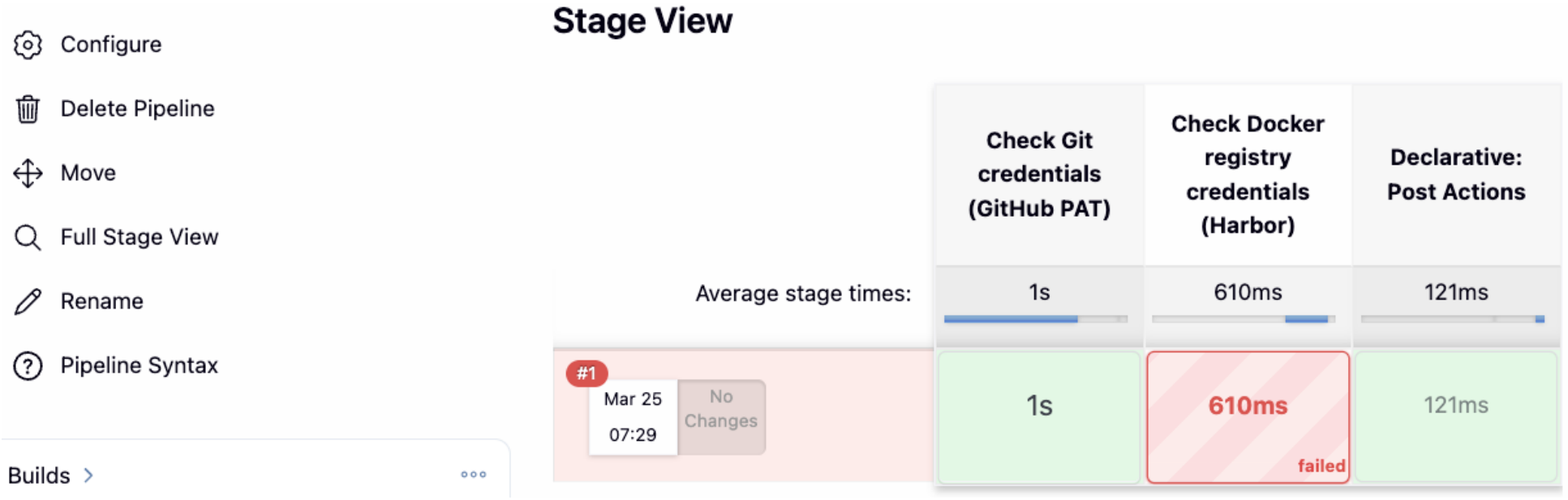
[검증 시험] Credentials 정보 검증하기





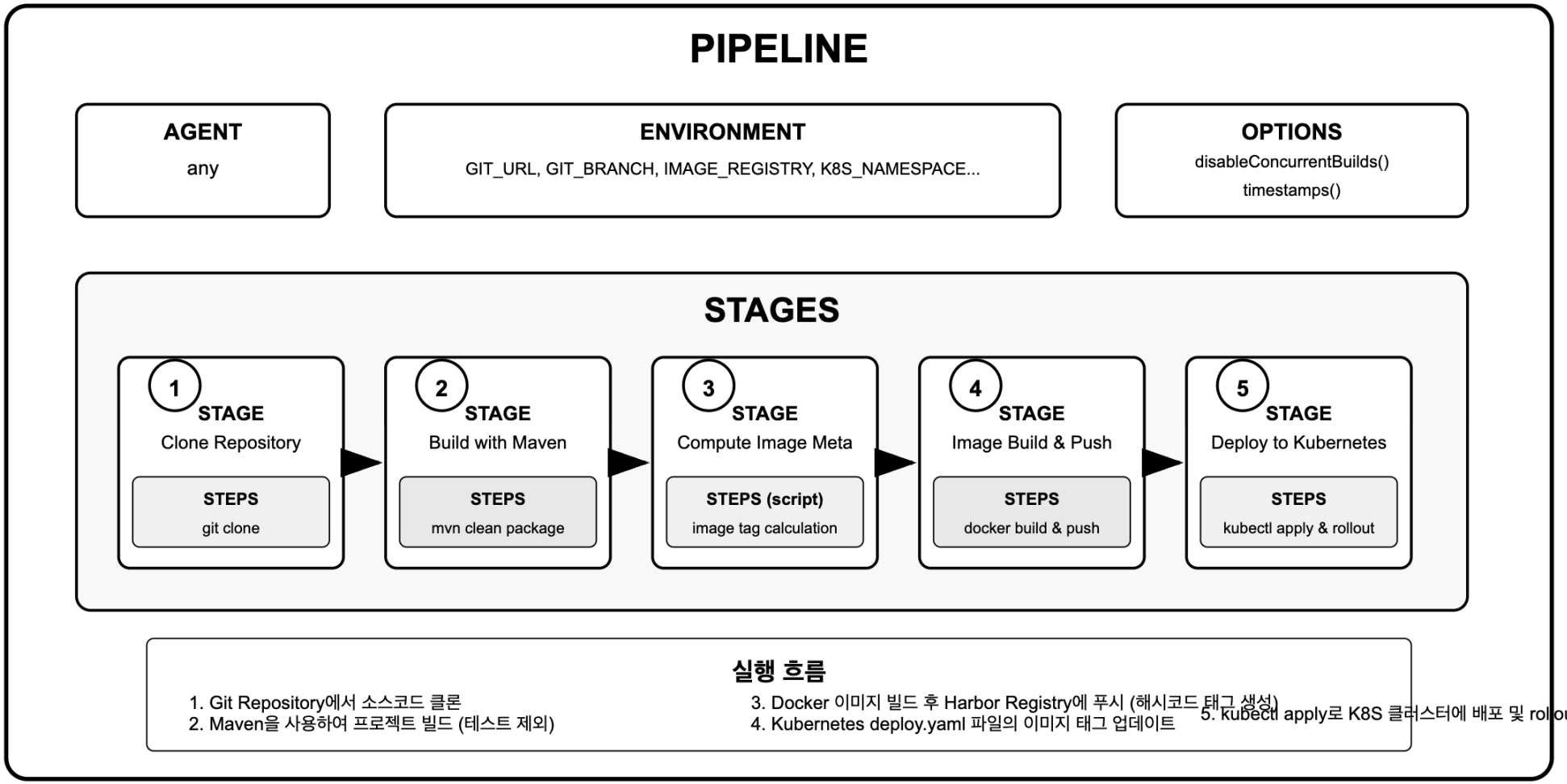
[검증 시험] Credentials 정보 검증하기

정상적으로 동작하는 경우 아래와 같으며 비정상적인 경우 skala-github-id 와 skala-image-registry-id 의 PAT 다시 등록



CICD 구성하기: pipeline script

Jenkins 기반의 나의 spring boot 코드를 컴파일, 컨테이너 이미지 build/push, kubernetes 배포까지의 파이프라인 흐름을 구성



CICD 구성하기: pipeline script

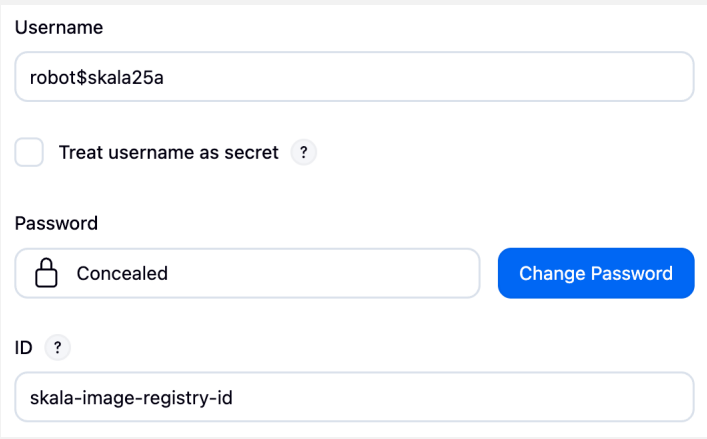
02.local/01.local-cicd/Jenkinsfile-CICD-docker 파이프라인 스크립트를 확인해보자

```
pipeline {
  agent any

  environment {
    // === 사용자 수정 영역 ===
    GIT_URL                = 'https://github.com/himang10/myfirst-api-server.git'
    GIT_BRANCH             = 'master'            // 또는 main
    GIT_ID                 = 'skala-github-id'   // GitHub PAT credential ID
    IMAGE_NAME             = 'skala-webserver'
    // ======================
    IMAGE_TAG              = '1.0'
    IMAGE_REGISTRY_URL     = 'skala-registry.skala-ai.com'
    IMAGE_REGISTRY_PROJECT = 'class-0'

    DOCKER_CREDENTIAL_ID   = 'skala-image-registry-id'  // Harbor 인증 정보 ID
    K8S_NAMESPACE          = 'class-0'
    DEPLOYMENT_NAME        = 'sk000-myfirst-api-server'
  }

  options {
    disableConcurrentBuilds()
    timestamps()
  }

  stages {
    stage('Clone Repository') {}

    stage('Build with Maven') {}

    stage('Image Build & Push (BuildKit)') {}

    stage('Update K8s Manifests') {}

    stage('Deploy to Kubernetes') {}

  }
}
```

[참고] withCredentials

Jenkins에 저장해 둔 자격증명(credentials)을 Pipeline 블록 안에서만 안전하게 꺼내 쓰게 해주는 Step

```
pipeline {
    agent any

    stages {
        stage('Login') {
            steps {
                withCredentials([
                    usernamePassword(
                        credentialsId:  'skala-github-id',
                        usernameVariable: 'HARBOR_USER',
                        passwordVariable: 'HARBOR_PASS'
                    )
                ]) {
                    sh 'echo "$HARBOR_USER"'
                    sh 'docker login -u "$HARBOR_USER" -p "$HARBOR_PASS" amdp-registry.skala-ai.com'
                }
            }
        }
    }
}
```



→ withCredentialsStep 이 나중에 실행할 블록 (Closure)을 전달

- **credentialsId**: 'harbor-creds ' → Jenkins에 미리 등록된 자격증명 이름
- **usernameVariable**: 'HARBOR_USER ' → username 값을 이 환경변수에 넣음
- **passwordVariable**: 'HARBOR_PASS ' → password 값을 이 환경변수에 넣음
- 블록 안에서는 $HARBOR_USER, $HARBOR_PASS 사용 가능
- 블록 밖으로 나오면 원칙적으로 더 이상 사용하지 않음

실습: CICD 파이프라인 구성하기

실습 코드 위치 : 04.jenkins/02.local/01.local-cicd

[CICD script] Jenkinsfile script 등록하기

Jenkins에 Jenkinsfile 스크립트를 등록하는 방법은 2가지 있음

1. 파이프라인 스크립트를 Jenkins에 직접 등록 (Pipeline script)

   스크립트 검증 시

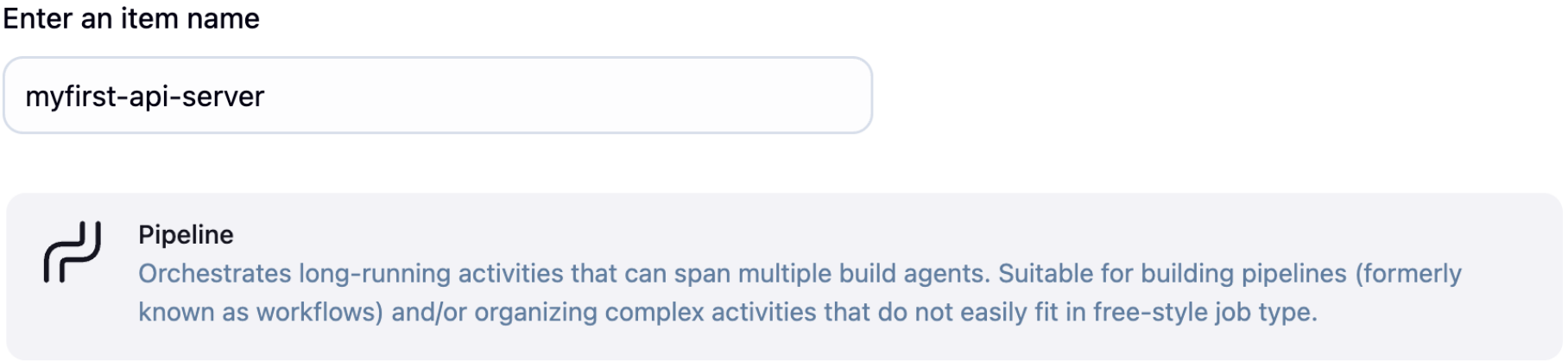
2. SCM으로 부터 파이프라인 스크립트를 불러오는 방법 (Pipeline script from SCM)

   검증된 스크립트를 파이프라인 구성 시

[Pipeline Script] 스크립트 등록 (1/2)

자신의 앱을 위한 파이프라인 구성하기

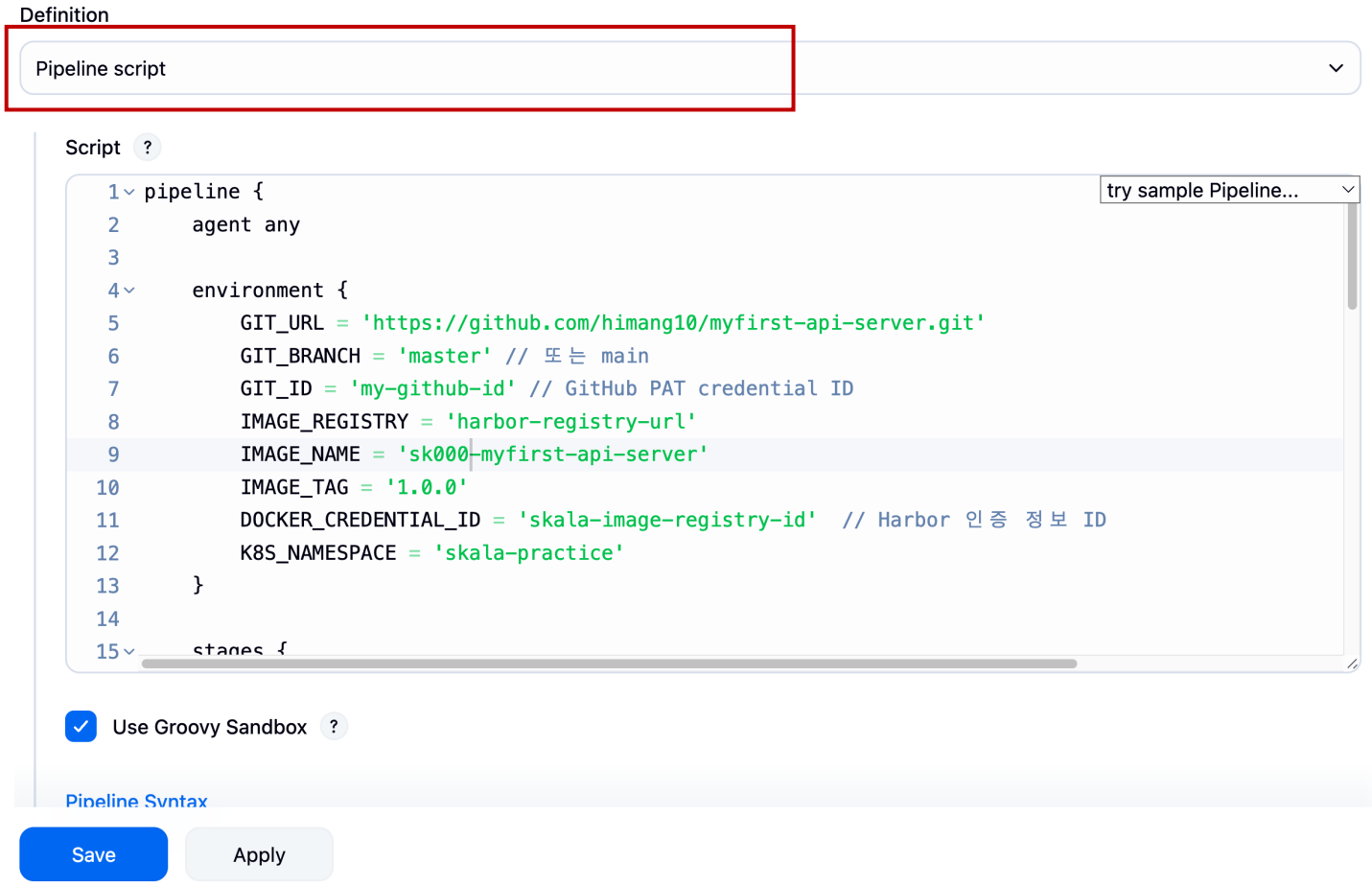
[메뉴] + New Item 클릭
└ Enter an item Name: “myfirst-api-server” 등록
├ Select an item Type에서 “Pipeline” 선택
└ OK 클릭



[Pipeline Script] 스크립트 등록 (2/2)

1단계: Pipeline script 를 적용

02.local/01.local-cicd/Jenkinsfile-CICD-docker 내용을 Script복사하고 저장

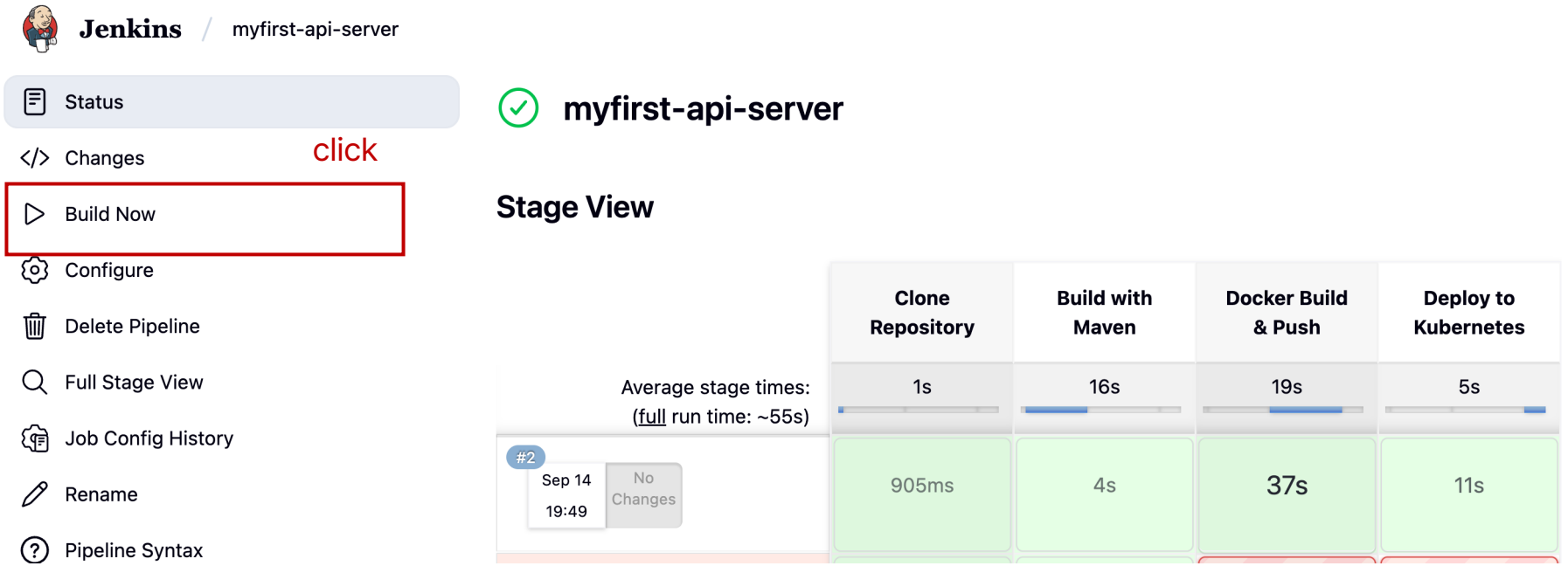


[Pipeline Script] 스크립트 실행 (1/2)

1단계: Pipeline script 를 적용

S	W	Name ↓	Last Success	Last Failure	Last Duration	# Issues
✓	☁	my-app	4 hr 33 min #2	4 hr 37 min #1	4.6 sec ▷	-
✓	☁	myapp-docker	4 hr 7 min #4	4 hr 14 min #2	3.9 sec ▷	-
✓	☁	myfirst-api-server	4 min 39 sec #2	8 min 26 sec #1	55 sec ▷	-

click



[Pipeline Script] 스크립트 실행 (2/2)

kubernetes에 배포된 현황을 확인한다.

1. 배포 현황 확인하기

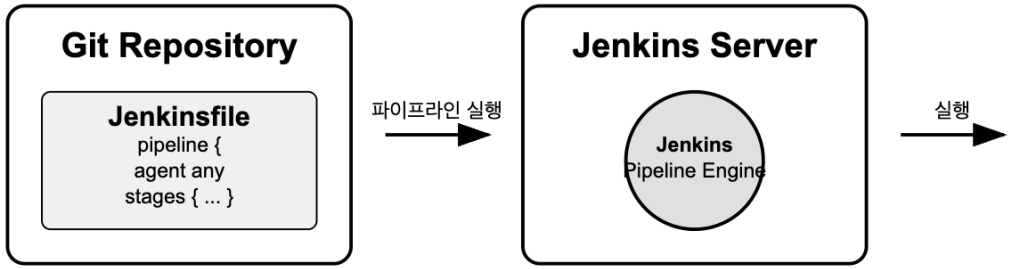
```
kubectl get pod -l app=sk000-myfirst-api-server

kubectl describe pod -l app=sk000-myfirst-api-server

kubectl get service -l app=sk000-myfirst-api-server
```

2. 배포 자원 삭제하기

```
kubectl delete deploy sk199-myfirst-api-server

kubectl delete service sk199-myfirst-api-server
```

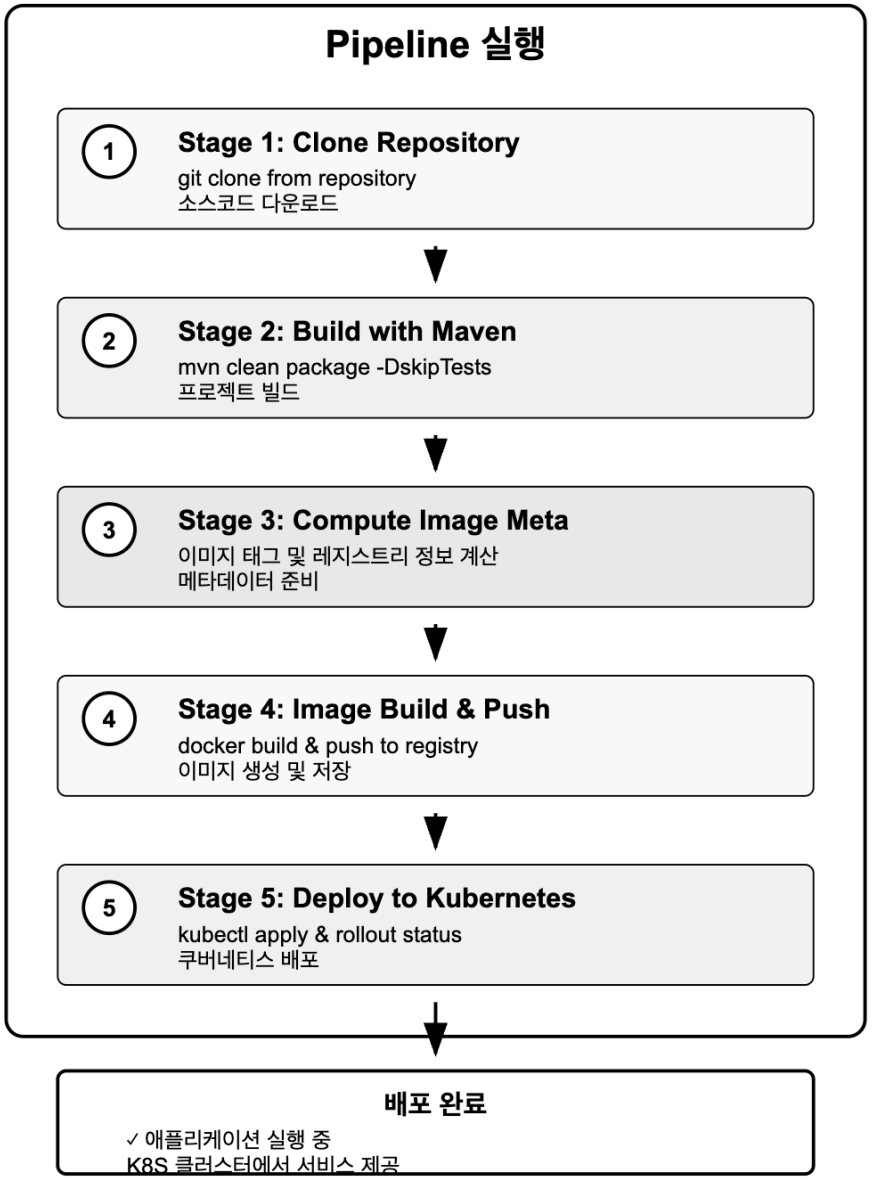
CICD 구성하기: script from SCM

Jenkinsfile을 git repository에 등록되어 있는 Jenkinsfile을 기반으로 파이프라인을 실행하기 위한 방법을 가이드한다



실행 과정

1. **트리거 발생**
   - Git push, 스케줄, 수동 실행 등
2. **Jenkinsfile 파싱**
   - Git repository에서 Jenkinsfile 읽기
   - Pipeline 구조 분석 및 실행 계획 수립
3. **순차적 Stage 실행**
   - 각 Stage는 순서대로 실행
   - 실패 시 파이프라인 중단



[CICD from SCM] Jenkinsfile을 git repo에 복사

파이프라인 구성 시 git repository에 등록되어 있는 Jenkinsfile을 pull 후 이것을 기반으로 파이프라인 실행을 위해 myfirst-api-server git repository에 Jenkinsfile이 등록되어 있어야 한다.

1. jenkinsfile/02.local/01.local-cicd/Jenkinsfile-CICD-docker 파일을 myfirst-api-server git repo에 복사하기

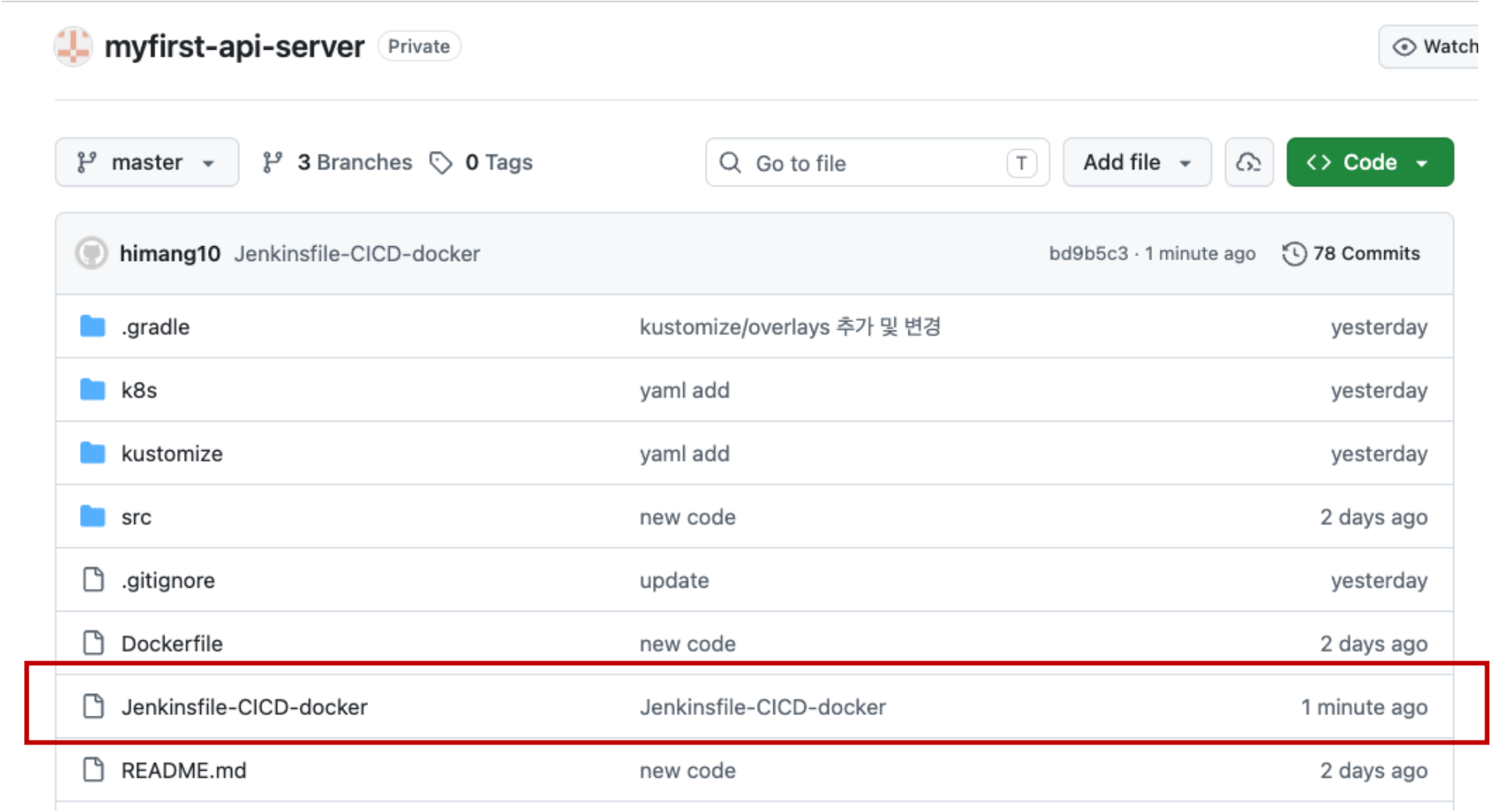
```
# cp {Jenkinsfile-CICD-docker 위치}  {본인 myfirst-api-server 디렉토리}
예) cp Jenksfile-CICD-docker ~/workspace/myfirst-api-server/
```

2. myfirst-api-server 로 이동해서 Jenkinsfile-CICD-docker 파일을 git repository로 push 하기

```
> git add .

> git commit –m “add Jenkinsfile”

> git push origin main
```

[CICD from SCM] Jenkinsfile을 git repo에 복사

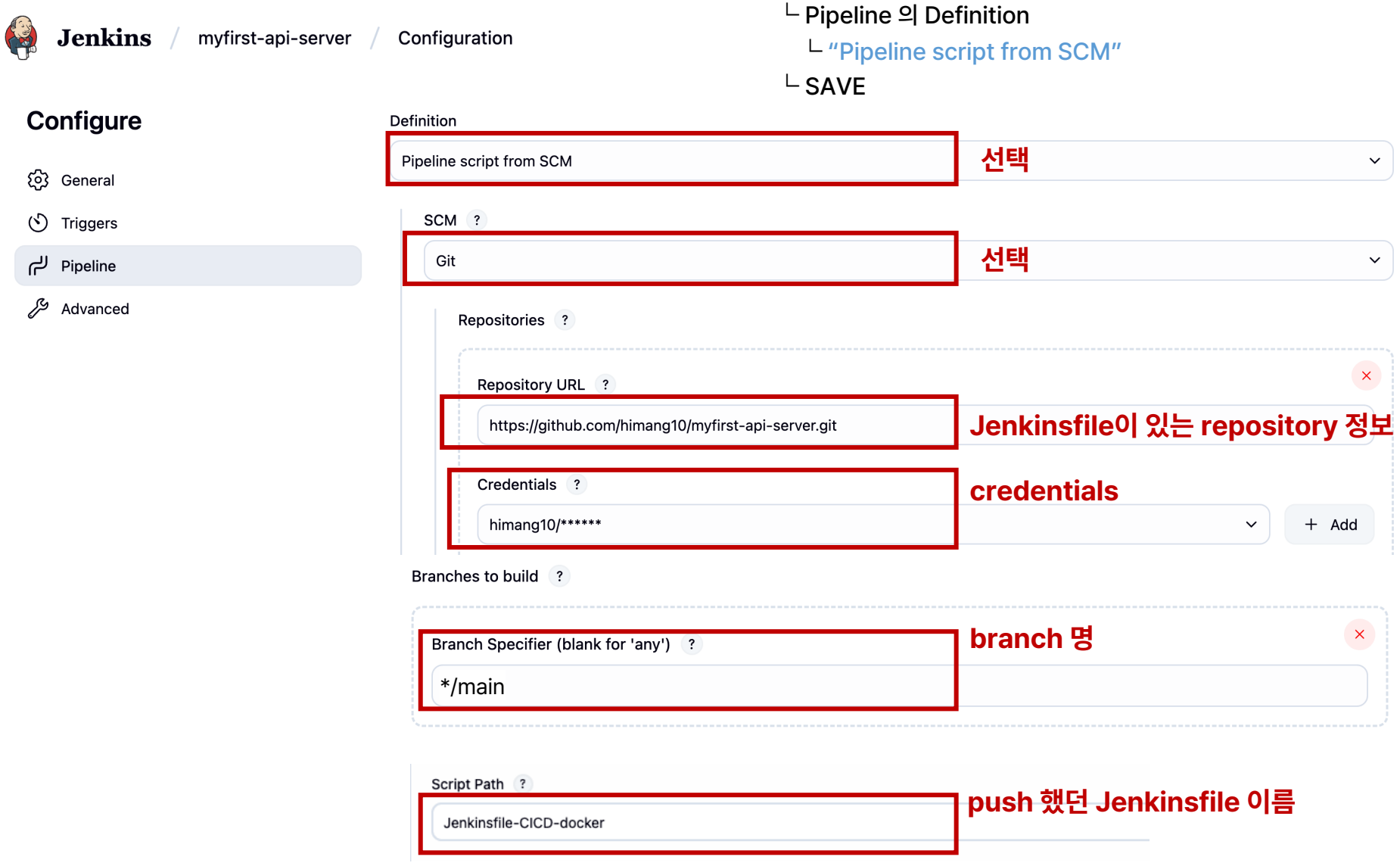
자신의 myfirst-api-server git repo에 Jenkinsfile-CICD-docker가 올라가 있어야 함



[CICD from SCM] 스크립트 등록

Pipeline script from SCM 으로 설정하기

[메뉴] Configure 의 General
└ Pipeline 의 Definition
└ "Pipeline script from SCM"
└ SAVE



[CICD from SCM] 스크립트 실행

2단계: Pipeline script from SCM 으로 설정하기

S	W	Name ↓	Last Success	Last Failure	Last Duration	# Issues
✓	☁	my-app	4 hr 33 min #2	4 hr 37 min #1	4.6 sec ▷	-
✓	☁	myapp-docker	4 hr 7 min #4	4 hr 14 min #2	3.9 sec ▷	-
✓	☁	myfirst-api-server	4 min 39 sec #2	8 min 26 sec #1	55 sec ▷	-

click

Jenkins / myfirst-api-server

- Status
- Changes
- Build Now
- Configure
- Delete Pipeline
- Full Stage View
- Job Config History
- Rename
- Pipeline Syntax

click

myfirst-api-server

Stage View

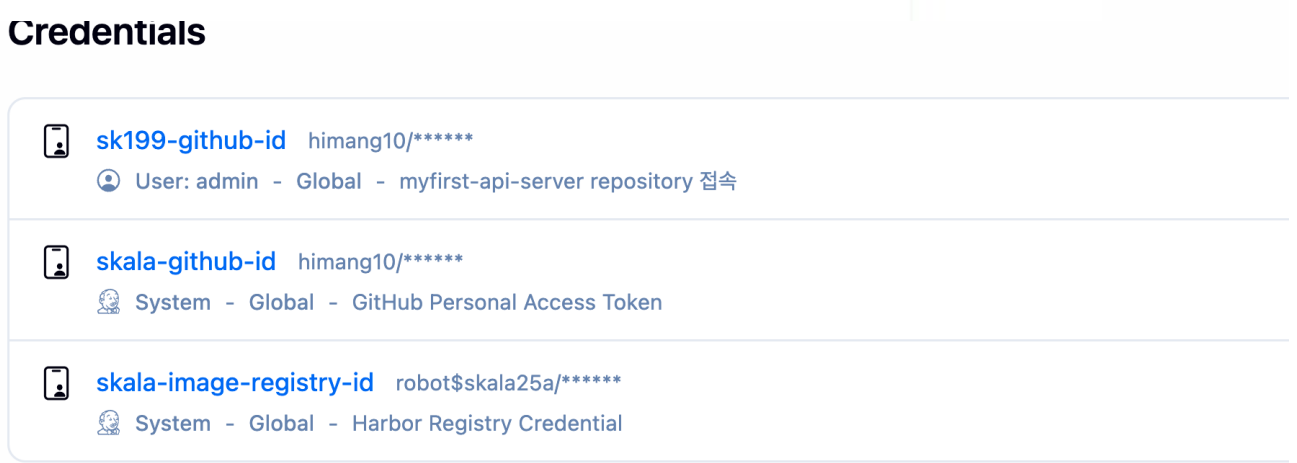
	Clone Repository	Build with Maven	Docker Build & Push	Deploy to Kubernetes
Average stage times: (full run time: ~55s)	1s	16s	19s	5s
#2 Sep 14 19:49 No Changes	905ms	4s	37s	11s

실습 끝

[둘러보기] Jenkinsfile-CICD-docker

Jenkinsfile에서 사용하기 위한 환경 변수 선언하기

```
environment {
  // === 사용자 수정 영역 ===
  GIT_URL                = 'https://github.com/himang10/myfirst-api-server.git'
  GIT_BRANCH             = 'master'            // 또는 main
  GIT_ID                 = 'skala-github-id'   // GitHub PAT credential ID
  IMAGE_NAME             = 'skala-webserver'
  // ========================
  IMAGE_TAG              = '1.0'
  IMAGE_REGISTRY_URL     = 'skala-registry.skala-ai.com'
  IMAGE_REGISTRY_PROJECT = 'class-0'

  DOCKER_CREDENTIAL_ID   = 'skala-image-registry-id'  // Harbor 인증 정보 ID
  K8S_NAMESPACE          = 'class-0'
  DEPLOYMENT_NAME        = 'skala-myfirst-api-server'  // kustomize/base/deployment.yaml
}
```



실습: builah 파이프라인 구성

실습 코드 위치 : 04.jenkins/02.local/01.local-cicd

[둘러보기] Jenkinsfile-CICD-buildah

```
// 로그인/빌드/푸시/정리
stage('Image Build & Push (Buildah)') {
  steps {
    withCredentials([usernamePassword(
      credentialsId: "${DOCKER_CREDENTIAL_ID}",
      usernameVariable: 'REG_USER',
      passwordVariable: 'REG_PASS'
    )]) {
    sh '''
        set -eux
        REG_HOST="${IMAGE_REGISTRY_URL}"
        IMAGE_REF="${IMAGE_REGISTRY_URL}/${IMAGE_REGISTRY_PROJECT}/${IMAGE_NAME}:${IMAGE_TAG}-${BUILD_NUMBER}"

        # 1. 로그인
        echo "Login to $REG_HOST as $REG_USER"
        sudo buildah login -u "$REG_USER" -p "$REG_PASS" "$REG_HOST"

        # 2. 빌드
        sudo buildah bud --storage-driver=vfs --platform "linux/amd64" -t "${IMAGE_REF}" .
        # 3. 푸시
        sudo buildah push --storage-driver=vfs "${IMAGE_REF}"
        # 4. 정리
        sudo buildah rmi -a || true
    '''
    }
  }
}
```

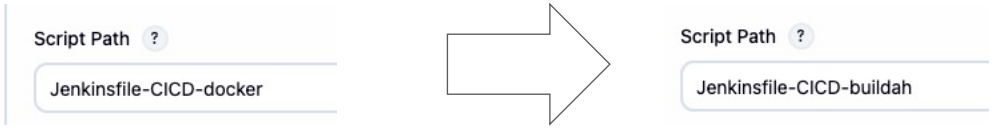
Jenkinsfile-CICD-buildah 실행해보기

기존 Jenkinsfile-CICD-docker 를 jenkinsfile/02.local/01.local-cicd/Jenkinsfile-CICD-buildah 로 대체해보자

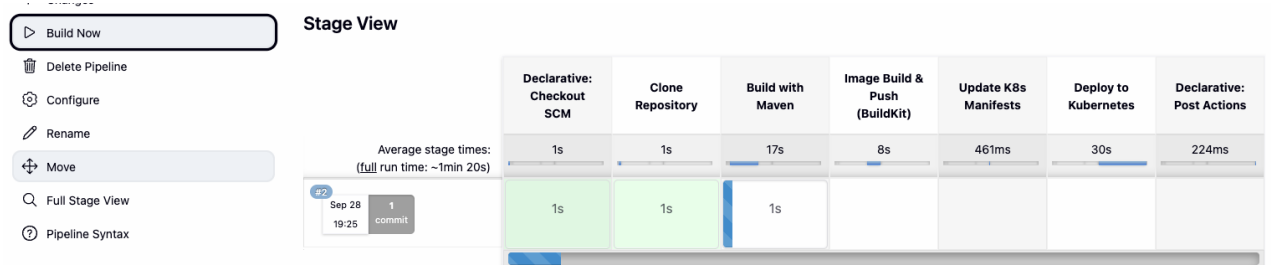
- jenkinsfile/02.local/01.local-cicd/Jenkinsfile-CICD-buildah를 myfirst-api-server repository에 push하기

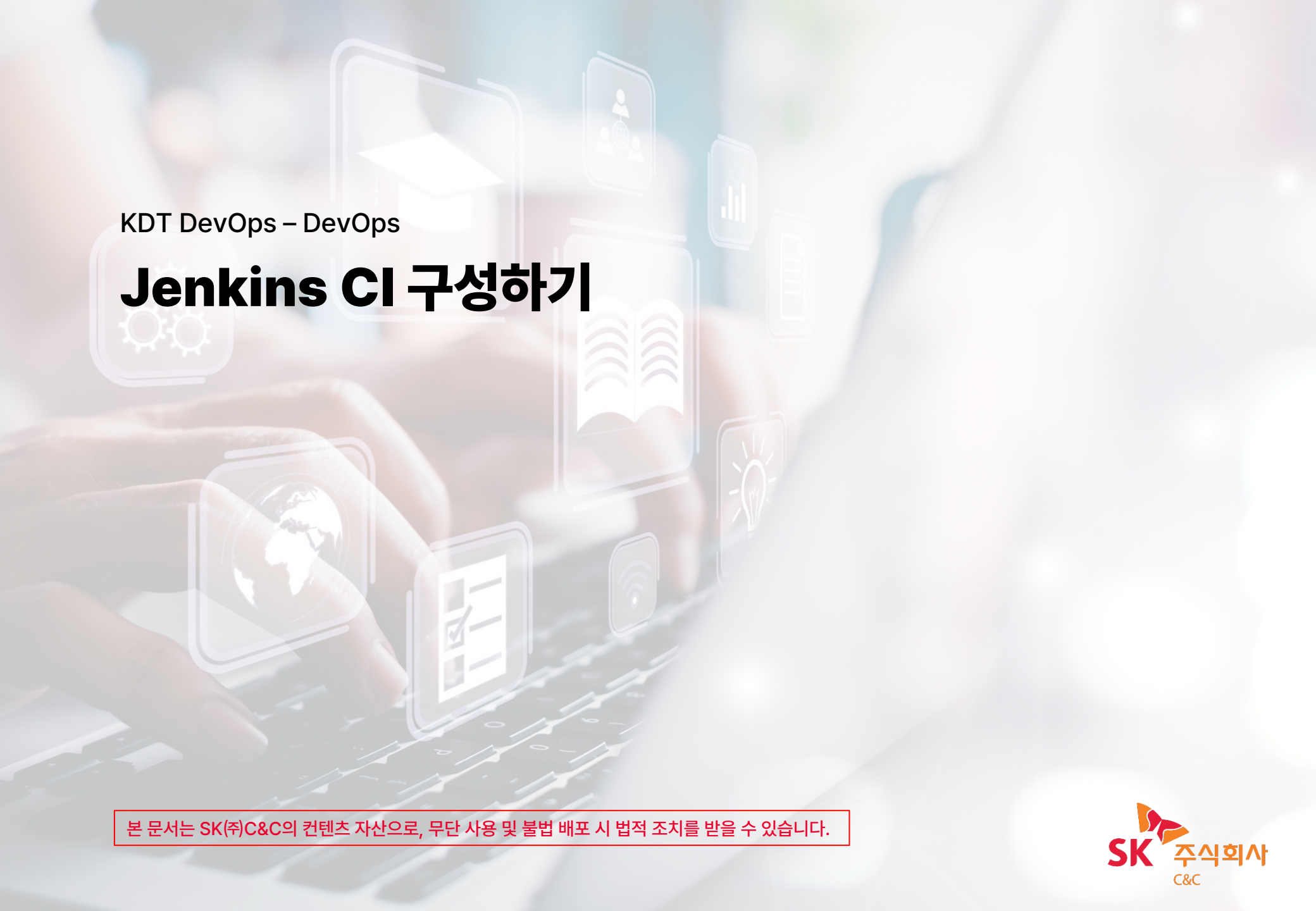
.gradle	kustomize/overlays 추가 및 변경
k8s	yaml add
kustomize	yaml add
src	new code
.gitignore	update
Dockerfile	new code
Jenkinsfile-CICD-buildah	Jenkinsfile-CICD-buildah
Jenkinsfile-CICD-docker	Jenkinsfile-CICD-docker

- myfrist-api-server 파이프라인의 Script Path를 아래 처럼 변경
  Jenkinsfile-CICD-docker → Jenkinsfile-CICD-buildah



- 빌드 실행해보기





KDT DevOps – DevOps

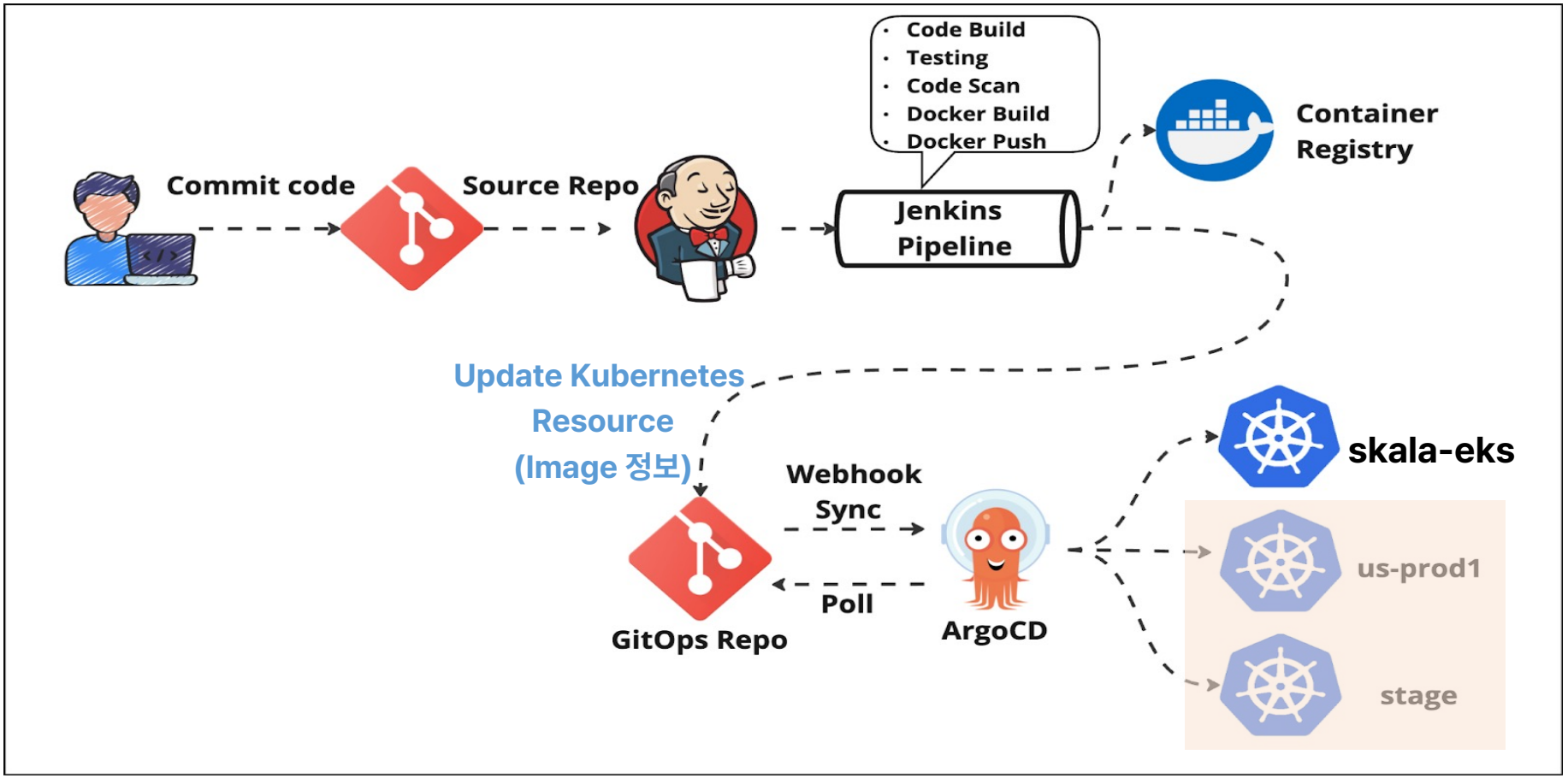
Jenkins CI 구성하기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

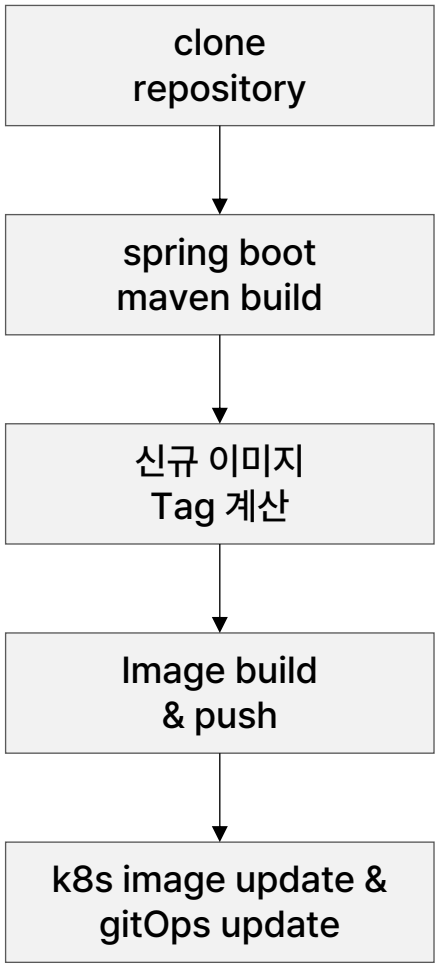
Jenkins CI 구성하기

Jenkins를 이용해서 1) git clone, 2) maven build, 3) build container ,
4) update image info를 포함한 Deploy 업데이트를 제공하여 ArgoCD를 통한
변경된 Image를 가진 컨테이너를 재 배포하도록 구성하기



Jenkinsfile-CI-docker 흐름

1. Jenkinsfile 복사하기



`stage('Clone Repository')`

spring boot source 를 git repository로 부터 clone 하기

`stage('Build with Maven')`

maven clean install –skipTests 를 통해 spring boot의 ./target/*.jar 빌드하기

`stage('Compute Image Meta')`

새롭게 생성되는 컨테이너 이미지를 위한 이미지 태그 생성

예) 1.0.0-ea0f6f578576

`stage('Image Build & Push (docker)')`

docker 를 통해 빌드 및 푸시 실행

예) amdp-registry.skala-ai.com/skala25a/sk000-myfirst-api-server:1.0.0-1-ea0f6f578576

`stage('GitOps update')`

새롭게 빌드된 이미지 정보로 ./k8s/deploy.yaml 갱신

업데이트된 ./k8s/deploy.yaml을 gitops branch에 push 실행

향후 argoCD에서 사용하는 IaC로 제공

[둘러보기] Jenkinsfile-CI-docker

1. Jenkinsfile 환경 구성

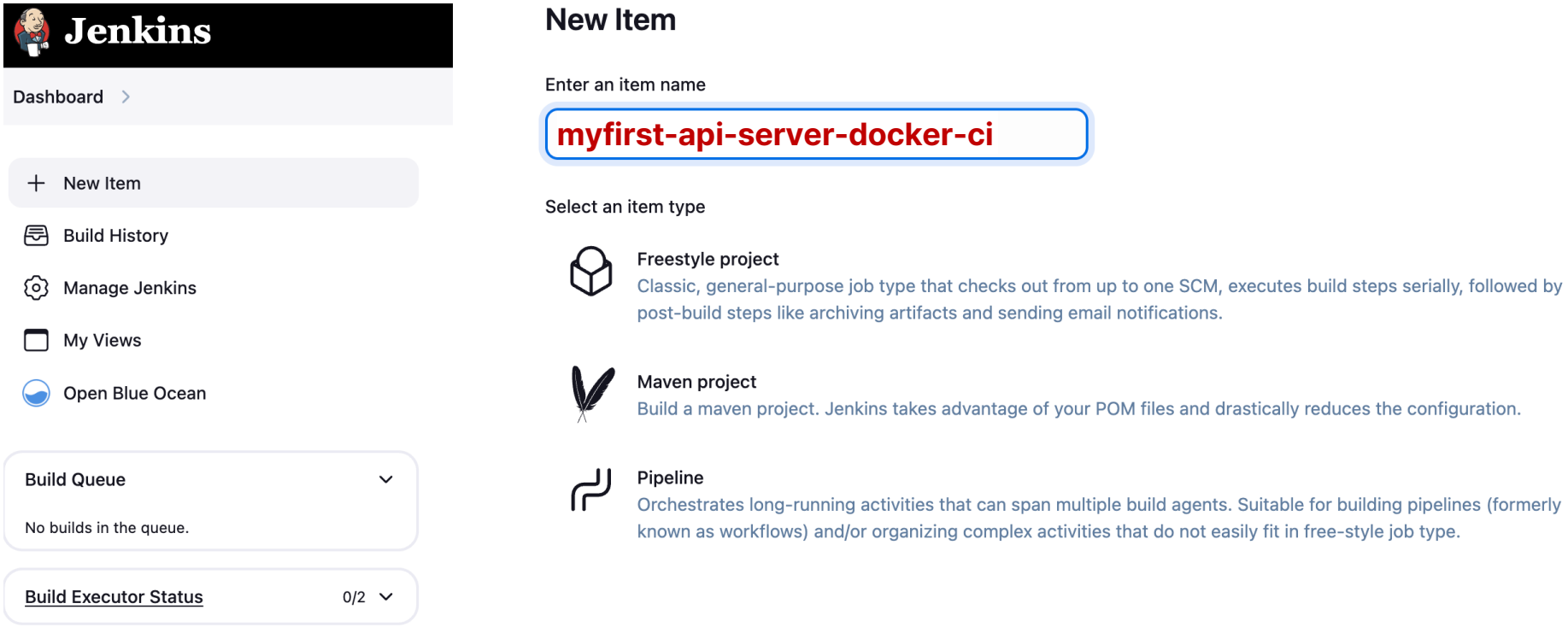
```
environment {
  // === 사용자 수정 영역 ===
  GIT_URL                = 'https://github.com/himang10/myfirst-api-server.git'
  GIT_BRANCH             = 'master'             // 또는 main
  GIT_ID                 = 'skala-github-id'    // GitHub PAT credential ID
  IMAGE_NAME             = 'skala-webserver'
  // ======================
  IMAGE_TAG              = '1.0'
  IMAGE_REGISTRY_URL     = 'skala-registry.skala-ai.com'
  IMAGE_REGISTRY_PROJECT = 'class-0'

  DOCKER_CREDENTIAL_ID   = 'skala-image-registry-id'  // Harbor 인증 정보 ID
  GITOPS_BRANCH          = 'gitops'
  GITOPS_MANIFEST_DIR    = 'kustomize/overlays/dev'
}
```

실습: CI 구성하기

실습 코드 위치 : 04.jenkins/02.local/01.local-cicd

[실습] Jenkinsfile-CI-docker 구성

각 스탭 별 트린트하는 파이프라인 구성하기



[실습] Jenkinsfile-CI-docker 등록

1단계: Pipeline script 를 적용

Jenkins / myfirst-api-server / Configuration

[메뉴] Configure 의 General
- Pipeline 의 Definition
  - "Pipeline script"
- SAVE

Definition: Pipeline script

Script

```
pipeline {
    agent any

    environment {
        GIT_URL = 'https://github.com/himang10/myfirst-api-server.git'
        GIT_BRANCH = 'master' // 또는 main
        GIT_ID = 'my-github-id' // GitHub PAT credential ID
        IMAGE_REGISTRY = 'harbor-registry-url'
        IMAGE_NAME = 'sk000-myfirst-api-server'
        IMAGE_TAG = '1.0.0'
        DOCKER_CREDENTIAL_ID = 'skala-image-registry-id'  // Harbor 인증 정보 ID
        K8S_NAMESPACE = 'skala-practice'
    }

    stages {
```

try sample Pipeline...

Use Groovy Sandbox

Pipeline Syntax

Save | Apply

[실습] Jenkinsfile-CI-docker 실행

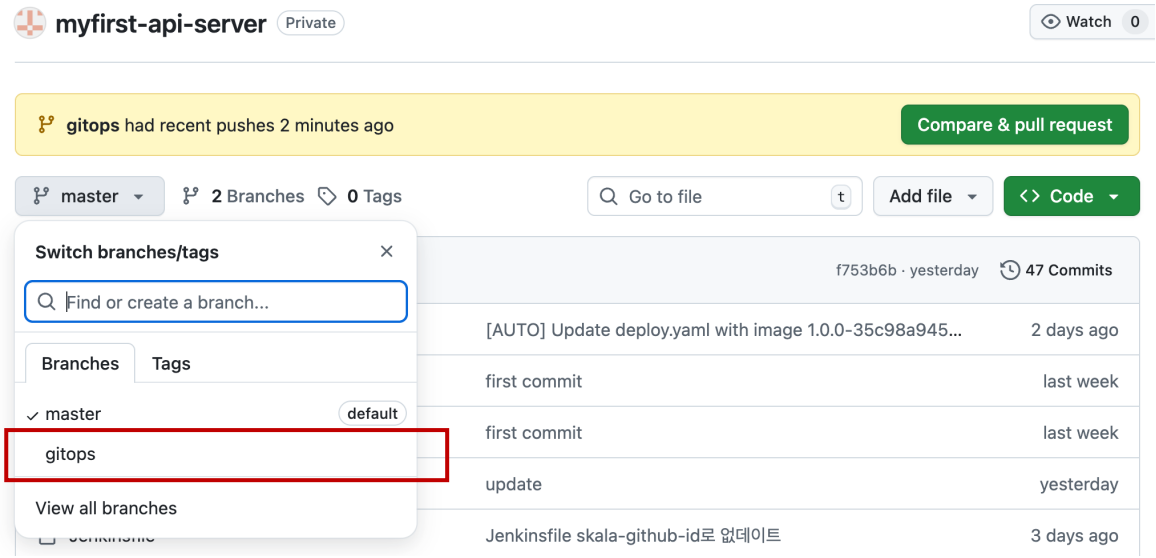
1단계: Pipeline script 를 적용

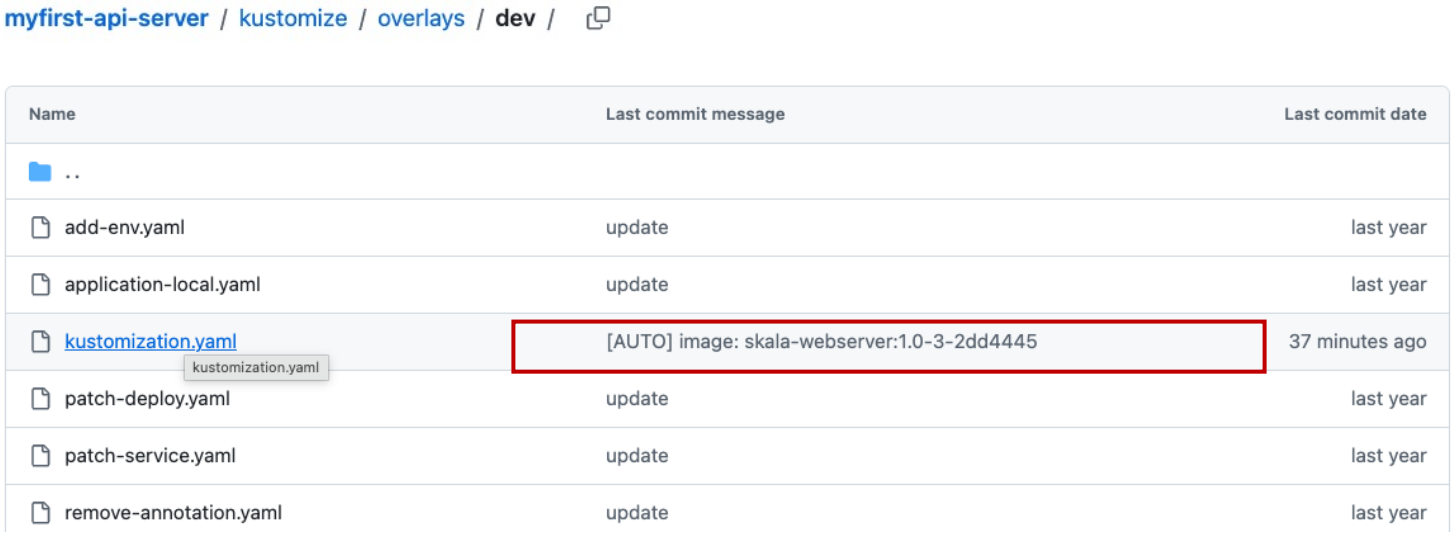
S	W	Name ↓	Last Success	Last Failure	Last Duration		F	# Issues
✓	☀	my-app	2 hr 22 min #2	N/A	6.4 sec	▷	☆	-
✓	☀	myfirst-api-server	1 hr 40 min #12	1 day 23 hr #7	37 sec	▷	☆	-
✓	☀	myfirst-api-server-docker-ci	1 min 34 sec #1	N/A	27 sec	▷	☆	-

	Clone Repository	Build with Maven	Compute Image Meta	Image Build & Push (docker)	GitOps Update	Declarative: Post Actions
Average stage times: (full run time: ~25s)	1s	5s	276ms	12s	4s	190ms
#3 Mar 25 22:27 No Changes	2s	5s	270ms	12s	8s	170ms

[실습] git repository에 push 결과 확인

gitops repository에 업데이트된 결과 확인하기





[실습] from SCM으로 동작하게 하기

아래의 Jenkinsfile을 대해 각각 “Pipeline script 방식” 과 “script from SCM” 으로 동작시켜보자

✓	☀	myfirst-api-server-buildkit-ci	3 min 12 sec #1	N/A	35 sec	▷	-
✓	☁	myfirst-api-server-docker-ci	51 min #3	57 min #2	22 sec	▷	-
✗	☁	myfrist-api-server-buildah-ci	N/A	2 min 8 sec #1	31 sec	▷	-

대상

jenkinsfile/02.local/02.local-ci/

- Jenkinsfile-CI-docker
- Jenkinsfile-CI-buildkit
- Jenkinksfile-CI-buildah

KDT DevOps – DevOps

Local Jenkins PollSCM 구성하기

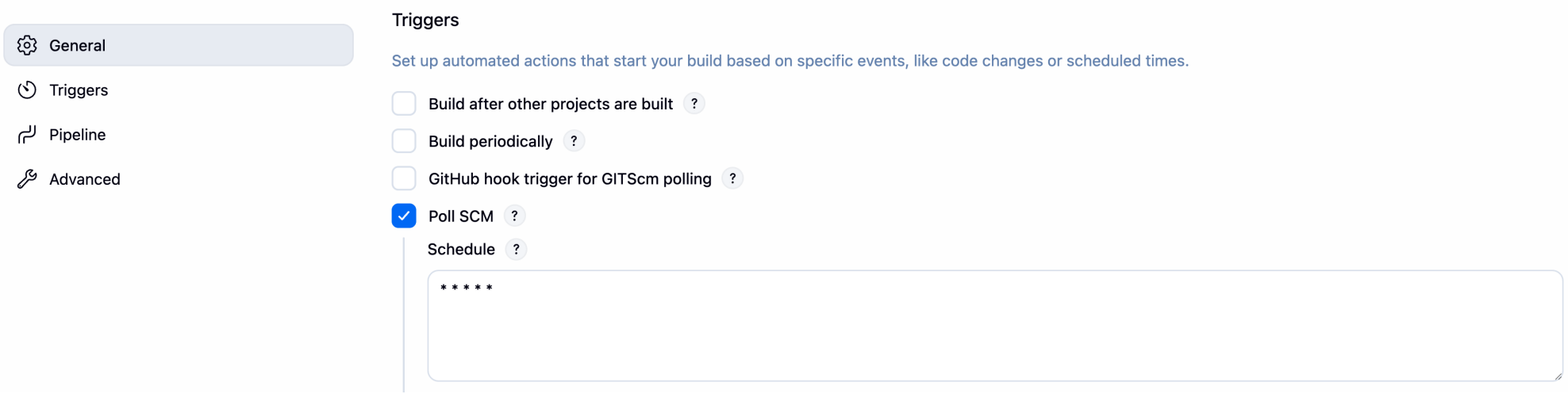
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Jenkins Build Trigger

github에 코드 변경이 될때마다 자동으로 Jenkins 파이프라인이 자동으로 실행되도록 webhook 등록하기

파이프라인에 Triggers 설정하기



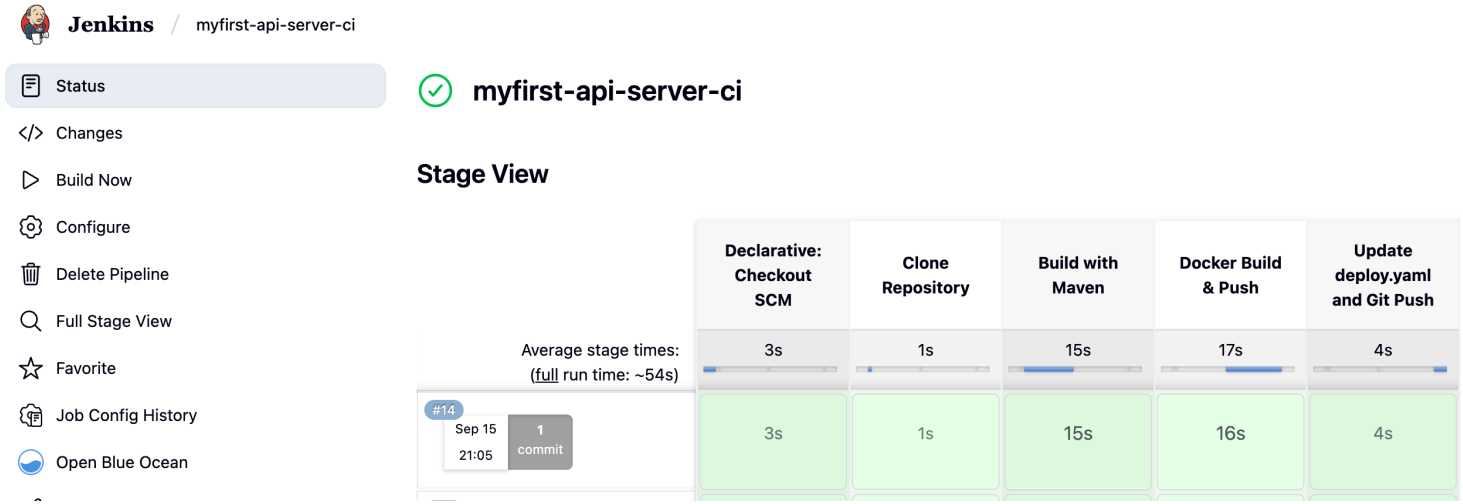

```
# 매 1분마다:
* * * * *

# 15분마다 (예: 7분, 22분, 37분, 52분):
H/15 * * * *

# 매시간 전반 30분 동안 10분 간격으로 (세 번, 예를 들어 4분, 14분, 24분에):
H(0-29)/10 * * * *

# 평일 오전 9시 45분부터 오후 3시 45분까지 매 2시간마다 매시 45분에 한 번씩:
45 9-16/2 * * 1-5

# 평일 오전 8시부터 오후 4시 사이 2시간 간격으로 한 번씩 (예: 오전 9시 38분, 오전 11시 38분, 오후 1시 38분, 오후 3시 38분):
H H(8-15)/2 * * 1-5

# 12월을 제외한 매월 1일과 15일에 하루 한 번씩:
H H 1,15 1-11 *
```

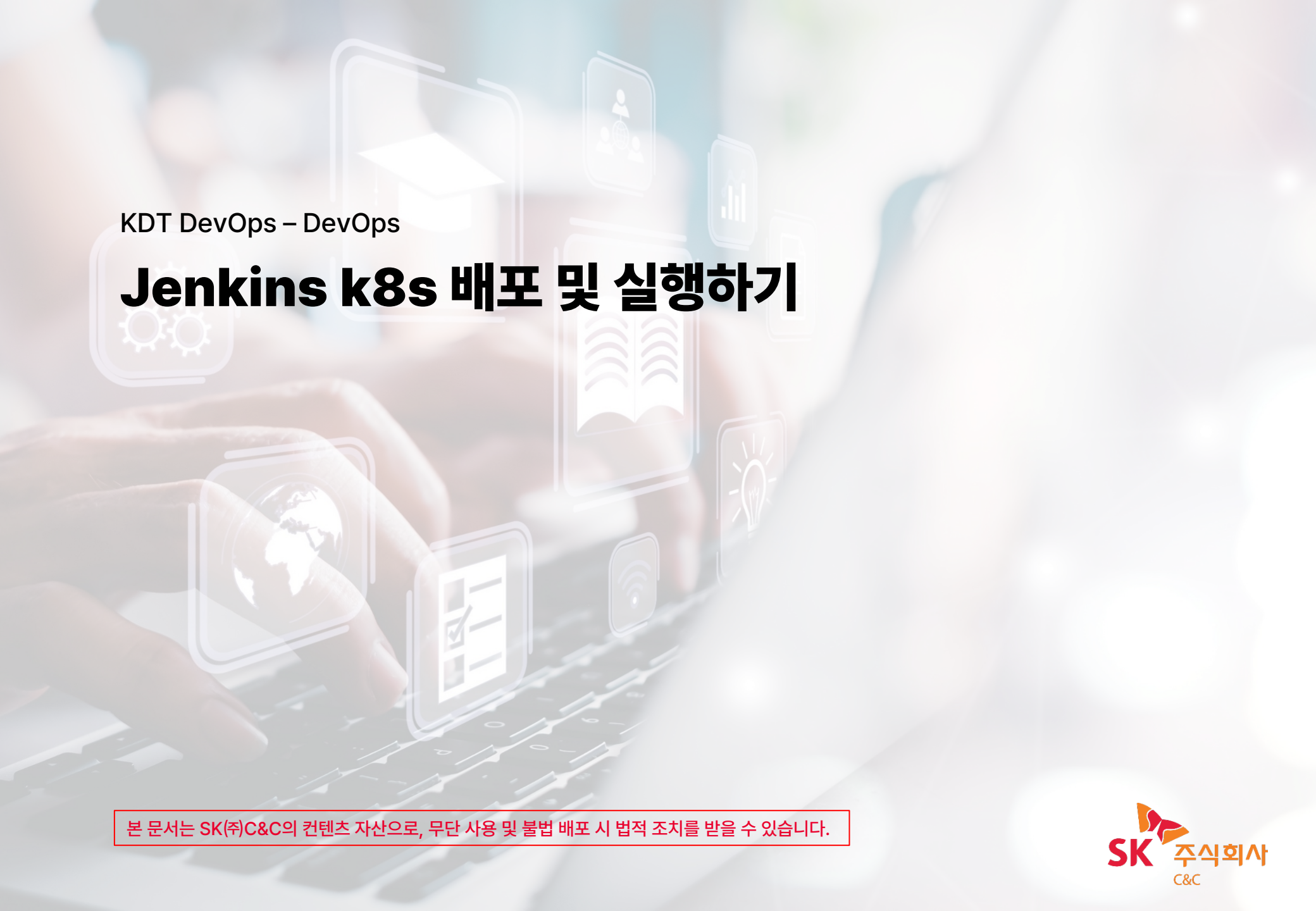
git repository에 READM.md update 확인

1. 자신의 git repository에 파일 업데이트 후 push 하기

```
echo “test README.md” >> README.md

git add README.md

git commit –m “update”

git push origin main
```

2 pollSCM 동작 확인하기 (1분 내 빌드 시작)





KDT DevOps – DevOps

Jenkins k8s 배포 및 실행하기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Kubernets에 나의 Jenkins 배포하기

자신이 가지고 있는 Jenkins를 배포해보자

위치: 04.jenkins/

조별로 한개씩 배포해보자
각 조에서는 하나의 Jenkins Server를 k8s에 배포

1. 04.jenkins 이미지를 사용

이미지 생성 과정이 시간이 많이 걸려서

`skala-registry.skala-ai.com/library/skala-custom-jenkins:1.0`

사용

2. kubernetes 환경에 배포하기

```
# 리소스 배포하기

kubectl apply -f  ./k8s
```

[참고] Kubectl config 로딩 우선 순위

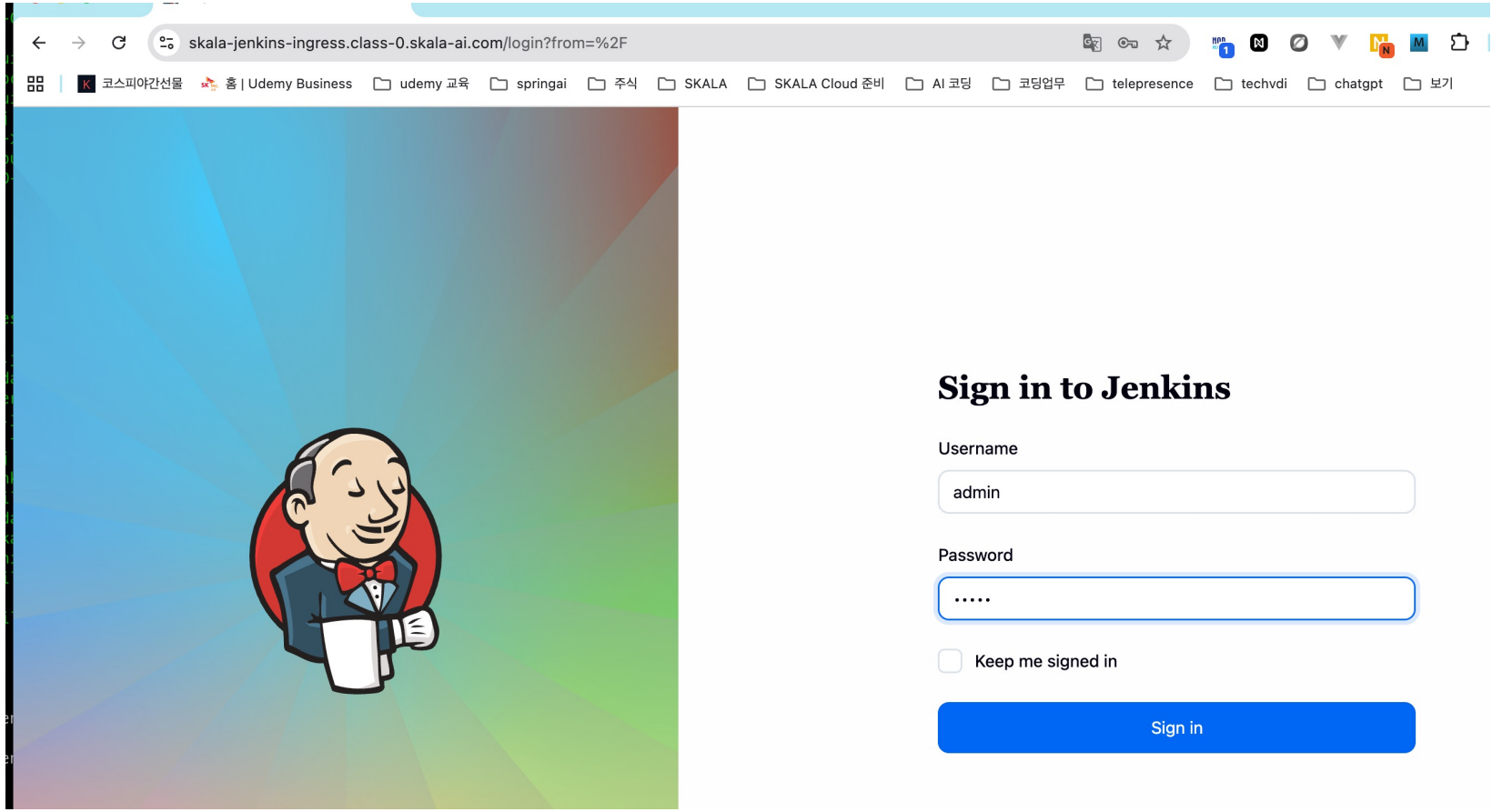
PRI	대상	설명	주 사용 환경
1	--kubeconfig 옵션	kubectl --kubeconfig /path/config ... 처럼 명령행에서 직접 지정한 kubeconfig 파일을 가장 우선 사용	로컬/CI/CD/스크립트
2	KUBECONFIG 환경변수	환경변수에 지정된 kubeconfig 파일(여러 개 가능)을 사용	로컬 터미널, Jenkins, 자동화 환경
3	~/.kube/config	기본 kubeconfig 파일. 위 두 개가 없으면 기본적으로 이것을 사용	일반적인 로컬 개발 환경
4	--context 또는 current-context	kubeconfig 파일을 찾은 뒤, 그 안에서 어떤 context를 사용할지 결정 **kubectl config current-context** **kubectl --context my-dev get pod**	여러 클러스터/사용자 전환 시
6	ServiceAccount	보통 로컬 kubectl 의 기본 우선순위는 아니고, Pod 내부에서 kubeconfig 없이 in-cluster 방식으로 접근할 때 사용 /var/run/secrets/kubernetes.io/serviceaccount/token /var/run/secrets/kubernetes.io/serviceaccount/ca.crt	Kubernetes Pod 내부

skala-admin-sa 사용

Jenkins 접속하기

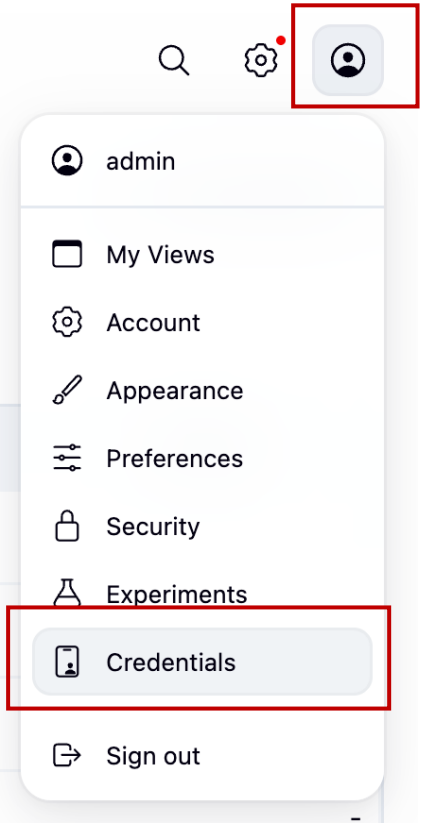
kubernetes에 배포되어 있는 Jenkin에 접속

https://sk000-jenkins-ingress.class-0.skala-??.com

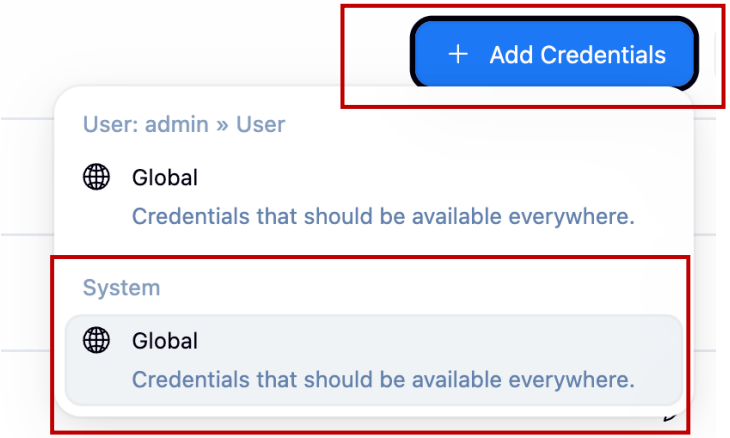


자신의 github-id 등록하기

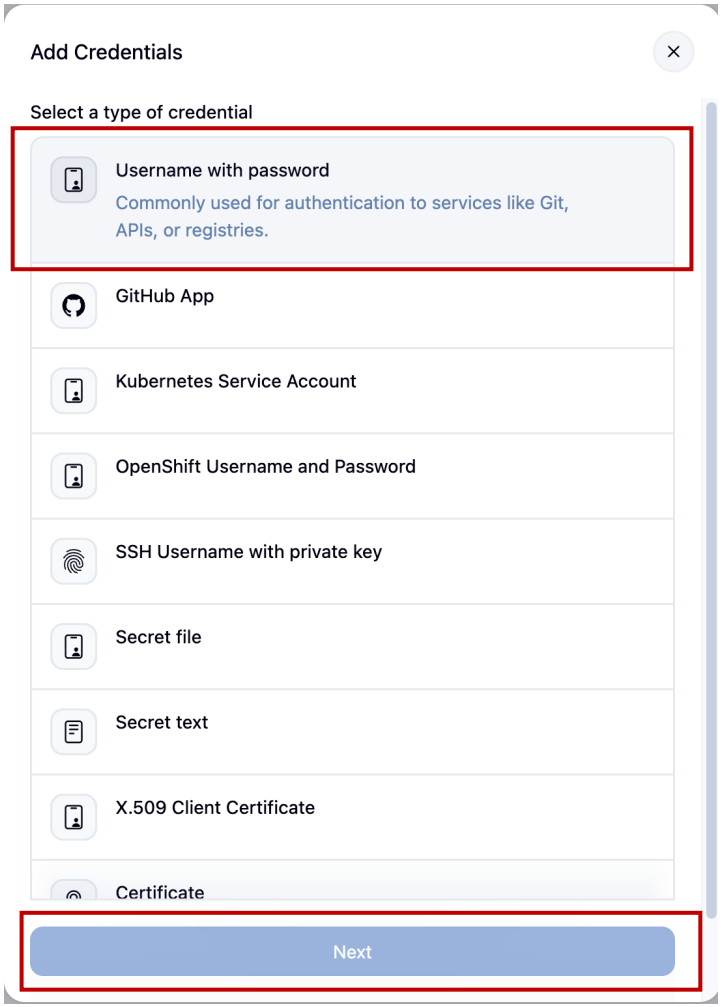
1. 오른쪽 계정 아이콘의 Credentials 선택



2. 오른쪽 Add Credentials 선택 후 System/Global 선택

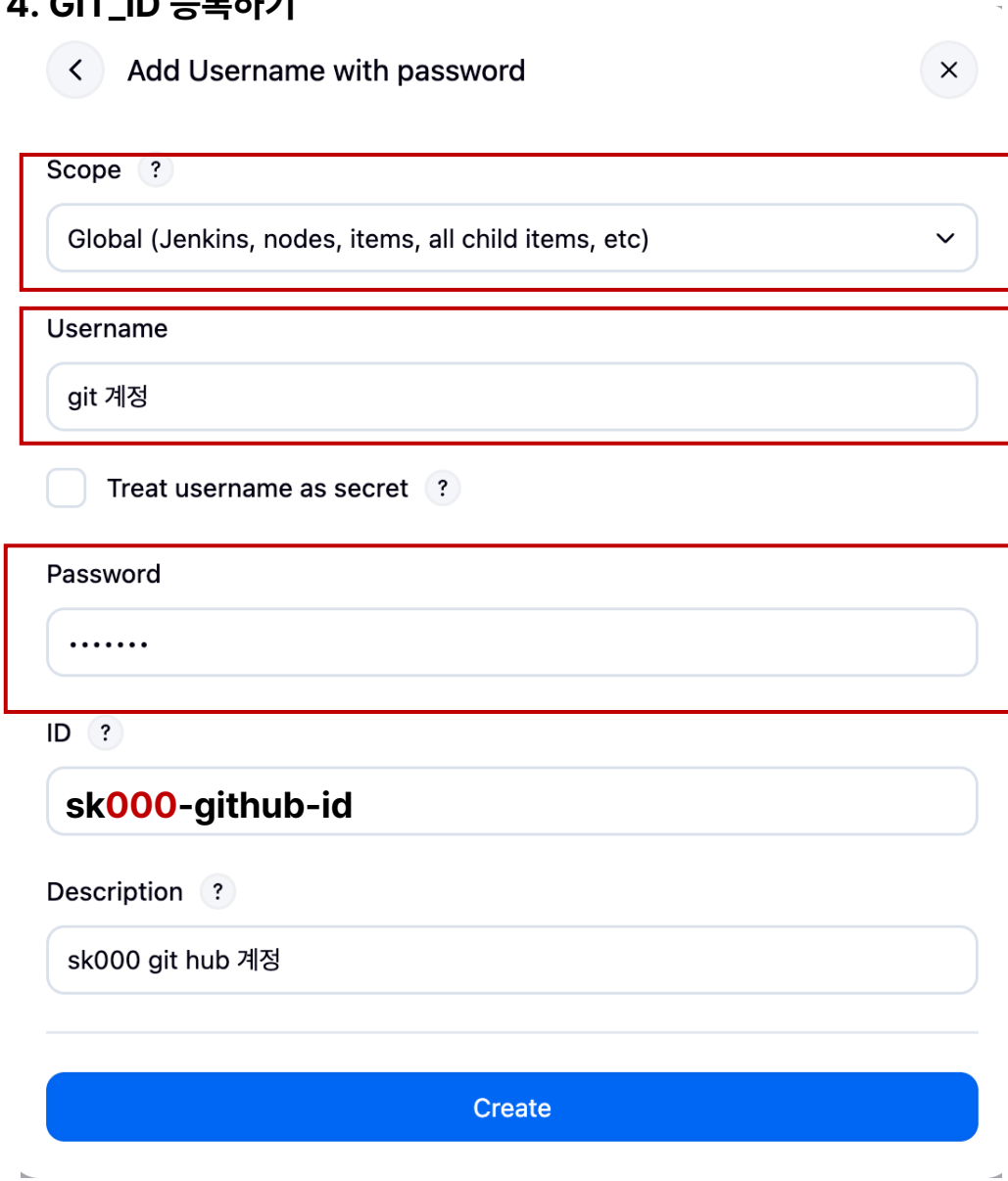


3. Username with password 선택



자신의 github-id 등록하기

4. GIT_ID 등록하기

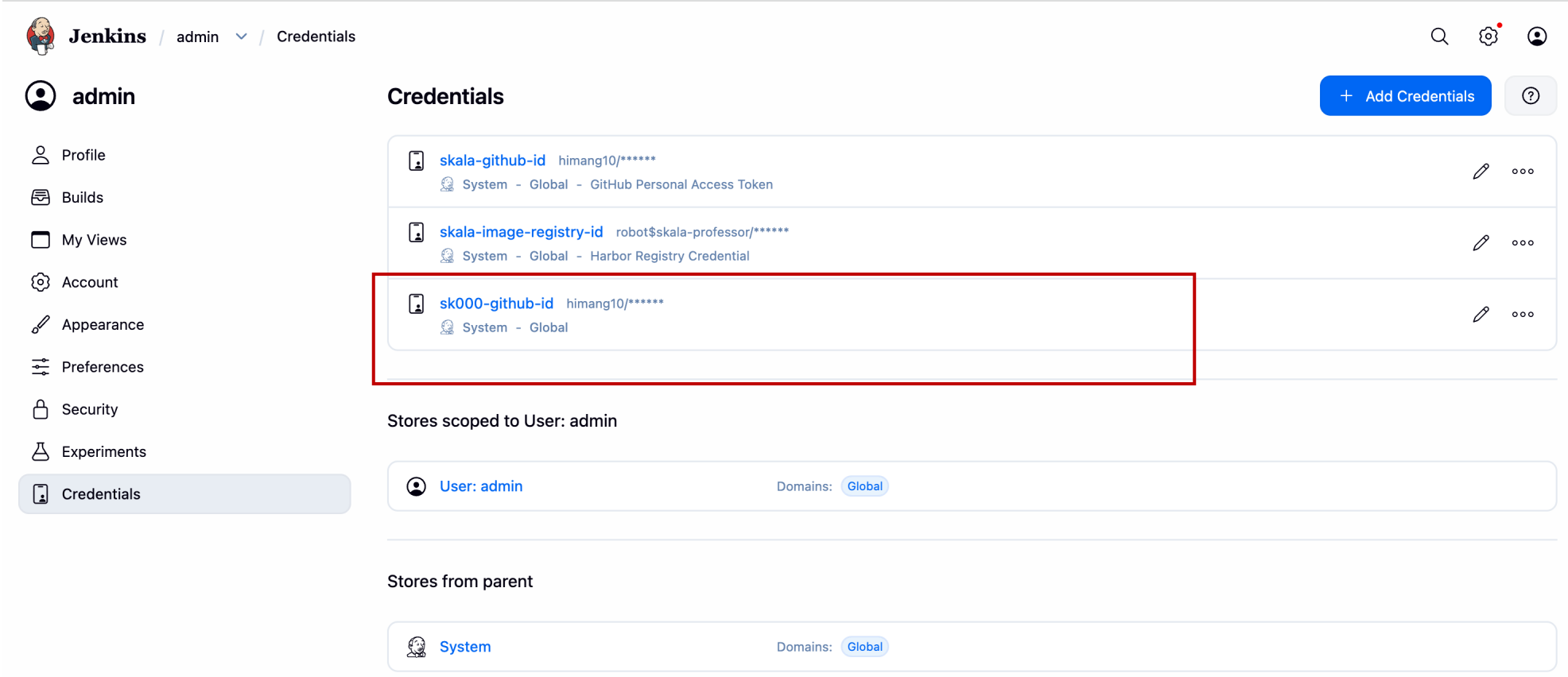


myfirst-api-server 에 대한 github 계정 작성

Personal Access tokens 을 여기에 복사

Jenkinsfile 내에서 사용하기 위한 GIT_ID

자신의 github-id 등록하기



Jenkins 파이프라인 실행하기

Jenkins Controller : Buildah CICD

CICD buildah 파이프라인을 등록 후 실행해보자

```
# jenkins controller에서 동작하는 buildah CICD 실행
jenkinsfile/02.local/01.local-cicd/Jenkinsfile-CICD-buildah
```

```
environment {
  // === 사용자 수정 영역 ===
  GIT_URL                = 'https://github.com/himang10/myfirst-api-server.git'
  GIT_BRANCH             = 'master'            // 또는 main
  GIT_ID                 = 'skala-github-id'   // GitHub PAT credential ID
  IMAGE_NAME             = 'skala-webserver'
  // ======================
  IMAGE_TAG              = '1.0'
  IMAGE_REGISTRY_URL     = 'skala-registry.skala-ai.com'
  IMAGE_REGISTRY_PROJECT = 'class-0'

  DOCKER_CREDENTIAL_ID   = 'skala-image-registry-id'  // Harbor 인증 정보 ID
  K8S_NAMESPACE          = 'class-0'
  DEPLOYMENT_NAME        = 'skala-myfirst-api-server'  // kustomize/base/deployment.yaml의 metadata.name과 동일해야 함
}
```

앞에서 등록한 sk000-github-id를 여기에 등록하기

Stage View

	Clone Repository	Build with Maven	Image Build & Push (Buildah)	Update K8s Manifests	Deploy to Kubernetes	Declarative: Post Actions
Average stage times: (full run time: ~1min 27s)	4s	31s	26s	528ms	17s	1s
#2 Oct 11 17:39 No Changes	4s	16s	20s	524ms	15s	1s
#1 Oct 11 17:34 No Changes	5s	46s	32s	533ms	19s	1s

Jenkins 파이프라인 실행하기

Jenkins Controller : Buildkit CI

CI buildkit 파이프라인을 등록 후 실행해보자

```
# jenkins controller에서 동작하는 buildah CI 실행
jenkinsfile/02.local/02.local-ci/Jenkinsfile-CI-buildkit
```

```
environment {
  // === 사용자 수정 영역 ===
  GIT_URL                = 'https://github.com/himang10/myfirst-api-server.git'
  GIT_BRANCH             = 'master'            // 또는 main
  GIT_ID                 = 'skala-github-id'   // GitHub PAT credential ID
  IMAGE_NAME             = 'skala-webserver'
  // ======================
  IMAGE_TAG              = '1.0'
  IMAGE_REGISTRY_URL     = 'skala-registry.skala-ai.com'
  IMAGE_REGISTRY_PROJECT = 'class-0'

  DOCKER_CREDENTIAL_ID   = 'skala-image-registry-id'  // Harbor 인증 정보 ID
  K8S_NAMESPACE          = 'class-0'
  DEPLOYMENT_NAME        = 'skala-myfirst-api-server'  // kustomize/base/deployment.yaml의 metadata.name과 동일해야 함
}
```

앞에서 등록한 sk000-github-id를 여기에 등록하기

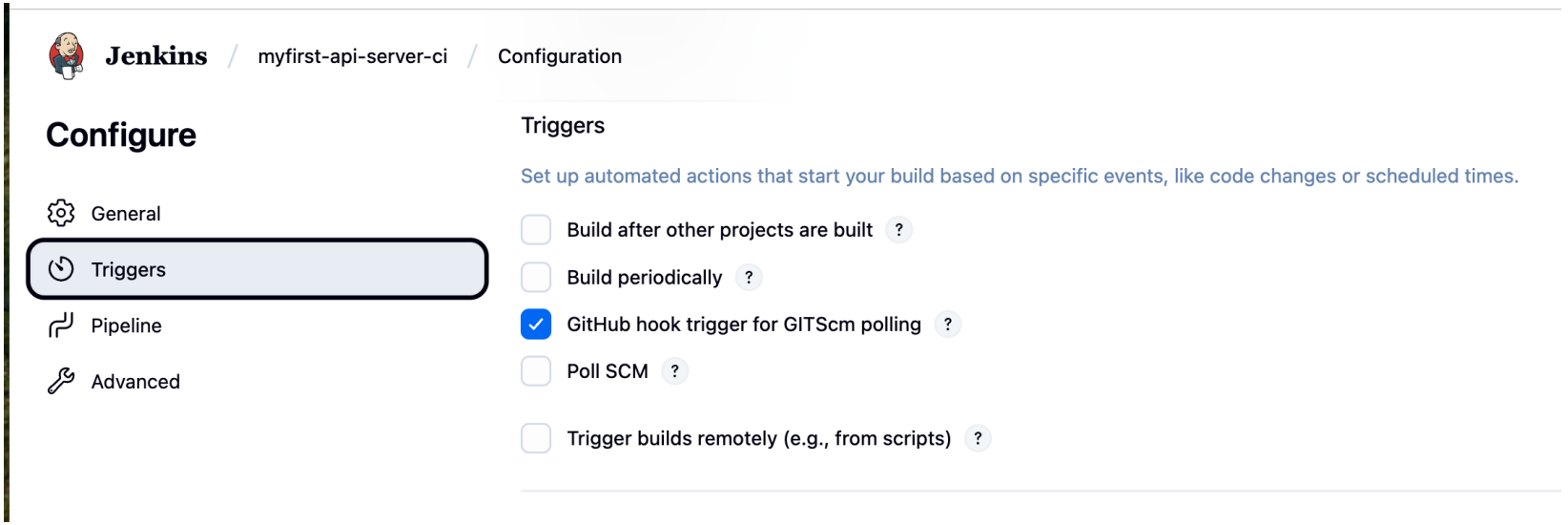
Stage View

	Clone Repository	Build with Maven	Image Build & Push (BuildKit)	GitOps Update	Declarative: Post Actions
Average stage times: (full run time: ~1min 0s)	4s	16s	30s	4s	1s
#1 Oct 11 17:42 No Changes	4s	16s	30s	4s	1s

Jenkins Build Trigger

github에 코드 변경이 될때마다 자동으로 Jenkins 파이프라인이 자동으로 실행되도록 webhook 등록하기

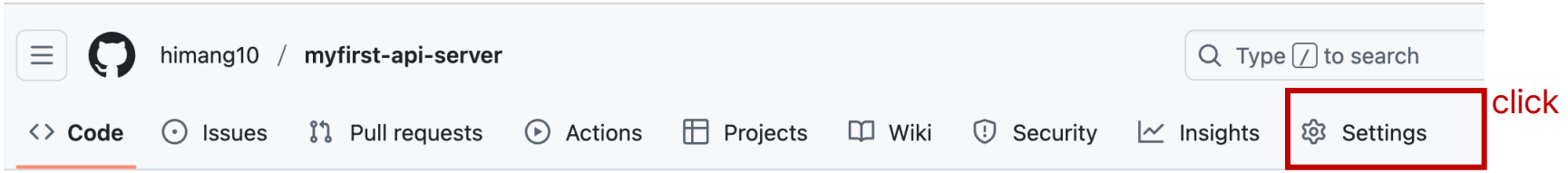
파이프라인에 Triggers 설정하기



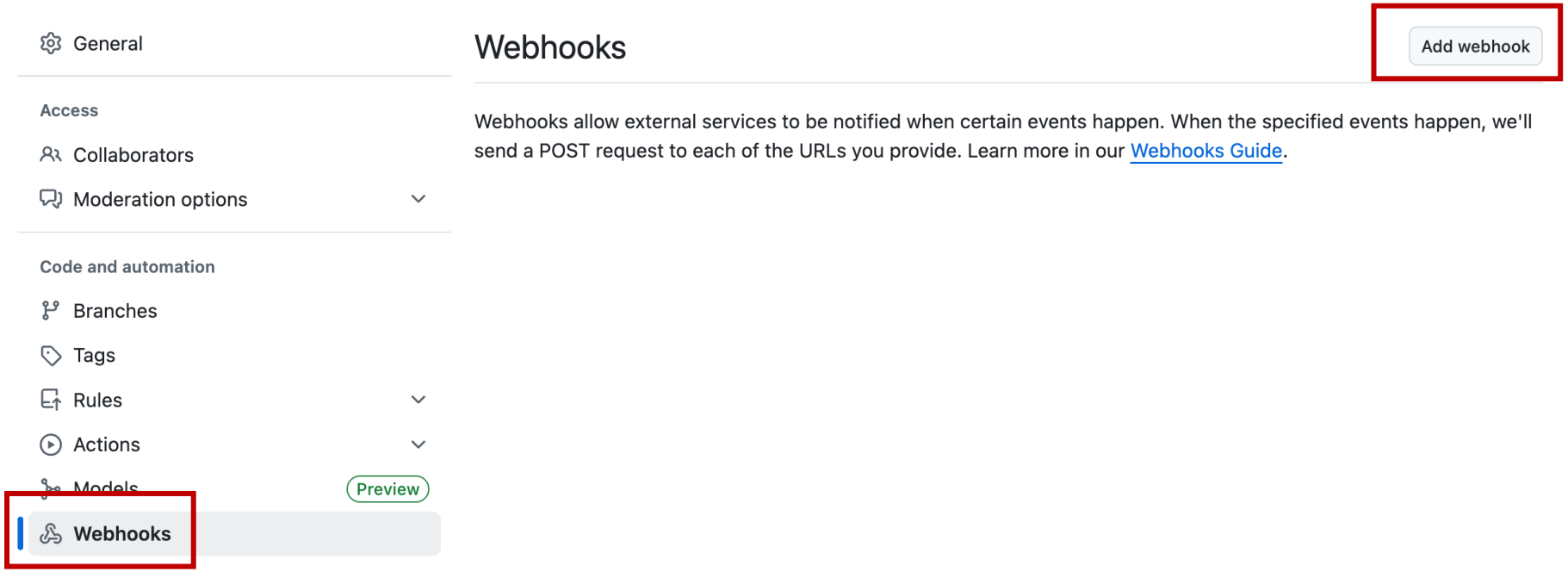
github에 webhook 등록하기

github

1. git repository의 setttings



1. webhooks 선택 및 Add webhook



github에 webhook 등록하기

github

Payload URL *

반드시 githbu—webhook 다음에 "/" 을 넣어야 합니다..

https://sk000-Jenkins-ingress.**class-0**.skala-**??**.com/github-webhook**/**

Content type *

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

Enable SSL verification Disable (not recommended)

Which events would you like to trigger this webhook?

Just the push event.

Send me **everything**.

Let me select individual events.

Active

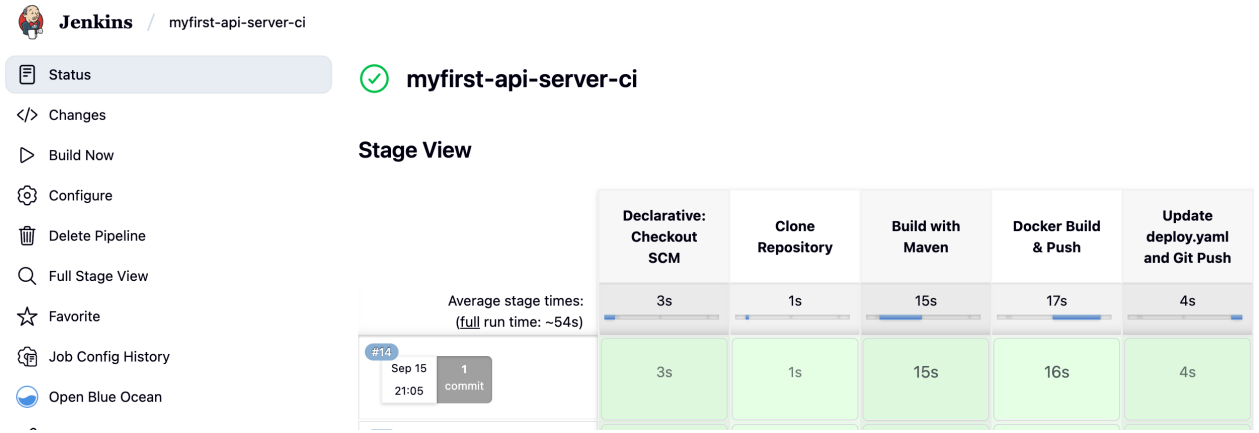
We will deliver event details when this hook is triggered.

Add webhook

git repository에 READM.md update 확인

1, Jenkins pipeline 확인하기

https://sk000-jenkins-ingress.class-0.skala-??.com



2. 자신의 git repository에 파일 업데이트 후 push 하기

```
echo “touch README.md” >> README.md

git add README.md

git commit -m “update”

git push origin main
```

KDT DevOps – DevOps

GitOps 기반 CD

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

개발자 중심 , 즉 개발자가 스스로 운영

기존 운영 방식의 문제

- **수동 배포**: 콘솔 클릭, 쉘 스크립트 실행 → 사람에 의존 → 오류 잦음.
- **불일치**: 개발 코드와 실제 운영 환경 설정이 다르게 흘러감 → 재현 어려움.
- **이력 추적 부재**: 운영자가 직접 수정하면 누가, 언제, 무엇을 바꿨는지 알기 어려움.
- **복잡성 증가**: 멀티 클러스터, 마이크로서비스 환경에서는 수동 관리가 불가능에 가까움.

Git의 특징

- **버전 관리**: 모든 변경사항을 Commit 기록으로 남김.
- **협업 표준**: 개발자들이 이미 익숙하게 사용.
- **SSOT(Single Source of Truth)**: Git에 정의된 것이 곧 진실 → 운영 환경은 Git과 동기화.
- **Rollback 용이**: 문제가 생기면 Git의 이전 버전으로 되돌리면 됨.

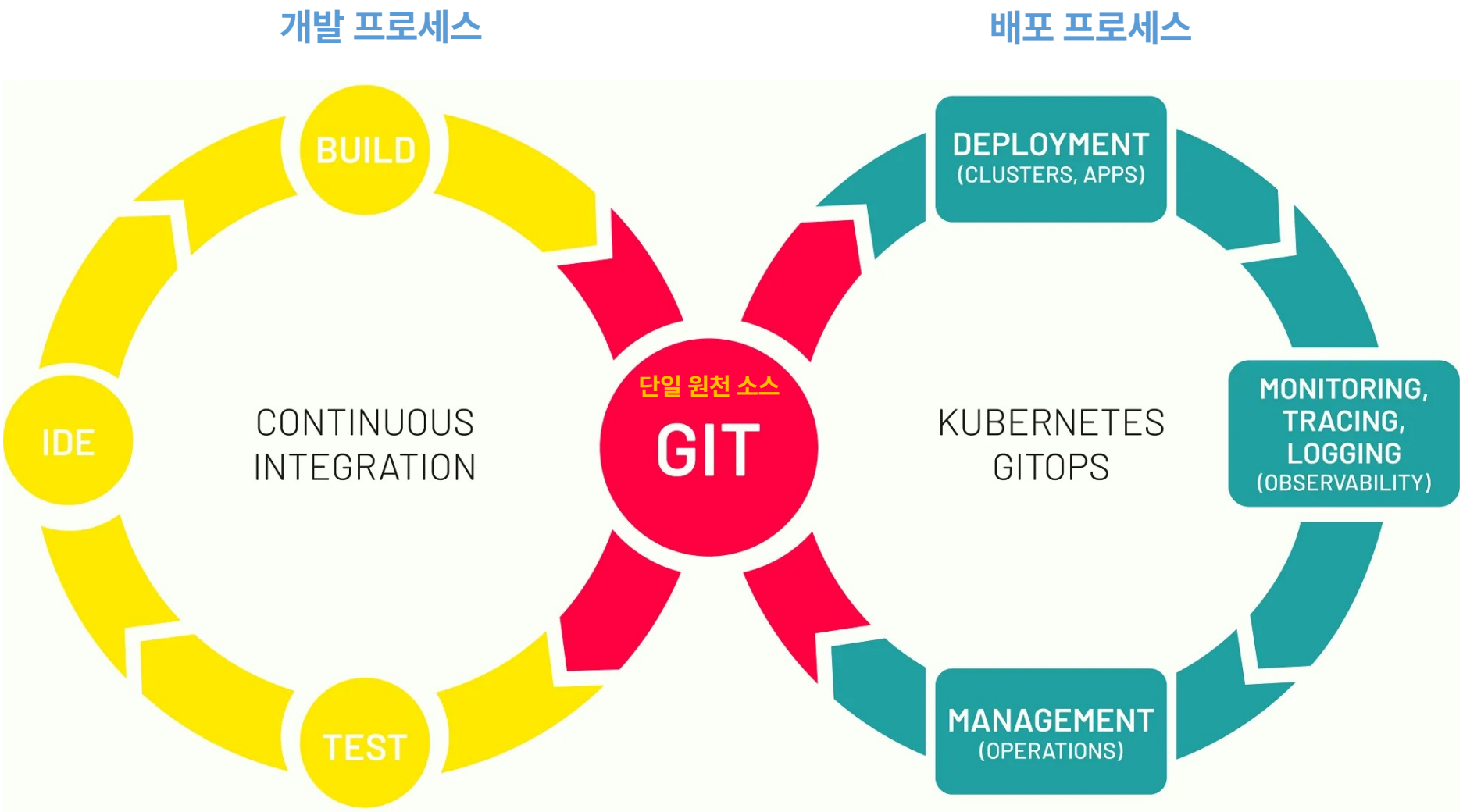
운영에 Git을 도입한 목적

목적: 운영 환경을 개발 코드처럼 관리해 **일관성과 추적성을 확보하고 자동화를 가능**하게 하기 위해

- **운영 환경을 코드화(IaC)** 하여 Git에 저장 → 선언적으로 관리.
- **운영 변경도 PR/리뷰 프로세스**를 거쳐 투명하고 안전하게.
- **자동화 도구(ArgoCD, Flux)**가 Git 상태와 실제 클러스터 상태를 자동으로 동기화

GitOps 란?

- Git을 통해 코드와 인프라를 함께 관리하고, 선언적 형상 관리로 개발과 운영을 잇는 새로운 방식



GitOps

Git을 기반으로 Operation을 자동화하는 경우 다음 6가지 측면에서 장점을 가질 수 있습니다

생산성
productivity

CICD를 단순화해서 배포 시간 최소화.
개발자가 Git에 코드를 커밋하면 자동으로 빌드, 배포까지 진행

향상된 사용자 경험
enhanced Experience

사용자는 git이라는 익숙한 툴을 통해 복잡한 시스템을 쉽게 다룰 수 있도록 지원
사용자는 git push 만으로 배포 완료

```
git commit –m “update config”
git push
```

안정성
Stability

모든 변경 사항은 git 하나에서만 이루어지므로 충돌 없이 시스템이 항상 일관된 상태를 유지하도록 지원
예) 직접 서버에 로그인해서 설정 수정하면, 누가 뭘 바꿨는지 알기 어렵고 충돌 가능성 발생. 그러나, git만을 통해 변경하므로 이런 충돌이 사람짐

신뢰성
Stability

문제 발생 시 git의 이전 커밋으로 롤백하면 자동적으로 그전 안정 상태로 자동 복구

예) 서비스 설정 실수로 인한 장애 시

```
git revert HEAD
git push
```

일관성
Consistency

변경을 위한 하나의 진입점
모든 것은 git 저장소의 커밋을 통해 구동
모든 것이 한 곳에서 설정이 되는 구조
Single Source of Truth (단일 진실 소스)

이를 통해 모든 변경이 기록, 추적, 자동화

보안 및 감사
Security / Audit

git은 보안과 서명 기능을 지원
누가 언제 어떤 변경을 했는지 추적 용이
→ 감사 로그 자동 제공

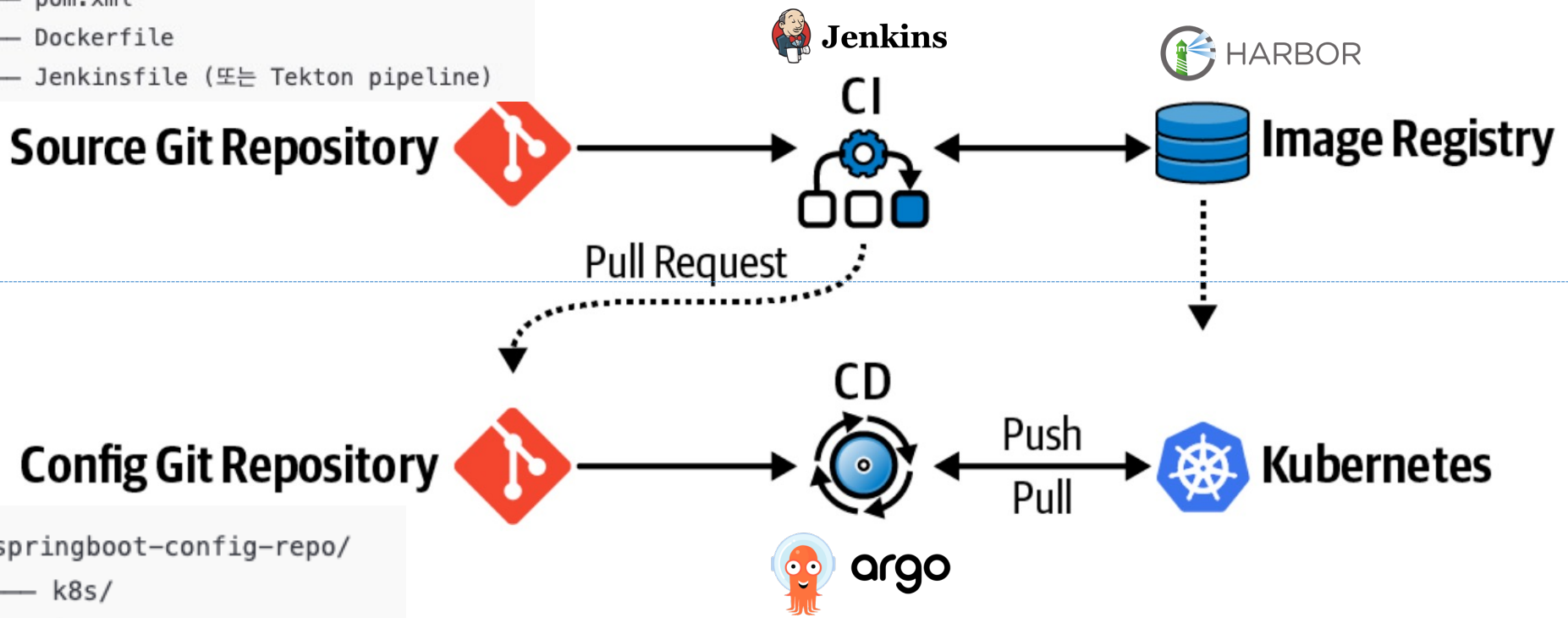
ArgoCD 란?

Kubernetes 환경에서 GitOps 방식으로 애플리케이션 배포를 자동화하고 관리하는 도구

Git에 저장된 원하는 상태 (Desired State)와 실제 K8S Cluster의 상태 (Acutal State)를 지속적으로 비교하고 둘을 일치시키는 도구

```
springboot-app-repo/
├── src/
│   └── main/java/...
├── pom.xml
├── Dockerfile
└── Jenkinsfile (또는 Tekton pipeline)
```

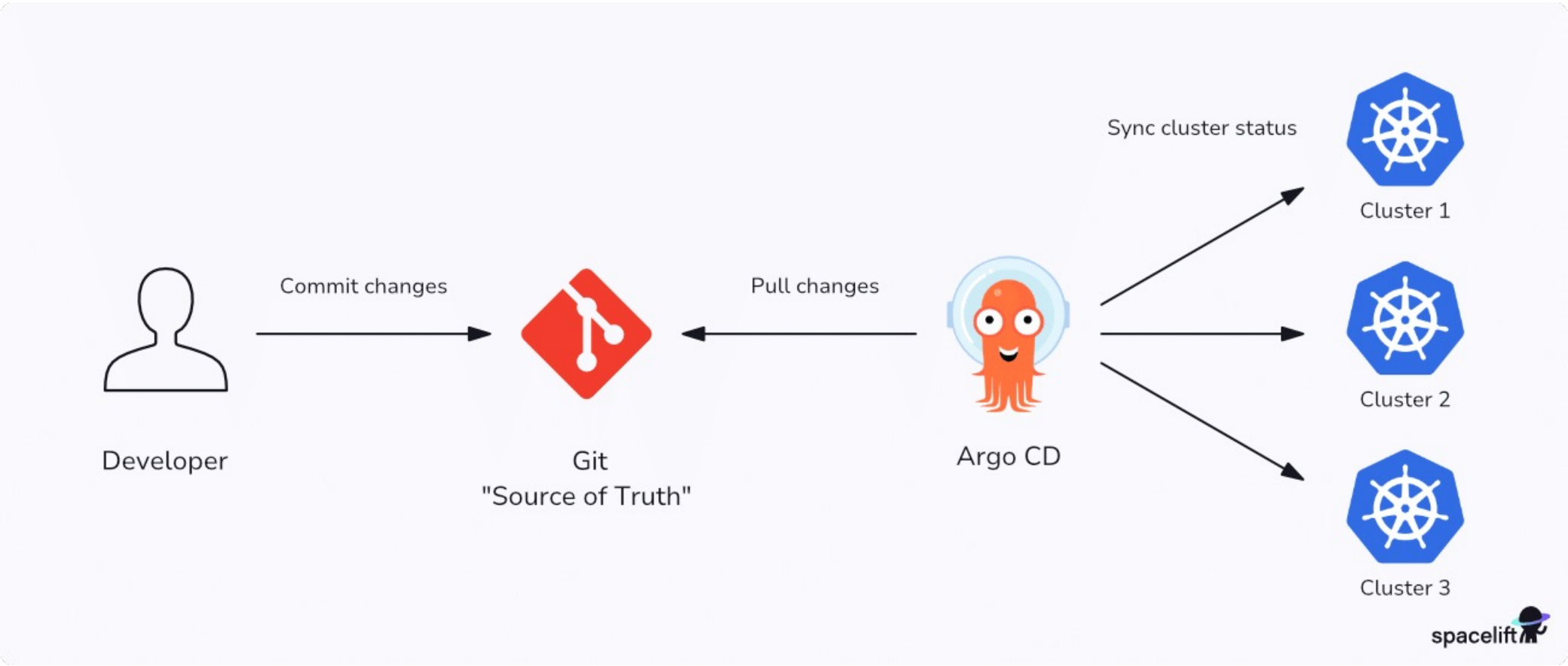
Application deployment model




```
springboot-config-repo/
└── k8s/
    ├── deployment.yaml
    ├── service.yaml
    └── ingress.yaml
```

ArgoCD 기반 Multi-Cluster 배포

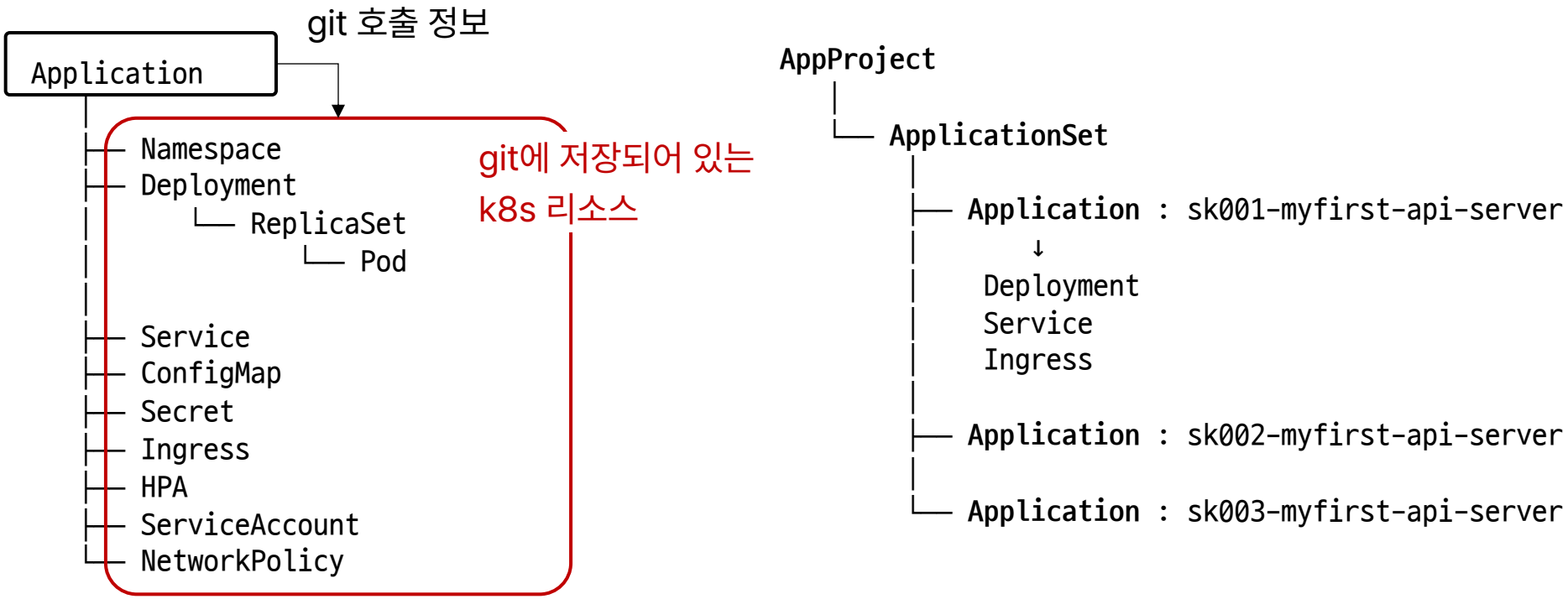
GitOps 솔루션 ArgoCD



ArgoCD Custom Resource Definition (CRD)

ArgoCD에서 정의하고 있는 사용자 정의 리소스 유형

리소스	역할	주요 용도
Application	하나의 애플리케이션 배포 단위	Git → Cluster 동기화
ApplicationSet	여러 Application을 템플릿으로 자동 생성	여러 Application 자동 생성
AppProject	Application들을 논리적으로 묶고 권한/배포 대상 제한	허용 Repo, Cluster, Namespace, Resource, Role 정의



ArgoCD 주요 Resource

ArgoCD에서 정의하고 있는 사용자 정의 리소스 유형

구분	k8s Resource	역할	주요 용도
Cluster	Secret	배포 대상 Kubernetes Cluster	API Server 주소, 인증정보 저장
Repository	Secret	Git/Helm Repository	Git URL, ID/PW/Token/SSH Key 저장

Cluster

```
apiVersion: v1
kind: Secret
metadata:
  name: cluster-skala-2026
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: cluster
type: Opaque
stringData:
  name: skala-2026
  server: https://kubernetes.default.svc
  config: |
    {
      "tlsClientConfig": {
        "insecure": false
      }
    }
```

Git Repository

```
apiVersion: v1
kind: Secret
metadata:
  name: class-0-repo
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
type: Opaque

stringData:
  name: class-0-repo
  project: class-0
  url: https://github.com/himang10/xxx.git

  username: git-user
  password: github-token
```

Custom Resource : AppProject

- Application들을 그룹화하여 공통 정책과 권한 범위를 적용하는 리소스

```yaml
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: class-0
  namespace: argocd
spec:
  clusterResourceBlacklist:
  - group: rbac.authorization.k8s.io
    kind: ClusterRoleBinding
  - group: rbac.authorization.k8s.io
    kind: ClusterRole
  destinations:
  - namespace: class-0
    server: https://kubernetes.default.svc
  - name: skala-2026
    server: https://xxxx.eks.amazonaws.com
    namespace: class-0-*
  namespaceResourceBlacklist:
  - group: rbac.authorization.k8s.io
    kind: RoleBinding
  - group: '*'
    kind: Role
  sourceRepos:
  - '*'
```

주요 구성 요소

sourceRepos

- 어떤 git/Helm Repo에서 소스를 가져올 수 있는지 제안
- '*' 는 모두 가능
- 'https://github.com/회사명/* ' 적용 가능

destination

- 어떤 k8s Cluster, Namespace에 배포할 수 있는지 제한
- local cluster 포함한 외부 클러스터 등록 가능
- 외부 클러스터 등록시에는 Secret Cluster 정보 등록 필요
- name 과 server 중 한개 이상 지정 필요

clusterResourceBlackList

- ClusterRole, CRD 처럼 클러스터 전체 범위의 리소스 중 생성하면 안되는 것을 지정

namespaceResourceBlackList

- Role, RoleBinding처럼 Namespace 범위의 리소스 중 생성하면 안 되는 것 지정

ArgoCD Application (1/3)

- ArgoCD에서 하나의 배포 단위(Deployment Unit) 를 의미
- git repo에 선언된 k8s 메니페스트(deploy, service, ingress, pvc etc) 와 실제 k8s 클러스터에 리소스를 연결해 주는 객체

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
  finalizers:
  - resources-finalizer.argocd.argoproj.io
spec:
  project: class-0
  source:
    repoURL: https://github.com/example/my-repo.git
    path: k8s
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: class-0
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: false
    syncOptions:
    - PrunePropagationPolicy=background
    - Replace=true
```

주요 구성 요소

project

- AppProject 이름

source (소스)

- **repoURL**: git repo, helm repo를 지정
- path: repo 내 path 정보
  - kustomiz/overlays/dev 또는 k8s
- targetRevision: git branch 정보 (앞의 gitops)

destination (대상)

- 배포할 Kubernetes 클러스터와 namespace를 지정

ArgoCD Application (2/3)

ArgoCD의 동기화 정책 (.spec.syncPolicy)은 Manual (기본) 또는 Automatic으로 설정 지원

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
  finalizers:
  - resources-finalizer.argocd.argoproj.io
spec:
  project: skala
  source:
    repoURL: https://github.com/example/my-repo.git
    path: k8s
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: my-namespace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: false
    syncOptions:
    - Validate=false
    - ApplyOutOfSyncOnly=true
    - PrunePropagationPolicy=background
```

1. Manual Sync

spec.syncPolicy: {}

2. Automatic Sync

automated 필드 추가시 자동 동기화 실행함

spec.syncPolicy.automated

- **prune** : true
  Git에 없는 리소스는 k8s에서 자동 삭제
- **selfHeal** :true
  클러스터 내 리소스가 변경되었을 때 Git 상태로 자동 복원
  예) git - replicas: 2, k scale deploy myapp --relicas=5
  ??

- **allowEmpty** (default: false)
  - application path 전체 삭제 시 차단
  - path 전체 (k8s or kustomize/overlays/dev) 삭제 시 어떻게 할 것인가?
  false이면 prune과 상관없이 유지

ArgoCD Application (3/3)

ArgoCD의 동기화 정책 (.spec.syncPolicy)은 Manual (기본) 또는 Automatic으로 설정 지원

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
  finalizers:
  - resources-finalizer.argocd.argoproj.io
spec:
  project: skala
  source:
    repoURL: https://github.com/example/my-repo.git
    path: k8s
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: my-namespace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: false
    syncOptions:
    - ApplyOutOfSyncOnly=true
    - PrunePropagationPolicy=background
```

3. Sync Options

k8s 리소스와 sync 시 세부동작 방식을 조정하는 옵션

- **Apply Out Of Sync Only**
  - true이면 변경된 리소스만 apply 실행
  - default = false

- **Prune Propagation Policy**
  - forground: 부모 삭제전에 자식부터 삭제 (일관성 보장)
  - default=foreground
  - kubectl 에서

```
# Foreground 방식 (자식 먼저 삭제, 부모는 마지막)
kubectl delete deploy my-app --cascade=background
# Foreground 방식 (부모 먼저 삭제, 자식는 백그라운드로)
kubectl delete deploy my-app --cascade=background
# Foreground 방식 (부모만 삭제, 자식은 orphan 리소스로 남김)
kubectl delete deploy my-app --cascade=orphan
```

Background가 유용한 경우

- 리소스가 많아서 삭제 완료 대기 시간이 긴경우
- 빠르게 다음 작업으로 넘어가야 할 경우
- 자식 리소스가 잠깐 남아 있어도 문제가 없는 경우

ArgoCD ApplicationSet

- ApplicationSet은 여러 개의 Application을 자동 생성·관리 해주는 상위 리소스

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: sk000-my-app
  namespace: skala-argocd
spec:
  generators:
  - list:
      elements:
      - environment: dev
        path: k8s/dev
        server: https://aaa.eks.amazonaws.com
        namespace: sakala-dev
      - environment: prod
        path: k8s/prod
        server: https://ccc.eks.amazonaws.com
        namespace: sakala-prod
  template:
    metadata:
      name: sk000-my-app-{{environment}}
      namespace: skala-argocd
    spec:
      destination:
        namespace: '{{namespace}}'
        server: '{{server}}'
      project: skala
      source:
        path: '{{path}}'
        repoURL: https://github.com/himang10/myapp.git
        targetRevision: gitops
```

주요 구성 요소

generators

여러 개의 Application을 만들기 위한 입력 데이터 목록을 만들어내는 부분

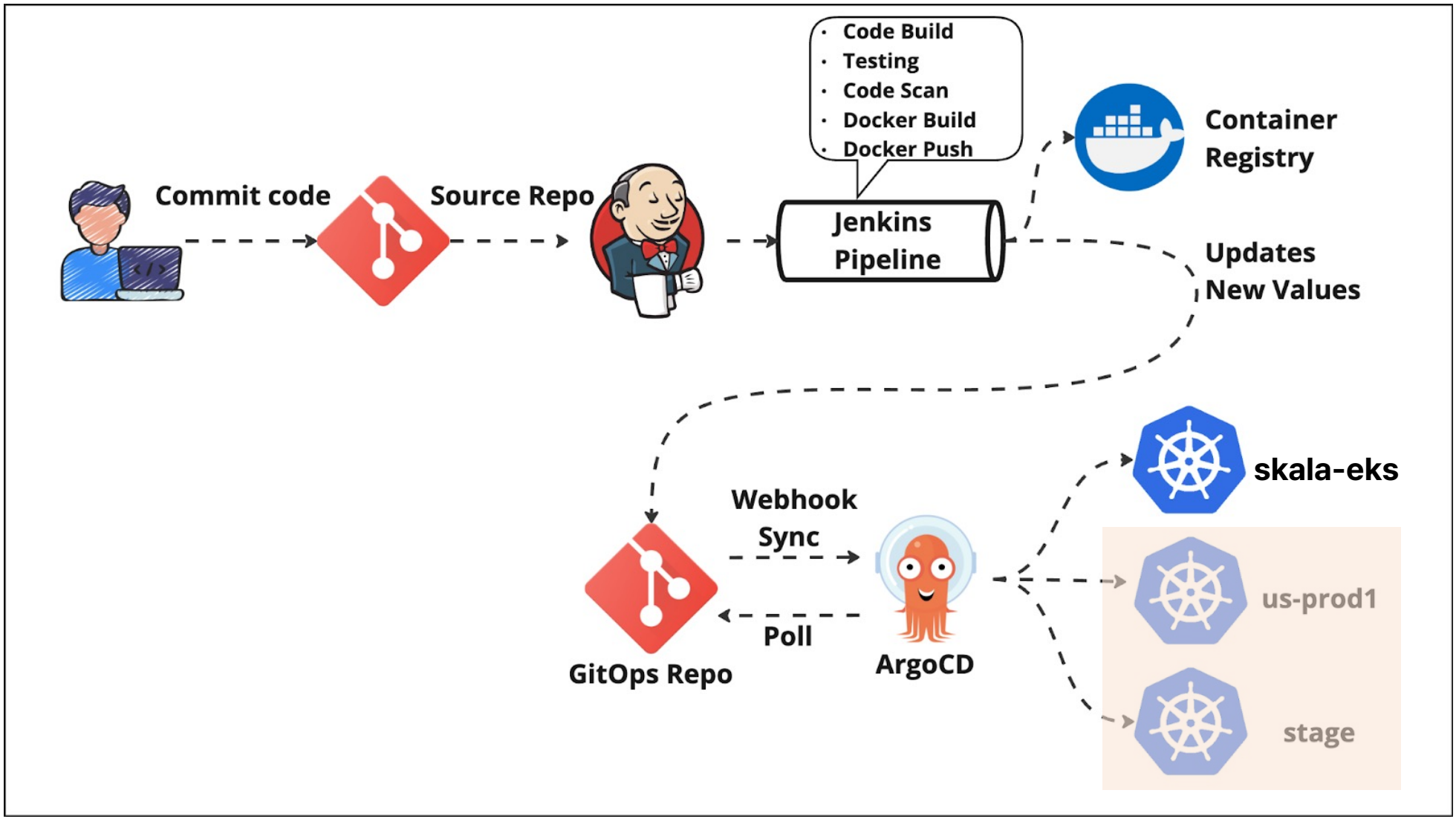
- **List Generator**
  element 리스트 기반 n개의 Application을 구성
- 각 element마다 템플릿 변수 {{environment}}, {{path}}, {{server}}, {{namespace}} 값이 달라짐
- 두개의 EKS API 서버를 가리키고, 경로만 dev/prod 지정

template

generator가 만든 값을 넣어서 실제 Application 모양을 정의하는 틀

- List 변수 값 기준 여러개의 Application을 구성
- 구성된 Application에 따라 여러개의 Application을 배포

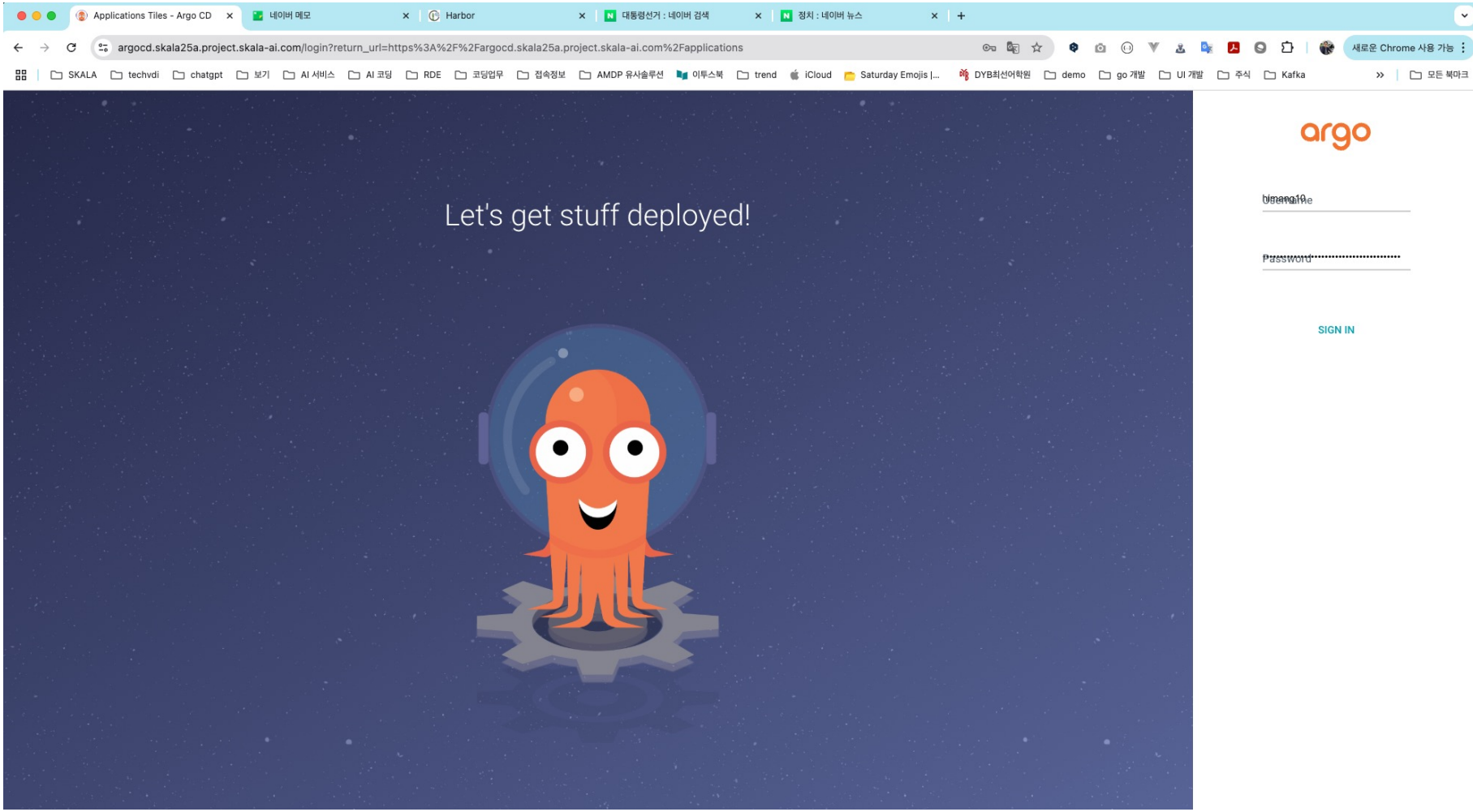
파이프라인 구성 흐름


Code Build
Testing
Code Scan
Docker Build
Docker Push
Container Registry
Commit code
Source Repo
Jenkins Pipeline
Updates New Values
skala-eks
Webhook Sync
Poll
GitOps Repo
ArgoCD
us-prod1
stage

ArgoCD 접속하기

ArgoCD 접속: https://skala-argocd.skala-??.com

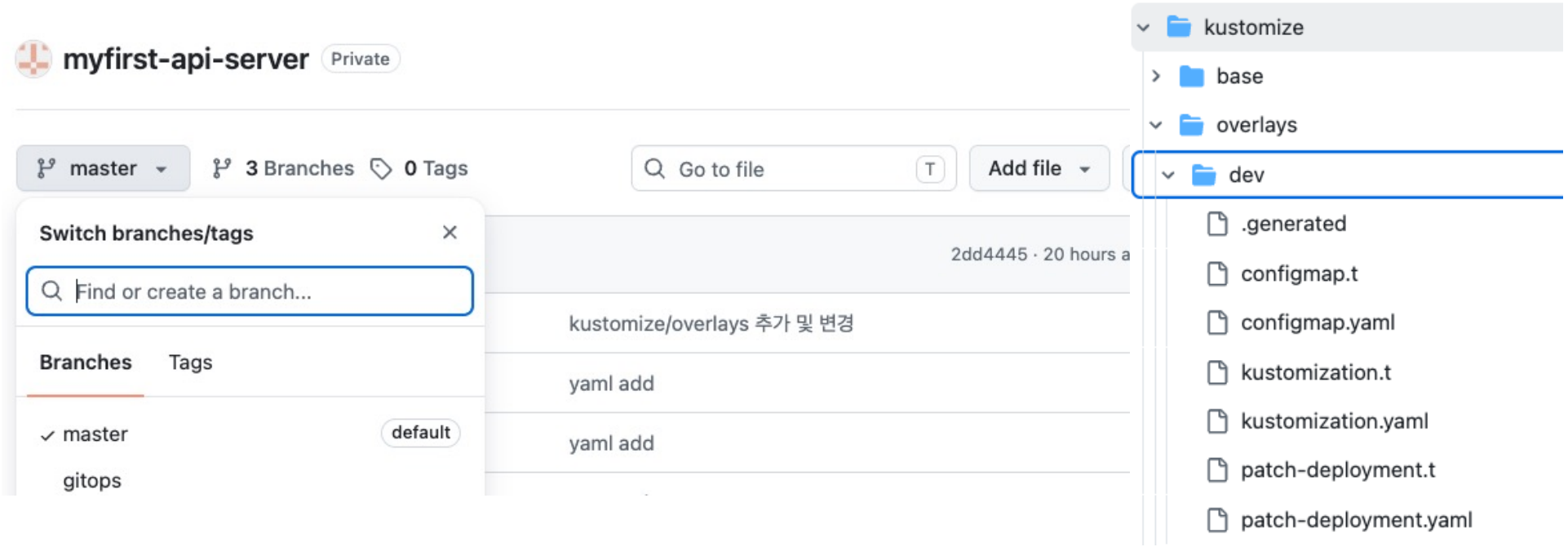
id: , **password:**



myfirst-api-server 컨테이너를 k8s 에 자동 배포하기

앞서 Jenkins 과정을 통해 myfirst-api-server git repo를 구성하고

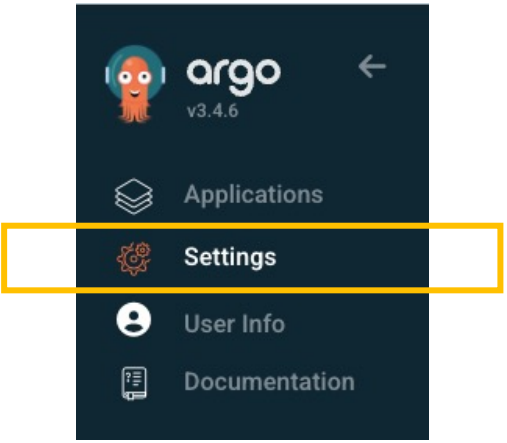
gitops branch 에 업데이트했던 kustomize 리소스를 기반으로
Application이 자동 배포되도록 구성하기



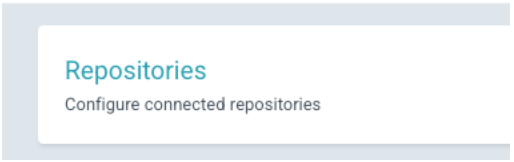
1. Repository 등록하기 (1/3)

git repository 를 등록해보자

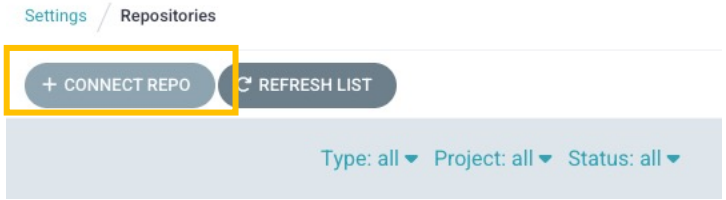
1. Settings 선택하기



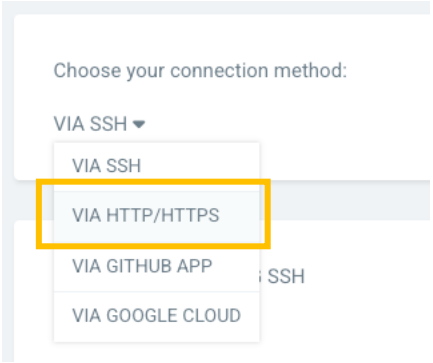
2. Repositories 선택



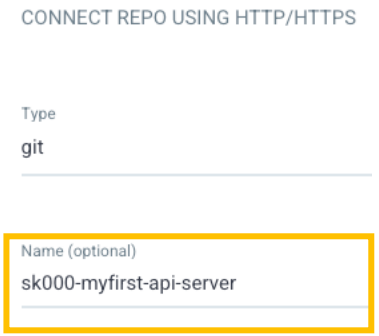
3. Connection repo클릭



4. VIA HTTP/HTTPS 선택



5. 이름 등록 (sk000 처럼)



1. Repository 등록하기 (2/3)

6. Git Repository 정보 등록

Project

class-0 프로젝트 (AppProject) 명

Repository URL

https://github.com/himang10/myfirst-api-server.git git repository URL

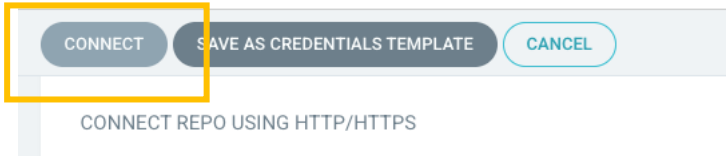
Username (optional)

himang10 git rep 계정 이름

Password (optional) PAT Token

예시) github_pat_xxxx

7. CONNECT 클릭



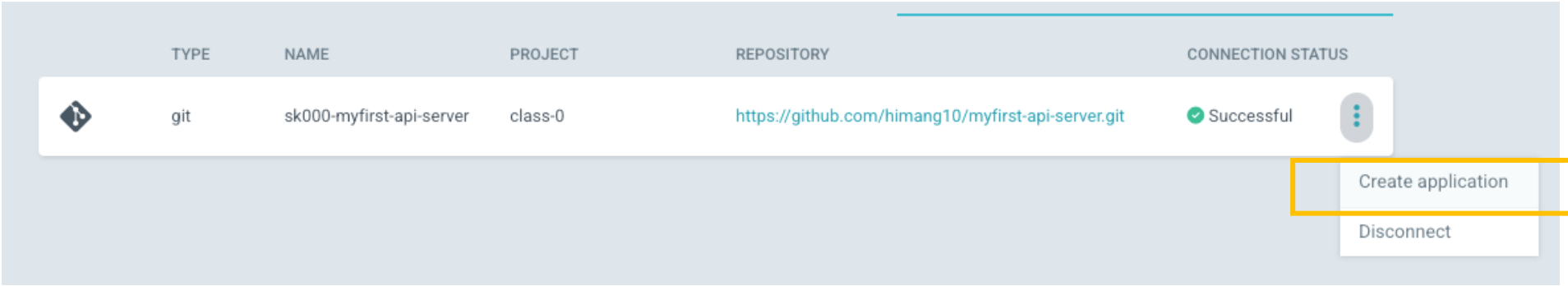
1. Repository 등록하기 (3/3)

8. Repository 가 정상적으로 생성된 경우

2. Application 등록하기 (1/4)

Repository기반으로 나의 Application 등록하기

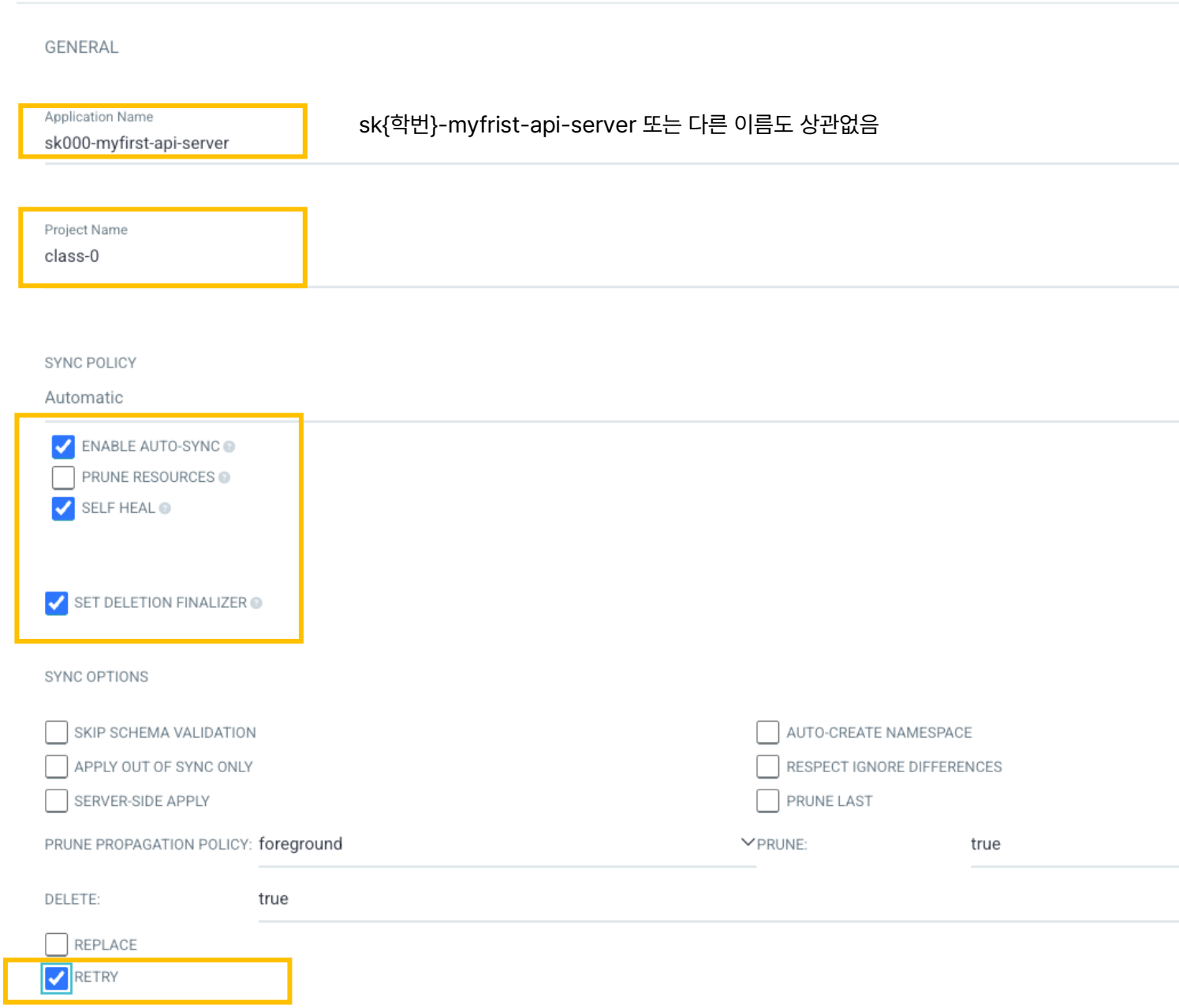
1. Create Application



2. Application 등록하기 (2/4)

2. Application yaml 구성하기

2.1. General 설정



2. Application 등록하기 (3/4)

2.2. Git Repo 내 branch / path 설정 (Source)

SOURCE

Repository URL

https://github.com/himang10/myfirst-api-server.git

Revision

gitops

kustomize branch 정보

Path

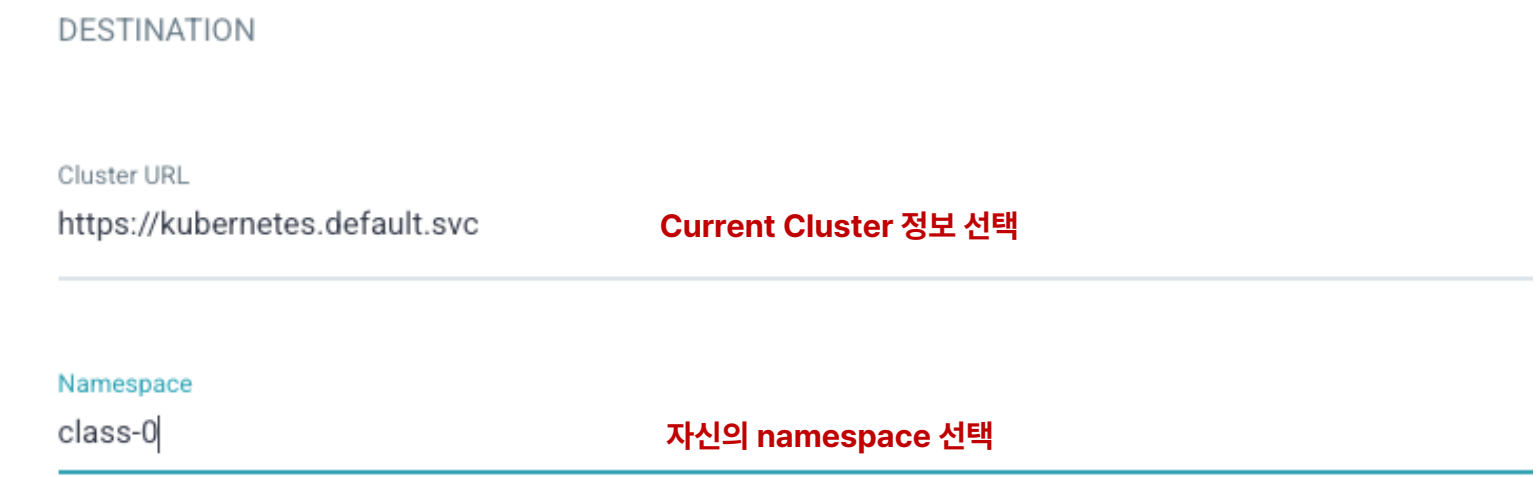
kustomize/overlays/dev

kustomize path 정보

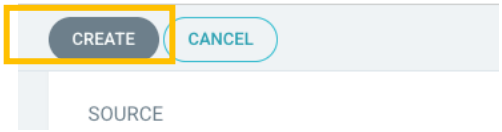
kustomize/overlays/dev

2. Application 등록하기 (4/4)

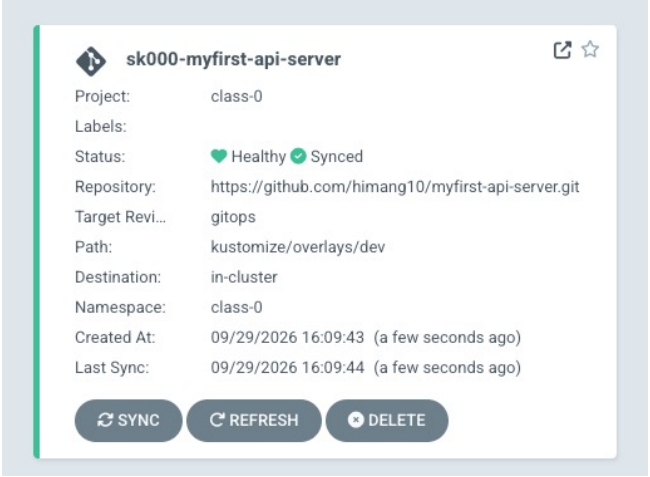
2.3. 배포를 위한 Target Cluster / namespace 정보 설정

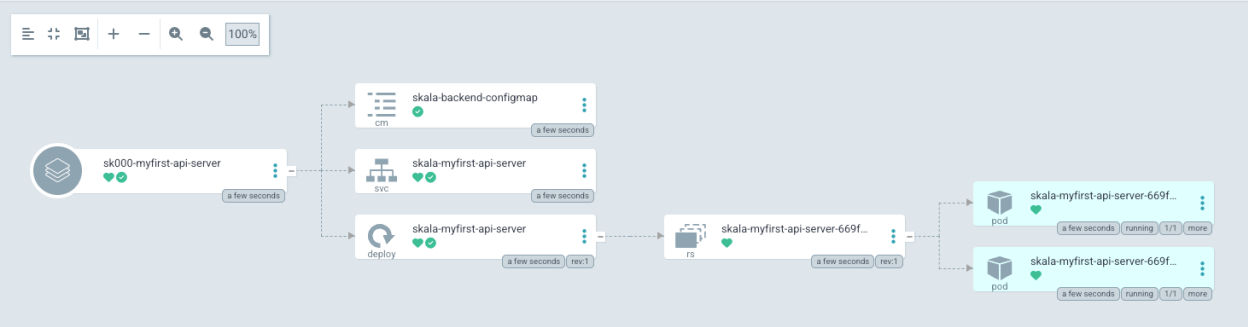


생성



3. 생성된 결과 확인 (1/3)





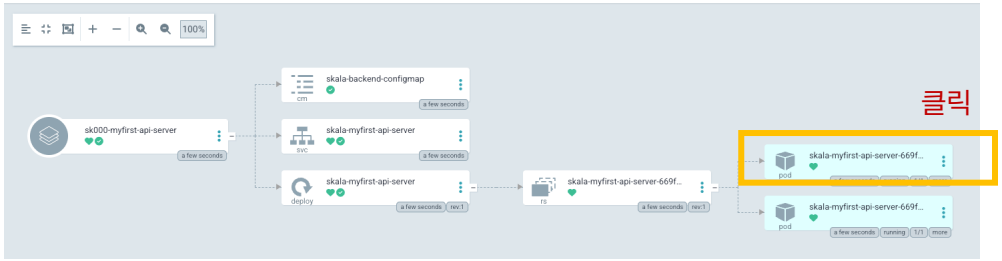
배포 현황 확인하기

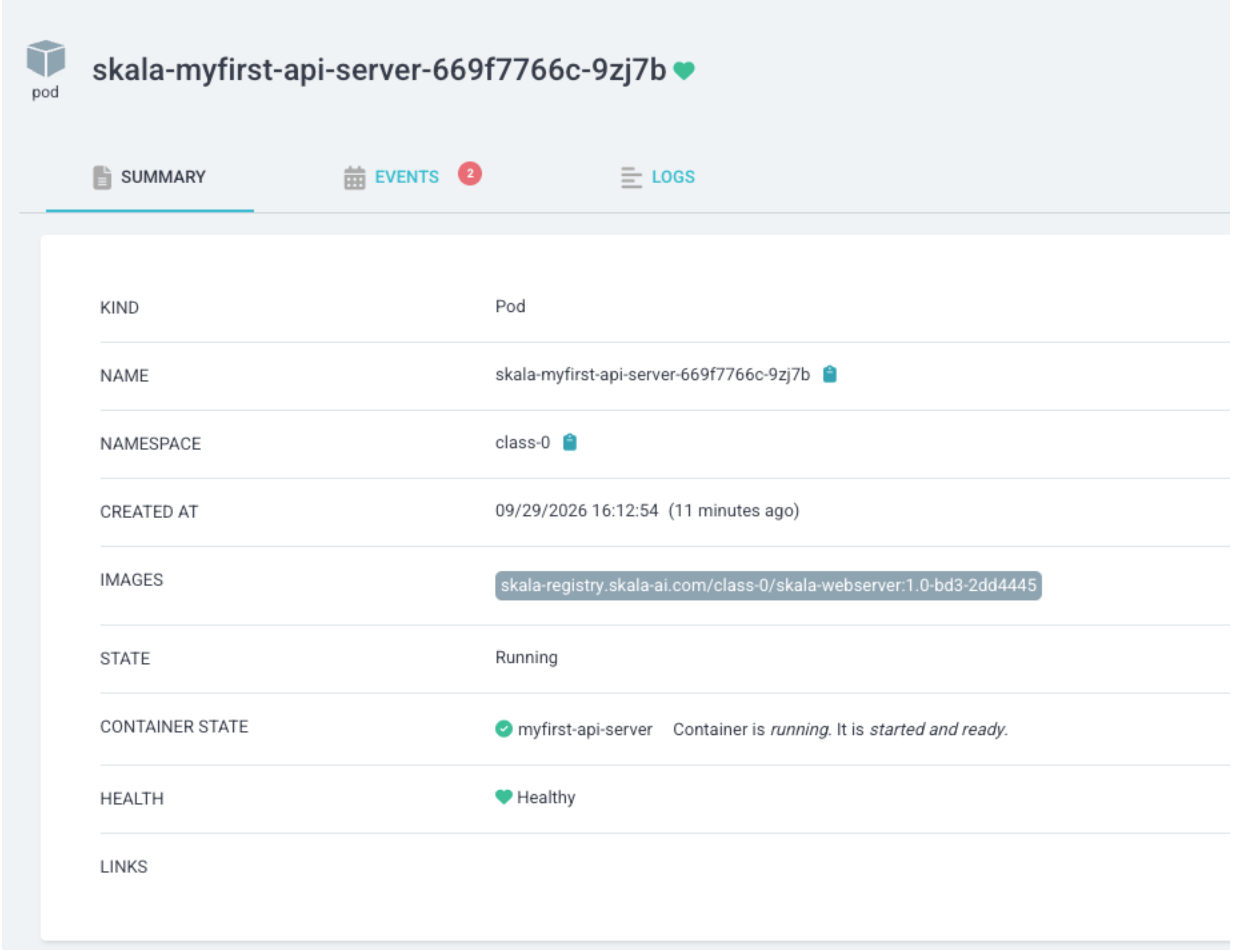
```
> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-554ccc5bfd-dgcc8   1/1     Running   0          78m
```

deployment 를 제거해보고 복구 확인해보기

```
> k get deploy skala-myfirst-api-server
NAME                        READY   UP-TO-DATE   AVAILABLE   AGE
skala-myfirst-api-server    1/1     1            1           99m
```

3. 생성된 결과 확인 (2/3)


100%
skala-backend-configmap
sk000-myfirst-api-server
skala-myfirst-api-server
skala-myfirst-api-server
skala-myfirst-api-server-669f...
skala-myfirst-api-server-669f...
skala-myfirst-api-server-669f...
클릭


pod
skala-myfirst-api-server-669f7766c-9zj7b
SUMMARY
EVENTS
2
LOGS
KIND
Pod
NAME
skala-myfirst-api-server-669f7766c-9zj7b
NAMESPACE
class-0
CREATED AT
09/29/2026 16:12:54 (11 minutes ago)
IMAGES
skala-registry.skala-ai.com/class-0/skala-webserver:1.0-bd3-2dd4445
STATE
Running
CONTAINER STATE
myfirst-api-server
Container is running. It is started and ready.
HEALTH
Healthy
LINKS

3. 생성된 결과 확인 (3/3)

배포 현황 확인하기

```
> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-554ccc5bfd-dgcc8   1/1     Running   0          78m
```

deployment 를 제거해보고 복구 확인해보기

```
> k get deploy sk000-myfirst-api-server
NAME                       READY   UP-TO-DATE   AVAILABLE   AGE
sk000-myfirst-api-server   1/1     1            1           99m

> k delete deploy sk000-myfirst-api-server
```

```
> k get pod -l app=sk000-myfirst-api-server
NAME                                        READY   STATUS    RESTARTS   AGE
sk000-myfirst-api-server-554ccc5bfd-c5xh4   1/1     Running   0          78m
```

Deployment를 삭제했는데 다시 Deployment 와 Pod 가 살아 있는 이유?

4. jenkins에서 CI 파이프라인을 실행해보기

Jenkins myfirst-api-server CI 파이프라인을 실행 해보기

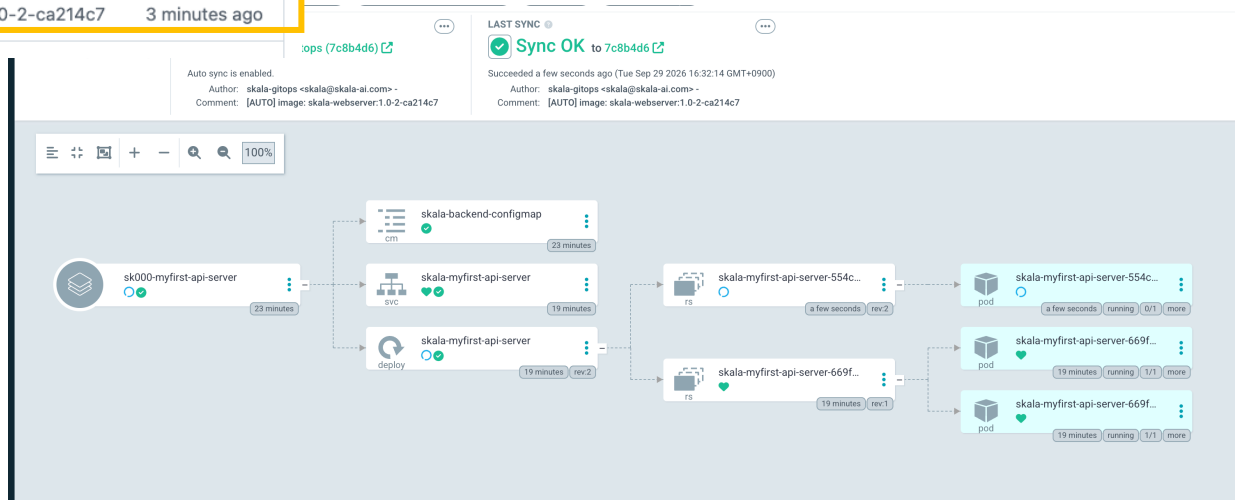
파이프라인 실행 후

1. git repository의 gitops branch가 갱신되었는지 확인
   (deploy Container tag 정보 확인)
2. argocd Application에서 새로운 버전 동작 확인

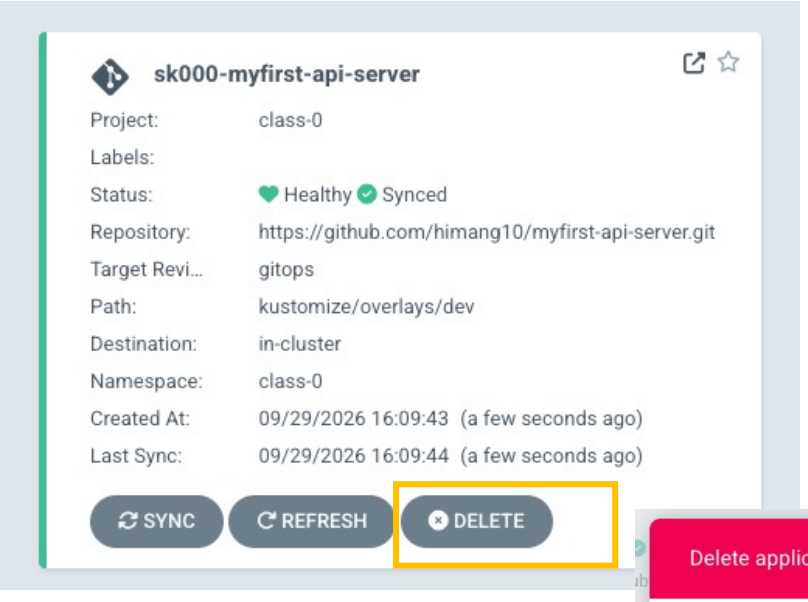
Name	Last commit message	Last commit date
..		
.generated	yaml add	2 days ago
configmap.t	kustomize/overlays 추가 및 변경	2 days ago
configmap.yaml	yaml add	2 days ago
kustomization.t	env.properties added	2 days ago
kustomization.yaml	[AUTO] image: skala-webserver:1.0-2-ca214c7	3 minutes ago

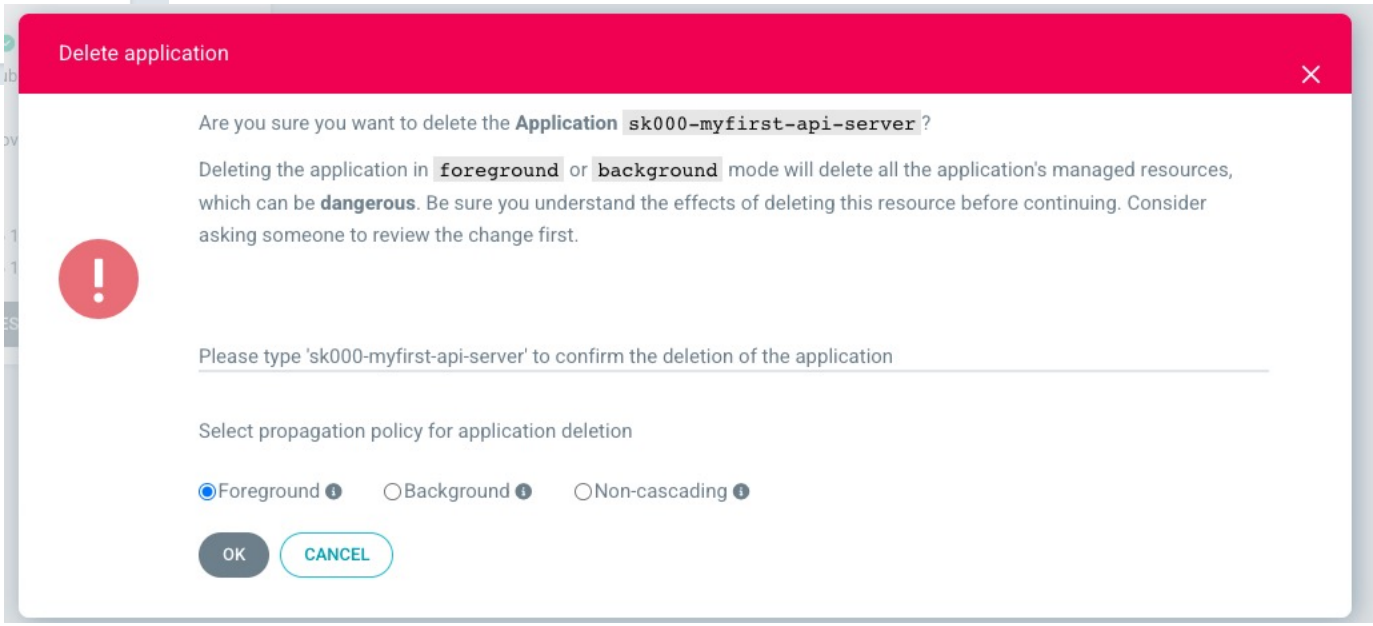
최종 gitops 업데이트 후

최대 3분 (180 초) 뒤 자동 동기화 실행



Application 삭제 하기





ArgoCD Cluster Secret 구조

소스코드 위치: 05.argocd/cluster.yaml

새로운 Cluster를 등록하는 방법은 Console로 지원하지 않고 k8s secret를 사용해서 등록해야 합니다.

클러스터 등록을 위한 Secret

```
vi ~/.kube/config
```

KUBECONFIG File

```yaml
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJC
    server: https://96BD83E8CE5CE0396D006BC5CEB350B0.gr7.
  name: arn:aws:eks:ap-northeast-2:881490135253:cluster/s
contexts:
- context:
    cluster: arn:aws:eks:ap-northeast-2:881490135253:clus
    namespace: skala-practice
    user: skala-admin-sa
  name: skala-edu-cluster
current-context: skala-edu-cluster
users:
- name: skala-admin-sa
  user:
    token: eyJhbGciOiJSUzI1NiIsImtpZCI6Imx5ZkVoa3FGTGVGcV
```

devops/argocd/cluster.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: skala-admin-cluster
  namespace: skala-argocd
  labels:
    argocd.argoproj.io/secret-type: cluster
type: Opaque
stringData:
  name: skala-2025
  server: https://96BD83E8CE5CE0396D006BC5CEB350B0.gr
  config: |
    {
      "bearerToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6Im>
      "tlsClientConfig": {
        "insecure": false,
        "caData": "LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS
      }
    }
```

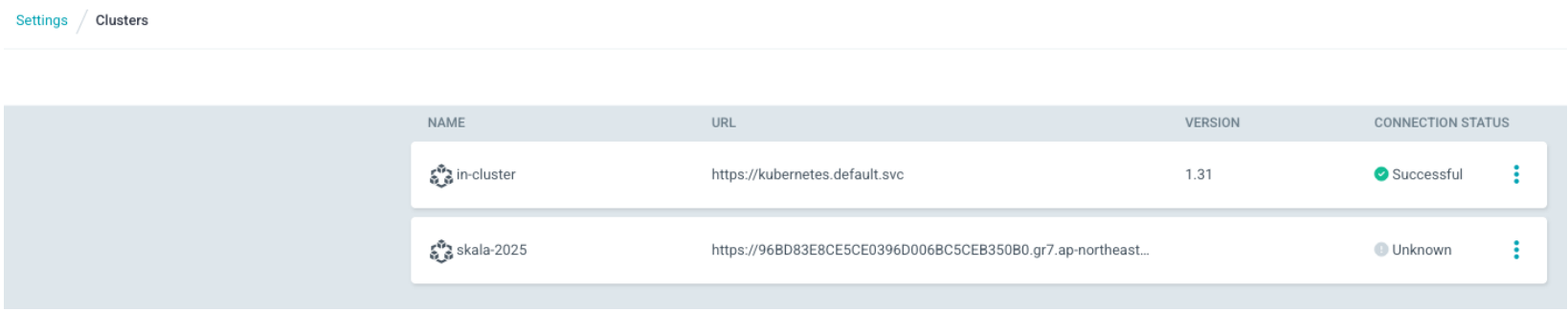
labels 에 argocd.argoproj.io/secret-type: cluster로 설정 시 자동 cluster로 등록

ArgoCD Cluster Secret 구조

소스코드 위치: 05.argocd/cluster.yaml

기본 클러스터는 in-cluster로 ArgoCD가 배포되어 있는 클러스터 정보가 표시

메뉴 : Settings > Clusters >



[수동 배포] git repository 등록하기

소스코드 위치: 05.argocd/repo/git-repository.yaml

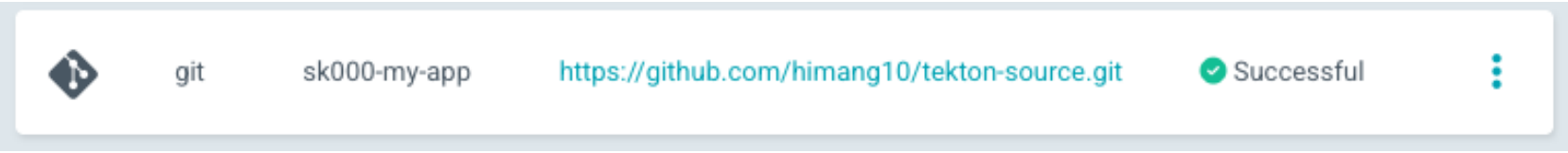
Repository를 등록하는 방법은 2가지로 제공한다. 1) Yaml 이용, 2) Console 이용

git-repository.yaml apply

```
> k apply -f git-repository.yaml
> k get secret -n argocd | grep sk000
NAME                TYPE     DATA   AGE
sk000-git-repo      Opaque     3     53m
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: skala-myapp-repo
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
type: Opaque
stringData:
  name: skala-myapp
  type: git
  url: https://github.com/himang10/myfirst-api-server.git
  username: himang10
  project: class-0
  password: github_pat_11AHJLYJI0K43BG3Qp0PjL_blj67YylYddPtGz
```

메뉴 : Settings > Repository >



[수동 배포] Application 배포하기 (1/3)

my-app 을 위한 CD 파이프라인을 구성을 위한 Application 생성

소스코드 위치: 05.argocd/application/application.yaml

```
> kubectl apply -f application/application.yaml

> kubectl get application -n argocd | grep sk000
sk000-myapp   Synced        Healthy
```

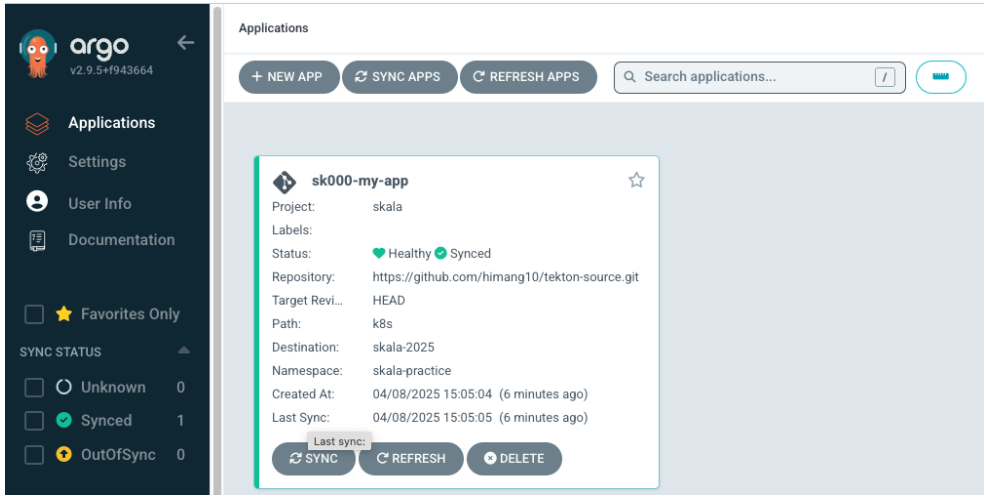
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: skala-myapp
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io/foreground
spec:
  destination:
    namespace: class-0
    server: https://kubernetes.default.svc
  project: class-0
  source:
    path: k8s
    repoURL: https://github.com/himang10/myfirst-api-server.git
    targetRevision: gitops
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
    - PrunePropagationPolicy=background
```

Application 삭제 시 k8s Resource 함께 삭제

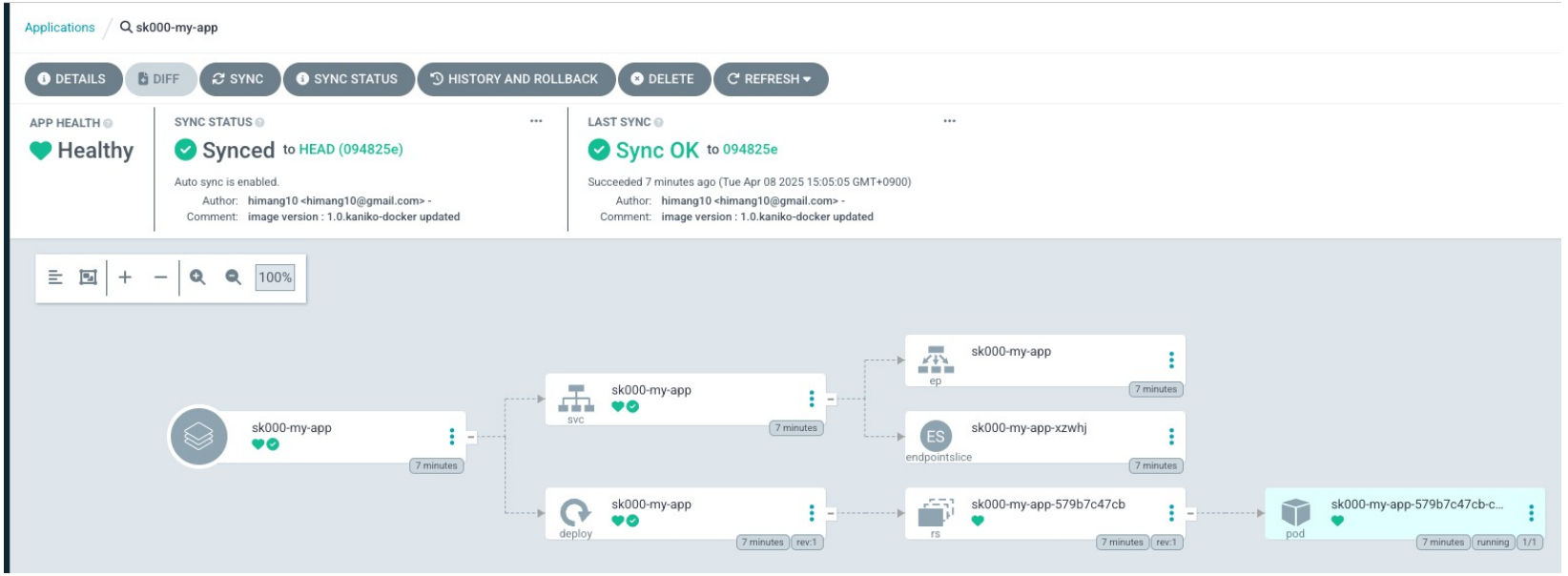
자동 동기화 실행

[수동 배포] Application 배포하기 (2/3)

my-app 을 위한 CD 파이프라인을 구성을 위한 Application 생성



기능 확인하기

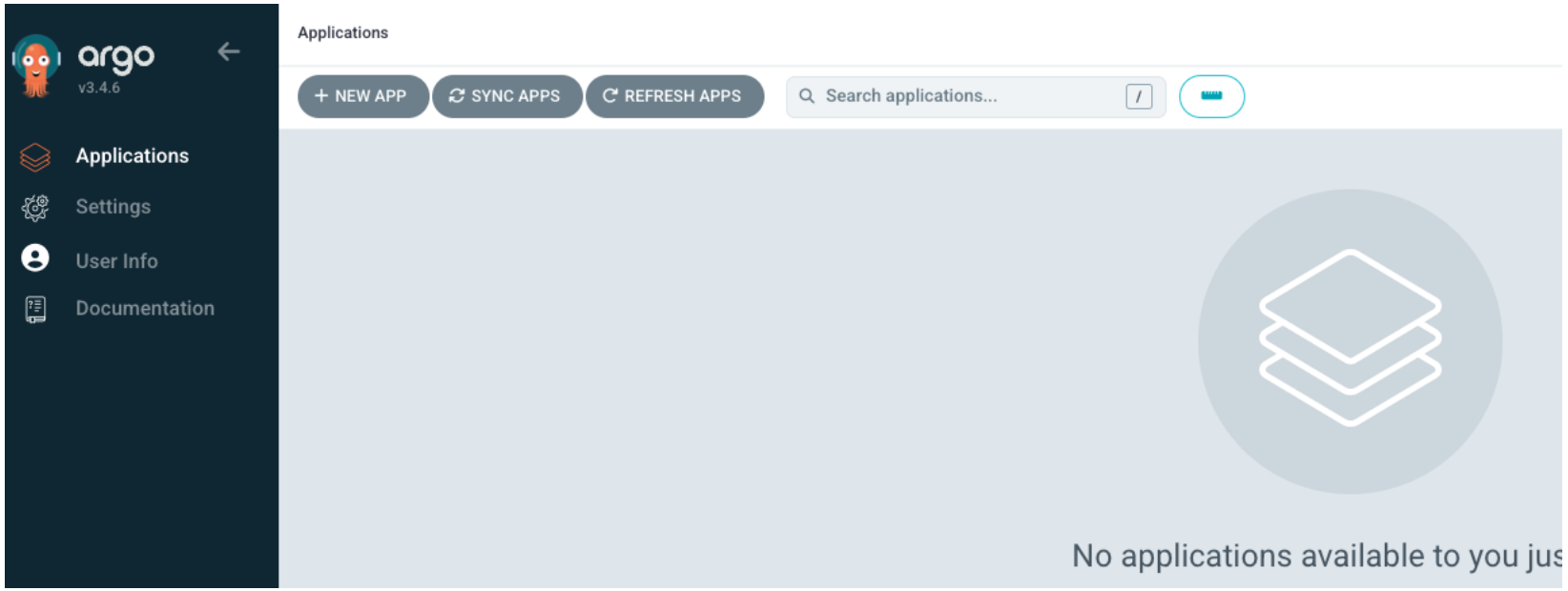


[수동 배포] Application 배포하기 (3/3)

Application을 삭제하기

```
> k delete -f application/application.yaml
```

화면 확인



[수동 배포] Kustomize Application 배포하기 (1/3)

자신의 myfirst-api-server을 kustomize 로 배포하기

소스코드 위치: 05.argocd/application/application-kustomize.yaml

```
> kubectl apply -f application/application-kustomize.yaml

> kubectl get application -n argocd | grep sk000
sk000-myapp-kustomize   Synced        Healthy
```

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: skala-myapp-kustomize
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io/foreground
spec:
  destination:
    namespace: class-0
    server: https://kubernetes.default.svc
  project: class-0
  source:
    path: kustomize/overlays/dev
    repoURL: https://github.com/himang10/myfirst-api-server.git
    targetRevision: gitops        # 예: HEAD, main, gitops 등
    kustomize: {}
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
    - PrunePropagationPolicy=background
```

ApplicationSet 구조 이해

단일 git repository를 기준으로 여러 Target 에 배포하는 방식 소스코드 위치: 05.argocd/application/example-applicationset.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: sk000-myfirst-app-set
  namespace: skala-argocd
spec:
  generators:
  - list:
      elements:
      - environment: dev
        path: k8s/dev
        server: https://96BD83E8CE5CE0396D006BC5CEB350B0.gr7.ap-northeast-2.eks.amazonaws.com
        namespace: skala-dev
      - environment: prod
        path: k8s/prod
        server: https://96BD83E8CE5CE0396D006BC5CEB350B0.gr7.ap-northeast-2.eks.amazonaws.com
        namespace: skala-prod
  template:
    metadata:
      name: sk000-myfirst-api-server-{{environment}}
      namespace: skala-argocd
    spec:
      destination:
        namespace: '{{namespace}}'
        server: '{{server}}'
      project: skala
      source:
        path: '{{path}}'
        repoURL: https://github.com/himang10/myfirst-api-server.git
        targetRevision: gitops
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
        - PrunePropagationPolicy=background
```

[참고] Helm Chart Application CR로 등록하기

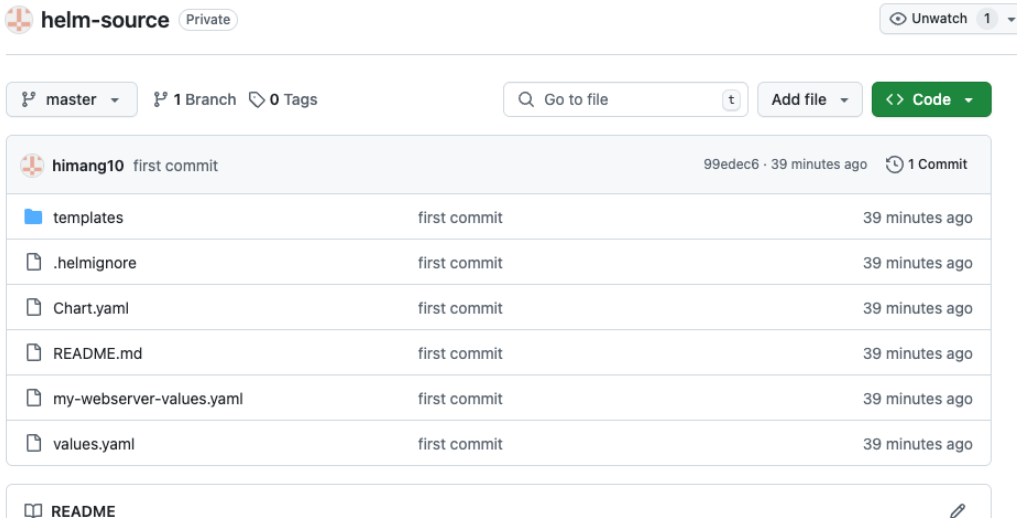
my-app 을 위한 CD 파이프라인을 구성을 위한 Application 생성

소스코드 위치: 05.argocd/application/application-helm.yaml

cloud/devops/argocd/helm-application.yaml

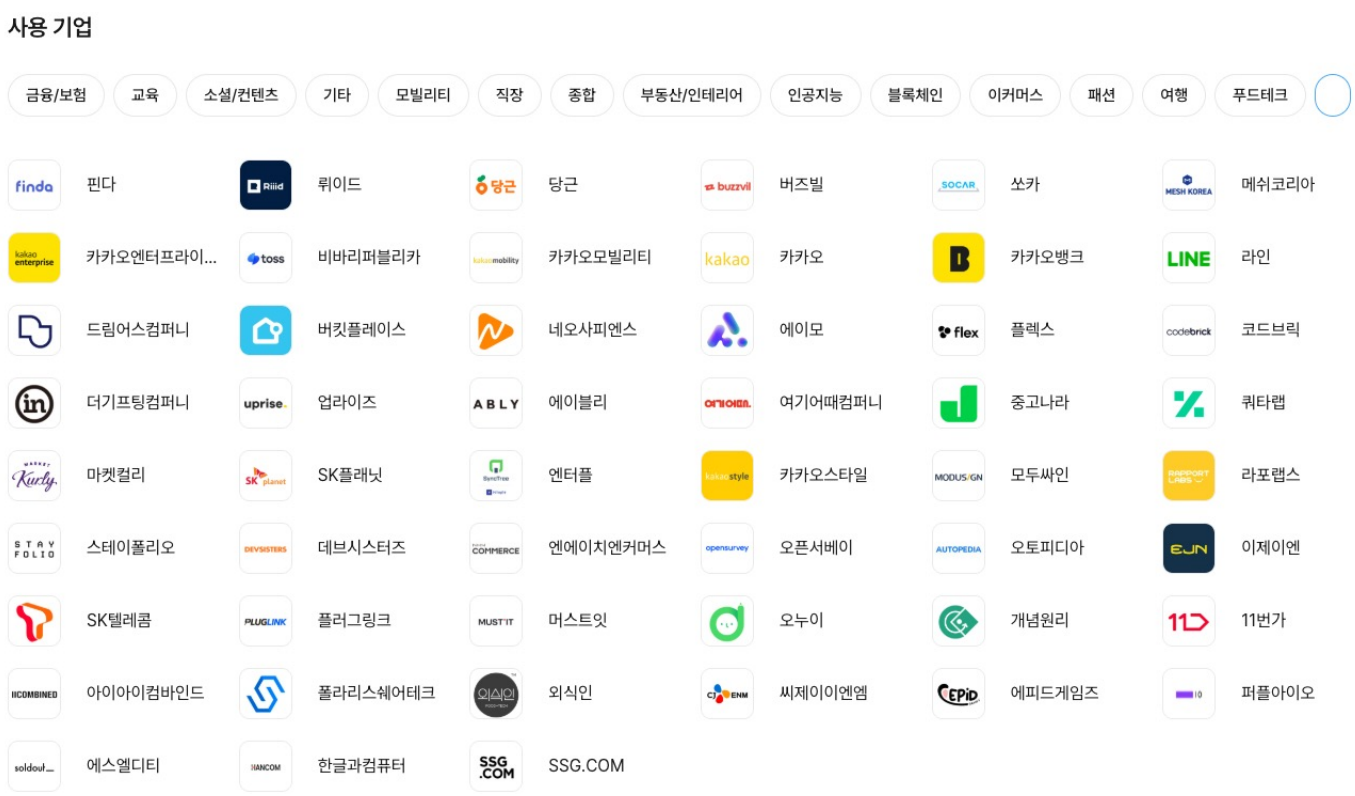
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: sk000-helm-my-app
  namespace: skala-argocd
spec:
  destination:
    namespace: skala-practice
    server: https://96BD83E8CE5CE0396D006BC5CEB350B0.gr7.ap-northeast-2.eks.amazonaws.com
  project: skala
  source:
    path: .
    repoURL: https://github.com/himang10/helm-source.git
    targetRevision: HEAD
    helm:
      valueFiles:
        - my-webserver-values.yaml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
    - PrunePropagationPolicy=background
```

helm내 custom values



[참고] Argo CD 적용 사례

수 많은 기업에서 사용 중인 사례



출처 : codenary.co.kr/techstack/detail/argocd

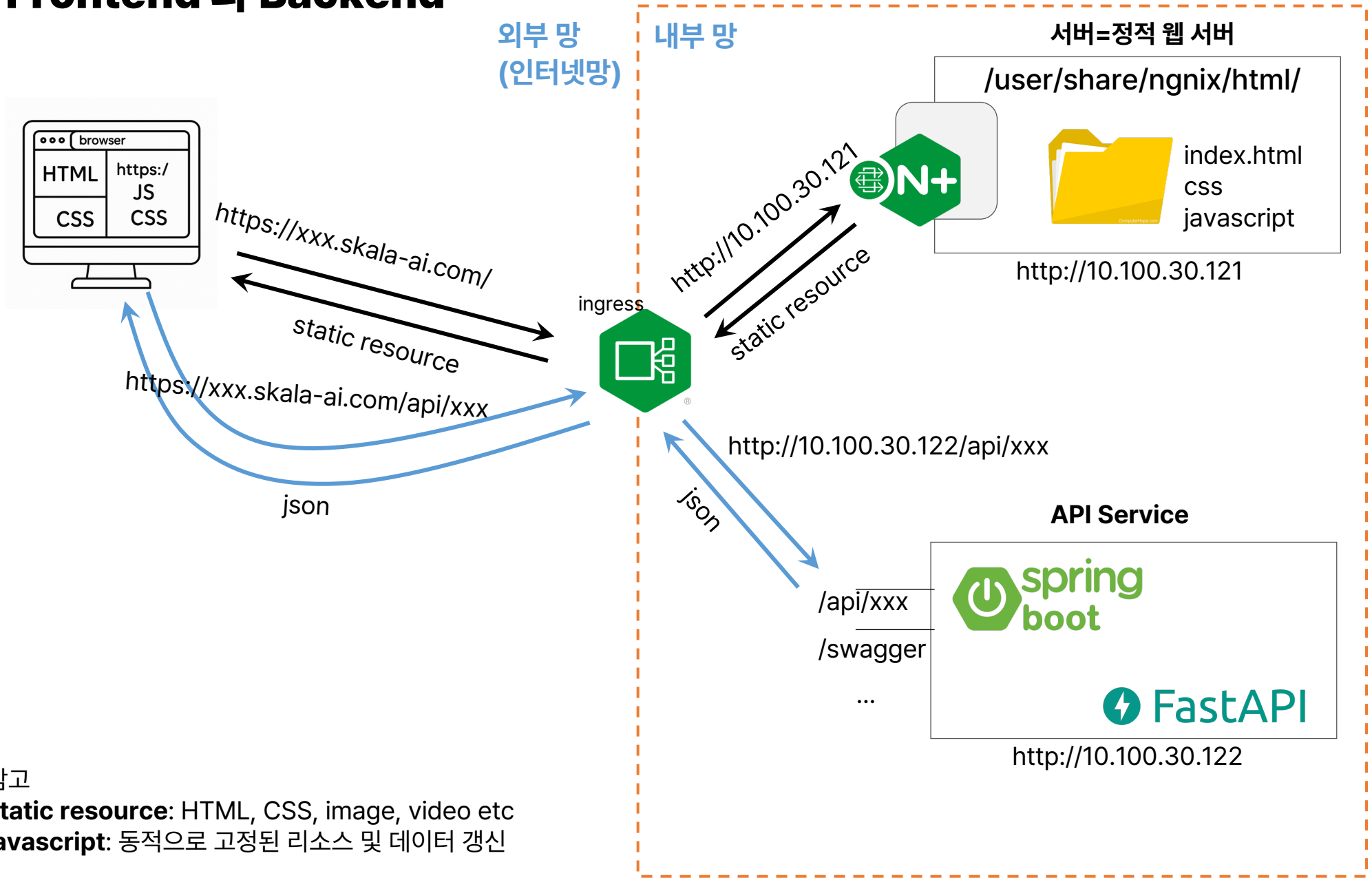
KDT DevOps – DevOps

나의 파이프라인 구성하기 실습

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Frontend 와 Backend



참고
static resource: HTML, CSS, image, video etc
javascript: 동적으로 고정된 리소스 및 데이터 갱신

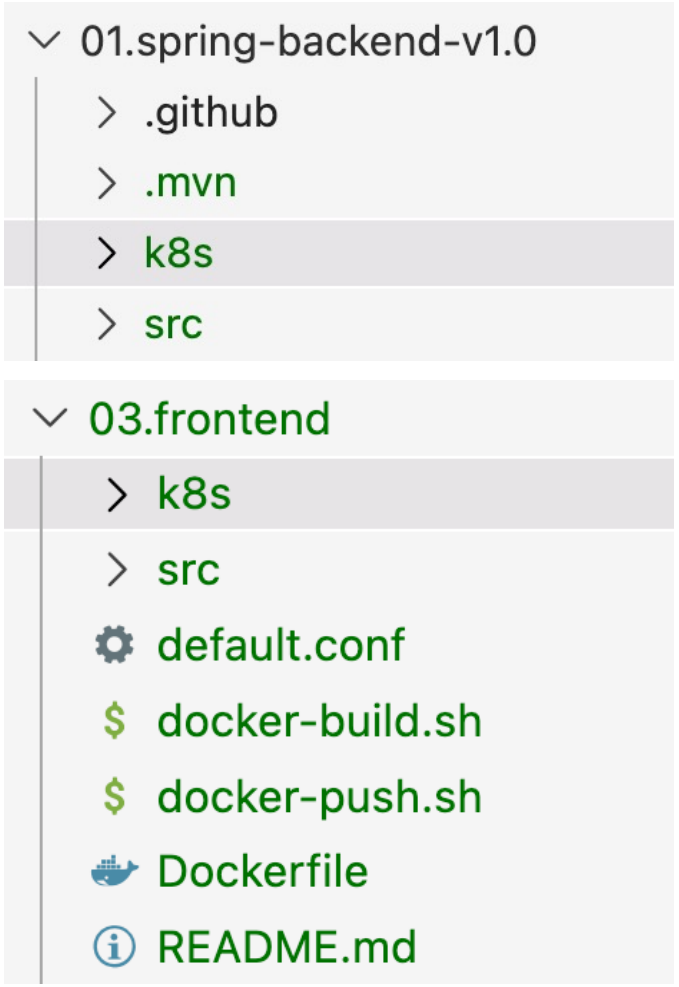
나의 앱을 위한 파이프라인 구성 실습

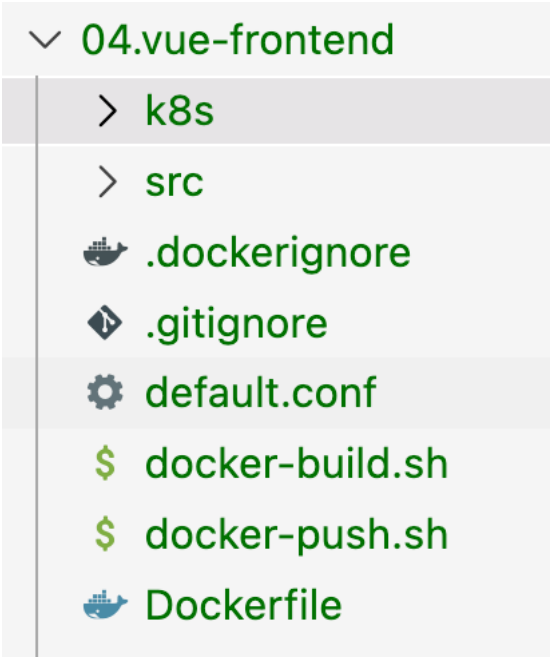
나의 앱을 위한 Jenkins + ArgoCD 파이프라인을 만들자

1. Frontend(Vue, React), Backend (Spring, Python) 등
2. 새로운 Git Repository를 만들자
3. CI Jenkinsfile 구성하기
4. ArgoCD 구성하기

[참고] 자신의 Frontend Backend가 없는 경우

00.sample-container/





수고하셨습니다.

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C